START FROM NOWHERE

The Build Playbook

How one person and a set of AI sessions built a multi-exam test-prep platform, every defect hit on the way, and the recipe for doing it again for something else

Built 2026-10-19 from commit 235fea9 · 221 commits over 30 days · 214 recorded defects · 17 chapters
This document is generated. Edit the chapters in docs/playbook/ and the ledger in data/playbook/incidents.jsonl, then run python3 src/build_playbook.py.

1. How to Use This Book
2. The Stack, Priced, and What It Would Cost in Open Source
3. From an Empty Repository to a Live Site
4. The Build System
5. The Interface: One System, Defined Once
6. The Algorithm: Adaptive Selection and Honest Scoring
7. Content at Scale, Without Lying About It
8. The Data Layer
9. Payments
10. Business Intelligence
11. Tests and Guards
12. Running the Build as an AI Loop
13. What the Ledger Says About Itself
14. The Defect Ledger
15. The Checklist
16. Adapting This to a Different Business
17. The Bootstrap Pack: Starting a New Project With This Loaded

How to Use This Book

This is the record of building one platform, written so it can be used to build a different one.

The platform is Start From Nowhere, a test-preparation site with five adaptive exam trainers, a college and business-school rankings library, a blog, a forum, subscriptions through two payment processors, and an admin console. It was built between 2026-09-19 and 2026-10-19, which is 30 days, across 221 commits, by one owner directing a series of AI coding sessions. As of this build it is 90 Python files, 112 JavaScript files, 24 TypeScript edge functions, 35 migrations and 65 documents: 2154 tracked files in total.

None of those numbers were typed. They are measured from the repository every time this document is built, which is the first thing worth copying.

What is in here

The book has four kinds of chapter, and they age differently.

The recipe chapters say how to stand up each part of the system: the infrastructure, the build, the interface, the algorithm, the content pipeline, the database, payments, business intelligence. These are the slowest to rot, because they are mostly about sequence and tradeoff rather than about any particular API.

The defect ledger is every bug, error and misfire that cost real time, with what was seen, why it happened, how it surfaced, what fixed it, and what now stops it recurring. Each entry ends with a lesson stated without reference to this codebase, because that is the part that transfers.

The analysis chapter is computed from the ledger. It counts how defects were actually found, which failure modes dominate, and which guards have already failed more than once. A handwritten guide cannot contain this chapter, because it has to be recounted every time the ledger changes.

The bootstrap pack is the part you use to start something new. It is a set of ready-to-paste artefacts for opening a fresh Claude project on a different business idea, with everything this build learned already loaded.

How to actually use it

Starting something new. Go straight to the last chapter, The Bootstrap Pack. It gives you three files and the order to use them in. You do not need to read the rest first; the pack carries the distilled rules.

Building a specific part. Read the matching recipe chapter, then read the ledger entries for that area. The recipe says what to do; the ledger says what will go wrong. The second is worth more.

Reviewing work before it ships. Use The Checklist. It is generated from the ledger, so every line on it exists because something went wrong once, and nothing is on it for completeness.

How this document stays true

It is built, not maintained. Three sources go in:

1. `docs/playbook/*.md`, the prose. Written by a person; this is the judgement.
2. `data/playbook/incidents.jsonl`, the defect ledger. One JSON record per incident,

carrying the commit that fixed it and the file that now guards it.

1. The repository itself, harvested at build time for every figure in the text.

`python3 src/build_playbook.py` assembles them and writes Markdown, HTML, PDF and Word. Three things fail the build rather than shipping quietly:

- An unresolved `{{PLACEHOLDER}}`, which means a fact the document wanted no longer exists.

- An incident citing a commit hash or a guard file that cannot be found.
- An em dash or en dash, because the house style bans them and the document about the

house rules should not break them.

That last one is a joke with a serious point behind it. A rule that is only enforced by attention is a rule that decays. Every rule worth having should have something that notices when it is broken.

Adding to it

When something breaks, add a record to `data/playbook/incidents.jsonl` before you fix it, while you still remember what you believed was true five minutes ago. That belief is the most valuable field and the first one you lose.

The record needs: what was seen, why, how it surfaced, the fix, the guard that now catches it, and the lesson stated so it makes sense to someone who has never seen this code. The `guard` field is what makes the ledger adaptive: the checklist chapter is generated from those fields, so writing the record is the only step. Nothing has to be copied anywhere.

If two records name the same guard, the analysis chapter will say so, and that means the guard did not hold and needs rebuilding rather than trusting.

The Stack, Priced, and What It Would Cost in Open Source

The whole platform runs for roughly the price of a couple of lunches a month. That is not a boast, it is the design constraint that produced most of the interesting decisions in this book, and it is reproducible.

What is actually running

LAYER	CHOICE	WHY THIS ONE	MONTHLY AT THIS SCALE
Hosting and CDN	Cloudflare Workers with static assets	Global edge, generous free tier, and a real runtime in front of the files when you need one	$0 to $5
Domain	Registrar of choice at cost	Nothing clever here	about $1 amortised
Database, auth, storage	Supabase (managed Postgres)	Row Level Security means the database enforces authorisation, not the app	$0 to $25
Serverless functions	Supabase Edge Functions (Deno)	Same project, same secrets store, no second vendor	included
Payments	Stripe Checkout, Apple In-App Purchase	Checkout removes PCI scope entirely; Apple is mandatory if you ship an iOS app	percentage of revenue
Email	Transactional provider of choice	Deliberately last. Do not build this before you have users	$0 at low volume
Source and CI	GitHub with Actions	The deploy hook and the test runner live in the same place as the code	$0
Error and product telemetry	Written in house, about 200 lines	Covered below; this is the surprising one	$0
Analytics	First-party beacon, written in house	Covered below	$0
Charts	Written in house, one file	Covered below	$0

Total fixed cost at the scale this platform runs: under thirty dollars a month, most of which is the database.

The three things not to buy

These are the decisions that saved the most money and, more importantly, the most complexity. Each replaced a product that would have been reasonable to buy.

Analytics. A first-party beacon is a `fetch` on page load carrying path, referrer, UTM parameters, a session id from `sessionStorage`, device class and time on page, into one table. Country and a salted address hash are added by a database trigger so the raw address is never stored. That is a few dozen lines. What you get that you cannot buy: the data is in your own database next to your users, so a question like "which landing pages precede a subscription" is a join rather than an integration project. What you give up: nothing an ad blocker was not already taking from the paid product.

Error monitoring. A global `error` and `unhandledrejection` handler posting to an insert-only table, with a clustering view and a human triage queue. The clustering matters more than the capture: raw errors are noise, and the value is in a human marking each cluster real, fixed, will not fix, or noise, so the queue gets quieter as it learns rather than louder. Nothing in it edits code, and that is deliberate.

Charts. A charting library is two hundred kilobytes and a second thing to pin in your Content Security Policy. Five chart forms plus a stat tile, built as inline SVG from numbers, is about six hundred lines and smaller than the smallest library's minified core. The chapter on the interface explains why the colours in it are computed rather than chosen.

The rule behind all three: **buy the thing that would be dangerous to get wrong, build the thing that is mostly a shape.** Payments are dangerous. Authentication is dangerous. A bar chart is a shape.

The open-source equivalent of every paid piece

If the managed services are unavailable, too expensive, or politically impossible, here is the substitution, and honestly what it costs you.

MANAGED	OPEN SOURCE	WHAT CHANGES
Cloudflare Workers	Caddy or nginx on a small VPS; Nixpacks or Dokku for deploys	You now own TLS renewal, log rotation and the box. Budget half a day to set up and an hour a month forever
Supabase	Postgres plus PostgREST plus GoTrue, or the self-hosted Supabase compose file	You get the same API surface. You also get backups, upgrades and connection pooling as your job. This is the biggest jump in operational load of anything in this table
Supabase Edge Functions	Deno Deploy, or a plain Node or Deno process behind the reverse proxy	Straightforward. The awkward part is secrets, which you will end up managing with something like SOPS or a mounted env file
Stripe Checkout	There is no real substitute, and you should not want one	Self-hosting payments means PCI scope. Do not
GitHub Actions	Forgejo Actions, Woodpecker, or Drone	Fine. Slower to set up, one fewer vendor
Managed Postgres backups	pgBackRest or WAL-G to object storage	Works well and is genuinely worth learning even on a managed service, because it is also your migration path off it
Supabase Auth	Keycloak, Authentik, Ory Kratos	All three are more capable and considerably more work. Keycloak in particular is a serious piece of infrastructure
Cloudflare Web Analytics	Plausible, Umami, Matomo, all self-hostable	Umami is the lightest. But see the point above: once you have a first-party beacon into your own database, a second analytics product is mostly duplication
Playwright	Already open source	No change
Anything for charts	ECharts, Chart.js, Observable Plot	All good. Weigh the bundle against the six hundred lines
Anything for PDF	Headless Chromium print-to-PDF, WeasyPrint, Typst	Chromium is already on the machine if you run browser tests. That is how this document is

MANAGED	OPEN SOURCE	WHAT CHANGES
		produced

The honest summary of that table: the only irreplaceable paid component is the payment processor. Everything else is a convenience purchase, and the convenience is real but bounded. The thing you are actually buying from a managed database is not Postgres; it is somebody else being on call for it.

What the architecture buys you

Two structural choices shaped everything else, and both are worth copying.

Static pages, built ahead of time, with no runtime framework. Every page on the site is a self-contained HTML file with inline CSS and inline JavaScript, generated by a Python build. There is no server rendering, no hydration and no client-side router. The consequences compound: pages are fast because there is nothing to boot; the whole site can be served from a CDN for nothing; there is no dependency tree to keep patched; and the security surface of a page is what you can read in that page. The cost is that anything dynamic has to be deliberate, which turns out to be a feature.

The database enforces authorisation, not the application. Row Level Security on every table, and every privileged read behind a `SECURITY DEFINER` function that checks an admin table. The app cannot grant itself anything, because the app is a browser, and a browser is not trusted. This is what makes it safe to ship a site with no backend of your own.

Both decisions have the same shape: **push the guarantee down to the layer that cannot be bypassed.** A rule enforced in JavaScript is a suggestion.

From an Empty Repository to a Live Site

The order below is the one that worked, and the order matters more than any individual step. Most of what went wrong in this build went wrong because something was done before the thing that would have caught it.

Phase 0: the rules file, before any code

Write the project's standing rules into a file the AI session reads on every turn. On this project that is `CLAUDE.md`, 65 documents in, and it is still the highest leverage file in the repository.

It is not documentation. It is the constitution, and it should contain only things that are load-bearing and that you would otherwise have to say again:

- The house style rules that a build can enforce. Ours bans em dashes and en dashes

outright, everywhere, including commit messages. That sounds trivial. It means a single grep decides whether copy was written by hand or pasted from somewhere, and the build fails on a violation.

- The sourcing rules. Ours: never state a statistic, price or external fact from memory;

every published figure carries a source, a year and a URL; unverifiable means null, never a guess; and a named list of banned sources.

- **Never invent a value.** Every identifier, hash, id, count, price, date and URL that

goes into a tool call, a commit or a page must be read from real output first. This rule exists on this project because a forty-character hex string was once typed straight into a merge call as though it were a commit SHA. The API rejected it as malformed, which was luck. A well-formed guess would have merged something else.

- The communication rule. Ours: if a link or a service cannot be reached, say so

immediately and ask, before doing the work another way. Never quietly substitute a screenshot or a memory for the source that was pointed to.

Every one of those was written after something went wrong. Start with them anyway.

Phase 1: the build, before the content

Write the thing that turns sources into pages before you have many pages. Ours is `src/build.py`, and on day one it did almost nothing. What matters is that it exists, so that every guard you later need has somewhere to live.

By the end it does all of this, and each item was added the day it was needed:

- Injects shared header, footer and CSS into every page from one place
- Parses every inline script with node and fails on a syntax error
- Fails on an em or en dash in any hand-edited document or item bank
- Fails on any asset larger than the platform's per-file limit
- Counts what it produced and asserts the counts agree across artefacts
- Writes the sitemap from the same directory scan that writes the pages

None of these are clever. All of them exist because of an entry in the defect ledger.

Phase 2: one page, deployed, for real

Deploy the simplest possible page to the real domain before building anything else. Not a staging environment: the actual production host, on the actual domain, through the actual pipeline.

Everything you learn here is cheap now and expensive later. The per-file size limit. Which directories the asset uploader walks. Whether your headers file applies to responses your own code generates. Whether `www` and the apex are the same origin. That last one cost this project a split consent state and a diluted ranking signal, and it is ten lines of redirect in the right place.

Phase 3: the data layer, with the guarantees tested

Before the interface, get the database right, because it is the thing you cannot casually change later.

1. Every table has Row Level Security enabled, from creation. Not later.
2. A table with no policy denies everyone, which is the correct default for anything only

your server should touch.

1. Every privileged read goes through a `SECURITY DEFINER` function that checks admin

membership. Clients never read a privileged table directly.

1. Revoke from `PUBLIC` as well as from `anon` and `authenticated`. Revoking from the roles

you can name leaves `PUBLIC` holding the grant, and it will not show up in the place you think to look.

1. Test the guarantee, not the code. Write a SQL smoke test that asserts what must be true

and rolls itself back. Two trigger bugs on this project were found exactly that way and would not have been found by reading.

Phase 4: the interface system, before the interfaces

One file that defines the type scale, the tracking ramp, the spacing grid, one page width, one gutter and one reading measure, injected everywhere. On this project that is `src/partials.py`, currently 57 tokens.

The reason to do this early is arithmetic. This site reached thirty-seven font sizes, twenty-eight spacing values and eight page widths before the system landed, and pulling them back to one scale was a multi-day job that touched everything. Doing it on day one costs an hour.

Phase 5: the product

Now build the thing. By this point the build catches syntax errors, the database refuses unauthorised reads, the type system stops the interface drifting, and you can deploy.

Phase 6: instrumentation, before growth

Before spending anything on acquisition, be able to answer: how many people arrive, what they do, where they stop. That is the beacon, the funnel and the error queue. All three are small. All three are useless retroactively, which is the entire argument for doing them before you need them.

Phase 7: payments

Last, and only when there is something worth paying for. The payments chapter has the details. The one scheduling note: the event ledger that lets you answer "how much revenue arrived and how much left" has to exist **before** the first real subscriber, because it cannot be reconstructed afterwards from a table you overwrite on every webhook. This project learned that in the right order by luck rather than judgement.

The shape of the whole thing

Read backwards, the order is: make failure loud, make authorisation impossible to bypass, make the interface consistent, then build features, then measure, then charge. Each phase makes the next one cheaper to get wrong.

The Build System

The build is the most valuable thing in the repository, and it is the part people skip.

What it is

`python3 src/build.py` takes sources in `src/` and `data/` and writes every page of the site. Nothing in the deployed output is hand-edited; the generated directories are gitignored so there is no temptation. Sources of truth live in exactly one place, and the build is the only thing that reads them.

The build currently produces five exam trainer applications, a landing page, a rankings library of 94 business schools and a separate college library, exam guide pages, pricing, a blog, a forum shell, and the legal pages. It also produces this document.

Why a build rather than a framework

A framework gives you composition and gives up legibility. A build gives you a directory of finished files you can open, diff, grep and serve from anything. At this scale that trade is heavily in favour of the build, for a specific reason: **the output is inspectable, so guards can run on the output.**

That turns out to be where most defects were caught. Not in the source, in the artefact. Reading the built CSS rather than the template is what eventually found a nested comment that had killed the palette on over a thousand pages. Reading the built page is what found a stale count in a meta description that Google was showing.

The guards, and what each one is for

Every one of these exists because of an incident. They are listed here in the order they would catch a problem.

Parse every generated script. The build writes JavaScript into HTML. It runs node over each inline script and refuses to write the page on a syntax error. One unescaped quote inside a `onclick` string once took down the entire trainer, because a parse error kills the whole script, not the one handler.

Ban the characters you said you would ban. Em dashes and en dashes fail the build in any hand-edited document or item bank. It is a one-line check that makes a style rule real.

Assert the size of every collection. Item counts per category, page counts, sitemap entry counts, school counts. Not the contents, the size. Deletion by shadowing, a filter dropping rows, a loop over an empty list: none of these raise anything, and all of them show up instantly as a count that moved.

Assert how many things you checked. A guard that matched `\d{2,4}` silently stopped checking the two largest item counts when they passed ten thousand, and kept passing. Any guard that selects a subset should report the size of the subset it selected.

Assert the platform's hard limits. Cloudflare rejects a static asset over 25 MiB. The build now fails on any asset over that, because a deploy-time rejection is an expensive place to learn a number your build already knew.

Compare inputs to outputs at every filter. A length filter measuring the wrong unit silently dropped sixteen valid items. Every stage that can reject should say how many it rejected.

Derive, never type. Any number in copy that describes the size of something is computed from that thing at build time. A typed number has a half-life.

Reproducibility

Three consecutive builds must produce identical output. This is worth enforcing because the failure is subtle: this project's item generator seeded from Python's `hash()`, which is randomised per process, so every build produced a different bank and the pinned statistics in the test file drifted run to run. It looked like a flaky test. It was a broken promise. `zlib.crc32` fixed it.

If your build has any randomness, seed it from something stable and assert the reproducibility, or you will eventually delete a test that was telling you the truth.

The test ladder above the build

The build is the first rung. Above it, in the order they run:

- `src/review_bot.js`
- `src/smoke.js`
- `src/smoke_billing.js`
- `src/smoke_business.js`
- `src/smoke_calculator.js`
- `src/smoke_charts.js`
- `src/smoke_community.js`
- `src/smoke_consent.js`
- `src/smoke_daily.js`
- `src/smoke_dates.js`
- `src/smoke_fit.js`
- `src/smoke_funnel.js`

- `src/smoke_games.js`
- `src/smoke_guide.js`
- `src/smoke_ios.js`
- `src/smoke_items.js`
- `src/smoke_launch.js`
- `src/smoke_load.js`
- `src/smoke_offline.js`
- `src/smoke_pages.js`
- `src/smoke_playbook.js`
- `src/smoke_rankings.js`
- `src/smoke_redirect.js`
- `src/smoke_render.js`
- `src/smoke_sharing.js`
- `src/smoke_signup.js`
- `src/test.js`
- `src/weekly_audit.js`

28 test files in total. The layering is deliberate:

1. **The build** catches structural problems in the artefact.
2. **Engine tests** run the domain logic headlessly, once per exam.
3. **Smoke tests** drive a real browser against the real built site at desktop and phone

width, and assert behaviour rather than markup.

1. **Review bots** play the product adversarially and report what only use reveals.
2. **A weekly audit** re-checks contrast, links and metadata across every page.

Rung three is where the most valuable failures show up, and rung four is where the embarrassing ones do.

The Interface: One System, Defined Once

The type and spacing system

One file holds the whole visual system and is injected into every page. Currently 57 tokens covering:

- A type scale, `--t-100` to `--t-900`
- A tracking ramp, `--tr-900` to `--tr-caps`
- A four-pixel spacing grid, `--s1` to `--s10`
- One page width, one gutter, one reading measure
- The full colour palette, in both themes

Reach for a token before a number. A raw pixel value in a template is how this site reached thirty-seven font sizes, twenty-eight spacing values and eight page widths.

Two type rules worth stealing outright:

Type tightens as it grows and never opens up. Negative tracking on headings, scaling from about -0.032em at hero size to 0 at body size. The single positive value is +0.06em, reserved for small uppercase labels.

Three faces, no more. A serif for headings, one sans for everything else, and a mono for figures that sit in a column, never for words. This site once ran a fourth face as a second sans, and the two were close enough that the result read as a mistake rather than as a choice. Sites that do this well are lopsided on purpose: Stripe runs roughly a hundred declarations of its sans to two of mono.

The layout traps

Three CSS behaviours caused more wasted hours on this project than anything else. They are not obscure. They are just silent.

Grid and flex children need `min-width: 0`. Without it a child holds its min-content width and pushes the page sideways on a phone. This broke pricing, the rankings controls and the blog submit form, separately, each time looking like a different bug.

CSS comments do not nest. The first `*/` closes the outer comment. A template placeholder left inside an opening comment killed the colour palette on over a thousand pages and was chased as a design problem for hours.

An undefined custom property is silent. CSS does not treat `var(--nope)` as an error; it falls back and paints. A page here referenced nine colour tokens that existed nowhere, and the legally required Do

Not Sell button rendered as white text on a transparent background on a white page, invisible from the day it shipped. **Audit computed colour on the rendered page, not authored colour in the source.**

And one more, for completeness: `overflow: hidden` makes an element the scroll container that `position: sticky` resolves against. A sticky header that will not stick almost always means an ancestor picked up an `overflow`.

Dark mode, and the way it actually breaks

A dark mode does not break by having a wrong colour. It breaks by **missing** one. A token defined only inside a media query falls back to whatever it inherits, and the result is usually legible enough to pass a glance and wrong enough to matter.

The structure that avoids it:

1. Define the **complete** palette on bare `:root`. Every token, no exceptions.
2. Redefine **only** those tokens under `@media (prefers-color-scheme: dark)`, guarded as `:root:not([data-theme="light"])`.
1. Redefine them again under `:root[data-theme="dark"]`, so an explicit choice wins in both directions.

Three states, not two: light, dark, and follow the system. A two-state toggle cannot express "follow the system", and that is the most common way a dark mode is got wrong.

Then test it by parity: read both `:root` blocks and assert every colour token defined in one is defined in the other. Select by whether the **value** is a colour, not by whether the **name** starts with a colour-ish prefix. Classifying by name caught a font size on this project.

Finally, resolve the theme before first paint, in a tiny inline script in the head. A theme applied after paint is a flash of the wrong one.

Colour in data display

This is the part most teams get wrong, and it is measurable rather than a matter of taste.

Brand colours are usually wrong inside a chart. This site's navy, teal and secondary navy all sit below the chroma floor and read as grey in a plot; its blue against its violet is a deuteranopia delta-E of 0.4, which means two series a colourblind reader cannot distinguish at all. Brand colour belongs on the page around the chart. Inside the plot, the job is telling series apart.

Validate the palette under simulated colour vision deficiency, across every pair, not just neighbours. The categorical set here passes protanopia, deuteranopia and tritanopia on all pairs in light mode.

A dark palette has about half the lightness range to work with. Roughly 0.48 to 0.67, against 0.43 to 0.77 for light, so hue has to carry more of the separation, and hue is exactly what collapses under deuteranopia. Measured, this palette tops out at three slots distinguishable by colour alone on dark. That is only workable with secondary encoding, so secondary encoding is not optional: every chart with two or more series ships a legend, small sets are direct labelled, and every chart has a table view.

Status colours are reserved and never used for a series. A chart with four series and a red one implies the red one is bad.

Never encode identity by colour alone. Two games on this site signalled right and wrong only by colour until an accessibility pass; outcomes now carry a word as well, which also survives greyscale printing and forced-colors mode.

Chart form is a claim

The form you choose asserts something about the data.

A line claims the values in between existed. Money movement on this site was charted as a line, and the line between $13.32 on one day and $4.99 on the next passed through every value in between, none of which happened. Discrete events are bars.

A funnel drawn as a tapering polygon encodes the number twice, once in width and once in area, and area is the one people misread. A ranked list of bars with the drop-off written between them says the same thing without the illusion.

Every chart should be able to become its own numbers. A table view under each chart is the accessibility backstop, and it is also what a person actually wants when they are trying to copy a figure out.

Accessibility, as a set of habits

- Announce outcomes in a live region, not just in colour or motion.
- A flipped card, a revealed tile, a state change: say what it is.
- Contrast is measured on the rendered page, in both themes, on every page. A near miss

is a fail; the grey carrying most of the secondary text here measured 4.83:1 on white and 4.41:1 on the tinted cards, and only the second one is obviously wrong.

- No horizontal scroll at 390 pixels. Assert it in a test rather than checking it by eye,

and measure the component, not the document, or a wide child can hide behind a narrower page.

The Algorithm: Adaptive Selection and Honest Scoring

This chapter is about a test-preparation engine, but the transferable part is the last section, and it applies to any product that shows a person a number about themselves.

The shape of the engine

One engine serves unknown exams (see src/engine.js). Adding an exam is a registry entry plus a tagged bank, not a fork. That was not free: the single most useful refactor in the project was making the section keys, skill lists and copy resolve from an exam id injected at build time, instead of being hardcoded. Before it, one application had been shipping a score card headed with the wrong exam's name and telling students to calibrate at the wrong test maker's site.

If you have two of something, resolve the differences from data, not from conditionals. A conditional on `if (exam === 'sat')` treats "not SAT" as "the original one", and that assumption breaks silently the moment there is a third.

Ability estimation

Three-parameter logistic item response theory, fitted by Fisher scoring, producing an ability estimate and a standard error. Alongside it, an Elo-style per-skill rating that moves faster and is what the learner actually sees at the skill level.

The two exist for different jobs and it is worth being clear about which is which. The IRT estimate is for the score band, because it has a defensible error term. The Elo rating is for selection and for the skill table, because it responds quickly enough to feel like it is paying attention.

Item selection, and the two bugs in it

Selection targets an item near the learner's current ability, with a damping factor so a run of luck does not send the difficulty somewhere absurd.

Bug one: selecting on a per-skill counter instead of measured ability. Early on, a skill with fewer than six answers targeted a fixed starting rating rather than the learner's measured ability in that section. A strong student answering a new skill got beginner items. The fix targets the section's measured ability until the skill has its own evidence.

Bug two: re-serving items the learner had already answered. The candidate pool did not exclude seen items, so a short session could serve the same question twice. The fix filters to unseen candidates and falls back to the full pool only when unseen is empty. The same fix had to be applied separately to passage groups, because a passage drags its sibling questions along with it and the group was not being filtered.

Both were found by a **review bot**: a script that sits the exam repeatedly against the real engine and reports what only playing reveals. It measures estimate recovery, band coverage across ability levels, whether selection actually adapts, bank reach, starving skills, and whether a deliberately planted weakness is diagnosed. That last one is the sharpest test in the suite, because it has a known right answer.

Measuring a stochastic system on one seed is an anecdote. A change here looked like it dropped band coverage from 88 percent to 72. Across three seeds the baseline was 52, 88 and 80 against 72, 80 and 72 for the change. The spread between seeds was larger than the effect, and the regression did not exist.

Scoring, and the honesty constraints

This is the transferable part.

The product reports an estimated **range** on the exam's own scale, never a single number, never called a predicted score, and never shown before a minimum evidence threshold. The band half-width is the standard error of the ability estimate with a floor, so it can narrow with evidence but never collapses to a point.

Four rules sit behind that, and every one of them is a rule about not overclaiming:

1. **A range, not a number.** A single number implies a precision the estimate does not

have. The interval is the honest object.

1. **A floor on the interval.** Even with a lot of evidence, the interval never gets

narrower than the floor, because the model's error is not the only error.

1. **Nothing before the evidence threshold.** An estimate from four answers is noise with

a decimal point.

1. **The method is published, and the calibration is named as ours.** The scale anchoring

is this product's calibration, not the test maker's, and the page that explains the method says so and stays in step with the code.

Two further rules are specific but generalise:

- **Never mix incomparable scales.** Two editions of one exam use different ranges and are

not convertible; they are compared by percentile only, with the concordance cited. Any product with a legacy unit and a current one has this problem.

- **Never present an internal threshold as official.** A routing cut chosen here is ours

and is labelled ours.

The pattern underneath all of it: **when you show someone a number about themselves, the number is a claim, and the claim has an error bar whether or not you print it.** Printing it is the difference between a product people trust and one they catch out.

Content at Scale, Without Lying About It

This platform ships over a hundred thousand practice items across 54 bank files. Almost all are generated. The chapter is about how to do that without producing a number that is technically true and substantively false.

Generators, not text

An item is produced by a schema: a template plus a parameter space plus a rule for generating distractors. The schema knows how to make a correct answer and how to make wrong answers that are wrong for a reason.

Every key is computed, never asserted. The generator does not record which option is correct; it computes the answer from the parameters and then places it. An asserted key is a key that can be wrong, and a wrong key in a practice bank is the worst possible defect because the learner concludes they are wrong.

The counting problem

The hard part of generated content is not generating it. It is counting it honestly. This project got the count wrong in four distinct ways.

The dedup key must be canonical under every transformation the item legitimately undergoes. Answer position is randomised per draw, and the key hashed choices in order, so one question with its options rearranged counted as two. Sort before hashing.

Identity is not always the choices. For reading items, identity is the passage plus the question asked. Four passages produced five hundred "distinct" stated-idea items that differed only in which of the passage's other sentences came along as distractors. A student who has answered one has answered all of them.

An over-broad key deletes as silently as a narrow one inflates. Two different conditional questions produced the same stem, hashed identically, and half the inference items in any corpus would have vanished without a word.

A filter that drops must say so. A length guard measuring the wrong unit silently removed sixteen valid items. Every rejection stage reports its rejection count.

The honest yield number for reading items here, once both dedup bugs were fixed, is about six stated-idea and two inferred-idea questions per passage. That means five hundred inference items needs

roughly a hundred and seventy passages. **That number is the entire cost of the category, and it is recorded rather than discovered later.**

Quality ratchets

A generated bank drifts toward exploitable patterns unless something measures them. Two that mattered here:

The length tell. In one section the longest option was the correct answer 81 percent of the time. A test-wise student does not need to read the question. The generator now draws distractors so that the key's length rank is uniform, and a ratchet in the test suite fails if the tell reappears. It is down to chance.

Comparable development. Each added clause in a distractor is chosen to leave the option wrong for the reason it was already wrong, so length and elaboration carry no signal about correctness.

A ratchet is a pinned number that may only move one way. If a fix makes a statistic better, pin the better number. This is the cheapest possible regression test for anything statistical, and it works where an exact assertion cannot.

Shipping a large bank to a browser

At a hundred thousand items the bank is tens of megabytes, and this is where content scale becomes a performance problem.

- The whole bank as a blocking script meant **20.3 seconds to the first question on

throttled 3G**. Nobody noticed, because nobody had measured it.

- The fix is a strided starter bank, small and covering every category, plus an async

remainder that pushes into the same array. Time to first question fell to 7.6 and 11.0 seconds, and blocking download from 11.3 MB to 1.2 MB.

- Platform limits bite here. Cloudflare rejects a single static asset over 25 MiB, so the

remainder is chunked, and the build fails on any asset over the limit.

- When you change a file naming scheme, grep for the old name as a **string**, including

inside regular expressions. A service worker precache pattern of `bank_rest\.js$` does not match `bank_rest1.js`, and the failure is silent degradation of offline support.

The sourcing discipline for facts

Generated practice items are one thing. Published facts about the world are another, and they need a different regime.

- **Every figure carries a source, a year and a URL.** Not a citation in prose: fields, on

the record, validated at build time.

- **Unverifiable means null, and null renders as a dash.** Never a guess, never a

plausible round number.

- **A named list of banned sources**, enforced by the validator. Forums, aggregators,

crowd-edited encyclopaedias and coaching-company blogs. A build fails if one appears.

- **One file per entity**, so a fix touches one file and a validator can walk them all.

This project holds 94 school files under that scheme.

- **When a source cannot be reached, say so immediately and stop.** Two exams on this site

are blocked, and recorded in the roadmap as blocked, because the test makers' pages return bot challenges. The rows that exist for them cite a banned source and are marked for replacement rather than quietly reused. Being blocked and saying so is a better state than being unblocked by a guess.

The Data Layer

16 tables across 31 migrations, with the database rather than the application as the enforcement point.

Row Level Security as the default posture

1. **Enable RLS on every table at creation.** Not in a later pass.
2. **No policy means no access**, which is exactly right for anything only your server

should touch. This project's telemetry, error and billing tables all have RLS on and no policies at all; the service role bypasses RLS, and admins read through functions.

1. **Public read where public read is the point**, scoped: forum rows that are not hidden,

for instance.

1. **Open insert with server-side stamping.** The forum takes posts from anyone with no

account. The client cannot set the author, the country or the address hash; triggers do. A client-supplied field is a claim.

The `SECURITY DEFINER` pattern

Every privileged read is a function that checks admin membership and then reads. Clients never touch the underlying table.

Two rules that are easy to get wrong:

Revoke from `PUBLIC`, not just from `anon` and `authenticated`. A leading `=X/postgres` in `pg_proc.proacl` means `PUBLIC` holds `EXECUTE`, and revoking from every role you can name changes nothing. Verify with the platform's advisors or by reading the ACL. Never verify by reading your own migration.

Never take an id from the request body. The cancel-subscription and portal functions here read the caller's id from their own RLS-scoped row. A function that accepts an id and acts on it is a function that acts on anyone's row.

Triggers, and the two that bit

Postgres fires same-timing triggers in alphabetical order by name. On this project `profiles_sharing_eligibility` sorted before `profiles_sync_age_tier`, so eligibility was judged

against a stale age tier and **silently revoked every legitimate adult opt-in**. If two triggers on one table have an order dependency, encode it in the name with a numeric prefix.

`UPDATE OF <columns>` fires on the columns named in the statement, not on what actually changed, and not on what an earlier `BEFORE` trigger wrote into `NEW`. So `update profiles set birth_year = 2011` ran the sync, stored a minor's tier, and skipped the eligibility check entirely. If you need "when this value changed", fire on every update and compare `OLD` to `NEW` yourself.

Both were found by a SQL smoke test that asserted the **guarantee** and rolled itself back. Neither would have been found by reading the trigger.

Ordering, idempotency and the event problem

Webhooks do not arrive in order. Notifications do not arrive in order. This is not an edge case, it is the normal condition, and the consequences are severe: a `deleted` landing after an `updated` resurrects a cancelled plan, and the reverse downgrades a paying customer.

The discipline, which applies to any external event source:

1. **Never write the event payload.** The payload tells you *which* thing changed. Re-read

the current state from the source of truth and write that.

1. **Guard on a monotonic field.** A notification older than what you last wrote is

discarded. Apple provides `signedDate`; most systems provide something.

1. **Terminal states are terminal.** A refund or a revocation is never overwritten by a

late renewal.

1. **Put idempotency in the database, not the handler.** A unique index on the natural key

is correct under concurrency. A check-then-insert in application code is not.

Retention

- Unlinkable telemetry: 400 days here, on a scheduled job rather than a trigger, because

a delete on every insert puts a table scan in the path of the user's action.

- The retention window should match the longest window your console can query, so nothing

the interface can ask for is ever missing.

- **A billing ledger is not telemetry.** It is a financial record and it is kept.

Migrations as a readable record

Every migration in this repository opens with a comment explaining what it does and, more importantly, **why the obvious alternative was rejected**. Read six months later, the "why not" is the part you need; the "what" is in the SQL.

Keep the applied migrations mirrored in the repository, and keep a manifest with checksums. Applying through a console and forgetting to mirror is how a schema and its history diverge, and the divergence is invisible until somebody tries to rebuild.

Privacy as a data-model decision

Two ideas here are worth copying wholesale.

Unlinkability by construction. The item telemetry table has no user column, no session column, no device column and no address column, **by design, and must never gain one**. That absence is precisely why it needs no consent gate. Privacy enforced by a schema is stronger than privacy enforced by a policy document, because a schema cannot be forgotten.

Keep what someone told you separate from what you were told about them. Self-reported attributes live on the profile. Third-party appended attributes live in a separate table that cascades on deletion. They are never merged, so what a person said about themselves stays distinguishable from what was bought about them, forever.

Payments

5 edge functions handle money here: `apple-notifications`, `cancel-subscription`, `create-checkout-session`, `create-portal-session`, `stripe-webhook`.

The shape

Hosted checkout, not a card form. The processor's hosted page removes PCI scope entirely, and there is no version of building your own that is worth the saving.

The webhook is authenticated by the processor's signature rather than by your own auth, which means it must be deployed with JWT verification **off** and the signature check **on**. Get that pairing wrong in either direction and you have either an endpoint that rejects every legitimate call or one that accepts every illegitimate one.

Every column the webhook writes is service-role only, left out of the column grants entirely, so no browser session can forge a plan. **A signed-in user was once able to grant themselves a paid plan on this project**; that is what the grant hardening is for.

Two processors, and why the plan column stops being the answer

The moment a second payment source exists, "what has this person paid for" cannot be a column, because it becomes a race: whichever webhook fired last wins, and a user who subscribed on iOS gets downgraded the next time a card event lands on their row.

The resolution here: the original column stays exactly as it was, owned by the original processor and untouched by anything new, and a function derives the **effective entitlement** across every source. The application reads the function. Nothing about the live, money-taking path changed, which was deliberate.

Generalisation: when a second source of truth appears, do not teach the first one to share. Add a derivation on top and move readers to it.

Promises that are load-bearing

Write these down and test them, because they are the ones that turn into complaints.

- **The free plan never asks for a card.** Tested in both signed-in and signed-out states.
- **Cancelling is one click, in the account page, through your own function.** Not through

the processor's hosted portal, which may not be configured, and which puts the most important retention moment on somebody else's page.

- **The exception you cannot engineer around.** An Apple-sourced subscription is Apple's:

there is no API to cancel on a user's behalf and the guidelines require pointing at Settings. So the button opens Apple's subscription management and explains why, rather than appearing to work and not. **Which path applies is decided by the entitlement record, never guessed from the platform the page happens to be open on.**

- **A refund window, stated, plus your own errors always refunded.**
- **Access continues to the end of the paid period.**

The ledger, and the mistake to avoid

The most consequential payments decision in this project was made late and should have been made first.

The profile row holds what an account pays **right now**, and it is overwritten on every webhook. That means the instant a subscriber cancels, the evidence they ever paid is gone. It is enough to gate access and useless for running a business: it cannot answer how much revenue arrived this month, how much left, what share of trials converted, or what came back out as refunds.

Build the append-only billing event ledger before the first real subscriber. It cannot be reconstructed afterwards.

The design that works:

- One row per fact, normalised across processors so a chart reads one vocabulary.
- **Cash and recurring value in separate columns.** An annual plan is $99.99 of cash on one

day and $8.33 of monthly revenue. A dashboard that adds those together is wrong in both directions.

- A `kind` column of your own verbs, not the processor's event names.
- `mrr_delta_cents` on every row, so new plus expansion minus contraction minus churn is

the movement bridge by construction, and cannot be off by a rounding rule applied in one place and not another.

Four traps, all of which would have produced a plausible wrong number:

Two event types announce one new subscription. Hosted checkout completion and subscription creation both fire, with different event ids, in no guaranteed order. Logging both doubles new revenue. A

partial unique index on (source, subscription id, kind) for the once-per-lifetime kinds lets the database decide which arrived first.

The first invoice is also an invoice. Counting every paid invoice as a renewal double-counts every new subscriber as a returning one. Only a cycle invoice is a renewal. The cash still counts.

The refunded amount is cumulative. A charge reports total refunded to date, not the amount of this refund, so a second partial refund counts the first one again. Use the most recent refund entry.

The cancellation write erases what you need to record the cancellation. Read the prior state before writing, or churned revenue is always zero.

Sandbox is not money. Test transactions carry real-looking prices. A sandbox row in a revenue chart is a fictional dollar, and it will be believed.

Know the units. One processor reports price in milliunits: 4990 means $4.99. Reading it as cents overstates revenue by a factor of ten, which is an error that looks like success.

Secrets

Secret keys and signing secrets live in the platform's secret store. Never in the repository, never in an environment file that is committed, never in a log line. This is the one rule in this book with no nuance attached to it.

Business Intelligence

The three streams

Keep them separate and know what each can answer.

Site events. A first-party beacon on every page: path, referrer, UTM, a session id from `sessionStorage`, device class, duration. Country and a salted address hash added by a trigger so the raw address is never stored. Behind a consent banner that honours Global Privacy Control and can be reopened from any page footer.

Product telemetry. What happened inside the product. Here that is item telemetry with **no user, session, device or address column, by design**. It answers "is this item too hard" and cannot answer "what did this person do", and that is the trade that lets it run without a consent gate.

Billing events. Money. Covered in the payments chapter.

They are deliberately not joined. Nothing here attributes a subscription to a visit. That is a design choice with a real cost, and it should be stated on the page rather than quietly left as a gap the reader assumes is filled.

Read through functions, always

No client reads an analytics table. Every admin view is a `SECURITY DEFINER` function gated on admin membership, returning exactly the shape the page needs.

One read per page, not six. Six reads can disagree with each other: the tile says eleven paying and the chart says twelve because a webhook landed between the two queries. One statement, one snapshot, one story.

The numbers not to compute

This is the part that separates a dashboard from a decoration.

A churn rate needs a denominator held at the start of the window. If your ledger is younger than the window, you cannot supply one. The counts and the lost revenue are real; the ratio would be a guess wearing a percentage sign. This dashboard prints the counts and explicitly declines the rate, and says why, and starts printing it automatically once the ledger is old enough.

A conversion rate must exclude the undecided. A trial still running is not a failed trial. Counting it in the denominator produces a rate that rises on its own as trials mature, which is worse than no rate at all.

Trial revenue is never revenue. State the pipeline figure separately and label it as conditional.

Gross and net are different numbers. Show refunds beside charges, not subtracted from them. Netting hides the month where a fifth of revenue came back.

A snapshot take rate is not a cohort conversion rate. Say which one you are showing.

Say what the data covers

Every derived figure is "since the log began". Print that date on the page. A figure that silently covers less than its label claims is the hardest kind of wrong to catch, and it is the failure the whole discipline exists to prevent.

The triage loop

Errors and moderation both work the same way, and the pattern is worth naming.

1. Capture into an insert-only table.
2. Cluster by signature, and attach the evidence to the cluster: the real message, the

real stack, the builds, the pages, how many separate sessions hit it.

1. **A human decides.** Real, fixed, will not fix, or noise.
2. The decision is stored, and it mutes the cluster permanently.

The queue gets quieter as it learns rather than louder, and the stored labels are exactly what a future model would train on. Nothing in it edits code, and that is deliberate: trust is the binding constraint on automated repair, so the system proposes and explains, and a person decides.

Never fabricate

No invented user counts, testimonials, efficacy claims or seeded activity. On a forum, house-authored posts say they are house-authored. This is not only an ethics rule; a fabricated number in a dashboard eventually gets used for a decision, and by then nobody remembers it was a placeholder.

Tests and Guards

28 test files, and the interesting thing about them is not what they assert. It is that the analysis chapter can count how defects were **actually** found, and the answer reshapes where you put effort.

The ladder

Rung 1: the build. Structural guards on the artefact. Parse every generated script. Assert every collection size. Fail on platform limits. Compare inputs to outputs at every filter. Cheapest rung, catches the most.

Rung 2: domain logic, headless. The engine, run without a browser, once per variant. Fast enough to run on every change.

Rung 3: the real browser against the real built site. Playwright against the built output over a local server, at desktop and phone width, in both themes. **Assert behaviour, not markup.** A markup assertion breaks on every refactor and passes through every real bug.

Rung 4: adversarial play. A bot that uses the product the way a user would, repeatedly, and reports what only use reveals. Three real defects here came from nowhere else.

Rung 5: the periodic audit. Contrast, links and metadata across every page, weekly. Catches the slow rot that no single change is responsible for.

The failure modes of tests themselves

Five entries in the defect ledger are about tests being wrong, and that proportion is not an accident. **A broken test is invisible, because its output is identical to a working one.**

A negative assertion passes when the system is broken in the right way. "Nothing is sent when consent is refused" passed here because the test used the wrong storage key, so consent was never configured, so nothing was ever sent. It would have passed with the feature deleted. **Always pair a negative assertion with the positive case**, or you are asserting nothing.

A test can measure the wrong instant. A performance test waited for the browser load event, which waits for the async resource the optimisation moved off the critical path. A change that made the page usable nine times faster would have reported no improvement. **Measure the moment the user can act.**

A guard can narrow its own scope. A count check matching `\d{2,4}` stopped checking the two largest counts when they passed ten thousand, and kept passing. **Assert how many things you checked**, not only that the checks passed.

A test can measure something that was never rendered. A hidden element still answers `innerText` and `getComputedStyle`. Layout and contrast assertions passed here on a view that was never displayed. **Assert the thing is on screen before you measure its layout.**

A test can classify by name instead of by value. A theme parity check selected tokens by name prefix and flagged a font size as a missing colour.

Ratchets

For anything statistical, pin the number and allow it to move one way. If a fix improves a statistic, pin the better value. This is how the length tell in the item bank went from 81 percent to chance and stayed there.

A ratchet only works if it is stable. This project's ratchets drifted run to run because the generator seeded from a randomised hash, and a flaky ratchet gets deleted. **Seed from something stable and assert reproducibility first.**

Write the expected number before the code

The dashboard test here carries a fixture of two subscriptions and a comment block computing, by hand, what every tile must show. The test knew $159.84 was the right answer before the page existed, which is how it caught the page printing $160.

If you cannot write down the expected output before you write the code, you do not yet understand the requirement.

Wire it into CI, and prove it ran

The browser suites on this project **had never actually run.** Playwright was a declared dependency in a repository where nothing had installed it. The tests existed, the command existed, and the coverage was zero.

A test that is not wired into CI is a test that does not exist. Prove a suite runs in the place it is supposed to run.

Render it and look at it

Three defects here were found by taking a screenshot: a chart unreadable in dark mode, a line chart implying values that never existed, and a dashboard that was not on screen. None of them would have

been caught by an assertion, because none of them were violations of anything anybody had thought to assert.

Budget for looking. It is the cheapest test there is and the only one that catches things you have not imagined.

Running the Build as an AI Loop

221 commits in 30 days, one owner, a series of AI sessions. This chapter is how that was actually run, including the parts that did not work.

The division of labour

The owner decides: what to build, what it means, what is acceptable, and every irreversible or outward-facing action. **The session does**: the building, the measuring, the testing, and the reporting of what it found.

The failure mode at each end is worth naming. An owner who reviews every line becomes the bottleneck and the throughput collapses to the speed of reading. A session that decides product questions on its own produces something coherent that is not what anyone wanted.

The rules file is the highest-leverage artefact

One file, read every turn, holding only load-bearing rules. Not documentation, not architecture notes. The test for whether a line belongs in it: **would you otherwise have to say this again?**

What earns a place:

- Style rules a build can enforce.
- Sourcing rules, with a named list of banned sources.
- **Never invent a value.** Every id, hash, count, price, date and URL must be read from

real output before it goes into a tool call, a commit or a page.

- **Report a blocked resource immediately** and ask, rather than quietly working around it.
- The handful of architectural facts that are expensive to rediscover.

Every one of these was written after something went wrong. Write them at the start anyway; the list above is transferable as-is.

Ratchets, not reviews

The owner cannot read every line. So the correctness that matters is encoded in things that fail loudly: build guards, pinned statistics, browser suites, review bots. The review budget then goes to the few decisions that are genuinely judgement, rather than being spread thinly over everything.

This is the same principle as the data layer. **Push the guarantee down to the layer that cannot be bypassed.**

What a session should report

Not a list of files changed. The useful report is:

1. What was built, in one paragraph.
2. **What broke on the way, and why.** This is the most valuable part and the easiest to

omit, because it reads as failure. It is the opposite.

1. What is now guarded against, and where.
2. What was left undone, and whether that is a decision or a blocker.
3. Numbers, with the command that produced them.

The commit messages in this repository are written that way, which is why a defect ledger could be reconstructed from them 30 days later. **Write the commit message as though someone will need to mine it. Someone will.**

Failure modes observed in this project

Reporting a regression from one seed. A stochastic measurement compared before and after on a single seed showed an 88 to 72 percent drop. Across three seeds the spread between seeds was larger than the effect. Run more seeds before reporting.

Pausing to report when told to keep working. A session that stops to summarise every few minutes converts working time into reading time. If the owner has said to keep going, keep going, and batch the reporting.

Treating a policy document as a blocker. A missing stylesheet reference was reported as a privacy concern because the file lived in a directory named after the privacy page. Be precise about what kind of problem you have found; a 404 is a 404.

Typing a value that looked right. A forty-character hex string was typed into a merge call as though it were a commit SHA. It was rejected as malformed, which was luck; a well-formed guess would have merged the wrong thing. `git rev-parse` costs one call.

Adding a guard that did not guard. Several ledger entries are guards that were too narrow, too broad, or measuring the wrong thing. After writing a guard, break the thing on purpose and confirm the guard fails.

The cadence that worked

Long uninterrupted sessions with standing authorisation, punctuated by the owner redirecting priorities, beat short supervised ones by a wide margin. The standing authorisation that made it work was specific: merge your own work once the checks are green, do not check in on each item, and stop immediately when told to.

The thing that makes that safe is not trust. It is that the checks are real, the destructive actions still require asking, and the record of what happened is written down well enough to audit later. Which is what this book is.

What the Ledger Says About Itself

214 recorded defects, over 30 days of building. This chapter is computed from the ledger every time the document is built, so it cannot fall out of step with it.

How defects were actually found

HOW	COUNT	SHARE
Found by reading the code or the output	121	57%
Found by measuring something	48	22%
A test caught it	23	11%
Found by rendering it and looking	10	5%
Found by a review bot or an adversarial pass	5	2%
A build guard caught it	5	2%
A person hit it	2	1%

This is the most useful table in the book. 212 of 214 defects, 99 percent, were caught by something other than a person hitting them in production. The single largest category is not a clever tool: it is reading the built output instead of the source that produced it. The second is measuring a number nobody had measured before. Neither requires infrastructure, and both are habits rather than tools.

Read that percentage with the bias it carries. This ledger is written by the people who found the defects, so it counts what was caught and cannot count what was not. A defect a user hit and nobody recorded does not appear here. The honest reading is not "97 percent of all defects were caught early"; it is "of the defects we know about, almost all surfaced through one of these five habits", which is still the useful claim, because it says where to spend attention.

By severity

SEVERITY	COUNT
Wrong data shown or stored	101
Degraded	53
Cosmetic	31
Silent loss	26
Site down	3

Silent loss is the dominant failure mode, at 26 of 214. Not a crash, not an error page: something quietly did less than it claimed. A loop over an empty list, a filter that dropped rows, a guard that stopped checking, a table that never received a write. None of these announce themselves, and none are caught by error monitoring, which is why the guard ladder in this book is built around asserting counts rather than catching exceptions.

By area

AREA	COUNT
Content generation	100
Tests and guards	38
Front end	16
Build system	14
Search and metadata	12
CSS and layout	9
Scoring and selection	7
Infrastructure and deploy	6
Payments	5
Database	4
Interface and data display	3

Guard coverage

208 of 214 defects produced an automated guard. 6 did not, and are carried by attention alone, which means they are the ones most likely to recur.

Carried by attention:

- **INC-0010** CSS comments do not nest, and one placeholder killed the palette on 1451 pages
- **INC-0019** The audit reported a working link as broken
- **INC-0025** A stale edge cache made a fixed 404 look like a live 200
- **INC-0034** A line chart interpolated between discrete money events
- **INC-0036** charge.amount_refunded is cumulative, so partial refunds double-count
- **INC-0045** The error reporter reported its own failures, in a loop

Guards that did not hold

The same guard named by two incidents is a guard that did not hold the first time. These are the places to spend effort.

- weekly_audit checks computed colour on every page (INC-0050, INC-0048)

Lessons learned more than once

95 of 214 incidents record that they repeat an earlier lesson, 129 links in all. This is the count the guard table above cannot produce: a repeat here means the lesson did not transfer, whether or not the same guard was named.

LESSON FIRST RECORDED IN	REPEATED BY	TIMES
INC-0088 A shipped schema answerable at 68 percent by picking the shortest option, under the size at which anything is checked	INC-0089, INC-0098, INC-0099, INC-0101, INC-0117, INC-0122	6
INC-0104 Half the school pages opened with a data note pasted into a sentence, and the structured data repeated it	INC-0105, INC-0131, INC-0134, INC-0135, INC-0145	5
INC-0130 The GRE exam guide credited ETS with a combined 260 to 340 score that the ETS page it cited does not describe	INC-0133, INC-0155, INC-0160, INC-0177, INC-0207	5
INC-0133 School pages published figures that the articles they cite do not contain, one of them a GRE total split in half	INC-0140, INC-0143, INC-0150, INC-0151, INC-0154	5
INC-0150 Columbia's MBA class was credited with five years of work experience, a figure the cited article gives for its 46-student MBAxMS cohort, and the source check passed it because the article does say five years	INC-0152, INC-0159, INC-0172, INC-0174, INC-0182	5
INC-0064 The guard against a blind counter was itself blind to three exams	INC-0067, INC-0082, INC-0085, INC-0088	4
INC-0069 A bank a student can play at 88 percent, inside a section the check passed	INC-0079, INC-0085, INC-0086, INC-0088	4
INC-0074 A corpus field written for one grammatical slot was spliced into another	INC-0075, INC-0087, INC-0093, INC-0096	4
INC-0137 A post queued to publish the next day said the LSAT trainer was still in development, eleven days after it went live	INC-0138, INC-0162, INC-0170, INC-0207	4
INC-0082 Nine published exam facts cite test prep companies, in the one published corpus with no source	INC-0124, INC-0125, INC-0130	3

LESSON FIRST RECORDED IN	REPEATED BY	TIMES
validator		
INC-0092 A schema threw away three draws in four, and the counter that knew was read by nobody	INC-0098, INC-0099, INC-0101	3
INC-0099 Every generated reading comprehension question shipped without its passage	INC-0100, INC-0101, INC-0103	3
INC-0105 Twelve average salaries were published as medians, and the social queue credited schools with figures they never published	INC-0106, INC-0124, INC-0142	3
INC-0119 Length corrections appended clauses that repeated what the answer choices already said	INC-0120, INC-0121, INC-0194	3
INC-0136 The LSAT guide said most test takers choose between remote and in-person testing, after LSAC had moved almost everyone into test centers	INC-0137, INC-0180, INC-0184	3
INC-0146 The source check's browser read returned Stanford's class profile without a single figure, because the figures are drawn in an embedded chart that renders only when scrolled into view	INC-0156, INC-0158, INC-0183	3
INC-0050 A landing-page icon referenced a colour token that did not exist	INC-0018, INC-0139	2
INC-0055 A new browser suite hardcoded this machine's browser directory and crashed in CI	INC-0067, INC-0110	2
INC-0059 The item counter missed a whole bank file because it assumed a quoting style	INC-0064, INC-0067	2
INC-0067 The browser path fix covered two suites and three others kept crashing	INC-0070, INC-0104	2
INC-0083 The rules digest promises to be prompt sized and its generator grows without bound	INC-0084, INC-0129	2
INC-0086 A finished generator module that nothing imported, and two of its four schemas produced nothing	INC-0090, INC-0126	2

LESSON FIRST RECORDED IN	REPEATED BY	TIMES
INC-0087 The same corpus field in two grammatical slots, in a schema written the same day the guard was read	INC-0093, INC-0096	2
INC-0090 Two of a schema's three question forms could not build, and the only sign was an item count	INC-0092, INC-0126	2
INC-0097 Every reading inference question asked about something the passage never mentions	INC-0102, INC-0114	2
INC-0098 The table said 31.0 and the explanation said 31, because the fix covered the table only	INC-0099, INC-0101	2
INC-0101 A new field-by-field copier reproduced the passage loss defect four days after the ledger recorded it	INC-0102, INC-0103	2
INC-0103 A 139 page section shipped with no route into it from the sitemap	INC-0108, INC-0109	2
INC-0115 A field written to open a sentence was printed mid-sentence in a reading stem	INC-0179, INC-0187	2
INC-0124 A blog post put the ACT science add-on at $4 in its table and $5 in its own FAQ, and nothing compared a post's prices with the sourced ones	INC-0125, INC-0132	2
INC-0125 Every College Scorecard figure in the MBA library cited the program's own website as its source URL	INC-0130, INC-0133	2
INC-0151 Rice's school page credited Poets&Quants with an average GMAT of 693 for the Class of 2027, a number the article never prints; the figure's own note said it was read from a search snippet	INC-0154, INC-0157	2
INC-0154 The weekly source check listed Duke's GMAT medians and Michigan Ross's work experience as missing from their sources for weeks, but its 30 findings mixed real errors with pages it cannot read, so none was ever worked	INC-0155, INC-0157	2

LESSON FIRST RECORDED IN	REPEATED BY	TIMES
INC-0156 The source check read mba.com's bot challenge page as the page it cites and reported the GMAT fact's numbers as missing from it	INC-0158, INC-0175	2
INC-0158 The source check filed six school pages as probably built by JavaScript; five were something else, and three of them showed newer figures than the library published	INC-0161, INC-0184	2
INC-0162 The blog build read a fee table as one sentence and refused a correct post for calling the SAT trainer unfinished, over College Board's seat Waitlist	INC-0165, INC-0169	2
INC-0171 Thirty published blog posts set their section headings in sentence case, against the house rule that headings are Title Case	INC-0176, INC-0178	2
INC-0174 Two GMAT facts passed the source check on numbers that meant something else: score validity cited the retake policy page, where "5 times" supplied its 5, and the score release's "3 to 5 days" matched a 3 in the page's helpfulness counter	INC-0177, INC-0182	2
INC-0014 A hardcoded count in the meta description went stale, and Google showed it	INC-0128	1
INC-0023 www and the apex were two origins, so consent and rankings split in half	INC-0127	1
INC-0044 The longest option was the correct answer 81 percent of the time	INC-0091	1
INC-0072 A distractor was replaced and the explanation went on naming the old one	INC-0119	1
INC-0080 A build step that fails while the build exits zero, and a verification run that was a remembered subset	INC-0095	1
INC-0081 A student who always answers 1 scores 98 percent on a schema, and no check looked at the answer itself	INC-0123	1

LESSON FIRST RECORDED IN	REPEATED BY	TIMES
INC-0089 A ratchet that trips on sampling noise gets re-recorded rather than read	INC-0123	1
INC-0093 Seven variable names were plural and every sentence built around them said was	INC-0096	1
INC-0110 A browser suite outside CI had failed on its first click since the consent dialog shipped	INC-0113	1
INC-0117 Three reading schemas could be answered by matching names, because every distractor came from a different passage	INC-0122	1
INC-0128 The trainer button on four exam guides and on the study guide hub opened the GMAT trainer	INC-0206	1
INC-0132 The GRE guide still quoted a $220 fee eight weeks after ETS's fees became $249	INC-0136	1
INC-0135 Arizona State's school page said its class entered in 2027, because the label parser took the first year it found	INC-0147	1
INC-0138 The exam guides hub told search engines only the GMAT and SAT trainers were live, eleven days after all five were	INC-0162	1
INC-0139 Every blog page drew the site header without the design tokens, so its logo sat against the screen edge on phones	INC-0163	1
INC-0142 Every school's employment rate was labelled and scored as a three month figure, though 15 of 70 were measured at six months, four months, a year or a reporting date, or at a timing their notes do not give	INC-0144	1
INC-0152 The source check read text inside HTML comments as page text, and confirmed Arizona State's 43 percent women from a table row the school had commented out	INC-0173	1
INC-0155 The GMAT guide cited mba.com's Official Score Reports page for the rule on five free score reports within 48	INC-0160	1

LESSON FIRST RECORDED IN	REPEATED BY	TIMES
hours, a rule that page never states		
INC-0172 The ACT exam guide and a live post said the ACT can be taken up to 12 times, after ACT's page had come to say there is no limit, and the source check matched the 12 in the page's K-12 menu	INC-0174	1
INC-0176 Headings outside blog post bodies broke the Title Case rule on every kind of page, because the build checked only the headings inside posts	INC-0178	1
INC-0184 Vanderbilt's median salary and 39 schools' Bloomberg ranks stayed on Bloomberg's 2025-26 edition after its pages moved to 2026-27, and nothing could see it: Bloomberg answers most reads with a bot challenge, and the source check never reads a rank	INC-0185	1
INC-0203 A live admissions post said most full-time MBA programs run three rounds a year, which 21 of the 32 programs whose deadlines we verified do not, and its other claims about rounds cited nothing	INC-0208	1

The largest family runs to 78 incidents: INC-0055, INC-0059, INC-0064, INC-0067, INC-0069, INC-0070, INC-0079, INC-0081, INC-0082, INC-0085, INC-0086, INC-0088, INC-0089, INC-0090, INC-0092, INC-0097, INC-0098, INC-0099, INC-0100, INC-0101, INC-0102, INC-0103, INC-0104, INC-0105, INC-0106, INC-0108, INC-0109, INC-0110, INC-0113, INC-0114, INC-0117, INC-0122, INC-0123, INC-0124, INC-0125, INC-0126, INC-0130, INC-0131, INC-0132, INC-0133, INC-0134, INC-0135, INC-0136, INC-0137, INC-0138, INC-0140, INC-0142, INC-0143, INC-0144, INC-0145, INC-0146, INC-0147, INC-0150, INC-0151, INC-0152, INC-0154, INC-0155, INC-0156, INC-0157, INC-0158, INC-0159, INC-0160, INC-0161, INC-0162, INC-0165, INC-0169, INC-0170, INC-0172, INC-0173, INC-0174, INC-0175, INC-0177, INC-0180, INC-0182, INC-0183, INC-0184, INC-0185, INC-0207. Every one of them is the same shape, a correction applied to the instances in hand rather than to the pattern, and it is the most expensive habit this ledger records.

Incidents that name an earlier one without claiming to repeat it. Each was read and ruled on: these are the cases where the earlier guard or practice worked, or its test was reused, which is the opposite of a repeat. They are listed so the ruling stays visible rather than becoming an omission.

- INC-0013 names INC-0012
- INC-0068 names INC-0039

- INC-0076 names INC-0075
- INC-0080 names INC-0077
- INC-0084 names INC-0059, INC-0064, INC-0074
- INC-0087 names INC-0075
- INC-0115 names INC-0114
- INC-0116 names INC-0114
- INC-0117 names INC-0114
- INC-0119 names INC-0120
- INC-0122 names INC-0079
- INC-0127 names INC-0126
- INC-0129 names INC-0128
- INC-0130 names INC-0132
- INC-0131 names INC-0130
- INC-0143 names INC-0105, INC-0140
- INC-0152 names INC-0146
- INC-0155 names INC-0156
- INC-0156 names INC-0155
- INC-0161 names INC-0133, INC-0154
- INC-0164 names INC-0163
- INC-0165 names INC-0163
- INC-0166 names INC-0163
- INC-0169 names INC-0126
- INC-0171 names INC-0170
- INC-0172 names INC-0170
- INC-0173 names INC-0174
- INC-0174 names INC-0173
- INC-0181 names INC-0057, INC-0079, INC-0104, INC-0105, INC-0109, INC-0110, INC-0130, INC-0170, INC-0171, INC-0172, INC-0173, INC-0174, INC-0175, INC-0180
- INC-0186 names INC-0117
- INC-0187 names INC-0186
- INC-0189 names INC-0039, INC-0117
- INC-0190 names INC-0119, INC-0189
- INC-0191 names INC-0189
- INC-0192 names INC-0117
- INC-0193 names INC-0189

- INC-0194 names INC-0193
- INC-0196 names INC-0166
- INC-0197 names INC-0196
- INC-0198 names INC-0109
- INC-0207 names INC-0138, INC-0203
- INC-0208 names INC-0207
- INC-0211 names INC-0196
- INC-0214 names INC-0203, INC-0208

Where defects concentrate

Files named by three or more incidents. This is not the same signal as the list above: a file that is the natural home for many checks will appear here without any one of them having failed. It says where the work has been, and where a reader new to the codebase should look first.

- `src/check_sources.py`, 22 incidents (INC-0130, INC-0132, INC-0133, INC-0136, INC-0140, INC-0150, INC-0152, INC-0154, INC-0155, INC-0156, INC-0158, INC-0159, INC-0161, INC-0168, INC-0172, INC-0173, INC-0174, INC-0175, INC-0177, INC-0180, INC-0182, INC-0184)
- `src/build_banks.py`, 17 incidents (INC-0003, INC-0007, INC-0008, INC-0009, INC-0011, INC-0079, INC-0081, INC-0086, INC-0088, INC-0089, INC-0090, INC-0091, INC-0092, INC-0120, INC-0123, INC-0126, INC-0127)
- `src/build.py`, 15 incidents (INC-0001, INC-0002, INC-0017, INC-0027, INC-0059, INC-0060, INC-0063, INC-0064, INC-0067, INC-0076, INC-0080, INC-0131, INC-0148, INC-0196, INC-0212)
- `src/build_blog.py`, 11 incidents (INC-0124, INC-0137, INC-0162, INC-0170, INC-0171, INC-0188, INC-0197, INC-0203, INC-0204, INC-0206, INC-0208)
- `src/test.js`, 10 incidents (INC-0004, INC-0038, INC-0039, INC-0040, INC-0043, INC-0044, INC-0069, INC-0085, INC-0192, INC-0194)
- `src/gen/g_rc.py`, 9 incidents (INC-0097, INC-0114, INC-0115, INC-0117, INC-0122, INC-0169, INC-0179, INC-0186, INC-0187)
- `src/validate_schools.py`, 9 incidents (INC-0125, INC-0141, INC-0143, INC-0144, INC-0147, INC-0151, INC-0153, INC-0157, INC-0185)
- `src/build_rankings.py`, 8 incidents (INC-0014, INC-0049, INC-0118, INC-0135, INC-0142, INC-0145, INC-0167, INC-0200)
- `src/page_checks.py`, 7 incidents (INC-0134, INC-0138, INC-0139, INC-0163, INC-0176, INC-0178, INC-0207)
- `src/bank_emit.py`, 6 incidents (INC-0062, INC-0066, INC-0068, INC-0073, INC-0119, INC-0189)
- `src/review_bot.js`, 5 incidents (INC-0022, INC-0026, INC-0051, INC-0061, INC-0077)
- `src/gen/framework.py`, 5 incidents (INC-0074, INC-0075, INC-0078, INC-0087, INC-0096)

- `src/smoke_playbook.js`, 4 incidents (INC-0054, INC-0084, INC-0129, INC-0165)
- `src/bank_repair.py`, 4 incidents (INC-0070, INC-0071, INC-0072, INC-0121)
- `src/weekly_audit.js`, 3 incidents (INC-0050, INC-0048, INC-0018)
- `src/smoke_consent.js`, 3 incidents (INC-0005, INC-0205, INC-0211)
- `src/smoke_items.js`, 3 incidents (INC-0006, INC-0099, INC-0210)
- `src/smoke_redirect.js`, 3 incidents (INC-0023, INC-0024, INC-0047)
- `src/build_playbook.py`, 3 incidents (INC-0057, INC-0065, INC-0083)
- `src/gen/g_act_sci.py`, 3 incidents (INC-0093, INC-0094, INC-0098)

The Defect Ledger

Every entry here happened. Each one is a record of something that broke, how it was found, what fixed it, and what now stops it coming back. The last field is the one that matters for a different business: the transferable lesson, stated without reference to this codebase.

They are grouped by the part of the system, and within a group by date. The `guard` field feeds the checklist chapter automatically, so nothing here has to be copied anywhere by hand.

Content generation (100)

INC-0003. Item banks were different on every build because Python randomises hash()

2026-09-16, Wrong data shown or stored, `0d6c357` *PR #33*

- **What was seen.** Two builds of the same commit produced different banks, and the per-section bias figures pinned in the test file drifted run to run.
- **Why.** build_banks.py seeded each category with abs(hash(exam + skill)). Python randomises string hashing per process by default.
- **How it surfaced.** A pinned test figure would not stay pinned. (A test caught it)
- **Fix.** Seed from zlib.crc32, which is stable across processes.
- **What stops it now.** three consecutive builds must produce identical output in `src/build_banks.py`
- **Cost.** a flaky ratchet that would have been disabled eventually
- **Lesson.** Any generator that claims reproducibility must be seeded from something stable across processes. hash() is not, in Python, and the failure shows up as a flaky test rather than as a wrong answer.

INC-0004. A shadowed variable silently deleted 3000 items

2026-09-16, Silent loss, `aecccd9` *PR #37*

- **What was seen.** ACT Maths dropped from 4500 items to 1500 and the build reported success.
- **Why.** Adding an ACT English mapping declared a second ACT_MAP that shadowed the existing one. Nothing errors when a map is smaller than it used to be.
- **How it surfaced.** Comparing the reported category counts against the previous build. (Found by measuring something)
- **Fix.** Merge the maps properly and read the section per skill.
- **What stops it now.** the build prints per-category counts and the test file pins them in `src/test.js`
- **Cost.** 3000 items, caught before merge

- **Lesson.** Deletion by shadowing is invisible. Any collection whose size is a fact about the product needs its size asserted, not just its contents.

INC-0039. 225 of 302 correct answers sat at position A

2026-09-16, Wrong data shown or stored, `81eecec` *PR #30*

- **What was seen.** Three quarters of the correct answers in a new bank were the first option.
- **Why.** Items were authored with the key written first and the position never randomised.
- **How it surfaced.** Counting the key positions, a check nobody had run. (Found by measuring something)
- **Fix.** Randomise the key position per draw.
- **What stops it now.** the test suite pins the key-position distribution in `src/test.js`
- **Cost.** a bank that could be beaten without reading it
- **Lesson.** In any set of multiple-choice content, count where the answers are. A positional tell makes the whole set worthless to a test-wise user, and it is invisible item by item.

INC-0044. The longest option was the correct answer 81 percent of the time

2026-09-16, Wrong data shown or stored, `425a8bb` *PR #35*

- **What was seen.** A student who picked the longest option without reading the question scored 81 percent against a chance level of 20.
- **Why.** Distractors were shorter than keys because a correct answer is naturally more qualified, and nothing measured the resulting length distribution.
- **How it surfaced.** Measuring the accuracy of a strategy that ignores the question entirely. (Found by measuring something)
- **Fix.** About ninety items had distractors rewritten to carry comparable development, each added clause chosen to leave the option wrong for the reason it was already wrong. 81 percent to 38, then to chance in a later pass.
- **What stops it now.** a ratchet pins the length tell and fails if it rises in `src/test.js`
- **Cost.** a section that could be beaten without reading it
- **Lesson.** Test your content against the strategies a lazy adversary would use, not only against whether it is correct. Measure the score of a rule that ignores the question.

INC-0007. The dedup key counted a reshuffled question as a new one

2026-09-17, Wrong data shown or stored, `fc4114c` *PR #39*

- **What was seen.** Bank counts were inflated. The same question with its options rearranged looked distinct.
- **Why.** canon() hashed the choices in the order they appeared, while the answer's position is randomised per draw.
- **How it surfaced.** Sorting the choices before hashing, which immediately collapsed the counts. (Found by reading the code or the output)
- **Fix.** Sort the choices inside canon().
- **What stops it now.** canon() sorts, and the build reports distinct counts per category in `src/build_banks.py`
- **Cost.** four ACT categories reported far above their real size
- **Lesson.** A deduplication key must be canonical under every transformation the item legitimately undergoes. Ask what varies per draw before you hash.

INC-0008. Four passages produced five hundred fake distinct questions

2026-09-17, Wrong data shown or stored, `61c3ed2` *PR #40*

- **What was seen.** A reading generator with four passages reported five hundred distinct stated-idea items.
- **Why.** The identity of a reading item is the passage plus the question asked, not the choices offered. The same stem with the same key varied only in which distractors came along.
- **How it surfaced.** Reading the generated output rather than its count. (Found by reading the code or the output)
- **Fix.** canon() lets a generator declare that its identity is the passage plus the question.
- **What stops it now.** per-category distinct counts printed at build time in `src/build_banks.py`
- **Cost.** would have shipped a bank five hundred deep and four questions wide
- **Lesson.** The same inflation arrives through a different door every time you close one. When you fix a dedup bug, ask what else shares an identity.

INC-0009. Two conditionals hashed identically and half the inference items would have vanished

2026-09-17, Silent loss, `61c3ed2` *PR #40*

- **What was seen.** Half of a passage's inference questions disappeared from the corpus.
- **Why.** Both conditionals produced the stem 'which of the following can be properly inferred from the passage', so they hashed to the same value and one was dropped as a duplicate.
- **How it surfaced.** Measuring true yield per passage. (Found by measuring something)
- **Fix.** Each conditional names the case it asks about.

- **What stops it now.** yield per passage is measured and recorded, not assumed in `src/build_banks.py`
- **Cost.** half the inference items in any corpus
- **Lesson.** Dedup can be wrong in both directions. An over-broad key deletes real content as silently as a narrow one inflates it.

INC-0011. A length guard silently dropped sixteen valid items

2026-09-17, Silent loss, `61c3ed2` *PR #40*

- **What was seen.** Sixteen items never appeared in the bank.
- **Why.** The guard measured characters, and the items were legitimately longer than the limit in characters while being normal in content.
- **How it surfaced.** Counting inputs against outputs. (Found by measuring something)
- **Fix.** Measure what the guard actually cares about.
- **What stops it now.** input and output counts are compared at every filter stage in `src/build_banks.py`
- **Cost.** sixteen items
- **Lesson.** Every filter needs its rejection count reported. A filter that silently drops is indistinguishable from an input that was never there.

INC-0062. Correcting a length tell moves it one rank over, every time

2026-09-22, Degraded, `b958ed11194565fcd5d1fb5fe3ccddd699e305e7` *PR #66*

- **What was seen.** On two separate hand written banks, extending one distractor per item took the longest-is-key rate to near zero and left 25 of 35 and then 30 of 42 keys sitting second longest, so a reader picking the second longest option scored 71 percent on both.
- **Why.** Mechanical rather than careless. Extending exactly one distractor past the key moves every key from rank 5 to rank 4 by construction. The recorded ratchet measures only the two extremes, so a corrected bank passes it while carrying a stronger tell one position in.
- **How it surfaced.** Measuring the full length rank rather than only the extremes, after the extreme came back clean. (Found by measuring something)
- **Fix.** Three passes rather than one, extending a second and third distractor on overlapping subsets, and a shared measure() that prints the whole rank distribution and the best single-rank strategy so the artefact cannot hide behind a passing extreme.
- **What stops it now.** bank_emit.measure prints the full rank and the best single-rank score in `src/bank_emit.py`
- **Cost.** two banks that would have passed the ratchet carrying a 71 percent tell

- **Lesson.** A guard on the extreme of a distribution can be satisfied by moving the mass next to the extreme. When you correct for a measured bias, measure the whole distribution afterwards, not the statistic you were correcting.

INC-0066. Half the length tell correction did nothing and the build said it had worked

2026-09-22, Silent loss, `1fd4e0529fe06171c90528558b16358e25b71c6a` *PR #69*

- **What was seen.** The new GMAT reading bank corrected its length tell with a table that carries a named number of distractors past the key on each item, so the keys land at spread ranks rather than all at one. The table named 65 items. On 31 of them the authored clause was shorter than the gap it had to close, so the distractor stayed below the key and the item did not move. The run printed an improved distribution and no error, and the 31 were only found by measuring intent against outcome by hand.
- **Why.** extend() appends whatever clause the author supplies and checks only that the needle matches exactly one non key choice. Whether the choice ends up longer than the key, which is the entire point of the call, was never checked. The author sizes each clause by eye against a gap reported in a separate run, and an estimate made that way is wrong about half the time.
- **How it surfaced.** Comparing the intended lift count for each listed item against the key's actual rank after the run, in a scratch script written because the printed distribution was flatter than before but not as flat as the table should have made it. (Found by measuring something)
- **Fix.** check_lift() in bank_emit.py takes the same intent the tables express, one entry per item saying how many distractors were meant to pass the key, and exits naming every item whose key did not land at the matching rank and by how much the clause fell short. The generator calls it after the last extend pass, so a clause that does nothing fails the build instead of being averaged into a number that looks better.
- **What stops it now.** check_lift asserts each lifted item's key rank matches the number of distractors the table lifted in `src/bank_emit.py`
- **Lesson.** A correction table is a set of claims about outcomes, and an entry that quietly fails still counts as applied. Aggregate metrics hide this well: a table where half the entries work still moves the number in the right direction, which reads as success. State the per item intent in a form the machine can check, and every entry that did nothing says so by name.

INC-0068. Shuffling the answers twice put 37 percent of the keys at A

2026-09-22, Silent loss, `879f9584ae538423484f8990f3eb93b23dd320ae` *PR #70*

- **What was seen.** The new LSAT reading generator printed a key position distribution of 33, 15, 14, 17, 11 across 90 items where an even split is 18 each. Position A held 37 percent of the keys, against a chance rate of 20, which is the exact defect permute() exists to prevent (INC-0039) and which the engine test's answer position check would have failed.

- **Why.** The generator called E.permute(I) twice. An intermediate run had ended with permute and measure so the skill counts could be read, and the final block added its own permute before the extend pass, and the earlier pair was never removed. permute seeds a shuffle with crc32 of the item id, so the second call applies the same permutation again, composing it with itself. A permutation composed with itself is not uniform: it favours its own fixed points and short cycles, and with five choices that piles keys onto the position they started at, which is A because items are authored key first.
- **How it surfaced.** Reading the generator's own measure output after the length tell was cleared. The length rank line was flat and the key position line on the same screen was not, and it had been on the screen for several runs before it was read. (Found by measuring something)
- **Fix.** permute() now stamps each item it has shuffled and exits if it is handed one twice, naming the item. The duplicate call in mk_bank_lsat_rc3.py is removed. The measured distribution after the fix is 20, 19, 19, 16, 16 across 90 items, and the length rank is 18 in every position.
- **What stops it now.** permute refuses to shuffle an item it has already shuffled in `src/bank_emit.py`
- **Lesson.** A seeded shuffle is deterministic, which makes calling it twice look harmless: the same input gives the same output. What repeats is the permutation, not the randomisation, and a permutation applied to its own result is biased toward leaving things where they were. Any function whose value comes from being applied exactly once should refuse to be applied twice rather than relying on the caller to remember. The second lesson is about reading: the generator printed the defect on every run, above the line being watched.

INC-0069. A bank a student can play at 88 percent, inside a section the check passed

2026-09-22, Degraded, `879f9584ae538423484f8990f3eb93b23dd320ae` *PR #70*

- **What was seen.** bank_sat_rw.js ships 32 items on which the longest answer choice is the key 88 percent of the time, and a student who always picks the longest option scores 84 percent on that file against a chance rate of 25. Eighteen other hand written bank files across all five exams carry the same tell at lower strength, several above 50 percent. The engine test reported the SAT Reading and Writing section at 35 percent and passed it.
- **Why.** The length bias check groups items by section. A section mixes hand written items with generated ones, and the generated items are flat by construction because the schemas assemble choices mechanically. For SAT Reading and Writing the generated bank supplies 160 of 348 items at 7 percent, which pulls a hand written 88 percent down to a section figure of 35 and under the recorded tolerance of 36. The aggregate was not wrong; it was answering a question nobody needed the answer to. A student does not meet a section, they meet items, and the items arrive from one file at a time.
- **How it surfaced.** Measuring the same statistic per source file rather than per section, which was written to find out where the new LSAT and GMAT banks sat relative to the old ones and reported the old ones instead. (Found by measuring something)

- **Fix.** The check now runs per hand written source file as well as per section, with its own recorded baseline per file, so a file can only improve. Numeric answer banks are excluded because the separate numeric rank check covers them and character length is the wrong measure for a number. src/bank_repair.py applies an authored clause table to a hand written bank in place, by textual substitution inside the named choice, so the remaining files can be corrected without regenerating a file that was never generated.
- **What stops it now.** length bias is measured per hand written bank file against a recorded baseline, not only per section in `src/test.js`
- **Lesson.** An aggregate over a mixed population reports the population, and if part of that population is flat by construction it will hide the part that is not. The rule that follows is about what the unit of the measurement should be: measure at the grain the defect can exist at, which here is the file, because a file is written by one person in one sitting with one set of habits. The section was the grain the data was convenient at.

INC-0070. The second tool that appends a clause did not carry the first one's rule about the full stop

2026-09-22, Cosmetic, `879f9584ae538423484f8990f3eb93b23dd320ae` *PR #70*

- **What was seen.** Two repaired SAT items read 'the number of purchases. made' and 'sleep debt over the course of a week. that they do not fully repay at the weekend'. The clause had been appended after the sentence's own full stop.
- **Why.** bank_emit.extend has handled this since it was written: it strips a trailing period, appends the clause and puts the period back. bank_repair.repair was written months later for hand written banks that a generator never produced, and it inserts at the closing quote of the string literal, which is after the period. The rule lived in one implementation rather than in a place both could use, and the second implementation was written from the problem rather than from the first solution.
- **How it surfaced.** Reading the per item report after the repair, where the appended text appears at the end of each changed choice. (Found by reading the code or the output)
- **Fix.** repair() now inserts before a trailing period, matching extend(). Both behaviours are stated in one comment naming the other function, so the next tool that appends to a choice is written from a rule rather than from scratch.
- **What stops it now.** bank_repair.repair inserts before a trailing full stop, as bank_emit.extend does in `src/bank_repair.py`
- **Lesson.** Writing a second tool for the same job in a different context reproduces every detail the first one learned the hard way, unless the detail is written down somewhere the second author will look. This is INC-0067 seen from the other side: there the callers were not migrated to the helper, here the helper's behaviour was not carried into the second implementation. Both are the cost of a rule living in code rather than in a statement of the rule.

INC-0071. Clauses written from a truncated report were appended to the end of the wrong word

2026-09-22, Wrong data shown or stored, `879f9584ae538423484f8990f3eb93b23dd320ae` *PR #70*

- **What was seen.** Repaired choices read 'in chronological orderorder', 'return to them afterwardsards', 'how it would spend itintends to spend the money' and 'the bags the proposal would affectl would charge for'. Eleven choices across four bank files were garbled in a way that leaves the file valid JavaScript, leaves the item longer, and moves the measured distribution in the intended direction.
- **Why.** The report that shows which distractors need lengthening truncates each choice to fit a terminal line. Several clauses were written as continuations of the truncated text, on the assumption that the tool inserts at the point the needle ends. It does not: repair() appends at the end of the string literal, which is correct for a clause and wrong for a continuation. Nothing distinguished the two, because a continuation is just a clause whose first character is a letter.
- **How it surfaced.** Reading the source after a needle failed to match, which showed the previous pass had written 'would spend itintends to spend the money' into the same choice. (Found by reading the code or the output)
- **Fix.** repair() refuses a clause that does not begin with a space or punctuation, which is exactly the shape of a continuation. The four affected files were reverted and the repairs redone with clauses written to append rather than to continue.
- **What stops it now.** bank_repair.repair refuses a clause beginning with a letter or digit, which would run into the preceding word in `src/bank_repair.py`
- **Lesson.** A report that truncates its output invites the reader to write text that continues it, and a tool that appends will put that text somewhere else. Either the report should not truncate the field the caller has to write against, or the tool should refuse input shaped like a continuation. The cheap half is the refusal, because it is one condition and it cannot be forgotten, while remembering not to write continuations is a habit that has to hold every time.

INC-0072. A distractor was replaced and the explanation went on naming the old one

2026-09-22, Wrong data shown or stored, `879f9584ae538423484f8990f3eb93b23dd320ae` *PR #70*

- **What was seen.** SR001 offers unremarkable, unaccounted for, unrelated and undisturbed, and its wrong answer note begins 'Unnoticed reverses the point'. Unnoticed is not on the paper. A student reading the explanation after answering is told why an option they were never shown is wrong.
- **Why.** The length tell on vocabulary in context items cannot be fixed by appending a clause to a single word, so bank_repair.swap replaces the distractor with a longer one of the same register. It replaces the text in the choices array and nowhere else. An item is not just its choices: expl and wrong name particular options, and on this bank they name them by word.
- **How it surfaced.** Reading the item while looking at its shape for a different purpose, three commits after the swap was made. (Found by reading the code or the output)

- **Fix.** swap() refuses when the text being replaced appears anywhere else inside the same item, which is exactly where an explanation that names it would be. The one affected note is rewritten to name a distractor that exists.
- **What stops it now.** bank_repair.swap refuses to replace a choice whose text appears elsewhere in the same item in `src/bank_repair.py`
- **Lesson.** A record has parts that refer to one another, and a tool that edits one part by text is editing a graph while looking at a string. The cheap guard is not to check every reference but to refuse the edit when the old text occurs anywhere else in the record, because that is the only place a reference to it can be. Refusing on a false positive costs one rewritten table entry; not refusing ships an explanation about an option nobody saw.

INC-0073. check_lift was told a one clause entry lifted two distractors

2026-09-22, Degraded, `879f9584ae538423484f8990f3eb93b23dd320ae` *PR #70*

- **What was seen.** The GRE reading generator failed with three items named: GR104, GR128 and GR133 were said to have lifted 2 distractors and to belong at rank 3, and each sat at rank 4. Each of the three had exactly one clause in the table.
- **Why.** bank_emit.extend accepts a table entry as either one (needle, clause) pair or a list of them, and it is the only place that knows the difference. Every caller of check_lift then has to re-derive the count, and the obvious expression, len(v), reads a bare two element tuple as two lifts. The rule for reading the table lives in extend; the callers each hold a private copy of it, and one copy was wrong.
- **How it surfaced.** check_lift failed the generator run and named all three items with their shortfalls. (A build guard caught it)
- **Fix.** bank_emit.lift_counts(table) derives the intent from the table using the same tuple or list test extend uses, and every generator that had spelled it out by hand now calls it.
- **What stops it now.** bank_emit.lift_counts is the single reader of the table shape, so check_lift and extend cannot disagree about what an entry says in `src/bank_emit.py`
- **Lesson.** A guard that takes the intent as an argument is only as good as the argument, and an argument derived by hand from the same data the guard is checking is a second implementation of the thing being checked. It fails in the direction that is hardest to see: too high an intent demands a rank the clauses cannot reach, and the author satisfies it by writing more clauses than the plan called for, which skews the distribution the other way while every check passes. Derive the intent from the data with the code that already reads it.

INC-0074. A corpus field written for one grammatical slot was spliced into another

2026-09-22, Wrong data shown or stored, `2c72da1fde8a3819ca9bda7eabe9fb75ac0754b7` *PR #71*

- **What was seen.** 160 of the 833 shipped GMAT v_pc items, 19 percent, offer a choice like "That shorten the time taken to settle a claim is the most urgent of the problems facing Calloway Insurance." It is not English. A student reading it can strike it out without considering the argument, which is a free elimination on a five choice item and makes the item easier than the rating it carries.
- **Why.** A PLAN scenario stores goal as a bare infinitive phrase, because the stem it was written for reads "X intends to <goal>". Two other templates splice the same field after "has tried to", where a bare infinitive is also right. One splices it into a subject position, "That <goal> is the most urgent of the problems facing X", where English wants a gerund or a noun phrase. The field is correct; the slot it was reused in is not, and nothing connected the two.
- **How it surfaced.** Reading the output of PlanAssume while counting the PLAN corpus in order to widen it. The category had been short since it was written and the shortfall was the reason to look. (Found by reading the code or the output)
- **Fix.** Every PLAN scenario gains goal_np, the same goal as a noun phrase, and the subject slot uses it. The bare infinitive stays where a bare infinitive belongs.
- **What stops it now.** framework.verify rejects a stem or choice where That or Whether is followed directly by a bare infinitive, which is what splicing an infinitive goal into a noun slot produces in `src/gen/framework.py`
- **Lesson.** A corpus field is written against the one sentence the author had in mind, and the schema that reuses it three templates later has no way to know which shape it is. The type system says str in both places. Two things follow. Store the field in every shape a template needs and name the shapes, rather than storing one shape and trusting the next author to notice. And guard the output, not the corpus: the generated sentence is the only place the mismatch becomes visible, and a cheap pattern over the rendered text catches a class that no check on the inputs can see.

INC-0075. Every correct answer on one GMAT schema was ungrammatical, and the guard written an hour earlier could not see it

2026-09-22, Wrong data shown or stored, `2c72da1fde8a3819ca9bda7eabe9fb75ac0754b7` *PR #71*

- **What was seen.** All 277 shipped cr_plan_eval items, a third of the GMAT Plan and Construct category, have a key reading "Whether what share of the traffic on Bridge Street stops there at all is what the measure would change." Every one of the eight scenarios produces one, across all six wh words. The student is asked to choose between four fluent distractors and one sentence that is not English, so the item is answerable without reading the argument and is rated as though it were not.
- **Why.** The same class as INC-0074 and the same corpus: check is stored as a wh clause because the explanation reads "Establish <check>, and one answer means ...". A second template put it after "Whether" and before "is what the measure would change", where a wh clause cannot go. INC-0074 was the same mistake with the goal field, found in the same reading, and the guard written for it tests for an infinitive after That or Whether. check is not an infinitive, so the guard passed it.
- **How it surfaced.** Rendering one item of each plan schema to read the output, immediately after fixing INC-0074 in the schema next to it. (Found by reading the code or the output)

- **Fix.** check is rewritten in all scenarios as a clause that reads after "Whether", the trailing "is what the measure would change" is dropped, and the explanation says "Establish whether". The guard is widened from infinitives to any corpus field spliced after That or Whether that does not read as a clause there, and it now runs over the key as well as the distractors.
- **What stops it now.** framework.check_clause_splice refuses a rendered sentence whose That or Whether is followed by a corpus field stored for another slot, tested for every plan field rather than for goal alone in `src/gen/framework.py`
- **Lesson.** A guard written from the instance in front of you covers that instance. INC-0074 was a bare infinitive in a noun slot, so the guard looked for bare infinitives, and the sentence one screen away in the same file was a wh clause in a clause slot and went straight through. The general defect was never the infinitive; it was that a corpus field carries no record of the grammatical shape it was written in, and any template may reuse it. So the guard has to be stated over the class, every field against every slot, not over the token that happened to be wrong first. The other half of this is where it was found: the distractor version was spotted first because it is louder, and the version in the key, which is three times as damaging, was found only because the first one prompted a second look. Reading one rendered item per schema would have caught both on the day they were written, and costs less than either fix.

INC-0076. str.capitalize() lowercased the rest of the sentence, and took the proper nouns with it

2026-09-22, Wrong data shown or stored, `2c72da1fde8a3819ca9bda7eabe9fb75ac0754b7` *PR #71*

- **What was seen.** 180 shipped generated items render a proper noun in lower case. 70 of them carry it in the correct answer: "The fenwick track is the only one in the county, so every competitive runner trains there whether they win or not" and "The halloran fund supplies the starting grant for nearly all of the laboratory's work". The stem two lines above spells both correctly, so the item contradicts itself on the page and the key is the choice that looks wrong.
- **Why.** Python's str.capitalize() is not what its name suggests to a reader in a hurry. It uppercases the first character and lower cases every other one. The generators use it twenty one times to render a stored fragment as the start of a sentence, which is right for a fragment of ordinary prose and wrong for any fragment containing a name. Four corpus fields contain one.
- **How it surfaced.** Reading one rendered item from each of the five schemas in a category before widening its corpus, which is the practice INC-0075 ended with. (Found by reading the code or the output)
- **Fix.** framework.upfirst raises the first character and touches nothing else, and every call site uses it. The four affected fields need no change, which is the point: the fragment was always correct and the renderer was destroying it.
- **What stops it now.** src/build.py fails if str.capitalize() appears anywhere under src/gen, the same way it fails on an em dash in `src/build.py`

- **Lesson.** A standard library function whose name is a plausible description of half of what it does will be used for that half. capitalize() reads as "make this the start of a sentence" and is in fact "make this the start of a sentence and flatten everything else", and the damage is invisible until a value happens to contain a capital. The guard is not a test that the output looks right, because the output looked right for every value that had no name in it. The guard is to ban the function: the correct one is three characters of slicing, the wrong one is never what a generator wants, and a lint catches it in the diff rather than in the bank.

INC-0078. A coefficient of one was printed as 1x on 2,353 shipped items

2026-09-22, Wrong data shown or stored, `2c72da1fde8a3819ca9bda7eabe9fb75ac0754b7` *PR #71*

- **What was seen.** 2,353 items across eight schemas and every exam that remaps them read "x squared - 1x - 30 = 0", "5(x - 7) = 1x - 6" and "2x + 3y = 10 and 1x + 5y = 12". The arithmetic is right and the key is right. The notation is not how anyone writes algebra, and on a test preparation product it tells the student the question was produced by a machine that does not know the convention.
- **Why.** Every schema that draws a coefficient formats the term itself, as the number followed by the variable. That is correct for every value except one and minus one, which are written as the bare variable. The rule is a convention of notation rather than of arithmetic, so no computation check could see it, and it was reimplemented at each of the sites that needed it, each time without the exception.
- **How it surfaced.** Reading one rendered item from each of the five schemas in the last category still under target, before widening them. (Found by reading the code or the output)
- **Fix.** framework.term renders a coefficient and a variable together and is the only place that knows the convention. Every site that built the string itself now calls it.
- **What stops it now.** framework checks its own rendered stems for a unit coefficient and drops the item, so a new schema that formats a term by hand fails rather than ships in `src/gen/framework.py`
- **Lesson.** Presentation rules travel with the value, and a value formatted at the point of use is formatted by whoever was writing that line. Eight schemas each wrote the same two characters and all eight omitted the same exception, which is not eight mistakes but one missing function. The give away is the shape of the defect: identical output in unrelated files means the knowledge was never in one place. The check that catches it cannot be on the arithmetic, because the arithmetic was always right, so it has to be on the rendered string.

INC-0079. A generated schema was playable at 100 percent by picking the third shortest option, and no check looked at generated schemas

2026-09-22, Wrong data shown or stored, `2c72da1fde8a3819ca9bda7eabe9fb75ac0754b7` *PR #71*

- **What was seen.** A student who always picks the third shortest option answers 861 of 861 sat_rw_apostrophe items correctly, and another 887 through the ACT remap, without reading a word. It is the largest single schema in Standard English Conventions on two exams. Measuring the rest found 39 of 92 generated schemas over the 1.8 times chance cap on one of the three figures, among them cr_plan_eval at 98 percent on one rank over 1,117 items, act_kol_redundancy at 100, act_s_changed at 97 percent shortest, and cr_plan_weaken at 65 percent longest.
- **Why.** INC-0069 added a length bias check per hand written source file, because a section figure hid a file at 88 percent. The same argument applies one level down and was not followed there. Generated items are checked only at the section level, and the section check runs on what the harness loads, which is the hand written banks plus the strided starter slice, so a schema contributes a few dozen items to a figure covering hundreds. The apostrophe schema is worse than an accident of drafting: its four choices are the four forms of one noun, and for a regular noun those forms are ordered by length by construction, so the key sits at the same rank on every item the schema can produce.
- **How it surfaced.** Widening five schemas failed the SAT Math ratchet, and measuring every generated schema to see whether that failure was real turned up the rest. (Found by measuring something)
- **Fix.** build_banks measures every generated schema over the whole bank rather than the starter, against a recorded table that can only come down. The apostrophe schema gains nouns whose plural is not the singular plus one character, so the four forms are no longer length ordered.
- **What stops it now.** build_banks fails on a generated schema over its recorded length bias, measured across the full bank per schema and per exam in `src/build_banks.py`
- **Lesson.** A check is scoped to a grain, and the grain is a claim about where a defect can live. INC-0069 moved the grain from the section to the file for hand written banks and stopped there, so the same defect went on living one level down in generated ones, where there are far more items. When a check finds something by being made finer, the question to ask immediately is what else is measured at the old grain. The second half is about which populations a check can see: this one ran on what the test harness loads, which is a sample chosen for a different purpose, and a sample chosen for a different purpose is not a population you can make claims about.

INC-0081. A student who always answers 1 scores 98 percent on a schema, and no check looked at the answer itself

2026-09-22, Wrong data shown or stored, `2c72da1fde8a3819ca9bda7eabe9fb75ac0754b7` *PR #71*

- **What was seen.** msr_count ships 1,867 items and the correct answer is 1 on 98 percent of them: the case sets it builds almost always contain exactly one compliant row. act_s_trend ships 820 and the answer is the rising option on 76 percent, because the studies it draws trend upward three times in four. gt_count is at 47 percent on 1, and three Data Sufficiency schemas sit at 42 to 45 percent on one of their five fixed statements against a chance rate of 20.

- **Why.** Three checks look at where the answer sits: the position among the choices, the length rank, and for numeric items the value rank. All three are about the answer's place in the set it was shown in. None looks at the answer itself. A schema whose parameters happen to produce the same answer over and over passes every one of them, because the answer moves around the choices perfectly well; it is only always the same answer.
- **How it surfaced.** Dumping a dozen msr_count items to work out why its value rank would not move after two new distractors, and noticing that every key was 1. (Found by reading the code or the output)
- **Fix.** build_banks measures the most common single answer value per schema and refuses one over its recorded figure, alongside the three place based figures. make_cases draws how many rows comply rather than leaving it to fall out of the violation draw, and the science studies draw their direction evenly.
- **What stops it now.** build_banks records and ratchets the most common single answer value per schema, which is the first check on what the answer is rather than on where it sits in `src/build_banks.py`
- **Lesson.** Every guard here measured the answer's place in its set, and a set of guards that all take the same kind of measurement shares a blind spot the size of everything else. The tell they could not see was the simplest one a student would find: the answer is the same answer. When adding the third check of a kind, the question worth asking is not whether it is stricter than the other two but what all three have in common, because that is what is going unmeasured.

INC-0082. Nine published exam facts cite test prep companies, in the one published corpus with no source validator

2026-09-22, Wrong data shown or stored, `5d7e45dc41329917f8576f90107e1eb36ebf2c89` *PR #73*

- **What was seen.** The five exam guide pages publish nine figures sourced to test prep and content marketing companies: The Princeton Review three times (the digital SAT score release window, and the two LSAT facts that logic games are retired and that the variable section can appear anywhere), Applerouth twice (the enhanced ACT rollout dates and Science becoming optional), plus Menlo Coaching for how the GRE is delivered, Achievable for the GRE fee reduction amount, UWorld College Prep for SAT superscoring, and Sallie Mae for the SAT retake limit. CLAUDE.md bans coaching site blogs outright as sources. Several of these are structural facts a student plans around, carried under a citation the house rules forbid.
- **Why.** Two things, and the second is why the first went unseen. The published figure corpora are data/schools, data/colleges and data/exams.json. validate_schools.py and validate_colleges.py enforce the source policy on the first two and run from their builds; data/exams.json has no validator at all and build_exams.py calls none, so nothing has ever read a source on it. Even had one existed, it would have passed all nine: BANNED_SOURCES lists the six sites CLAUDE.md names as examples, and CLAUDE.md bans those six AND the category coaching site blogs. The general clause was dropped in transcription, so the list refuses GMAT Club by name and accepts Kaplan.

- **How it surfaced.** Reading ROADMAP.md for open work rather than picking by judgement. It carried a note that sat.score_release cited The Princeton Review and was still open. Asking the obvious next question, which corpus that figure lives in and what validates it, found the file had no validator; walking every url host in it against the test makers' own domains found eight more. (Found by reading the code or the output)
- **Fix.** src/validate_exams.py applies the same contract to data/exams.json that the other two corpora have had: every published figure carries src, year and url, the url host must be the test maker's own domain or an explicitly allowed one, and banned sources are fatal. build_exams.py calls it before it renders. The banned list moves to one shared module so the three validators cannot drift, and gains the coaching site category CLAUDE.md bans alongside the six named sites. Each of the nine figures was then re-verified against the test maker's own page and recited to it, or removed where the maker does not publish it.
- **What stops it now.** validate_exams.py fails the build on a figure in data/exams.json missing src, year or url, on a banned source, and on a url whose host is not the test maker's own or explicitly allowed, so a plausible looking citation from a prep company cannot be added silently in `src/validate_exams.py`
- **Lesson.** Two lessons, and they compound. A rule copied into code by its examples loses the clause the examples were illustrating: CLAUDE.md bans six named sites and coaching site blogs, and the list kept the six and dropped the category, which is the half that generalises. And a validator gets written for the corpus that had the problem at the time, then quietly defines what is checked: two of three published corpora were enforced and the third had never had a source read, which is not a weaker check but an absent one. When a guard exists, the question is not only whether it is strict enough but which of the things it could be pointed at it is not pointed at.

INC-0085. One flashcard for a skill, for as long as anyone cared to look

2026-09-22, Degraded, `052472e3ac614a5aa268405b0b1f1004b7a1b097` *PR #75*

- **What was seen.** Counting cards per skill rather than per exam for the first time: LSAT explicitly stated information had 1 card and inference 1, ACT integration of knowledge had 1, GRE sentence equivalence had 2, and 9 more skills across the three exams had 2. A student drilling one of those skills saw the same card come round immediately. The totals looked reasonable, which is why nobody looked further: 30, 33 and 45 cards against the GMAT 120.
- **Why.** test.js asserts that every tracked skill has bank items, and for the deck it asserts only that each card names a real section. So the bank has a per-skill floor and the deck has none, and the deck was measured by its total, which is an average over twelve or fifteen skills and hides any one of them being empty.
- **How it surfaced.** Counting cards per skill while sizing up the roadmap item about bank parity. The per-exam totals had been on the dashboard all along and said nothing was wrong. (Found by measuring something)

- **Fix.** Every skill on every exam brought to at least eight cards: 176 new cards written from each exam's own published skill list, taking the decks from 340 to 531. test.js now asserts the floor per skill, on every exam, so a new skill cannot ship with a token card.
- **What stops it now.** test.js checks cards per skill against a floor for every exam, the same shape as the existing per-skill check on bank items rather than a check on the deck total in `src/test.js`
- **Lesson.** Two corpora side by side, one guarded per unit and one guarded only in total, is not two levels of rigour but one measurement and one blind spot. When a check exists for the bank, ask what else is shipped alongside it and counted only in aggregate.

INC-0086. A finished generator module that nothing imported, and two of its four schemas produced nothing

2026-09-22, Silent loss, `3a45d4d2a9d679e94e07163a2e11cb012940b6e2` *PR #77*

- **What was seen.** src/gen/g_rc.py is 339 lines of working reading comprehension generator with an authored passage corpus, and no file in the repository imports it. Its four schemas were therefore contributing nothing. Two of them could not have contributed anyway: rc_main and rc_caveat raised ItemError on every single draw, because they build their distractors from OTHER passages and need choices_n minus one of them, which is four, and the corpus held four passages in total. The two GMAT skills the module targets, v_st and v_inf, were the only two of the twelve with no generated items at all.
- **Why.** Two failures that hid each other. The module is not in the import list in build_banks.py, so nothing ever called it and nothing ever reported that it was silent. And the corpus is one short of the minimum its own schemas require, which is a data shortfall that reads as an exception rather than as a count: ItemError is the framework's ordinary way of saying this draw did not work, so a schema that raises it every time looks exactly like a schema that is merely fussy.
- **How it surfaced.** Looking for the largest remaining gap in the banks and finding that the LSAT had no generator. Listing what generators exist turned up a module for reading comprehension that no import statement mentions, and running its four schemas directly showed two of them returning nothing at all. (Found by reading the code or the output)
- **Fix.** Eight more passages, taking the corpus from four to twelve, which is past the minimum both schemas need and gives the distractor balance room to work. All four schemas now run with zero errors. The module is imported and wired into the GMAT plan for v_st and v_inf, which had no generated items, and its schemas are mapped onto the LSAT reading categories, which had none either.
- **What stops it now.** build_banks fails when a schema in a wired plan produces no items at all, so a schema that cannot draw is a build failure rather than a quiet zero in a category total in `src/build_banks.py`

- **Lesson.** A module that nothing imports fails no test, and an exception raised on every draw is indistinguishable from an exception raised on a hard draw. Both are silence, and a build that reports totals hears neither. Count what each schema contributed, not what the category holds, and treat a contribution of zero as a failure rather than as a small number.

INC-0087. The same corpus field in two grammatical slots, in a schema written the same day the guard was read

2026-09-22, Wrong data shown or stored, `51a41537e6f9b8877295392ab15a91d2e98f67ce` *PR #78*

- **What was seen.** A new flaw schema, cr_necsuff, renders its correct answer as 'treats a condition that is required for be admitted to the conservatoire as though meeting it were enough'. The goal field holds a bare infinitive because the stem needs one after may: 'Wen may be admitted to the conservatoire' is right. After 'required for' the same string needs a gerund. Caught before the schema was committed, by rendering one item from each of the three new schemas and reading them.
- **Why.** One field written against the slot the author had in mind and reused in a second slot that takes a different form. That is INC-0074 exactly, and INC-0074's own lesson says what to do instead: store the field in every shape a template needs and name the shapes. I read that record while building the recurrence counting earlier in the same session and then wrote the schema with one shape anyway.
- **How it surfaced.** Rendering one item from each new schema and reading it, which is the practice INC-0075 ended with. It cost three items of output and found the defect before the file was committed. (Found by rendering it and looking)
- **Fix.** The scenario carries goal and goal_ing, both written out, and each template slot names the one it needs. No transformation between them, because the transformation is where the assumption hides.
- **What stops it now.** check_clause_splice and the class level guard from INC-0075 already cover the wh clause and infinitive cases at emit time; the practice that caught this one, rendering an item per schema before widening its corpus, is the guard that works on a shape nobody has written a check for yet in `src/gen/framework.py`
- **Lesson.** Reading the record does not prevent the defect; the practice does. This one was written hours after its own lesson was read closely enough to be catalogued as a recurrence, and it was caught by rendering three items rather than by remembering. Budget the render, not the recollection.

INC-0088. A shipped schema answerable at 68 percent by picking the shortest option, under the size at which anything is checked

2026-09-22, Wrong data shown or stored, `144c1b209e20e910682c987f214eb4c9e36a0e86` *PR #79*

- **What was seen.** rc_infer ships 40 items to GMAT v_inf and the correct answer is the shortest of the five choices on 68 percent of them, against a chance rate of 20 and a cap of 36. It merged and went live without the bias check ever looking at it. The same schema over the long passage corpus is at 62 percent on 16 items. The cause is structural: the key is a modus tollens conclusion, a single clause, and the distractors are the converse and inverse of a conditional plus sentences lifted from the passage, all of which run longer.
- **Why.** check_bias opens with 'if n < 50: continue', so a schema contributing fewer than fifty items is not measured at all. The threshold is defensible for the cap it applies, because 36 percent on a handful of items is within sampling noise, but skipping is the wrong response to that: at 40 items a rate of 68 percent is nearly eight standard errors above chance and is not noise by any reading. The check switched off where it should have widened.
- **How it surfaced.** Measuring the four reading schemas directly after noticing that adding them did not move the 'schemas measured' figure the build prints. The count staying at 188 while four schemas were added is what gave it away. (Found by measuring something)
- **Fix.** The cap widens for small samples instead of the check switching off. Below ten items nothing is measured, because even a perfect run there is thin; from ten up the cap is the ordinary one or chance plus two and a half standard errors of a proportion at that sample size, whichever is larger. At 40 items that is 36 percent and rc_infer fails; at 8 items it is 55 and a schema is not condemned for noise. rc_infer itself gained distractors at the key's length, since its key is short by the nature of the inference it tests.
- **What stops it now.** check_bias measures every schema of ten items or more, with a cap widened by the sampling error at that size, so a small schema is held to a fair standard rather than to none in `src/build_banks.py`
- **Lesson.** A size threshold on a check is a silent exemption, and it grows as the corpus does: every schema written from a small authored corpus falls under it by construction, which is exactly the population most likely to carry a structural tell. When a measurement cannot be trusted at a small sample, widen the tolerance to what the sample supports rather than declining to measure, and notice when a count of what was measured does not move after you add something to measure.

INC-0090. Two of a schema's three question forms could not build, and the only sign was an item count

2026-09-22, Silent loss, `9056b8daa4fe87508a82591b79adb7b8a9fd181f` *PR #82*

- **What was seen.** The GRE shipped 48 items of the trigonometry schema where the SAT shipped 96, and the key was the smallest of the five options on half of them. The schema listed five wrong answers and its SAT and ACT copies built without complaint. Checking the other banks the same way found it again in the quadratic roots schema: the sum question never built at five choices, so the GMAT and the GRE carried that schema's product and greater-solution questions and none of its sum questions. Between them, two whole question forms were missing from three exams.

- **Why.** Two entries in a distractor list can be the same arithmetic without looking alike. Swapping the opposite and adjacent sides and inverting the ratio both turn a tangent into its reciprocal. The sum of the roots IS the negated coefficient of x and their product IS the constant term, so on a sum question 'flip the sign' and 'read the coefficient off the equation' are one wrong answer wearing two labels, and so are 'give the product' and 'read the constant term'. Once the duplicates are dropped the list is shorter than the exam asks for, make() raises, and the draw is discarded. That is unremarkable on its own: ItemError is the ordinary way a schema says not this draw, the next draw is a new set of numbers, and the category still fills from its neighbours. But when the collision is in the SHAPE of the wrong answers rather than in the numbers, every draw of that question form is discarded and the form is absent from that exam.
- **How it surfaced.** A geometry schema failed the answer bias check for a reason that only made sense if a whole question form were missing, so the schema was built directly at four and five choices and what came back was counted. (Found by measuring something)
- **Fix.** The wrong answers are written out per question form instead of as parallel lookups keyed by the form, which is what hid the collision: the list reads as five entries whatever the form is. Each form now carries six or seven distinct wrong answers where five choices need four. The trigonometry schema went from 48 distinct items to 242 and builds all three of its functions at both widths; the quadratic roots schema stopped discarding two draws in five. Widening the polynomial expansion schema the same way cleared a third.
- **What stops it now.** check_starved draws a fixed number of times from every schema at every exam's choice count and fails the build on one that loses more than a twentieth of its draws for want of distinct wrong answers, with the sixteen already over it recorded in STARVED_DEBT so the table has to shrink rather than grow in `src/build_banks.py`
- **Lesson.** A generator's wrong answers are written as labels and read as labels, and nobody looks at the values two labels produce. Where the question type makes two misconceptions arithmetically identical the list is shorter than it reads, and because a discarded draw is the ordinary way a schema says no, a whole question form can vanish from an exam leaving nothing behind but a number nobody has a reference for. Count what a schema actually produces at each width it has to serve, and compare the widths against each other.

INC-0091. Every wrong answer a schema could think of was bigger than the right one

2026-09-22, Degraded, `9056b8daa4fe87508a82591b79adb7b8a9fd181f` *PR #82*

- **What was seen.** Four generated schemas put the correct answer at the same value rank on 39 to 47 percent of their items against a 36 percent line, and on the five choice exams the key was never the largest of the options and was the smallest on half of them. The rectangle schema, the box volume schema, the trigonometry schema and the polynomial expansion schema, each measured on its own draws rather than on the mix the bank happened to ship.

- **Why.** The characteristic errors for these questions all run one way. On an area question every slip, adding the sides or subtracting them or computing the perimeter, is smaller than the product. On a volume question every slip but doubling is smaller than three edges multiplied. On a trigonometry question every slip either inverts the ratio or trades a leg for the longer hypotenuse, so all of them are larger. The lists were written by naming misconceptions, which is the right way to write them, and nobody asked which side of the answer the resulting numbers land on. make() aims for a value rank drawn uniformly, but it can only place the key among the candidates it is handed, so with five candidates above it and one below it the key can occupy two ranks out of five and no shuffling changes that.
- **How it surfaced.** Measured each schema on its own seeded draws once a change elsewhere pushed them over the line, which separates what the schema does from what the bank happened to draw. (Found by measuring something)
- **Fix.** Each of the four now offers wrong answers on both sides of the key, taken from real slips rather than invented to fill a side: counting grid lines rather than the squares between them and adding the perimeter to the area sit above an area key, squaring the rectangle on its longer or its shorter side sits below a perimeter key, the sum of the three face areas sits below a volume and cubing the longest edge sits above it, and dividing by the sum of the two legs or by the perimeter is the one common trigonometric slip that lands below the ratio. The worst value rank across the four went from 47 percent to 30, and smallest is key on the trigonometry schema from 49 percent to 11.
- **What stops it now.** check_bias already measures which value rank the key holds for every schema at every width; what this change adds is that none of the four is recorded as debt any more, so the check holds all of them to the ordinary cap in `src/build_banks.py`
- **Lesson.** A list of misconceptions is a list of labels and a student sees numbers. Where every characteristic error runs the same direction the key sits at a predictable place in the ordered options however carefully the item is shuffled, because the shuffler can only place it among the candidates it is handed. When writing wrong answers, sort them next to the right one and look at which side they fall on.

INC-0092. A schema threw away three draws in four, and the counter that knew was read by nobody

2026-09-22, Silent loss, `be6b0736906afbb45e883a29de8c8752ed707465` *PR #83*

- **What was seen.** The exponential growth schema discarded 69 to 75 percent of everything it built, on all four exams that carry it, and the absolute value schema discarded 48 percent on the two that ask for five choices. Twenty nine schema and exam pairs lose draws this way. None of it appeared anywhere: not in the per category report, not in the bias check, not in the starvation check written the same day, and not in any test. The banks still filled, because a category that comes up short is topped up by the schema's neighbours.

- **Why.** Two causes, one on top of the other. The item cause is that a distractor which does not LOOK like the answer is never chosen, so make() keeps only candidates whose rendered shape matches the key's, and drops the draw when too few remain. The exponential schema multiplies a starting amount by a fraction raised to a power, so its key renders as a whole number on one draw, two decimal places on the next and a fraction on the one after, while its wrong answers render differently again: the pool of same shaped candidates is empty most of the time and the question type has no stable shape to hold them to. The reporting cause is the one that let it sit: make() counts these rejections into self.shape_misses, and shape_misses is written in exactly one place and read in none. A counter incremented for nobody is not a measurement, and the number it held was the largest single loss in the bank.
- **How it surfaced.** Counting what every schema produces per draw at each exam's choice count, after a smaller version of the same loss was found and guarded the day before. The guard written then matched on one rejection message, and this is the other one. (Found by measuring something)
- **Fix.** Eleven schemas, and the same two questions each time: does this question have one shape, and does it offer four wrong answers of that shape. The exponential growth schema asks for the quantity to the nearest whole number, which is how a real exam asks it. The absolute value schema is built from its two solutions rather than from the coefficients, so both come out whole. The percent question draws a base that makes the answer whole. The perpendicular slope schema takes both parts of its slope bigger than one and coprime, so the given slope and its negative reciprocal are both proper fractions. The distribute schema gains an off by one pair, which cannot change the shape of the answer it is built from and so supplies two same shaped candidates on every draw whichever kind it is. The ACT science table prints its readings to one decimal place, because a measurement column with uniform precision is both how a real table looks and the only way its readings can be offered as choices. The slope schema also lost a wrong answer that was minus the slope under a second name. Nothing was recorded to make any of this pass.
- **What stops it now.** check_discarded counts AssemblyError, which is the exception make() raises when it cannot assemble what a schema offered, rather than matching the wording of one of the two messages; it draws a fixed number of times per schema per exam so the figure is exact, fails the build above a twentieth, and holds what is still over in DISCARD_DEBT. shape_misses is gone: a counter nothing reads is worse than none in `src/build_banks.py`
- **Lesson.** A counter that nothing reads is not instrumentation, it is a comment that looks like instrumentation, and it is worse than nothing because it answers the question 'is anyone watching this' with a yes. Every time a guard is written against one symptom, ask what the same failure looks like arriving another way, and count the whole category rather than the instance that prompted it.

INC-0093. Seven variable names were plural and every sentence built around them said was

2026-09-22, Cosmetic, `cbd9af348b8a1e7a8e467196d6d2e979096baa00` *PR #84*

- **What was seen.** Eight of the ACT science schemas shipped sentences like 'when the minutes in the dye bath was 30 minutes' and 'the particles remaining is what the students measured'. Seven of the thirty study scenarios name a variable with a plural noun, and every template that puts a variable name in front of a verb assumes a singular one, so roughly a quarter of the science section read as though written by someone who does not speak English. It was on the page, in stems and in explanations, and no check looks at whether a sentence agrees with itself.
- **Why.** A scenario stores its variables as a name and a unit, and the name is dropped into whatever slot a template needs: the object of a preposition in one, the subject of a verb in another. Most names are singular noun phrases and the templates were written against those. Nobody wrote down that the templates need a singular, so seven scenarios with names like Days of curing and Seeds germinating out of 100 were added over time and each one broke every sentence that used it as a subject.
- **How it surfaced.** Rendering nine hundred items from every schema in the module and searching the text for a plural name followed by a singular verb, while fixing a different fault in the same file. Reading one item of each schema would not have found it: a schema picks a scenario at random and twenty three of the thirty are innocent. (Found by measuring something)
- **Fix.** Every variable name is now a singular noun phrase, which cost seven renames and no template changes: Minutes in the dye bath became Time in the dye bath, Days of curing became Curing time, Seeds germinating out of 100 became Number germinating out of 100. Four of them also had their unit in the name, so the table header read 'Days of curing (days)', and that is gone too. Each scenario now declares the grammatical number of both its variables, and the check refuses a plural one, so the requirement the templates have always had is written down where the next scenario is added.
- **What stops it now.** g_act_sci.check_names() refuses a scenario whose variables are not both declared singular, and renders items from every schema across every scenario looking for a name in front of a verb that disagrees with it; build_banks runs it in `src/gen/g_act_sci.py`
- **Lesson.** A template is a promise about the grammar of what goes into it, and the promise is invisible: the code says name and the sentence needs a singular noun phrase. Whenever a stored string lands next to a verb, an article or a plural, write the requirement down beside the data rather than in the template, and make adding a new row state that it meets it. Renaming the data to fit one grammar is usually cheaper and always safer than teaching the templates to handle two.

INC-0094. A reading of 100.2 for a quantity the table itself calls out of 100

2026-09-22, Wrong data shown or stored, `cbd9af348b8a1e7a8e467196d6d2e979096baa00` *PR #84*

- **What was seen.** One ACT science study measures seeds germinating out of 100, and on a fifth of its draws the table's first row read 100.2. Every question built on that table then asked about a number that cannot exist, and the ones asking for a trend or a comparison were still answerable, so nothing looked broken. The same table also reported a count of seeds to one decimal place.

- **Why.** A study's readings are a base and a step scaled by a jitter drawn per draw, and the only thing checked about the result is that it is positive and that the column is monotone. A quantity with a natural ceiling has no way to say so: this one tops out at 100 by its own name, and at the largest jitter the base alone cleared it. The decimal is the same gap from the other side, a quantity that can only be a whole number rendered by a formatter that gives every reading one decimal place.
- **How it surfaced.** Reading one rendered item from each of the renamed scenarios while fixing their grammar, which is the first time anyone had read this scenario's numbers rather than its trend. (Found by reading the code or the output)
- **Fix.** A scenario may declare a ceiling, and build_study refuses a draw that passes it, the same way it already refuses a non positive reading. The germination study declares 100 and its base comes down so the ceiling is headroom rather than a filter. Its measurement is a germination rate in percent rather than a count of seeds, because a rate is what a reading to one decimal place actually is.
- **What stops it now.** build_study rejects any reading above a scenario's declared ceiling, on both studies, the same check that already rejects a non positive one in `src/gen/g_act_sci.py`
- **Lesson.** Generated data gets checked for the properties the questions need, monotone and positive and distinguishable, and not for the properties the world needs. A quantity whose name contains its own bound is stating a constraint that the generator has no way to hear unless someone writes it down as data. When a field's name says out of 100, or percent, or per 100, that is a ceiling and the generator should be told.

INC-0096. Three more stored phrases in front of a verb that did not agree with them

2026-09-22, Cosmetic, `d7ec4a71aaf8ed28e38d45d840ea1846fa83863d` *PR #85*

- **What was seen.** Two causal reasoning scenarios shipped 'Officials concluded that the cycle racks was responsible for the change' and the same for the traffic islands, and one necessary condition scenario shipped 'The rule says that two independent reviews is needed'. Three corpus entries across two modules, in schemas that between them carry several thousand items.
- **Why.** The same shape as the ACT science scenario names fixed hours earlier: a stored noun phrase dropped into a template that writes a singular verb after it. The templates were written against the entries that existed, every one of them singular, and later entries were added by reading their neighbours rather than the sentence they would end up inside.
- **How it surfaced.** Rendering items from every schema in every exam and searching the text for a plural word in front of a singular verb. The search returns 292 candidate phrases and 289 of them are correct English, because the plural word is usually not the subject: 'the sum of the solutions is 12', 'each of its players is', 'Every one of the drivers was'. Reading the sentences is what separated them, and a check that fires on the pattern would be useless. (Found by measuring something)

- **Fix.** Three renames. The cycle racks became the cycle parking and the traffic islands the traffic calming, both of which match the installation each scenario already describes. The two independent reviews became a second independent review, which is the same necessary condition and reads correctly in all four slots that field lands in.
- **What stops it now.** framework.plural_head names the shape, and each module asserts it of the fields that land in a bare subject slot: the causal scenarios' short name and the necessary condition's requirement. Only those fields, because the same test applied to a field that lands anywhere else is 99 percent false alarms in `src/gen/framework.py`
- **Lesson.** Fixing an instance of a defect is the moment to sweep for the rest of it, and the sweep is worth running even when it is too noisy to become a check. A hundred to one false alarm rate is useless in CI and perfectly workable for one person reading the hits once, and it found three defects that no guard would have. What ships from the sweep is the narrow check on the fields that actually failed, not the broad one that found them.

INC-0097. Every reading inference question asked about something the passage never mentions

2026-09-22, Wrong data shown or stored, `4beeeda2c23720a47ca80348d2122ee2f2e7597b` *PR #86*

- **What was seen.** 'Which of the following can be properly inferred from the passage about the Sweetwater jurisdiction?' The passage is about water rights doctrine in nine western jurisdictions and never names Sweetwater, or the Karoo section, or Agnes Thorne, or the falling-tone item. 800 of 800 inference items sampled asked about a subject that appears nowhere in the text they were asked about. The explanation then told the student 'The passage states that the Sweetwater jurisdiction had no mining district of any kind', which the passage does not say either. The question type is the one that tests valid inference, it runs on both the GMAT and the LSAT, and not one of its items could be answered by reading.
- **Why.** The question is a modus tollens with three parts: a universal the passage establishes, a particular case, and the conclusion that follows. The case is the part the QUESTION supplies, which is why the corpus stores it separately from everything that goes into the prose. The stem template asks what can be inferred about the case and never states it, so the premise the whole item turns on was held in the data, used to compute the key and to write the explanation, and never shown to the person answering. The explanation compounded it by attributing the case to the passage in the same sentence pattern as the universal, which made the item look internally consistent to anyone reading the explanation rather than the passage.
- **How it surfaced.** Reading one rendered inference item end to end, passage included, while looking at the corpus for an unrelated reason. Every existing check passes: the key is genuinely the only valid conclusion, the distractors are genuinely the converse and the inverse, the choices balance, and nothing in a bank of strings can tell that a noun phrase in the stem is absent from the passage beside it. (Found by reading the code or the output)

- **Fix.** The stem supplies the case, which is what a conditional question does: 'If the Sweetwater jurisdiction had no mining district of any kind, which of the following can be properly inferred from the passage?' The explanation says the passage states the universal and the question adds the case, because that is what each of them is. Nothing in the corpus changed; the premise was always there and was simply never printed.
- **What stops it now.** g_rc.check_subjects renders items from every reading schema and fails when a stem names a subject that appears in neither the passage nor the stem's own premise, which is the property that was violated in `src/gen/g_rc.py`
- **Lesson.** A generated item is checked as data, and this one was correct as data: the logic was valid, the key was right, the distractors were the intended errors. What was missing was a fact about the RELATION between two strings, that a name in the question also occurs in the passage, and no property of either string alone can see it. When a question and its source are assembled from separate fields, write down what has to be true of them together, because every check that looks at one field at a time will pass.

INC-0098. The table said 31.0 and the explanation said 31, because the fix covered the table only

2026-09-22, Cosmetic, `c59d046c09369c1a24bfa19301b1268fd255fc56` *PR #87*

- **What was seen.** An ACT science table printed a reading of 31.0 and the explanation beside it wrote the same reading as 31, in six of the twelve schemas in the module. A student checking the explanation against the table finds two spellings of one measurement.
- **Why.** Earlier the same day a formatter was added that prints every reading to one decimal place, because a reading that lands on a whole number rendered as 6 beside a neighbour rendered as 7.2 is visibly a different kind of thing and the choices could not be assembled from them. It was applied to the table, to the one schema whose choices are readings, and to that schema's explanation, which is where the evidence was. Every other schema in the module renders readings in prose, and those were left on the formatter that drops the decimal. The change was scoped to where the failure had been measured rather than to the quantity it was about.
- **How it surfaced.** Searching every schema's explanation for figures that appear nowhere in the table, stem or choices, which is a list of derived values and therefore almost all legitimate. Reading it turned up a column printed as 6.2, 12.4, 18.6, 24.8, 31, where the table says 31.0. (Found by measuring something)
- **Fix.** Every reading rendered in prose uses the reading formatter, in all twelve schemas. Settings keep the other one, because a setting is a value a student dialled in and 2 mM is right where 2.0 mM is not. The two formatters now say in their names and their docstrings which quantity each belongs to, so the next call site is a choice rather than a coin toss.
- **What stops it now.** check_readings renders items from every schema in the module and fails when a reading appears in the text in a form the table does not use in `src/gen/g_act_sci.py`

- **Lesson.** A fix scoped to where the evidence was is a fix scoped to the sample, not to the defect. When the change is to how a QUANTITY is written, the unit of work is the quantity and every place it is written, not the place where the failure happened to be visible. Finding the other call sites costs one search; the fix that covers a subset leaves a difference the reader can see and the measurement cannot.

INC-0099. Every generated reading comprehension question shipped without its passage

2026-09-22, Wrong data shown or stored, `9928611571de97064408629c343ac56d8b34251b` *PR #88*

- **What was seen.** The owner was served a reading comprehension question on the live site with nothing to read: 'If the first season used fixed speakers, which of the following can be properly inferred from the passage?' and five options, on a page with no passage. 280 items are like this, every generated reading item the GMAT and the LSAT carry, across all eight reading schemas. The worst of them ask what the passage says and offer five options all drawn from passages, so without the text there is nothing to choose between.
- **Why.** The generator attaches the passage to the item as the field the app reads, and the emitter that writes the bank file names the fields it copies one by one. It copies passageHtml, which the data tables use, and has never copied passage, which the reading items use. So the item was right in memory, right in every test that builds items in process, and lost the moment it was written to the file that ships. The item also lost its passageId, which is what the app's game pools filter on to keep passage based items out of a view that has no passage.
- **How it surfaced.** The owner hit it in a live round and sent a screenshot. No check saw it: the browser suite renders one item per schema through the real session path and asserts the source renders, but it decides whether an item HAS a source by looking at passageHtml alone, so a reading item was read as an item with no source and the assertion was skipped rather than failed. (A person hit it)
- **Fix.** The emitter copies passage and passageId as it copies every other field, and passages are written once per file as constants and referenced by name, which is the convention the hand written banks already use, so 280 copies of a 300 word passage do not go into the payload. Carrying passageId turned out to change more than the game pools: the mock section picker takes a passage's questions as a group to keep them together, and a generated passage offers ten questions across three skills where a hand written one offers seven or eight across most of them, so three groups filled a 26 question reading section and two skills were never tested. The picker now keeps a slot for every skill still waiting for one, and when only part of a group fits it takes the part that asks something new rather than the part that sorts first.

- **What stops it now.** smoke_items asserts that every item whose type is RC or R carries a passage and puts it on screen, derived from the type rather than from a field that can be absent; build_banks refuses to write a reading item with nothing to read; to_js refuses any generator field it has not been told to write or to skip, which is the whole class rather than this one field; and the mock section check draws twenty sections instead of one, because a single draw passed while a third of them left two skills untested in `src/smoke_items.js`
- **Lesson.** A check that infers what to expect from the same data it is checking cannot fail on a missing field: absence reads as nothing to look for. Derive the expectation from something the data cannot erase, the item's TYPE, and then look for what that type requires. A field that is set in memory, used by every in-process test and copied by hand into the shipped format has no test at all between the two, so the copying should be checked as a whole rather than field by field. And a check that runs one random draw is not a check of a random process: this one was correct for months and simply never asked often enough to see the answer.

INC-0100. A blocked source turned a real check into a permanent warning, and the flagship exam published its structure table uncited for as long as the page existed

2026-09-22, Silent loss, `fce7f003a6953e6985c9ad985c9c9cc8ffa9303e` *PR #89*

- **What was seen.** The GMAT exam page published the section structure table, 21 questions in 45 minutes for Quantitative Reasoning and so on, with no source line under it. Every other exam on the site, SAT, GRE, LSAT and ACT, carried one. The figures themselves were right, confirmed later against the exam maker, so nothing looked wrong on the page: it simply asserted four numbers on our own authority, which is the thing the sourcing rule exists to forbid.
- **Why.** The check was already written and already correct: validate_exams knows the sections array is a set of published figures and knows sections_src is what sources it. It was deliberately downgraded to a warning, with a comment saying why, because www.mba.com serves this environment an Imperva challenge stub instead of the GMAT structure page, so the figures could not be verified from here. That reasoning was sound about the page and wrong about the fact. Unreachable was a property of one URL, not of the figure, and it was never retested against a different publisher. GMAC's own corporate site states all three sections with their question counts and times, is not behind the challenge, and is the exam maker rather than a reseller.
- **How it surfaced.** The warning printed on every build for as long as the page existed and was read as known and accepted, which is what a warning becomes once its reason is written down. Found only while looking for a reachable citation for something else, by trying the exam maker's own domain rather than the consumer site the record already pointed at. (Found by reading the code or the output)

- **Fix.** GMAT gains a sections_src block citing GMAC's exam overview page, read this session, which states 2 hours 15 minutes, 3 sections, 64 questions and no writing assessment, with Quantitative 45 minutes and 21 questions, Verbal 45 minutes and 23 questions, and Data Insights 45 minutes and 20 questions. The score scale cites GMAC's scores page and drops the phrase 10-point increments, which that page does not state and which always ending in 5 across 205 to 805 already says. Total time keeps its original citation, because the GMAC page confirms 2 hours 15 minutes but says nothing about the optional break, and a source may only be cited for what it states.
- **What stops it now.** The warning becomes an error in validate_exams, and build_exams refuses to render a structure table for any exam whose sections_src is missing a text, src, year or url. The check sits on the table, which is the thing being published, not on the citation, which is the thing that goes missing. Verified by removing sections_src and watching the build exit 1. in `src/build_exams.py`
- **Lesson.** A check downgraded because a source is unreachable carries an assumption with no expiry date on it, and the assumption is usually narrower than the downgrade. Blocked was true of one URL; it was applied to the fact. Before accepting that a figure cannot be sourced, try the organisation that publishes it and not just the page you first landed on. And a warning nobody can act on is not a lesser guard, it is a comment: if the condition is genuinely acceptable, record why and clear it, and if it is not, make it fail.

INC-0101. A new field-by-field copier reproduced the passage loss defect four days after the ledger recorded it

2026-09-22, Wrong data shown or stored, `fce7f003a6953e6985c9ad985c9c9cc8ffa9303e` *PR #89*

- **What was seen.** The diagnostic being built for the study guide selected one real item per scored skill and copied it into the page. It named the fields it copied, and it named passage. Multi-source reasoning and graphs-and-tables items do not use passage: they carry their material in passageHtml. So two of the twelve questions would have reached a student with no source material at all, including one reading 'How many of the five pending requests comply with the booking policy exactly as submitted?' with no policy and no requests on screen.
- **Why.** Exactly INC-0099's root cause in new code. A copier that enumerates the fields it carries is correct only for the item shapes its author had in mind, and silently drops everything else. The guard written for INC-0099, EMITTED_FIELDS in gen/framework.py, closes that class of bug for the bank emitter and only for the bank emitter, because it is a check on one function rather than on the idea. Four days later the same author wrote a second copier and made the same mistake, which is the strongest possible evidence that the guard was in the wrong place.
- **How it surfaced.** Caught in the build, before shipping, by reading the generated data rather than the generating code: a listing of the twelve selected items printed a passage column, and the two DI reading types showed false. Nothing automatic would have caught it, because smoke_items covers the trainer's render path and the diagnostic is a different page. (Found by reading the code or the output)

- **Fix.** The diagnostic carries passageHtml as well as passage and renders whichever is present, matching what the app does with the same items.
- **What stops it now.** build_guide refuses to emit a diagnostic item whose type is one that asks about source material unless that item is carrying some, with the expectation derived from the item's type and not from whether a field happens to be populated. Verified by removing passageHtml from the copier and watching the build refuse.
- **Lesson.** When a defect is about a KIND of code rather than a line of code, a guard bolted to the site of the failure does not generalise, and writing one feels like closing the case. Ask instead what the next instance of this code will look like and where it will live: here the answer was anywhere an item is copied out of the bank, and the durable check is that an item asking about source material has to be carrying it, wherever it is being rendered.

INC-0102. A table question asked which row rose the most when every row had fallen

2026-09-22, Wrong data shown or stored, `e32605f9d66884d57cef73a8894a9679d2f94bc8` *PR #93*

- **What was seen.** Item ZM4859 shipped live asking 'Which store had the greatest percent increase in units sold from Q2 to Q3?' over a table in which every store fell: Ashford down 22 percent, Belmont 55, Carlisle 40, Dunmore 40, Eastgate 19. The credited answer is Eastgate, which declined least. A student who reads the question correctly concludes that no store increased and that none of the five choices answers it, and is marked wrong for being right.
- **Why.** gt_leader_pct picks two columns at random, computes each row's percent change between them, takes the maximum, and writes the word increase into the stem unconditionally. Nothing checks the sign of the winner. When every row falls, the maximum is the least negative number and the stem asserts a direction the data does not contain. The correct handling already existed ninety lines above in the same file: gt_change computes rose = b > a and writes increase or decrease accordingly. The pattern was there to copy and was not copied.
- **How it surfaced.** Found while reading the shipped bank for something else. No check saw it, and none could have: every existing guard tests one string at a time, and both strings here are individually fine. The stem is a grammatical question and the explanation is arithmetically correct. What is wrong is the relation between the stem's direction and the data's. (Found by reading the code or the output)
- **Fix.** The stem's direction is derived from the data rather than assumed. When every row fell the item asks for the greatest percent decrease, the winner becomes the most negative row, and the absolute-change trap inverts with it: the row that lost the most in absolute terms from a larger base now has the smaller percent decrease. Mixed and all-rising tables keep the increase wording, which is correct for them. A draw whose winner is exactly zero is discarded rather than described in either direction.

- **What stops it now.** g_gmat_gt.check_directions() builds the schema repeatedly and fails any item whose stem says increase while no row rose, or says decrease while no row fell, reading both from the rendered item rather than from the generator's internals. Wired into build_banks beside check_names, check_readings and check_premises. Verified by running it against the generator as it shipped.
- **Lesson.** When the same file already solves a problem correctly, the second implementation is the one to distrust: the reference was available and was not used, so whatever made it easy to skip will make it easy to skip again. And a generator that writes a word describing the data, rather than computing it from the data, has an assumption in it that nothing will ever check unless the check is written against the relation rather than the field.

INC-0105. Twelve average salaries were published as medians, and the social queue credited schools with figures they never published

2026-09-26, Wrong data shown or stored, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** 12 of the 53 MBA programs with a salary figure publish only an average; several of their stat notes say so in as many words ('average salary, Class of 2023; median not published'). Every one was labelled a median: 'Median starting salary is $X' in the page lead, 'median starting salary $X' in the meta description, a 'Median base salary' table row sitting beside a note saying no median exists, 'Median salary' in the rankings detail view and the printed report. The social post queue in Admin went further in two ways: every salary post said graduates 'report a median starting salary' that was 'reported by the school', although 8 salary figures come from a secondary source, and every acceptance rate post said the figure was 'from the school's own class profile', although 9 of the 16 come from Poets&Quants (six), US News, GMAC or a university news office. The international share posts said 'Class composition is published by the school' and 'Sourced from the school' for all of them, although 7 of the 60 international shares come from a secondary source. And two acceptance rates whose own sources call them 'estimated' and 'around 22%' were posted as flat facts.
- **Why.** The field is named salary_median_usd, and every consumer took the name as the definition. The source ladder allows an average when a school publishes no median and the data entry recorded which one it was in the stat note, but nothing that renders the figure reads the note. The social templates were written as fixed sentences about the typical case, a school reporting its own median, and never looked at the figure's own source.
- **How it surfaced.** Found on 2026-09-26 while fixing INC-0104: routing every sentence through stat_kind meant asking what each figure's note says, and the salary notes answered 'average' twelve times. The social claims were found by grepping for every consumer of the same field. (Found by reading the code or the output)

- **Fix.** Every label derives the statistic from the figure's own note through stat_kind: a median is called a median, an average is called an average, and a note that says neither gets no adjective. The social queue names the figure's actual source instead of asserting the school's, and uses the same derived statistic. The public methodology says that the outcomes component uses the median where a school publishes one and the average where it publishes only that.
- **What stops it now.** build_rankings refuses to render a school page whose prose or table calls a figure a median when its stat note resolves to an average, or the reverse, and build_social refuses to emit a post whose text claims the school as the source of a figure hosted elsewhere. Each failure names the school and the field.
- **Lesson.** Any consumer that describes a value in words must read the field that records what kind of value it is, never the field's name. A field name is a hope about the data, not a description of it, and a schema that allows two kinds of value will be described everywhere as the kind it was named after.

INC-0106. Official figures from a school's own university were footnoted as secondary and 'indicative rather than official'

2026-09-26, Wrong data shown or stored, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** 29 figures across 10 MBA school pages carried the secondary-source asterisk and the footnote 'This school does not publish this figure on its own site, so it comes from a secondary source ... treat it as indicative rather than official', although every one comes from the school's own university: Cornell Johnson's class profile and employment figures (9 of them) from business.cornell.edu, Miami Herbert's class profile (8) from the university news office, tuition from the bursar, registrar or student accounts pages of Rice, Pitt, UC Irvine, UC San Diego, Kentucky, Georgia and Howard, Howard's salary and employment figures from the university's own news site, and Wharton's tuition, salary and employment figures from Wharton subdomains including its own career report. The same test drove the social queue, which would have said 'as reported by Wharton 2025 MBA Career Report' as if that were a third party.
- **Why.** is_secondary decides 'the school published this' by comparing the figure's host with the host of the program's website field, allowing only an exact match or one being a subdomain of the other. Universities publish from sibling hosts (bursar.rice.edu beside business.rice.edu, business.cornell.edu beside johnson.cornell.edu), which are neither, so the test read 'a different host' as 'a different publisher'.
- **How it surfaced.** Found on 2026-09-26 while fixing INC-0105: the corrected social post for Wharton credited 'Wharton 2025 MBA Career Report' as an outside source, and listing every figure the test called secondary showed 29 on the schools' own registrable domains. (Found by reading the code or the output)

- **Fix.** The test compares registrable domains (the university's domain, e.g. rice.edu, with a short list of two-label public suffixes handled), so a school's own offices count as the school and Poets&Quants, US News, GMAC, Bloomberg and BusinessBecause still count as secondary. Three official documents live on hosts no domain rule can recognise, USC Marshall's own storage bucket, the University of Washington's alias domain and Rice's career office uploads on a vendor CDN, so a school file may now list official_hosts, each with the evidence that it is the school's and the date that was checked; all three were verified on 2026-09-26 (Marshall's pages load their own images from the bucket, the alias redirects to foster.uw.edu, and Rice's career highlights page links its employment report from that upload directory). Checking Rice also found its program URL in the data file returning 404, used as the Official Program Site button and four citations; it moved to /rice-mba/full-time-mba, read from Rice's own homepage.
- **What stops it now.** build_rankings runs a self-check on every build before rendering: a school's sibling office (bursar.rice.edu beside business.rice.edu) must count as the school and a publisher (poetsandquants.com, usnews.com, gmac.com) must not, and the build prints how many figures it marks secondary and on which hosts, so a host that should not be there is visible in the log.
- **Lesson.** A provenance label is a factual claim and deserves the same checking as the number it annotates. A proxy that is right for the common case (one host per school) will be confidently wrong for every institution that publishes from more than one office, and the error flatters no one: it tells readers that official data is unofficial.

INC-0108. The college methodology page quoted its own checks from the day they were written, and the build had moved on

2026-09-26, Wrong data shown or stored, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** /colleges/methodology/ told readers that 80 percent of our top 25 appears in a published top 25, that 96 percent appears somewhere in a published ranking, and that the score correlates with admission rate at -0.41. The build that generated the page printed 76 percent, 100 percent and -0.38 on the same run. The page also states how many public colleges charge a non-resident premium, the largest premium, and how many private colleges report identical in-state and out-of-state tuition, all as literals.
- **Why.** The figures were typed into the page template as prose when the reweighting shipped, beside placeholders for the counts that were templated. validate_ranking.report computes all of them on every build and returns them to build_colleges, which used only the per-category agreement. Nothing compared the prose to the numbers the same build had just printed.
- **How it surfaced.** Found on 2026-09-26 by reading the build log beside the methodology page's source while working on college snippets: the log line 'score against admission rate: -0.38' sat directly under a template sentence saying -0.41. (Found by reading the code or the output)

- **Fix.** Every figure on the page that the build can measure is now filled from the measurement: top 25 agreement and recognition and the admission rate correlation from validate_ranking.report, and the public, premium and private tuition counts from the college data. Historical figures that describe the old weighting stay as prose and say they are historical.
- **What stops it now.** build_colleges fails if the methodology template loses any of the placeholders the measured figures fill ({AGR25}, {REC25}, {CORR}, {PUB100}, {PREMIUM_LINE}), which is how a typed literal would return, and fails if the ranking checks did not run, rather than rendering the page without them.
- **Lesson.** When a page publishes a number the build computes, the page should read it from the build. A figure typed in beside a placeholder is the one that will be wrong next month, and it will be wrong on the page whose whole purpose is to say how carefully the numbers are checked.

INC-0114. Reading inference questions credited a key that depended on a rule the passage never stated

2026-09-26, Wrong data shown or stored, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** An rc_infer item asked 'If the Kivalliq site recorded a snowpack of twenty-two centimetres, which of the following can be properly inferred from the passage?' and credited 'The Kivalliq site did not show the winter reversal.' The passage says the reversal appeared wherever the snowpack exceeded forty centimetres, which is the opposite direction: deep snow is enough for the reversal, not required for it, so a site with shallow snow may or may not show it. The rule that makes the key follow, that every site in the survey that showed the reversal had deep snow, is held in the corpus and never appears in the text. None of the forty universals the inference schema runs on appears in its passage. Read one by one against the prose, twenty of the forty credited a conclusion that the passage and the stem's premise do not establish together: the first conditional of tundra, guilds, reefs, roads, saltmarsh, cicadas, almshouse, riparian and phonotactic, the second of tenements, basalt and lichen, and both of scriptoria, pidgin, birdsong and assize. The failures are of four kinds: the passage states the converse of the rule (tundra, guilds, riparian); it states nothing like the rule (scriptoria, basalt, almshouse, phonotactic and others); the case is never placed inside the study the rule covers, so a rule about the eleven Canadian sites is applied to a site that may not be one of them (roads, saltmarsh, and the scoped ones above); or the step needs outside knowledge, that English is the lexifier of Krio or the year the lichen hypothesis was published. The explanation on all forty opened 'The passage establishes that' and then quoted the universal, which the passage does not contain. Birdsong's first key did not follow even with its rule stated. The schema runs on both the GMAT (v_inf) and the LSAT (lsat_rc_inf).

- **Why.** The module docstring says 'Each passage carries two conditionals', and the passage data does carry them, as fields beside the prose. text(p), which builds what the reader sees, assembles the old view, the two findings, the revision and the caveat, and never the conditionals. The generator then computes the key from the stored universal and the explanation quotes it, so every string the item is built from agrees with every other and the item looks valid to anything that reads the data. What the reader gets is the prose, and for half of the conditionals the prose was written to say something close to the rule rather than the rule. INC-0097 fixed the same shape one field over: the case was used to compute the key and never printed, and that fix printed the case. The universal had exactly the same property and was not looked at, because the fix answered the question 'is the case shown' rather than 'which premises is the reader shown'.
- **How it surfaced.** Found on 2026-09-26 while planning more passages to grow the thin reading categories. Checking what a new conditional would have to satisfy meant reading the existing forty against their rendered passages, and the first one checked, tundra, credited a conclusion the prose does not support. check_premises passed throughout, because it checks that the stem states a case and that the case is new to the passage, and says nothing about the rule. (Found by reading the code or the output)
- **Fix.** text(p) now states both universals in the second paragraph, before the revision, so the rule the key turns on is in the text the reader has and the explanation's 'The passage establishes that' is true. Every conditional whose conclusion is about what a case did rather than which group it belongs to now places the case inside the study its rule covers ('the Kivalliq site, one of the eleven in the Canadian survey, ...'). Reefs said shaded colonies produced 'almost none' of the proteins, which contradicts a rule that only colonies in strong light produced any, and now says none. Birdsong's first conditional, cicadas' first and the cases that needed outside knowledge were rewritten so the step is in the text. Conclusions that deny membership were reworded to say so in one form ('is not among', 'is not in').
- **What stops it now.** g_rc.check_premises now also fails when a universal does not appear verbatim in the rendered passage, and each conditional declares a scope: the phrase that limits whom its rule covers. When the conclusion is an outcome, the scope must appear in both the universal and the case, so the case is inside the rule; when the conclusion denies membership instead, it must say so in a recognised form, which needs no scope because it holds whether or not the case was ever in the study. The build fails on any of these, before a bank is written. in `src/gen/g_rc.py`
- **Lesson.** When a key is derived from premises held as data, each premise has to be found in what the reader sees, and a check should prove that by searching the rendered text rather than the data. Fixing the one missing premise that was noticed is not the same as asking which premises the reader is shown: list them all, and look for each one in the output.

INC-0115. A field written to open a sentence was printed mid-sentence in a reading stem

2026-09-26, Cosmetic, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** Every rc_stated item that asks about a passage's second finding opened 'According to the passage, A later survey of eleven sites across northern Canada found that', with a capital A after the comma. The built banks carried 20 of them for the GMAT (13 in bank.js, 7 in the deferred chunks) and 8 for the LSAT, one per passage. The same field joined to its neighbour with a fixed 'of' also produced 'A second team of carbon isotope records from three continents found that', a team made of records, and 'Later laboratory work of colonies' and 'A playback experiment of the same population' in the passages themselves.
- **Why.** ev2who is used in two slots. text(p) puts it at the start of a sentence, so it was written capitalised, and the stated-idea stem puts it after 'According to the passage,' where it needs lower case. The field was written for the first slot and never read in the second. The 'of' join is the same fault in a different form: the template supplies a preposition that suits most of the stored phrases and was never checked against all of them.
- **How it surfaced.** Found on 2026-09-26 while rewriting the passages for INC-0114, by searching the built banks for the stems that quote ev2who. (Found by reading the code or the output)
- **Fix.** ev2who is stored in the form the middle of a sentence wants ('a later survey') and the passage capitalises it where it opens a sentence. Capitalising is the safe direction: it cannot damage a proper noun, while lowercasing one is the fault this file was already fixed for twice. The three phrases that read wrongly before 'of' were rewritten ('a second team's study', 'a later laboratory study', 'a playback study').
- **What stops it now.** g_rc.check_corpus fails when ev2who opens with a capitalised article or other common word, which is the stored form that breaks the mid-sentence slot. in `src/gen/g_rc.py`
- **Lesson.** A stored phrase that goes into more than one slot has to be written for the hardest of them, and the transform has to run in the direction that cannot damage anything: capitalising a sentence opener is always safe, lowercasing one breaks proper nouns. When a field is reused, read the output of every slot it lands in, not the one it was written for.

INC-0116. A passage described zircon dating with the vocabulary of argon dating

2026-09-26, Wrong data shown or stored, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** The basalt passage said the extinction dates 'come from zircon crystals, which close to argon loss and so record the eruption rather than later heating'. Zircon is dated by uranium and lead, and the property that makes it useful is that it keeps its lead when heated later; closure to argon loss is the language of argon dating, which uses other minerals. The sentence was also the key of a stated-idea item, so a student was asked to select it as what the passage says.
- **Why.** The passage was written from memory and the two dating methods were conflated in one clause. Nothing checks the science in a passage, because a reading question is answered from the text and every check on the item passes whether the text is true or not.
- **How it surfaced.** Found on 2026-09-26 while reading every passage in full for INC-0114. (Found by reading the code or the output)

- **Fix.** The clause now reads 'which retain their lead when later heated and so record the eruption rather than any later event'.
- **What stops it now.** None that a machine can run. Passage science is checked by reading, and this record is the reminder that a passage is published content held to the same standard as a sourced figure even though no question depends on it being true.
- **Lesson.** Practice material is still published writing. A passage that is only there to be read carefully is still read by people who know the subject, and a question built on it can ask them to endorse the error.

INC-0117. Three reading schemas could be answered by matching names, because every distractor came from a different passage

2026-09-26, Wrong data shown or stored, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** An inference item stated 'If Providence adopted the building codes and filtration in the same year' and offered five conclusions, of which only the key mentioned Providence; the others were about the Arezzo panel, infant 14, the Wendle map and the Blakeney marsh. Choosing the option that shares the most names with the premise picked the key on 549 of 600 distinct seeded draws of rc_infer (91 percent) and 572 of 600 of rc_infer_long (95 percent). Main idea and caveat items had the same property with topic words instead of names: choosing the option whose words appear most in the passage picked the key on 92 percent of rc_main and rc_caveat draws and on 79 and 84 percent of their LSAT variants. Every one of these items also closed with 'Watch for: Every other choice states something the passage also says. They are true', which is the stated-idea note and is false of all three schemas, whose wrong choices the passage does not say.
- **Why.** INC-0088 found the inference key was the shortest option on 68 percent of items and fixed it by adding other passages' conclusions to the distractor pool, because they are the right length. They are also about a different subject, so the fix traded a length tell for a subject tell. Main idea and caveat were built the same way from the start: their only wrong answers were other passages' summaries, which are safely wrong because they describe a different subject, and safely wrong is the same as eliminable without reading. test.js measures length rank and nothing else, so the ratchet reported these schemas as clean. The 'Watch for' note was written once in the shared emit() for the schema it describes and inherited by the three it does not.
- **How it surfaced.** Found on 2026-09-26 while checking the INC-0114 fix by rendering an inference item and reading its choices, then measured across seeded draws of all six schemas. (Found by reading the code or the output)

- **Fix.** Wrong answers now come from the same passage, and at least one is always offered. Each conditional carries two authored near misses about its own subject, each claiming something neither premise establishes and at least one worded in the negative like every key; a negative one is always offered and the other joins the pool. Main idea always offers one of the two findings, quoted or summarised, as if it were the whole passage, with the closing limit, an overreach about every account of the topic and the reverse of the argument in the pool. Caveat always offers one of four misreadings that quote the passage: the first finding declared unreliable, the evidence from either study declared unreliable, or the revision declared shown everywhere. Other passages' summaries stay in the pool but can no longer fill every slot. Forcing an option pins the key's length rank, since a quoted finding is nearly always longer than the key, so balance() now builds a draw that includes a same-subject option for a target rank drawn from the ranks that can be built; with no same-subject options it samples exactly as before, so every other schema's bank is unchanged. The 'Watch for' note is written per schema.
- **What stops it now.** g_rc.check_tells draws every schema except stated idea across the corpus and fails the build when either shortcut finds the key, and only the key, on more than 40 percent of distinct draws: the choice repeating the most names from the premise, or the only negative choice naming the case, for inference; the choice whose words appear most in the passage, for main idea and caveat. Forty percent is twice a blind guess among five and far below the 79 to 95 percent recorded here. After the fix the measured rates were 0 to 8 percent. g_rc.check_premises also requires each near miss to name the case and at least one to be worded in the negative, because every key is. in `src/gen/g_rc.py`
- **Lesson.** A test measures what someone can get right without the skill, and there is more than one way to do that. Removing one tell does not make an item sound; it moves the question to the next shortcut. Distractors drawn from unrelated material are always wrong and therefore always free, so a wrong answer has to be wrong about the same thing the right one is about.

INC-0118. School pages said a program does not publish an acceptance rate when we had only not found one

2026-09-26, Wrong data shown or stored, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** Every school page without an acceptance rate, 75 of the 91 active programs, printed 'not published' in the class profile table and answered 'What is the acceptance rate at X?' with 'X does not publish an acceptance rate, and neither do most full-time MBA programs', in the page and in its FAQPage structured data. Of the 16 programs that do carry a rate, 8 come from the school's own site and 8 from a publisher, which is the point: a program without one on its own site can still report one that a publisher prints, and this session could not reach US News or Poets and Quants at all. Acceptance rate is the largest search intent on the school pages, 2,204 impressions in the owner's export. The same sentence ran through the /schools/ index (its Dataset description, the international column note, an FAQ answer and the methodology), the trainer's fit view ('not published' beside a school's missing figure, and twice in its help text) and a queued social post, and the index hard-coded 'Twenty' and 'sixty' as program counts in one of those sentences.
- **Why.** The copy states the conclusion of a search as a fact about the school. 'We did not find a rate from a source we trust and could read' became 'the school does not publish one', and the generalisation about most programs was never sourced. The branch that writes it runs for every program whose field is empty, so it asserts a negative about each of them without anything recording that anyone looked.
- **How it surfaced.** Found on 2026-09-26 while writing up why acceptance rates could not be filled: counting where the 16 existing rates come from showed half are publisher figures, so the absence of a rate on a school's own site does not mean the rate is unpublished. (Found by reading the code or the output)
- **Fix.** Every one of those places now says what is true: the figure has not been verified from the school or a tracked publisher, so none is shown, and none is estimated. School pages: the table reads 'not verified', the answer drops the unsourced claim about most programs, the footnote under secondary figures reads 'We have not found these figures on this school's own site', the empty-profile row and the template's dash note likewise. The /schools/ index, the trainer's fit view and the social post were reworded the same way, and the index's two program counts are filled from the data at build time. GROWTH.md and ROADMAP.md said 16 programs publish a rate; they now say the library holds one for 16, 8 from the schools and 8 from publishers. smoke_fit asserts the new label.
- **What stops it now.** build_rankings fails the build when any copy it writes for a school page (the lead, intro, description, FAQ answers, footnote and the rows it writes without data) says a figure is not published or that a school does not publish, release or report one, and it checks both page templates for the same once per build, with comments stripped first. Stat notes quoted from a named source ('median not published' in a specific employment report) are provenance about the document read and are not checked. The trainer's fit view is not guarded; smoke_fit asserts its label. in `src/build_rankings.py`
- **Lesson.** An empty field records that a search came up empty, not that the thing does not exist. Write the absence as what was done ('not verified from the sources we could read'), never as a fact about the world, because the world is the part nobody checked.

INC-0119. Length corrections appended clauses that repeated what the answer choices already said

2026-09-26, Wrong data shown or stored, `9b254b7fb5db916c3b78cd5a875dd5a665a75c64` *PR #97*

- **What was seen.** GRE item GR001 offered 'establish that literacy rates were higher before printing than is usually supposed than is usually supposed'. ACT item AR047 offered 'poorly made pianos require more frequent tuning than well made ones than well made ones do.' GMAT item V613 offered 'explaining why the Hanseatic League never established a court with power to compel its members of its own'. Across nine hand written bank files (ACT reading, GRE reading, GMAT reading, SAT reading and writing, LSAT reasoning and LSAT reading), 165 answer choices were garbled where a clause had been appended to them: 48 carried a run of words twice back to back, and 117 restated their own ending in other words or carried on from a point earlier in the choice than the place the clause landed. Every one was a distractor, so every one was a wrong answer a careful reader could reject for being garbled rather than for being wrong.
- **Why.** The length tell on hand written items is corrected by appending an authored clause to named distractors: bank_emit.extend() for banks emitted by a script, bank_repair.py for banks edited in place. A table entry names its choice by a needle, a substring of it, and the clause is appended at the end of the choice, not after the needle. Most of these clauses were written as though they would land straight after the needle: the needle stopped short of the end of the choice, and the clause said again, in its own words, the text that still followed the needle ('designed before depletion was understood' + ' had been observed anywhere'). Others were written against the full choice and opened with the words it already ended on. Neither tool compared a clause with the text it was appended to: extend() checked that the needle matched exactly one choice that is not the key, and check_lift() checked where the key landed afterwards, so an entry that repeats itself passed both. bank_repair.py compared only the clause's first three words, and its first word, with the end of the choice, which misses a restatement in other words. INC-0072's doubled word check looks for one word twice, and a phrase twice is a different string.
- **How it surfaced.** Found on 2026-09-26 while reading the GRE reading bank to wire generated passages into it: GR001's third choice ends in the same four words twice. A scan of every hand written choice for a run of two or more words repeated back to back found the word for word cases, and a diagnostic pass through extend() named the entries whose clause opened with, or overlapped, the choice's ending. After those were repaired, a second pass listed every append whose needle still stopped short of the end of its choice, 140 of 902, and reading each of the 140 found 27 more garbled choices that no word overlap test could see. (Found by reading the code or the output)
- **Fix.** Each defective choice now says it once. Where a clause restated text that followed its needle, that text came off the source choice so the clause carries on from the end; where a clause opened with the words the choice ended on, they came off one side. Where the shorter choice no longer carried the key to the rank its table sets, the clause was lengthened with content rather than padding, and check_lift confirms every placement. Every needle in the nine scripts now reaches the end of its choice, 902 appends in all. bank_lsat_rc2.js, which bank_repair.py had lengthened in place and whose script no longer reproduces it, is corrected in place. No key's text changed.

- **What stops it now.** bank_emit.extend() and bank_repair.py refuse a needle that stops short of the end of its choice, since that is where the clause goes, and refuse a clause that opens with the last words of the choice or shares a three word run with its last six words. build_banks.py and test.js fail on any stem, choice, statement or explanation that says a run of two or more words twice back to back with only a space between, whatever tool produced it; that check is how INC-0120 was found. in `src/bank_emit.py`
- **Lesson.** An edit that appends text has to read what it is appending to. A correction step that checks only its own goal (here, that the choice got longer) will happily achieve it by making the choice worse, and every check downstream measures the goal, so nothing notices. Check the seam: the words either side of where new text meets old text.

INC-0120. A generated word problem printed its rate unit twice

2026-09-26, Cosmetic, `9b254b7fb5db916c3b78cd5a875dd5a665a75c64` *PR #97*

- **What was seen.** SAT item ZS3711 read 'A pool contains 60 gallons of water and is filled at a constant rate of 3 gallons per minute gallons per minute.' Every pool version of the linear word problem schema (sat_alg_word) carried the unit twice: 136 SAT items and 151 ACT items in the generated banks, 4 of each in the starter slice that loads first and the rest in the deferred chunks, plus one ACT question of the day queued in the admin social queue.
- **Why.** The schema's pool template already ends '{r} gallons per minute.', and build() appends ' gallons per minute' to the rate whenever the setup mentions gallons, so both write the unit. Both halves were in the file from the commit that created the schema (4b25169). Nothing reads a generated stem as a sentence: the build checks that the arithmetic works, that the choices are distinct and that the key's length rank is not a tell, and a repeated phrase breaks none of those.
- **How it surfaced.** Found on 2026-09-26 while calibrating the repeated phrase scan written for INC-0119: it was meant for hand written answer choices and was run over every stem of every exam's bank as well, to see what else it would catch. (Found by reading the code or the output)
- **Fix.** The template writes the unit and build() no longer appends it, so the stem says it once. The rate goes through the same number formatter as before.
- **What stops it now.** build_banks fails the build when any generated item's stem or choice contains a run of two or more words said twice back to back, naming the items; a run made only of single letters, such as a coin toss sequence, is allowed. test.js runs the same check over every hand written stem and choice. in `src/build_banks.py`
- **Lesson.** When two pieces of code each supply part of a sentence, decide which one owns each word. A template and the code that fills it both writing the unit is invisible in either one read alone; only reading the output as a sentence shows it.

INC-0121. The banks lengthened in place carried the same garbled seams, and nothing recorded what had been appended

2026-09-26, Wrong data shown or stored, `c1fafa255d26d98f22f26fc8c71c9491389d777a` *PR #98*

- **What was seen.** Thirteen older hand written banks (ACT reading and science, LSAT reading, SAT reading and writing, GMAT verbal) had 293 distractors lengthened in place by bank_repair.py in one commit. Reading every one of those seams found 12 more garbled distractors in 6 of the files: 'at every hour measured hour that was measured', 'adopted numerical inflation targets during the 1990s during that decade', 'the manufacturer's pumps machines in more or less every respect', 'recommends correcting them held in European collections'. Two appended clauses had lost an apostrophe ('a single year of that company operation', 'the whole of a bottle life'), and one key, written that way from the start, read 'keeping the movement structural role'.
- **Why.** The same defect as INC-0119, in the files INC-0119's sweep could not reach. The banks emitted by a script keep their clause tables, so each append could be checked against the text it followed. bank_repair.py edits a bank in place, and the tables it was run with were one-off and never committed, so once the edit landed nothing recorded which text was original and which was appended. A text scan finds a phrase said twice back to back, but a clause written to follow a needle that stopped short does not repeat anything word for word, and without the table there was no seam to check. The apostrophes were dropped because a clause inserted into a single quoted literal cannot carry a bare one.
- **How it surfaced.** Found on 2026-09-26 while closing INC-0119: the commit that applied bank_repair.py was diffed item by item against its parent, which recovers every appended clause and where it met the choice, and each of the 293 seams was read. The seam rules flagged 4 (one of them ordinary parallel wording); reading found the rest. (Found by reading the code or the output)
- **Fix.** Each of the 12 now says it once, and the three missing apostrophes are restored. Where a repair would have moved a distractor to the other side of its key's length, the sentence was completed with content rather than cut, so every item keeps the length rank test.js records for its file.
- **What stops it now.** bank_repair.py now refuses a needle that stops short of the end of its choice and a clause that restates the words it follows (INC-0119), so an in-place edit can no longer produce this. What was appended before that guard existed is recoverable only from git, which is how this audit was done; the back to back scan in build_banks.py and test.js still covers the word for word case in every bank. in `src/bank_repair.py`
- **Lesson.** An edit tool that throws away its input leaves the diff as the only record of what it changed. Audit such edits from the commit that made them, and keep the table that drove a bulk edit in the repository, because the table is what lets the next check see the seam.

INC-0122. Reading questions asked once of each passage passed the length check on the luck of the draw, and two new passages re-rolled it

2026-09-26, Degraded, `a442f830c3e19edaca1e92908fea414ba0227086` *PR #103*

- **What was seen.** Adding two long reading passages (ferries, bridgepiers) failed the build: lsat/rc_caveat_long put 7 of its 14 keys at the middle length rank, 50 percent against a cap of 47 at that size. In the same build lsat/rc_main_long never had its key as the shortest choice (0, 1, 4, 4 and 5 keys at the five ranks, shortest to longest). Neither schema had changed; the two passages moved every draw that came after them. The GRE variants, given a rotation in PR 101 for this very reason, were not even either: 4, 10, 5, 7 and 6 of 32 caveat keys.
- **Why.** MainIdea and CaveatImplication ask one fixed question of each passage, so each ships one item per passage: 14 on the LSAT corpus, 32 on the GMAT and GRE one. The key's length rank was drawn at random for each item. Even with every rank open, 14 uniform draws put 7 on one rank about 6 percent of the time, and every rank was not open: a key longer or shorter than the options its passage offers cannot be built at some ranks (the juries caveat key only as the shortest choice, riparian's never as either of the two shortest, the new bridgepiers main idea key only as the longest, because it was longer than every option in its pool), so the draws piled into the ranks that were left. The rotation added for GRE in PR 101 was scoped to GRE because the GMAT and LSAT variants were passing at the time, and passing was the draw rather than the design. Rotation on its own also misses: where a passage cannot build its turn, _balance_own falls back to a random rank, and even a buildable turn is missed when twelve random picks of the forced option never find the one that works.
- **How it surfaced.** The build's schema answer bias check (INC-0079, INC-0088), on the first build with the two passages. The per rank counts come from rerunning the bank build with the bias function wrapped to print them, and the ranks each passage can build from counting its options directly. (A build guard caught it)
- **Fix.** Every schema that asks one question of each passage now assigns each passage its key's length rank before anything is drawn. The ranks a passage can build are computed from its own options with the same test _balance_own applies, and passages are placed most constrained first, each on the least used rank it can build, so the keys that ship are spread as evenly as the corpus allows rather than as evenly as the draw fell. When a rank is assigned, _balance_own tries every forced option before giving up on it instead of twelve random picks. This replaces the GRE rotation and covers the GMAT and LSAT variants too. The two new passages' main idea keys and the ferries caveat key were shortened to sit among the other passages' keys. The six schemas now plan 7, 7, 6, 6 and 6 keys per rank over 32 passages and 3, 3, 3, 3 and 2 over 14, every item lands on its planned rank, and the bias check, which orders two choices of equal length by position, measures them within a key or two of that.
- **What stops it now.** g_rc.check_spread() runs at the start of the bank build and fails when a schema asked once per passage cannot be spread to within one passage of even, naming the passages whose keys are too long or too short to be placed, so a passage that would pile the ranks up is caught when it is written, with the reason, rather than by the aggregate check some builds later. check_bias stays as the measurement of what actually shipped. in `src/gen/g_rc.py`

- **Lesson.** A check that passes on a random draw has told you about that draw. When a schema ships a handful of items, a property the whole bank needs, such as keys spread over every length rank, has to be assigned rather than sampled. And a fix proven on one variant of a shared class belongs on the class: the variants left alone were passing on luck, and the next unrelated change spends it.

INC-0124. A blog post put the ACT science add-on at $4 in its table and $5 in its own FAQ, and nothing compared a post's prices with the sourced ones

2026-09-26, Wrong data shown or stored, `5ad630fcac638d8ef029727e2a365a303080dfaf` *PR #106*

- **What was seen.** The comparison table in /blog/sat-vs-act-2026/, published September 26, 2026, gave the enhanced ACT's optional science section as '40 questions, 40 min, $4'. ACT's fee page says 'Science Add-On $5' (read September 26, 2026), data/exams.json records $5.00 against that page, and the same post's fee row and FAQ both say $5, so the page contradicted itself a few lines apart.
- **Why.** The fee was typed into the table rather than taken from the sourced record. build_blog.py checks a post's structure (title and description length, FAQ count, sibling links, dashes, banned sources) and never compares a figure in a post with the figure the site already holds for it, so two statements of one fee, one right and one wrong, passed every check.
- **How it surfaced.** Found on September 26, 2026 while reading the existing posts to plan the next blog batch, and confirmed against ACT's fee page the same day. A scan of every dollar amount in the 39 posts against the sourced data found no other. (Found by reading the code or the output)
- **Fix.** The table says $5, matching ACT's fee page, data/exams.json and the post's own fee row and FAQ.
- **What stops it now.** build_blog.py fails the build when a post states a dollar amount that appears nowhere in the sourced data: data/exams.json (every fee note and cost figure), the verified fact sheet in src/blog/EDITORIAL.md, and the school files in data/schools. A new price therefore has to be sourced in one of those before a post can print it. On the current posts it flags this figure and nothing else. in `src/build_blog.py`
- **Lesson.** A figure written twice on one page will eventually be written two ways. Where the site already holds a sourced value, a page that states it should be checked against that value, because a reader who meets $4 and $5 for the same fee trusts neither.

INC-0125. Every College Scorecard figure in the MBA library cited the program's own website as its source URL

2026-09-26, Wrong data shown or stored, `bb8ef13c00e42bfea89d5abee8ba0c9d7b560218` *PR #108*

- **What was seen.** The 337 College Scorecard figures in 87 MBA school files (median earnings one and four years after completing, median federal debt, and the national median for the field) all carried the program's own website as their url, although their src names the US Department of Education's College Scorecard field of study file. None of those pages holds the figure. The school pages cite the Scorecard in text and do not print these URLs, so no reader was sent to the wrong page, but the public data files recorded, for every one of them, a source that does not contain the number.
- **Why.** enrich_schools_federal.py set each figure's url to s.get('website'), the one URL the script had to hand, so that every figure carried the three fields the source policy asks for. validate_schools.py checks src, year and url on the profile figures and never looked at the federal block, so nothing compared a url with the source it is meant to point to. A url field that is present passes a presence check whatever it points at.
- **How it surfaced.** Found on September 26, 2026 while reading a school file to plan adding programs to the library: the federal figures' url was the program page. A count across the library found all 337 the same. (Found by reading the code or the output)
- **Fix.** Each federal figure now cites https://collegescorecard.ed.gov/data/, the Department's data page, which offers the Most Recent Cohorts field of study file the figures were drawn from (read September 26, 2026). The enrichment script writes that URL, so a rerun cannot put the website back.
- **What stops it now.** validate_schools.py now checks the federal block: every federal figure with a value needs src, year and url, and any figure in a school file whose source names the College Scorecard must have its url on collegescorecard.ed.gov. in `src/validate_schools.py`
- **Lesson.** A presence check on a source field proves the field is filled, not that it is true. Where a figure names its source, check that the URL belongs to that source, because the easiest value to fill a required field with is the wrong one that happens to be nearby.

INC-0130. The GRE exam guide credited ETS with a combined 260 to 340 score that the ETS page it cited does not describe

2026-09-26, Wrong data shown or stored, `4727c2e59c09a7d1bbe45f95e917b8cfab2ee836` *PR #117*

- **What was seen.** data/exams.json gave the GRE score scale as 'Verbal Reasoning and Quantitative Reasoning each scored 130 to 170, for a combined 260 to 340; Analytical Writing scored separately', cited to ETS's GRE scoring page, and the exam pages published it as that page's fact: in full in the /exams/ hub's structured data, and in part in the GRE guide's Score Scale tile. The page says three scores are reported, Verbal Reasoning and Quantitative Reasoning from 130 to 170 in 1-point increments and Analytical Writing from 0 to 6 in half-point increments, and mentions no combined score, 260 or 340. The site's own GRE posts already said the total is an informal sum that ETS does not report.

- **Why.** The sum is arithmetic that applicants and schools commonly use, and it was written into the fact as though the source said it, in the same sentence as the two numbers the source does give. validate_exams checks that every fact names a source, a year and a URL, not that the source says what the fact says, so a figure that looks right and sits beside a real citation passes every check there is.
- **How it surfaced.** Found on September 26, 2026 while planning a GRE score calculator: reading ETS's scoring page for the rule that the total is the sum found no total at all. (Found by reading the code or the output)
- **Fix.** The fact now says what the page says: three scores are reported, Verbal Reasoning and Quantitative Reasoning from 130 to 170 in 1-point increments and Analytical Writing from 0 to 6 in half-point increments. The check written as the guard then found the same gap elsewhere, and each case was corrected against its page. The SAT fee note named a 2025-26 testing year the page does not mention. The LSAT fee note gave a $248 prior fee and a same price for remote and test-center testing, neither on the page, which says the price is the same for first-time and repeat takers. The GRE law school fact said more than 100 ABA-approved schools, where ETS lists 128 in the United States and says nothing of ABA approval. Two ACT claims were real but cited to a page that does not print them, and two clauses, switching test modes up to the late deadline and four-year universities valuing the test, are on no ACT page and were removed, from a held post as well. The SAT score scale moved to College Board's page on what scores mean, which prints the ranges without JavaScript. The GRE fee was stale and is INC-0132.
- **What stops it now.** src/check_sources.py reads the page or PDF each exam fact cites and reports every number in the fact or its note that the source does not print. A number that is arithmetic on the source is declared with its working in a derived entry, and the check looks for the inputs instead. It runs weekly in the playbook workflow, rendering pages that build their text with JavaScript, and opens an issue when a fact no longer matches. in `src/check_sources.py`
- **Lesson.** A citation vouches only for what its source says, so check each figure against the source rather than checking that a citation is present. The claim most likely to slip through is the true looking one written next to real ones, arithmetic or common knowledge put in the source's mouth.

INC-0132. The GRE guide still quoted a $220 fee eight weeks after ETS's fees became $249

2026-09-26, Wrong data shown or stored, `4727c2e59c09a7d1bbe45f95e917b8cfab2ee836` *PR #117*

- **What was seen.** The GRE exam guide's Cost tile read $220, from data/exams.json, whose note gave China's fee as $231.30. ETS's fees page, the page the figure cited, says its fees are effective August 1, 2026 and lists the GRE General Test at $231.30 in China and $249 in all other areas of the world. A price is the figure the site's rules most insist on, and this one had been wrong on the page since August 1.

- **Why.** A fee is a figure its publisher changes on a date of its own choosing. The record carried the year it was read, 2025, and nothing ever read the page again. validate_exams checks that a source is named, and the blog's price guard (INC-0124) checks that a dollar amount in a post appears in the sourced data. Both passed, because both compare the site with itself: the posts agreed with the data while the source had moved.
- **How it surfaced.** Found by src/check_sources.py on its first run, on September 26, 2026: 220 was not on the page the fact cited, and the page's fee table said $249. (Found by measuring something)
- **Fix.** The fee is $249, with the date ETS says it took effect. China's $231.30 and the $100 fee reduction price stay, since the same page still prints both.
- **What stops it now.** src/check_sources.py runs in the weekly audit and reports any number in an exam fact that the cited page no longer prints, so a changed price surfaces within a week rather than whenever someone next happens to read the page. in `src/check_sources.py`
- **Lesson.** A figure with a source and a year can still go stale, because the source moves and the record does not. For anything a publisher changes on its own schedule, fees above all, read the live source on a schedule instead of trusting the record because it cites one.

INC-0133. School pages published figures that the articles they cite do not contain, one of them a GRE total split in half

2026-09-27, Wrong data shown or stored, `f0d4186c4d35925e0e2f48fa3b9cf2ebf9de2c94` *PR #119*

- **What was seen.** Columbia's school page gave its Class of 2027 a GRE Verbal average of 163 and a GRE Quant average of 163, cited to a Poets&Quants article that prints a combined GRE average of 326 and a 150 to 170 range for each section, and no section average at all: 163 is 326 halved. Miami Herbert's page gave a GMAT median of 650 and 40 percent women, cited to a University of Miami news story that mentions neither; it gives GPA, a combined GRE, the acceptance rate, class size, work experience and citizenship, which the page reports correctly.
- **Why.** The library was compiled across sessions, some of which ran out of research budget, and a figure that fills an empty field is easiest to take from arithmetic or from somewhere other than the page cited. validate_schools checks that each figure has a source, a year and a URL and that the source is allowed, not that the page says the number, so a figure invented beside a real citation passes every check, which is the gap INC-0130 found in the exam guides.
- **How it surfaced.** Found on September 27, 2026 by running src/check_sources.py over the school library for the first time, then reading each flagged source that is a static article, where a missing number means something. (Found by measuring something)
- **Fix.** The three unsupported figures are removed, since the rule is that an unverifiable figure is shown as a dash and never guessed. Figures that are honest arithmetic on their page, such as months of work experience given in years or a count given as a percentage, now declare their working in a derived entry so the check can verify the inputs.

- **What stops it now.** check_sources.py --schools reads every page the school library cites, sets aside the College Scorecard dataset figures, reports a page that shows none of its figures as unread rather than wrong, and reports every other number its page does not print. It runs weekly in the playbook workflow, rendering pages built by JavaScript, alongside the exam check. in `src/check_sources.py`
- **Lesson.** Run the source check on every corpus that cites pages, not just the one that broke. When a field is empty and the research is out of budget, leave it empty: a blank is honest, and a figure made to fit beside a real citation is the most convincing kind of wrong.

INC-0134. Two school pages wrote "a 18.8% acceptance rate", because the article was chosen before the number arrived

2026-09-27, Cosmetic, `2d4369d75871766abdb5db40f6d52880b0357a5f` *PR #120*

- **What was seen.** MIT Sloan's school page opened 'The Class of 2027 profile reports a median GMAT Focus of 675, a 18.8% acceptance rate', and Harvard Business School's read 'a median GMAT Focus of 685, a class of 943, a 11.3% acceptance rate'. Eighteen and eleven begin with a vowel sound, so both take 'an'. They are the only two of the 16 programs with a verified rate whose rate starts with one of those sounds, which is why the sentence read correctly everywhere else.
- **Why.** The opening paragraph in build_rankings.py writes f'a {rate}% acceptance rate', so the template chose 'a' before it knew the number. In English the article follows the sound of the next word, and for a numeral that is the number spoken aloud, so a rate beginning eight, eleven or eighteen breaks the sentence. INC-0104 met the same fault with words ('a average'), and its guard names those two words, so a number that reads the same way passed it.
- **How it surfaced.** Found on September 27, 2026 by reading MIT Sloan's rebuilt page after its class size moved to the Class of 2028 profile. (Found by reading the code or the output)
- **Fix.** The opening paragraph takes its article from the number, through one helper that knows which numerals are spoken with a leading vowel sound.
- **What stops it now.** page_checks.articles() reads every built page and fails the build on 'a' before a number spoken with a vowel sound (8, 11, 18, 80 to 89, 800, 8,000 and so on) or 'an' before one spoken without. build.py runs it on every section beside python_reprs, and build_blog.py on the blog. in `src/page_checks.py`
- **Lesson.** A template that puts a fixed word beside a value has decided something about a value it has not seen. Let the value choose the words that depend on it, and check the built page for the rule rather than for the word that broke last time: a guard that lists yesterday's mistake misses the same mistake spelled another way.

INC-0135. Arizona State's school page said its class entered in 2027, because the label parser took the first year it found

2026-09-27, Wrong data shown or stored, `2d4369d75871766abdb5db40f6d52880b0357a5f` *PR #120*

- **What was seen.** W. P. Carey's page read 'The profile of the class that entered in 2027 reports a class of 47, a 19% acceptance rate', one sentence after 'The Class of 2027 profile reports an average GMAT Focus of 690'. The school's page describes a single class, its incoming class of fall 2025, so the site split one class into two and dated one of them in the future.
- **Why.** class_subject() in build_rankings.py turns a free-text class label into a sentence. When the label held 'enter', 'incoming' or 'cohort' it returned the first four-digit number anywhere in the label, and ASU's label was '2027 (entered fall 2025)', which leads with a graduating year we had inferred rather than one the page prints. The GMAT figure's stat carried the same inferred 'Class of 2027', and a figure's own class name outranks the page label, so the one class came out under two names.
- **How it surfaced.** Found on September 27, 2026 while reading the flags from check_sources.py --schools: ASU's only flag was the number 2027, which the school's page never prints. (Found by measuring something)
- **Fix.** class_subject() takes the year written next to the entering word ('entered fall 2025', 'Fall 2025 entering class', 'cohort, 2025') and returns no class when no year sits beside it, rather than guessing. ASU's label is now the page's own wording, the inferred class name moved from the figure's stat into its note, and the women figure cites the current page, which prints it.
- **What stops it now.** build_rankings checks class_subject() against every label shape in the library before rendering, including '2027 (entered fall 2025)', and fails the build when any school's parsed entering year is later than the build year. in `src/build_rankings.py`
- **Lesson.** When a parser must pull one value out of free text, anchor it to the words that give the value its meaning, not to its position, and return nothing when nothing anchors it: a sentence that says less is better than one that states a guess as fact. Keep inferences out of the fields that hold a source's own words, because the inferred class name in a stat is what let one class be reported under two names.

INC-0136. The LSAT guide said most test takers choose between remote and in-person testing, after LSAC had moved almost everyone into test centers

2026-09-27, Wrong data shown or stored, `bb46090cc18ccf961615d6a2ead116943c5695d3` *PR #123*

- **What was seen.** The /exams/lsat/ guide's delivery fact read 'Most test takers choose between an online, live remote-proctored test and in-person testing at a Prometric digital testing center', cited to LSAC's LSAT FAQ page. LSAC's LSAT Remote Testing and Distance Exceptions page says that starting with the August 2026 LSAT the multiple-choice portion is moving toward in-center testing for almost all test takers, with limited exceptions, so the guide described a choice most readers no longer have.
- **Why.** The fact was accurate to the page it cites, and that page is out of date: LSAC's FAQ still describes the 2025-2026 testing year, fee included, while the change was announced on a different LSAC page. check_sources.py compares only the numbers in a fact with the numbers on its page, and this sentence has no number in it, so a page that went stale in its words could not show up as a mismatch.

- **How it surfaced.** Found on September 27, 2026 while checking a queued LSAT post against LSAC's pages: LSAC's navigation listed Remote Testing Exceptions, and that page contradicted the post and the guide. (Found by reading the code or the output)
- **Fix.** The delivery fact now quotes LSAC's Remote Testing and Distance Exceptions page and cites it, with the testing year it describes.
- **What stops it now.** check_sources.py now reports, for the exam guides, any cited page that describes itself as covering a testing, academic or school year that has already ended (LSAC's FAQ says 'For the 2025-2026 testing year'), so a fact resting on a page like that is re-read even when its numbers still match. in `src/check_sources.py`
- **Lesson.** A citation is only as current as the page it points to, and publishers leave old pages up. When a source describes itself as covering a period, check that the period is still running; a page about last year's test can be quoted perfectly and still be wrong about this year's.

INC-0137. A post queued to publish the next day said the LSAT trainer was still in development, eleven days after it went live

2026-09-27, Wrong data shown or stored, `bb46090cc18ccf961615d6a2ead116943c5695d3` *PR #123*

- **What was seen.** lsat-format-scoring-guide, set to publish on September 28, 2026, ended by saying an LSAT trainer 'is in development' and pointed readers to the LSAT guide's waitlist; the LSAT trainer has been live at /lsat/app/ since September 16. The same post gave the LSAT 'roughly 75 to 80 scored questions', a count LSAC does not publish, and repeated the remote-or-in-person delivery claim from INC-0136.
- **Why.** Posts are written ahead and published by date, so a post carries the state of the product on the day it was written. Nothing re-read a queued post when that state changed, and the build checks posts for dashes, links and held slugs, not for claims about which trainers are live.
- **How it surfaced.** Found on September 27, 2026 while adding a link from the post to the new LSAT percentile calculator, the day before it was due to publish. (Found by reading the code or the output)
- **Fix.** The post now says the LSAT trainer is live and links to it, drops the question count, and gives the delivery change as LSAC states it.
- **What stops it now.** build_blog.py fails when any post, published or queued, says a trainer that is live is in development, coming soon or on a waitlist, naming the post and the sentence. in `src/build_blog.py`
- **Lesson.** Anything written ahead of its publication date is a promise about the future made from the past. Re-check a queued post's claims about your own product at build time, where the product's real state is known, rather than trusting the day it was written.

INC-0141. Two school pages printed a source note cut off mid-word, because the research merge kept the first 300 characters of every note and nothing checked that printed text ends as a clause

2026-09-27, Cosmetic, `214f8da3401b8186dff51a54ce9bbff918edecf8` *PR #127*

- **What was seen.** Cincinnati Lindner's tuition note ended "tuition only, general, campus life and ITIE fees excluded; B" and UC Davis's ended "page marks figures as estimates subject to". Both are printed beside the tuition figure on the school's page, so each stopped partway through a word or a phrase in front of visitors.
- **Why.** data/research/merge_results.py joined a research result's stat and note and kept the first 300 characters, with a slice that cuts wherever the count runs out. Both notes are exactly 300 characters long, the longest in the library, and both entered the file in commit 4b25169 on September 19, 2026. build_rankings.py prints a figure's stat beside it, but validate_schools.py only checked a figure's src, year and url and whether the stat carried a dash, so a note that stopped mid-word passed every check.
- **How it surfaced.** Found on September 27, 2026 while rewriting the Cincinnati tuition entry for the school source check: its last clause was the letter B. A hand scan for other short last clauses found nothing more; the new validator check then found UC Davis, whose note ends on the word to, which the scan had not looked for. (Found by reading the code or the output)
- **Fix.** Both notes are rewritten whole from their pages, and the Cincinnati entry declares the working behind its two sums. merge_results.py keeps the whole note; a note too long for the page is for a person to shorten.
- **What stops it now.** validate_schools.py fails the build on a profile or scholarship stat whose last clause is one or two letters long, or that ends on a separator or on a word such as and, or, the, of, to. merge_results.py runs the same test and rejects a result that would store such a note. in `src/validate_schools.py`
- **Lesson.** Any field that is printed on a page is copy, whatever the schema calls it. A length cap that cuts at a character count will eventually cut a word in half and publish it, so shorten text by clause or leave it to a person; and check that printed text ends the way text ends, because a check that a field is present and sourced passes a field cut off halfway through.

INC-0142. Every school's employment rate was labelled and scored as a three month figure, though 15 of 70 were measured at six months, four months, a year or a reporting date, or at a timing their notes do not give

2026-09-27, Wrong data shown or stored, `2127341e6ab72debe85a7d0c7cac6740f476ebd4` *PR #129*

- **What was seen.** School pages and the rankings drawer printed every employment_rate_pct under the fixed label "Employed at 3 months", and the SFN Score's methodology says outcomes use 3-month employment. Of the 70 figures in the library, 9 are six month figures (Kogod, Babson, GW, Michigan State, Kellogg, SMU, Tennessee, UC Irvine, UNC), Tulane's is at four months, Northeastern's is within a year, and 4 notes give no timing in months (BC Carroll's reports to October 31; Portland State's, Wharton's and William & Mary's give none). Kogod's drawer read "Employed at 3 months: 94%, students reporting fully employed, in graduate school, or both, six months post-graduation", the label and its own note disagreeing on one line. The rankings table's Employ column, its sort, and the SFN Score's outcomes component used all 70 as if they were one measure, so a school reporting a later count was sorted and scored on a different measure from the rest.
- **Why.** The label and the methodology describe the field as it was meant to be filled, not as it was filled. Each figure's note records its timing, but nothing read the note: the label was a constant in PROFILE_FIELDS and the drawer, and sfn_score() took whatever value the field held. INC-0105 fixed the same shape of problem for salary (a median label on averages) by deriving the label from each figure's note, and the employment field was never given the same treatment.
- **How it surfaced.** Found on September 27, 2026 while checking the new whole program cost row in the rankings drawer: Kogod's drawer showed the three month label above a note that says six months. A scan of all 70 notes by the first timing each names found the rest. (Found by rendering it and looking)
- **Fix.** Each employment figure's timing is read from its note (three months or 90 days, four months, six months, a year, a reporting date, or none we have verified), and the school page, the drawer and the table say that timing. A figure its note places at another point is shown but neither scored nor sorted with the three month rates, as the methodology says; one whose timing is not verified is labelled so and still counts, so an unread report does not move a school on a guess. UNC's figure moves to the three month rate its report prints beside the six month one (88 percent). The SFN top three are unchanged; Michigan State, whose 93 percent is a six month rate, moves from 26th to 43rd, and Tulane, whose 100 percent is at four months, from 36th to 51st.
- **What stops it now.** build_rankings checks the timing reader against a table of note wordings on every build before any page is written, so a change that reads a six month note as three months, or a reporting date as none, stops the build. in `src/build_rankings.py`
- **Lesson.** A label fixed in code is a claim about every value the field will ever hold. When the data carries its own qualifier, such as a timing or a statistic, the label has to be read from the data, and anything that compares or scores the values has to use only the ones that share the qualifier the method names.

INC-0143. Six school figures kept their source description under the key reserved for our own commentary, so their pages printed them bare and the source check never read them

2026-09-27, Degraded, `8066d24aa5eec6aae6089f9940ea89851fa93c96` *PR #130*

- **What was seen.** Chicago Booth's and CMU Tepper's tuition, Michigan Ross's and Penn State Smeal's tuition, Emory Goizueta's acceptance rate and Vanderbilt Owen's salary each carried a description of the figure under note and none under stat. School pages print stat beside a figure and never note, so Michigan Ross's tuition showed without saying it is a 2025-26 non-resident rate, Penn State's without saying it is estimated tuition and fees for a one-year program, and Vanderbilt's salary lost the word median from its label, which is read from stat (INC-0105). The school source check leaves note out as commentary (INC-0133), so none of the numbers in those six descriptions had ever been checked against their pages.
- **Why.** The school schema uses two keys with deliberately different jobs: stat describes the figure as its source states it, and is published and checked; note is our commentary on how a figure was read, and is neither. Nothing enforced the difference, and six entries written before it was drawn put the description under note with no stat at all. Refreshing Booth's and Tepper's tuition in INC-0140's batch kept the key the entry already used.
- **How it surfaced.** Found on September 27, 2026 while listing tuition figures for a blog post: two of the highest showed no description. A scan of every sourced figure found the six with a note and no stat, and five more with both, whose notes are commentary as intended. (Found by reading the code or the output)
- **Fix.** The six descriptions move to stat, where the pages print them and the source check reads them; the five figures whose note is commentary keep it.
- **What stops it now.** validate_schools.py fails the build on a published figure that has a note and no stat, because a note is commentary beside a description, never a replacement for one. in `src/validate_schools.py`
- **Lesson.** When two fields hold similar text for different purposes, enforce the difference where the data is validated. A convention that only readers of the schema know is one the next edit will not follow.

INC-0144. BYU Marriott's two-year tuition total was published as tuition per year, and the fit card doubled it again

2026-09-27, Wrong data shown or stored, `8066d24aa5eec6aae6089f9940ea89851fa93c96` *PR #130*

- **What was seen.** BYU Marriott's school page showed "Tuition per year $63,968", beside a note saying "two-year program total". BYU's costs page prints $63,968 as the tuition for the entire two-year program for students who are not members of the Church, so the yearly figure was twice what it should be. The rankings tuition column and filter read it as a yearly rate, and the trainer's fit card, which assumes two years, showed $127,936 over two.
- **Why.** Until program_cost_usd was added on September 27, 2026, the library had only tuition_usd, a yearly field, and a research pass kept BYU's program total in it "with that caveat" rather than leave it blank. The caveat went in the note beside the figure, but every place that reads tuition_usd reads it as one year, so a label and a calculation contradicted the note. Nothing checked that a yearly field's own description did not call it a program total.

- **How it surfaced.** Found on September 27, 2026 while listing verified tuition figures for a blog post: the note read two-year program total under a column of yearly figures. A scan of every tuition note found no other; the one-year programs whose total is their year are labelled as one-year. (Found by reading the code or the output)
- **Fix.** BYU's figure moves to program_cost_usd, the field for a whole-program figure, with the members' total in its note, and tuition_usd is blank.
- **What stops it now.** validate_schools.py fails the build on a tuition_usd whose note calls it a program total, a total program or a whole program, unless the note says the program lasts one year. in `src/validate_schools.py`
- **Lesson.** A caveat in a note does not travel with a number: every consumer of the field reads the field's meaning, not the note. When a value does not fit the field's meaning, give it a field that does, or leave the field blank.

INC-0145. Wharton's page credited its Class of 2026 GRE scores to the Class of 2027, because the GRE sentence began with Its and followed whichever class came last

2026-09-27, Wrong data shown or stored, `cc6f1854fbb5995656e1cc739174defee0eddd4a` *PR #133*

- **What was seen.** Wharton's school page read "The Class of 2027 has an average GMAT Focus of 676. Its average GRE scores are 162 Quantitative and 163 Verbal." The GRE figures come from the Class of 2026 section of Wharton's own class profile page, where the Class of 2027's figures are drawn by JavaScript, so the sentence gave one class's scores to another.
- **Why.** The lead paragraph writes class size and GMAT through class_sentences(), which asks fig_subject() which class each figure describes and names it, so a record that mixes classes gets one sentence per class (INC-0104). The GRE sentence was written separately and began with Its, meant as the program. Placed after a class sentence, Its reads as that class, so whenever a record's last class sentence and its GRE figures describe different classes, the page misattributes the scores. Wharton's record is the one that mixes them: its GMAT Focus is Poets&Quants' Class of 2027 coverage and its GRE is the official page's Class of 2026 section.
- **How it surfaced.** Found on September 27, 2026 while checking which class Wharton's GRE figures describe before quoting them in a blog post: the page's own sentence put them in the Class of 2027. A scan of every school with both GRE figures found no other page affected today. (Found by reading the code or the output)
- **Fix.** The GRE figures join the class sentences, so fig_subject() names their class like any other: Wharton's page now says the Class of 2026 had average GRE scores of 162 Quantitative and 163 Verbal, beside a separate sentence for the Class of 2027's GMAT Focus.
- **What stops it now.** build_rankings fails the build when a lead paragraph contains a sentence beginning with Its, because a pronoun after a class sentence names that class whether or not the figure belongs to it. in `src/build_rankings.py`

- **Lesson.** A pronoun is a claim about what came before it. In generated prose the sentence before is chosen by the data, not by the writer, so name the subject of every sentence that carries a figure.

INC-0147. Six school records stored a bare year as their class, which the site printed as a Class of that year, so Notre Dame's average cohort size became "The Class of 2027 has 85 students"

2026-09-27, Wrong data shown or stored, `b1024a5805d4ec9f538dd4c4e09c2b672e271321` *PR #136*

- **What was seen.** Notre Dame Mendoza's school page said "The Class of 2027 has 85 students." Its program page prints "85 Average Cohort Size" and names no class; the only 2027s on it are application deadlines, and the record's own notes say its stats are not tied to a stated class year. Penn State Smeal's page said "The Class of 2025 profile lists an average GMAT of 657", citing the Smeal MBA Employment Report 2020, which covers the legacy two-year MBA rather than the One Year MBA the rest of the record describes.
- **Why.** class_subject() read a bare year in class_year as Class of that year, the shorthand data/DATA.md documented, and fig_subject() gives every figure without a Class of of its own the record's class. The research merge of September 19, 2026 wrote bare years into six records. Two meant the Class of that year (Maryland Smith and Texas A&M Mays, from Poets and Quants' class articles). The others meant something else: the year a page was read or the cycle it serves (Notre Dame 2027, Florida Warrington 2026), the class profile page's year on a record whose GMAT, GPA and salary come from a 2020 report on a program that no longer runs (Penn State Smeal 2025), and a year on a record with no class figures (Georgia Terry 2027). So pages that name no class got one, and figures from different documents shared it. Figures whose own stat said their page labels no class year took the record's class all the same.
- **How it surfaced.** Found on September 27, 2026 while choosing rows for a GPA post by the class each figure describes: Notre Dame's 3.3 sorted as the Class of 2027 although its notes say the program page names no class year. A scan of every record found five bare years, and the validator rule written for this found a sixth, Texas A&M Mays. (Found by reading the code or the output)
- **Fix.** class_year must say what its year is. Maryland Smith and Texas A&M Mays now read Class of 2026; Notre Dame and Florida Warrington read Typical class profile (no class year stated); Georgia Terry, which holds no class figures, has no label; Penn State Smeal reads One Year MBA class profile (class year not verified), and its GMAT, GPA and salary from the legacy program's 2020 report are removed. Notre Dame's class_size is empty, because an average cohort size is not a class's enrollment. class_subject() reads a bare year as no class, and fig_subject() gives no class to a figure whose stat says its page names none.
- **What stops it now.** validate_schools refuses a class_year that is only a year and says what to write instead. build_rankings' class self-check asserts that a bare year reads as no class, and that a figure described as having no class year on its page takes none from a record labelled Class of 2026 while one that names its class keeps it. in `src/validate_schools.py`

- **Lesson.** A shorthand in a schema is read by code with one meaning and written by people and tools with whatever meaning they had in mind. Where a value names what something is, require it to say so in full; a bare year cannot say whether it is a graduating class, an entering class, or the day a page was read.

INC-0150. Columbia's MBA class was credited with five years of work experience, a figure the cited article gives for its 46-student MBAxMS cohort, and the source check passed it because the article does say five years

2026-09-27, Wrong data shown or stored, `d24b47f6bf43ad598a908de5d67fcac94caf983f` *PR #140*

- **What was seen.** data/schools/columbia.json gave work_exp_years 5, "average", cited to Poets&Quants' article on Columbia's Class of 2027, and the queued November 5 post listed Columbia Business School at 5 years. The article's only work experience figure is in its paragraph on the MBAxMS cohort: "The incoming class includes 46 students with an average GPA of 3.54 and an average of five years of work experience." It gives none for the MBA class.
- **Why.** The research merge took the article's one work experience figure without the sentence around it, which is about a different program. check_sources.py then confirmed it: it asks whether a figure's number appears anywhere on its page, reading number words as numbers, and the article does say "five years of work experience", in the paragraph about the MBAxMS cohort. A check that finds the number cannot tell which program or class the sentence around it describes.
- **How it surfaced.** Found on September 27, 2026 by a test written to measure how weak a found-anywhere check is for small numbers: for every figure behind the GPA and work experience posts, look for its digits within 150 characters of its field's label on the page the check read. Columbia's 5 was the one figure whose digits never sit near experience; reading the article showed the figure it did print, as the word five, belongs to the MBAxMS cohort. (Found by reading the code or the output)
- **Fix.** Columbia's work experience is empty, with the reason in its notes, and the queued November 5 post and its fact block drop Columbia: 37 programs, 28 between 4.9 and 6 years, the middle program still at 5.1. William & Mary's 5 years now declares its working, 60 months divided by 12.
- **What stops it now.** check_sources.py --schools now asks where a school figure's number sits, digits or word. If it never sits within 200 characters of a word saying what it counts (GPA, experience, women, international and so on), the figure is reported as not found. If it does only near another program's name (MBAxMS, executive, part-time, evening, a master of science), it goes on a worth-reading list, because comparison tables and footnotes put other programs beside figures that are right: of the 13 it lists today, 12 are the right program's figures, and the 13th, Rice's international share, has no support on a page that now prints three-year averages. The self-check holds Columbia's paragraph, a JD/MBA footnote and a figure written as "from abroad". in `src/check_sources.py`
- **Lesson.** Finding a number on a page is not finding the fact. A check has to look for it beside the words that say what it counts, and notice when those words are about something else: another program, another class.

INC-0151. Rice's school page credited Poets&Quants with an average GMAT of 693 for the Class of 2027, a number the article never prints; the figure's own note said it was read from a search snippet

2026-09-27, Wrong data shown or stored, `26150577529d7e7a333b05d676afdee5ea716838` *PR #142*

- **What was seen.** data/schools/rice-jones.json gave gmat_classic 693, "average", cited to Poets&Quants' "Meet the Rice Business MBA Class of 2027" (2026), and the live page said "The Class of 2027 profile lists an average GMAT of 693 on the Classic 200 to 800 scale (Poets&Quants, Meet the Rice Business MBA Class of 2027, 2026)." The article's only GMAT figure for the class is "In terms of testing, the median GMAT came in a 700." It never prints 693 and names no edition. The figure's note read "Scale not labeled in snippet", and the record's own notes said Rice's class profile page printed an average GMAT of 693 with no edition label and was not recorded under the edition rule: the same number, refused under one citation and published under another.
- **Why.** The research merge filled Rice's GMAT from a search result's snippet and wrote the article the result pointed to as its source, keeping the snippet only in the figure's note, which is commentary and is never printed or checked. validate_schools warns on a source labelled search snippet, but it reads src, and this src named the article. Nothing at build time compares a figure with its page. The weekly source check does (INC-0133), and it listed "693: not in the source" among 35 open findings, but its report is a queue worked by hand. This session's plan for refreshing Rice was to keep the figure because it named its own class, without rereading the article.
- **How it surfaced.** Found on September 27, 2026 while moving Rice to the three-year averages its class profile page now prints: reading the article the record kept for the GMAT, to quote the class it names, showed a median of 700 and no 693 anywhere on the page. (Found by reading the code or the output)
- **Fix.** Rice's GMAT is empty, and its notes give both unlabelled figures and why neither is published: Poets&Quants' median of 700 for the Class of 2027 and the class profile page's average of 694 across the Classes of 2026, 2027 and 2028, neither naming an edition. The rest of Rice's profile moves to the page's three-year averages and says so. UNC Kenan-Flagler's and USC Marshall's GMAT notes, the other two that said snippet, now quote the sentence each article prints.
- **What stops it now.** validate_schools refuses a published figure whose stat, note or source says it was read from a snippet. A snippet is a search engine's excerpt, not the page, so the figure has to be read on its page, and its note rewritten to say what the page prints, before it can be published. in `src/validate_schools.py`
- **Lesson.** A search result's snippet is not the page it links to. A figure taken from one has to be read again on the page before it is published, and a record that admits the snippet in a note while citing the page looks fully sourced, which is why nobody rereads it.

INC-0154. The weekly source check listed Duke's GMAT medians and Michigan Ross's work experience as missing from their sources for weeks, but its 30 findings mixed real errors with pages it cannot read, so none was ever worked

2026-09-27, Wrong data shown or stored, `9bd9bb1fe5872b7b0a24043c7bce467b4011867b` *PR #147*

- **What was seen.** data/schools/duke-fuqua.json gave a median GMAT Focus of 665 and a median GMAT Classic of 720, cited to GMAC's article "Who Is In The Duke MBA Class Of 2027?" (2025), and Duke's school page printed both. The article prints neither: its only GMAT figures are a Class of 2027 middle 80 percent of 680 to 770 "under the previous scoring system" and its writer's estimates of about 615 and 735 on the current scale. Duke's own class profile page, now for the Class of 2028, gives only a middle 80 percent GMAT Focus range of 605 to 715, and the record's own notes said the two medians "were not confirmable". michigan-ross.json gave 5.8 average years of work experience, cited to a Poets&Quants article that prints no work experience figure, and filed Ross's GMAT as unlabelled although that article labels it: "Average GMAT 10th Edition 731" and "Average GMAT Focus 681". The same list held Washington University Olin's $130,000 median salary, shown with the year 2024 and a note naming the Class of 2024 employment report, which gives a median salary of $126,000 and calls the 81 percent beside it an offer rate, with 78 percent accepting a job; Darden's GMAT ranges, which our stat called middle 80 percent where the page prints only Low and High; Boston College's class size and acceptance rate and Wisconsin's women and international shares, which their pages no longer print; USC Marshall's 2024-25 tuition, a year behind its page; and William & Mary's tuition, dated fall 2024 by a page that names no year.
- **Why.** check_sources.py --schools prints every figure whose numbers it cannot find on the cited page and exits 1, and the weekly job opens an issue with that list. Nothing recorded a person's judgement about a finding, so each run printed the same list again. Fourteen of its 30 lines were right figures the check cannot read, drawn as graphics at Berkeley Haas, as chart data at Northwestern Kellogg, as images at Pitt Katz, or split across two fee pages at Cincinnati, and those can never leave the list; three more were flagged only for numbers in our own commentary. A list in which more than half the lines are known false alarms reads as noise, and the 13 real errors sat in it unworked from week to week. Duke's medians came from the research merge and were kept with a note that they could not be confirmed, which records the doubt and publishes the figure anyway.
- **How it surfaced.** Found on September 27, 2026 while working the check's list after Rice's GMAT (INC-0151): reading each of the 30 flagged figures against its page, in a browser for the pages the check cannot read, showed 13 wrong or out of date, 3 flagged only for our own commentary and 14 right. (Found by reading the code or the output)

- **Fix.** Duke's GMAT is empty, with Duke's own Class of 2028 range and the article's figures in its notes, and the queued posts' fact block drops the two medians. Michigan Ross's work experience is empty, its GMAT Focus average of 681 is added and both GMAT figures carry the article's edition labels. Olin's salary and employment rate are the Class of 2024 report's own: a $126,000 median salary and 78 percent accepting a job within three months, 81 percent with an offer. Darden's stats quote the page's own labels. Boston College's class size and acceptance rate and Wisconsin's women and international shares are empty, with what the pages used to print in their notes. USC's tuition is its 2026-27 rate and William & Mary's says its page names no year. Georgia Tech's, Foster's and Northeastern's reasoning about GMAT editions moves from the stat, which the school page prints, to the note. Cincinnati's surcharge page is cited beside its fee page, and the check reads both.
- **What stops it now.** data/source_triage.json records each flagged figure a person has read and found right: its value, the numbers the check cannot find, why, and the date it was read. check_sources.py --schools reports those apart from new findings and exits 1 only for a finding nobody has judged, so the weekly issue lists only what is new, and it names any entry the check no longer needs. validate_schools.py refuses an entry whose figure has gone or changed value, so a judgement cannot outlive the figure it was about. A figure worked from two pages names the second in also_urls, and the check reads both. in `src/check_sources.py`
- **Lesson.** A report that says the same thing every week stops being read. When a checker has blind spots, record each judgement about a finding it cannot settle, so that what it prints shrinks to what is new; a list that mixes known false alarms with real errors hides the errors about as well as no list at all.

INC-0155. The GMAT guide cited mba.com's Official Score Reports page for the rule on five free score reports within 48 hours, a rule that page never states

2026-09-27, Wrong data shown or stored, `58c9567d9d403a8a70a268cea4209520fc1d095a` *PR #148*

- **What was seen.** data/exams.json gmat.key_facts[5] said "Up to 5 free score reports can be sent if programs are selected within 48 hours of the Official Score becoming available" and linked it to https://www.mba.com/exams/gmat-exam/scores/official-score-reports. Read in a browser on September 27, 2026, that page describes the score report and its performance insights and mentions neither free reports nor 48 hours. The rule is on the page its Sending Your Score link opens: "If you would like to take advantage of the free score reports, you must send your (up to 5) free score reports within 48 hours of your Official Score being made available on mba.com." The fact was right and its citation sent a reader to a page that does not support it.
- **Why.** The fact was filed with the URL of the fact beside it, the report's performance insights, which that page does print; the rule on sending scores sits on a sibling page in the same section. mba.com answers a plain fetch with a bot challenge, so check_sources.py reported the page as unreadable on most runs and flagged "48: not in the source" only on runs whose browser read got through. A finding that appears on some runs and not others looks like noise, and nothing recorded that anyone had looked.

- **How it surfaced.** Found on September 27, 2026 while running the exam source check after INC-0154: the browser read of mba.com flagged 48 as not in the source, as two earlier runs that day had; reading the page showed no mention of free reports, and its Sending Your Score link led to the page that states the rule. (Found by reading the code or the output)
- **Fix.** The fact cites mba.com's Sending Your Score page, read in a browser on September 27, 2026, with 2026 as its year. Its wording already matched that page.
- **What stops it now.** check_sources.py reads mba.com in a browser and finds the fact's 5 and 48 on the page it now cites, so the exam check passes; when mba.com answers with its bot challenge instead, the page is reported unreadable rather than the fact wrong (INC-0156), so a miss on this fact is a real one. in `src/check_sources.py`
- **Lesson.** A fact filed next to another fact tends to inherit its citation. When two facts come from one section of a site, check that each one's own page says it, not the page its neighbour came from.

INC-0157. Eleven GMAT figures on ten school pages were filed under an edition their sources never name, by guesses from the class year, the last digit or how high the number was

2026-09-27, Wrong data shown or stored, `52e8d6caf9abeb828313f67a7e5be710a2b9d504` *PR #149*

- **What was seen.** Georgia Tech's 654, Washington Foster's 655, UC San Diego Rady's 645 and Arizona State's 690 were published as GMAT Focus figures, and Arizona State's 698, Texas A&M's 672, William & Mary's 620, Notre Dame's 680, USC Marshall's 742, UNC Kenan-Flagler's 710 and Cornell's 710 as GMAT Classic figures, though no source for any of them says which edition it is on. Several stats printed the reasoning beside the figure, and some of it was false: Arizona State's and WashU Olin's called 698 and 702 "above Focus scale cap", though the Focus scale runs to 805; William & Mary's said 620 "is not producible on the Focus scale", though an average can be any number; Cornell's notes said 710 "is not an attainable GMAT Focus total", though a median of an even number of Focus scores can be 710, and Cornell's own FAQ names "the GMAT" and "GMAT 10th Edition" as separate exams, so its plain "GMAT (median) 710" may be the current one. USC's notes said its official average of 742 names no edition and was not recorded, while the same 742 sat in gmat_classic cited to Poets&Quants, which names none either. At the same time 13 figures whose pages do name the edition carried stats that did not say so, and Alabama's and Rutgers' stats said no edition was labeled where the pages print "GMAT Traditional exam (older version)" and "GMAT (Classic)".

- **Why.** The research merges filed an unlabelled GMAT figure under whichever edition looked likely and wrote the reasoning into the stat. GMAC's own announcements rule out each reason: GMAT Focus testing began on November 7, 2023, and scores of either edition stay valid for five years, so any class that enrolled from 2024 on can hold both; an average or median can end in any digit; and neither scale tops out below 800. The library applied two rules at once, since Rice (INC-0151) and USC's own page had been left blank for want of a label. validate_schools checked a GMAT figure's range and never that its stat named the edition, and the school page's GMAT row already told readers that an unlabelled figure is never guessed.
- **How it surfaced.** Found on September 27, 2026 while fixing the source check's findings (INC-0154): Georgia Tech's stat called 654 "consistent with the Focus scale", which any average is. Listing every GMAT figure whose stat names no edition, or reasons about one, gave 29; reading each source found 13 labeled, 4 whose class took the test before the Focus Edition existed, 1 drawn on the Classic scale, and 11 that name no edition. (Found by reading the code or the output)
- **Fix.** The 11 figures whose sources name no edition are blank, each with a note quoting what its source prints. Every other GMAT figure's stat says which edition its source gives it in the source's words, or carries an edition_proof: Babson's and Kentucky's classes enrolled in 2018 and 2019, and Northeastern's and TCU's Classes of 2025 in fall 2023, before Focus testing began, each shown from the school's program page or the article's date and GMAC's announcement. WashU Olin's 702 stays, drawn on the page's 200 to 800 scale, without the false reasoning. The school page's GMAT row now says a figure printed without its edition is left out rather than guessed.
- **What stops it now.** validate_schools refuses a published GMAT figure whose stat does not show the edition its source gives it (the edition's name, its scale, or the other edition named beside it) unless an edition_proof gives the reason and the pages that show it, and refuses a stat or note that reasons about the edition, such as "consistent with", "scale cap" or "not labeled". in `src/validate_schools.py`
- **Lesson.** A figure with a unit that its source leaves out is not verified until the unit is. Deducing it from the number's shape feels like evidence and is not; either the source says it, or something outside the number proves it, or the figure stays blank.

INC-0159. Baylor's school page gave its whole MBA program's entering class of 80, with 15 part-time students in it, as the full-time class size, and a source note that named only the two full-time tracks

2026-09-27, Wrong data shown or stored, `9871ec6b11ca2f47fc09be06056a86cd15b0ea5b` *PR #153*

- **What was seen.** Baylor Hankamer's school page said the class that entered in 2024 has 80 students, and the figure's source line read "Total - MBA Program, entering 2024; includes Two-Year 53 and One-Year 12 tracks". Baylor's student profile gives the entering class as 53 in the two-year MBA, 12 in the one-year MBA and 15 in the part-time MBA: 80 counts the part-time students, and the two tracks the note names make 65. The acceptance rate of 24.4 percent was the same whole-program total, 121 accepted of 495 applications, and every one of the 15 part-time applicants was accepted; the two full-time tracks accepted 106 of 480. The GPA, women and international figures beside them are the whole program's too, and only the GPA said so.
- **Why.** The research merge took the page's Total - MBA Program row for the class size and the acceptance rate, and described the class size from the rows above it, naming two of the three tracks the total adds up. Nothing compared the description with the page: the source check could not read Baylor at all, because its pages answer 403 to a script, and nothing adds up the parts a description names.
- **How it surfaced.** Found on September 27, 2026 while testing whether the source check's browser read could open the pages that refuse its plain read: Baylor's rendered in Chromium, and its table did not add up to the note. (Found by reading the code or the output)
- **Fix.** Baylor's class size is 65, the two-year and one-year entering classes, and its acceptance rate 22.1 percent, 106 accepted of 480 applications to those tracks, each with its working in a derived entry and a description that gives the program's totals with the part-time students. The GPA, women and international figures say they describe the whole MBA program, part-time included. The source check now tries the browser when a site refuses its plain read with 401, 403 or 429, so Baylor's three pages are read every week, and it recognises Cloudflare's verification page and Bloomberg's unusual-activity page, which Columbia, Michigan Ross and Bloomberg answer the browser with, as challenges. Read at last, Baylor's pages showed two more loose ends, now fixed: its tuition of 67,756 is two semesters at 33,878 with no working declared, and its salary's description cites a median of 80,000 that only the tuition page prints, which is now read with it through also_urls.
- **What stops it now.** check_sources reads a page refused with 401, 403 or 429 in the browser before calling it unreadable, and its self-check holds Cloudflare's and Bloomberg's challenge text, Bloomberg's being long enough to pass for a page. Run on the library as it was, the check now reads Baylor's pages, lists the class size of 80 and the GPA as found only beside the part-time MBA's name, and reports the acceptance rate, the tuition and the salary description as numbers the pages do not print; the derived entries now make it find every input of Baylor's arithmetic on the page. in `src/check_sources.py`
- **Lesson.** A total is only as good as the rows it adds. When a page breaks a figure into parts, record which parts the figure counts, and prefer arithmetic a check can redo over a description it has to trust.

INC-0160. The GMAT guide's Cost tile showed $275 and $300 from a note that no check read, cited to a page that prints no price

2026-09-27, Degraded, `3525aa85563806e01ec2bd9cab7074f70b030615` *PR #154*

- **What was seen.** The GMAT exam guide's Cost tile read "$275 / $300". build_exams took both from the note of a cost figure whose value is empty, cited to mba.com's register page, and that page, read in a browser on September 27, 2026, prints no price, fee or cost at all. The prices are on mba.com's exam payment page, and only after a country is chosen. The figures were right, and that was luck: nothing had checked them.
- **Why.** Two parts of the site disagreed about what a figure publishes. build_exams falls back to the dollar amounts in a cost's note when the cost has no value, while validate_exams treats a figure with no value as unpublished and asks nothing of it, and check_sources reads only figures that have a value or text. The GMAT's cost was stored without a value because its US price depends on how the exam is delivered, which put the one published GMAT price where neither check looked.
- **How it surfaced.** Found on September 27, 2026 while collecting the GMAT's fees for a post: the register page the cost cites printed none, and the fee table appeared on the exam payment page only once United States was chosen. (Found by reading the code or the output)
- **Fix.** The GMAT's cost has a value, 275, the US fee at a test center, with the online fee in its note, and it cites the exam payment page, with the address that page loads its US table from in also_urls so the source check reads the table itself. The rescheduling, cancellation and score report fees from the same table are recorded as key facts. build_exams prints a cost only from its value, never out of a note. Two rules from GMAC's policies document are recorded beside them: rescheduling or cancelling by phone adds a US$10.00 fee, and a voucher or fee waiver covers the exam fee only.
- **What stops it now.** validate_exams refuses a figure with no value whose note carries a dollar amount, since nothing checks it, and holds also_urls to the rules for url: https, on the test maker's host. check_sources reads the payment table through also_urls, reading it once more when the browser's first read gets Imperva's challenge, which two of three reads on September 27, 2026 did not; that day's run verified every fee. in `src/validate_exams.py`
- **Lesson.** What a page publishes has to be what the checks read. When a builder falls back to a second field to fill a gap, that field is published, and every check written for the first field has to cover it too.

INC-0167. School pages called an acceptance rate its source gives as an estimate "reported"

2026-09-27, Wrong data shown or stored, `2ed6c9cea644c397b9223ab96f364497b47d8cc2` *PR #163*

- **What was seen.** Harvard Business School's page said "The reported acceptance rate is 11.3 percent" and answered its FAQ with "Its reported acceptance rate is 11.3%", though the figure's own source line, kept in the library, reads "Poets&Quants (The M7 By The Numbers 2026, described as estimated)". Adding Columbia's rate, which Poets&Quants marks "*Estimate; source: CBS", produced the same sentence for it: "The reported acceptance rate is 19.5 percent."

- **Why.** The school page builder writes every acceptance rate with one fixed word, "reported", in the profile sentence, the page lead, the meta description and the FAQ. Whether a figure is an estimate lives in its source and note, which the page prints in its table, but none of the sentences read it, so the table said estimate and the prose said reported.
- **How it surfaced.** Found on September 27, 2026 when Columbia's estimated acceptance rate was added and its page was read back sentence by sentence. (Found by reading the code or the output)
- **Fix.** Every sentence that states an acceptance rate reads the figure's own source and note: where either calls it an estimate, the page says "estimated" in the lead, the profile sentence, the meta description and the FAQ. The page's closing line, which said anything unverified shows a dash rather than an estimate, now says a dash, never an estimate of our own: a publisher's estimate is shown, and named as one.
- **What stops it now.** build_rankings' prose check fails a school page that calls an estimated acceptance rate "reported" in any sentence it writes about itself. in `src/build_rankings.py`
- **Lesson.** A qualifier recorded next to a figure has to reach every sentence that states the figure. When prose is generated from data, words like reported, median or estimated are part of the value, and a fixed word in a template silently overrides what the record says.

INC-0168. Emory's acceptance rate was shown beside counts that divide to a different rate, with nothing on the page saying so

2026-09-27, Wrong data shown or stored, `2ed6c9cea644c397b9223ab96f364497b47d8cc2` *PR #163*

- **What was seen.** Emory Goizueta's page showed a 32% acceptance rate with the source line "Poets&Quants (Class of 2027 admissions coverage: 1,581 applications, 450 admitted)". 450 of 1,581 is 28.5 percent, so the page's own source line contradicted its figure, and a reader who did the division had no way to know which to believe.
- **Why.** The Poets&Quants article, read on September 27, 2026, prints 32% in its table and its text, and also prints 1,581 applications and 450 admits in both; its table credits the data to Emory. Every earlier year in the same table matches its counts (509 of 1,383 is 36.8%, 432 of 1,104 is 39.1%), and the Class of 2027's yield, 128 of 450 or 28.4%, agrees with the admits, so one of the rate and the application count is misprinted and the article does not say which. The record kept the printed rate and put the counts in its source line as if they were its working.
- **How it surfaced.** Flagged on September 26, 2026 as an open question while the article answered 403 here; resolved on September 27, 2026 when the article could be read. (Found by reading the code or the output)
- **Fix.** The rate stays at 32%, as its source prints it. Its source line no longer implies the counts produce it, and its note says what the counts work out to and why the conflict cannot be settled from the article; the school's own class profile page prints no admissions figures.

- **What stops it now.** The source check now reads the 28.5 in the note as derived from the page's 450 and 1,581, so if the article changes either count or the rate, the check reports the figure. in `src/check_sources.py`
- **Lesson.** When a source contradicts itself, pick nothing silently. Show the figure it states, show the figures it contradicts, and say which cannot be settled; a reader given the conflict can judge it, a reader given one number cannot.

INC-0170. A published post said GMAC caps GMAT attempts across a lifetime, which its current retake policy does not say

2026-09-27, Wrong data shown or stored, `9a074f173c89e85a1c4450105c243c2e3193a5f8` *PR #165*

- **What was seen.** "Should You Retake the GMAT? A Decision Framework", live since August 25, 2026, said GMAC "limits how many times you can sit the GMAT within a 12 month window and across a lifetime". GMAC's retake policy page, read in the browser on September 27, 2026, sets two limits only: at least 16 calendar days between appointments and up to 5 exams in a rolling 12-month period. It names no lifetime limit. The site's own exam record and its GMAT cost post already gave the current rule.
- **Why.** The post described the policy from memory in general terms, without a source for the lifetime clause, and linked the GMAT home page rather than the policy it described. It was written before exam facts were mirrored with their URLs and read weekly, and a post's prose is not compared with those facts, so the claim sat unchecked when the page it would have cited said otherwise.
- **How it surfaced.** Found on September 27, 2026 while collecting the five exams' retake limits for a new post: a search of every post for GMAT retake claims turned up the lifetime clause. (Found by reading the code or the output)
- **Fix.** The post states GMAC's current limits with their source and links the policy page itself, and its FAQ answer gives the same figures.
- **What stops it now.** build_blog keeps a table of retired claims, each a pattern and the reason it no longer holds, and fails any post that makes one; a GMAT lifetime attempt limit is its first entry. in `src/build_blog.py`
- **Lesson.** A fact written into prose without its source cannot be rechecked, so it stays whatever it was the day it was written. State the figure with the page it comes from, or link the page and say nothing it does not say.

INC-0171. Thirty published blog posts set their section headings in sentence case, against the house rule that headings are Title Case

2026-09-27, Cosmetic, `9a074f173c89e85a1c4450105c243c2e3193a5f8` *PR #165*

- **What was seen.** A scan of every post's h2 and h3 on September 27, 2026 found the 30 posts published from August 17 to September 20 setting their headings as sentence-case questions ("What does GMAC's retake policy allow?", "Where does data sufficiency live now?"), while the posts queued after them use Title Case ("What Does the ACT Cost in 2026?"). A handful of later headings capitalise a small word mid-heading ("Submit Early, On Purpose").
- **Why.** The Title Case rule in CLAUDE.md covers section headings, but the blog build checks a post's structure, links, prices and trainer claims and never its headings, so the first batch of posts, written in sentence case, shipped that way and nothing flagged them when later posts followed the rule.
- **How it surfaced.** Found on September 27, 2026 while correcting one of those posts (INC-0170): its headings were plainly in sentence case, and a scan of all posts counted the rest. (Found by reading the code or the output)
- **Fix.** Every post heading is rewritten to Title Case by the rule's own terms: each word capitalised except a, an, the, of, and, or, to, for, by, in, on, at and vs, which stay lower case unless first, last, or first after a colon; words already carrying capitals, such as GMAT or GMAC's, keep them.
- **What stops it now.** build_blog fails any post whose h2 or h3 is not in Title Case by that rule, and names the heading and the words to change. in `src/build_blog.py`
- **Lesson.** A style rule that nothing checks holds only for the work written after someone remembered it. When a rule can be stated precisely enough to apply by hand, it can be checked by the build, and it should be, or the earliest work quietly keeps breaking it.

INC-0172. The ACT exam guide and a live post said the ACT can be taken up to 12 times, after ACT's page had come to say there is no limit, and the source check matched the 12 in the page's K-12 menu

2026-09-27, Wrong data shown or stored, `9a074f173c89e85a1c4450105c243c2e3193a5f8` *PR #165*

- **What was seen.** The ACT exam record's retake policy, shown on /exams/act/, read "Can be taken up to 12 times in total; ACT suggests most students retest only two to three times", and the SAT vs ACT post, live since September 26, 2026, gave "Up to 12 times in total" in its comparison table. The page both cite, read on September 27, 2026, says "There is no limit to how many times students can take the ACT test. On average it takes students 2-3 times to achieve their testing goals." The weekly exam source check had passed the figure every week.
- **Why.** The source check confirms that every number in a fact appears somewhere on the cited page. ACT's page no longer gives an attempt limit, but its navigation carries a "Students & Parents K-12" link, so 12 was found, and 2 and 3 were found in "2-3 times". A number matched anywhere on a page is not the fact found: the school check learned this and wants a figure beside its label (INC-0150), but the exam check still accepted a number from anywhere, the site menu included.
- **How it surfaced.** Found on September 27, 2026 while reading the five exams' retake pages for a post on retake limits. (Found by reading the code or the output)

- **Fix.** The ACT record, the SAT vs ACT post, the queued ACT format guide and the editorial fact sheet say what ACT's page now says: no limit, with 2 to 3 attempts the average ACT reports. The source check no longer reads a number out of a grade range such as K-12.
- **What stops it now.** check_sources' self-check holds a page that prints 12 only in "K-12", which must not confirm a fact's 12; and the blog build's retired-claims table (INC-0170) fails any post that gives the ACT a 12-attempt cap. in `src/check_sources.py`
- **Lesson.** A number's presence on a page is weak evidence for a fact, because pages are full of numbers that mean other things: menus, dates, grade ranges, footnote markers. A check that matches numbers should read them where the fact would be, and should expect the page to change what it says around them.

INC-0174. Two GMAT facts passed the source check on numbers that meant something else: score validity cited the retake policy page, where "5 times" supplied its 5, and the score release's "3 to 5 days" matched a 3 in the page's helpfulness counter

2026-09-27, Wrong data shown or stored, `b6727f6ba5f7be256111a4212877c1bc4980feaf` *PR #166*

- **What was seen.** /exams/gmat/ showed Score Validity 5 years citing GMAC's retake policy article, which says nothing about validity; GMAC's validity article, last updated October 30, 2023, says "GMAT scores are valid for five (5) years and available for reporting for up to 10 years." Its Score Release row read "Official Score Report is typically available in your mba.com account within 3 to 5 days (up to 20 days in some cases), with an email notification when it is ready". The article it cites, last updated September 9, 2026 and read on September 27, says "Your official score report should be available within five (5) days. Although not typical, it can take up to 20 days for your exam to be scored" and "Scores cannot be expedited", with no 3 and no email notification. The weekly exam source check had passed both.
- **Why.** The exam check confirmed a fact when each of its numbers appeared anywhere on the cited page. The retake article prints 5 in "up to 5 times within a rolling 12-month period", and the release article prints 3 only in its footer's "0 out of 3 found this helpful". INC-0172 met the same weakness through a K-12 menu link but guarded only grade ranges, so any other stray number still passed; and a fact with no text of its own, such as a validity figure, had no words for a number to sit beside. Both facts entered the record on September 19, 2026 (PR #45), ten days after the release article's last update, so whether it ever said 3 to 5 days cannot be told from here.
- **How it surfaced.** Found on September 27, 2026 by an audit of every exam figure found on its page only away from all of the fact's words: 21 numbers, 18 of them cells of percentile tables (ETS's Table 1B, LSAC's percentiles, ACT's national ranks) confirmed row by row, and these two plus GRE's validity (INC-0173). (Found by measuring something)
- **Fix.** The validity figure cites GMAC's validity article, and the score release says what the release article now says.

- **What stops it now.** check_sources now wants each number of an exam fact within 160 characters of one of the fact's own words (five letters or more, less words that every exam page uses), or of its field's label for a figure with no text (validity: valid, reportable, expire), or as a cell in a run of numbers, which is how a table prints it. Its self-check holds the retake article's "5 times", the helpfulness counter and a percentile table row. in `src/check_sources.py`
- **Lesson.** When a check fails the same way twice with different details, guard the shape rather than the instance. Two narrow guards covered a cohort paragraph and a grade range, and the third instance was a vote counter. The question every match should answer is whether the number sits where the fact would be, and that can be asked of every fact.

INC-0176. Headings outside blog post bodies broke the Title Case rule on every kind of page, because the build checked only the headings inside posts

2026-09-27, Cosmetic, `847a00a070c30c03bf268490fd5e56ec5bcddf3f` *PR #168*

- **What was seen.** A scan of every built page on September 27, 2026 found sentence-case headings outside post bodies: the blog template's "Keep reading" and "Frequently asked questions" on every post, the blog index's "Insight for your next big exam", headings in the trainer app such as "Review your section", "Custom drill" and "Which exam are you preparing for?", the FAQ questions on the SAT, ACT and LSAT test date pages ("When is the next LSAT?"), the rankings page's "Summary comparison" and "Sources and method", and the funding page's "Veterans and Military: the Post-9/11 GI Bill".
- **Why.** INC-0171 added a Title Case check to the blog build, and it reads the h2 and h3 in each post's own body, the only place its incident had looked. Every other heading is written in a template or a page generator, and nothing read those. The helper had also never met headings from the rest of the site: it lowered the first word of a subtitle after a question mark ("Should You Retake the GMAT? a Decision Framework"), the first word after a numbered prefix ("1. the Short Version"), a middle initial ("Richard a. Chaifetz") and the A of "Texas A&M", so it could not have been pointed at the whole site as it stood.
- **How it surfaced.** Found on September 27, 2026 while previewing a new post: its template's own headings were in sentence case, and a scan of every built page with the same helper counted the rest. (Found by reading the code or the output)
- **Fix.** Every heading in a template or page generator is set in Title Case. The helper moves to page_checks, where build.py can use it too; it treats the word after a colon, a question mark, an exclamation mark or a numbered prefix as first, and keeps a single-letter initial and a small word joined to other letters by & as written.
- **What stops it now.** build.py checks every h1, h2 and h3 on every built page against the rule, and the blog build does the same for blog pages, template included. Both skip a heading a script assembles, which has no fixed text to check, and the names of schools and colleges, which are proper names. in `src/page_checks.py`

- **Lesson.** A guard written for the instance found covers only where that instance was. When a rule applies everywhere, check where every page ends up, the built output, rather than one of the several places pages are written.

INC-0177. The LSAT guide said scores come out roughly three weeks after each administration, citing a dates page that says no such thing, and the source check never looked because the fact wrote its number as a word

2026-09-27, Wrong data shown or stored, `3519ae8d2ce6498eb1751b609494e9b987486f03` *PR #169*

- **What was seen.** /exams/lsat/ gave the LSAT's score release as "Scores are released on published dates roughly three weeks after each administration; processing can take up to three weeks from the date LSAT Argumentative Writing is completed", citing LSAC's LSAT dates page. Read on September 27, 2026, that page is a table of dates: its 2026-2027 releases fall 18 days after each administration's last test day, and it says nothing in words about three weeks. The processing sentence is on LSAC's LSAT scoring page, which the fact did not cite, and that page also says a score is released on its date only with an approved writing sample on file and no holds on the account.
- **Why.** The source check reads a fact's numbers as digits only, while it reads the page's numbers as digits and words, so a fact that writes its number as a word ("three weeks", "five sections", "all three") has nothing to check. 34 of the 109 exam facts write at least one number as a word. And a number that is only part of a date in a table, such as the 3 of 3/3/2027, counted as a table cell beside other numbers, so it would have confirmed the 3 had the check looked.
- **How it surfaced.** Found on September 27, 2026 while gathering the LSAT's score release for a post: the fact's first clause matched no sentence on either LSAC page. (Found by reading the code or the output)
- **Fix.** The LSAT score release now says what LSAC's pages say, citing the scoring page and the dates page: scores come out on the published release date for each administration, with an approved writing sample on file and no holds, and writing samples can take up to three weeks to process. The ACT cost note's "all three", a count of our own, now reads "with both add-ons".
- **What stops it now.** check_sources reads a fact's number words as numbers for the exam guides, as it already did for pages, and a number that is only part of a date (3/3/2027, 2027-03-03) counts as found but never as beside the fact's words or in a table. The self-check holds the old LSAT sentence against the dates page and fails unless it is flagged. in `src/check_sources.py`
- **Lesson.** A check that reads two sides must read them the same way. Reading number words on the page but not in the fact meant any figure a writer happened to spell out was exempt, and the exemption was invisible because the check reported nothing missing rather than nothing checked.

INC-0178. Buttons across the site named their actions in sentence case, against the house rule that a button naming a destination or action is Title Case, and no check read them

2026-09-27, Cosmetic, `39375d2c0ea0f60024ef7b50d25b93a368b2954e` *PR #172*

- **What was seen.** A scan of every built page on September 27, 2026 found 27 distinct labels on styled buttons and button links in sentence case: "Open the trainer" on the study guide pages, "Start a free round" on every blog post, "Take the diagnostic" and "Skip this one" in the diagnostic guide, and in the trainer app "Start drill", "Back to deck", "Study due cards", "Abandon section", "10 questions", "Coming soon" and more. The blog's call to action card was titled "Put this into practice".
- **Why.** The house rule names buttons alongside headings, but INC-0176's check reads h1 to h3 only, and nothing read a button's label. Buttons are written in several templates and in the trainer app's script strings, each in whatever case its author reached for.
- **How it surfaced.** Found on September 27, 2026 while closing out INC-0176, whose record listed buttons and card titles as still to do; a scan of built pages with the shared Title Case helper counted them. (Found by reading the code or the output)
- **Fix.** Every styled button and button link is labelled in Title Case, and the blog's call to action card title too.
- **What stops it now.** build.py and the blog build check the label of every <button> and <a> carrying the btn class on every built page, reading each tag with its quoted attributes whole so a script in an onclick cannot end it early, and skipping labels a script assembles. in `src/page_checks.py`
- **Lesson.** A rule with a list of places it applies needs a check for each place on the list. Finishing the headings made the buttons look done by association, and only reading the rule again, clause by clause, showed the gap.

INC-0179. A new reading passage's revision opened with a place name, and the stated-idea questions printed it as "ardley cheese"

2026-09-27, Cosmetic, `26cb4febc386d201c3984787bf154cffd7f4cbb5` *PR #174*

- **What was seen.** Reading every item generated from five new long passages on September 27, 2026, before they shipped, the LSAT stated-idea questions on the Ardley cheese passage offered "ardley cheese grew famous mainly because the railway let the farms reach the city markets quickly, and the abbey's recipe played at most a small part" as a choice, with the place name in lower case.
- **Why.** The stated-idea schemas print each of a passage's sentences as a choice with its first letter lowered, since the choices continue a stem ("According to the passage, Quillan found that"). That is right for a sentence that opens "The" or "Food historians" and wrong for one that opens with a proper noun. Every earlier passage happened to open its revision with a common word, and check_corpus, which checks a passage's shape, did not look at how its sentences open.
- **How it surfaced.** Found on September 27, 2026 by reading the items a new batch of passages generated, at LSAT length, before committing them. (Found by reading the code or the output)

- **Fix.** The revision now opens "the fame of Ardley cheese came mainly from the railway", which lowers safely.
- **What stops it now.** check_corpus refuses a passage whose lowered sentence opens with a word the same passage writes capitalised in the middle of a sentence, which is how a proper noun shows itself; run over all 62 passages it flags only the one found here. in `src/gen/g_rc.py`
- **Lesson.** Changing the case of text you did not write is only safe when you know what its first word is. A generator that lowers or raises a first letter needs a check that the word is one it may change, or the corpus has to be written so it never matters.

INC-0180. The GMAT's delivery fact said appointments are available year round with no fixed testing windows, words its cited page no longer carries, and the source check passed it because 20 of the 113 exam facts carry no number for it to look for

2026-09-27, Wrong data shown or stored, `767d7b9230bc51f1f3fcfa345586af7ac4cd97aa` *PR #175*

- **What was seen.** Reading mba.com's GMAT register page on September 27, 2026, for a post on testing at home, the page offers an online or a test center appointment and says nothing about appointments being available year round or about testing windows. The GMAT delivery fact in data/exams.json, cited to that page as mba.com 2025, says "At test centers or online, with appointments available year round (no fixed testing windows)", and the GMAT exam guide prints it under Delivery. The source check run the same day had reported 113 exam facts checked against 67 sources with none flagged.
- **Why.** check_sources.py compares the numbers in a fact with the numbers on its page. 20 of the 113 exam facts carry no number at all (the delivery, used_for and acceptance lines and several key facts), so for them there was nothing to compare, and they were still counted among the facts checked. INC-0136 found this blind spot on the LSAT's delivery fact and guarded one way a page goes stale, a page that names a testing year that has ended, which leaves a page that simply stops saying something undetected. The summary line counted a fact with nothing to check as a fact checked, so "0 flagged" read as 113 facts verified when 20 had not been read for anything.
- **How it surfaced.** Found on September 27, 2026 by reading the cited page by hand while researching a post on which exams can be taken at home. (Found by reading the code or the output)
- **Fix.** Every number-free exam fact was read against its page on September 27, 2026, and the 14 whose words had drifted now say what their pages say. The GMAT's delivery line cites mba.com's exam day pages: test centers and online, with the online exam unavailable in Mainland China, Cuba, Iran, North Korea and Sudan. Four facts had claims no page of theirs made and were re-cited: the GMAT's Quant calculator rule to GMAC's Data Insights guide, the SAT's uses to College Board's answer on what SAT scores are used for, the ACT superscore's arithmetic to ACT's superscore FAQs, and the LSAT writing sample reaching law schools to LSAC's law school reports page. The queued December 1 post and the SAT format post, which repeated two of those claims, were corrected with them.

- **What stops it now.** check_sources.py reads a fact with no number by its words: each content word must be on one of its pages, as the word, a plural or another tense, and the words that are not are reported like a missing number. The summary line counts the facts read by words apart. Its self-check holds the old GMAT delivery line against the register page's own text and fails unless year round and windows are reported; run over the data before the fix it reported 14 facts, and after it none, the GMAT delivery line's second page read on a retry after Imperva challenged the run's read. in `src/check_sources.py`
- **Lesson.** A check that finds nothing wrong has only checked what it could see. Count the things it had no way to test as unchecked rather than as passed, or a clean report will quietly include everything the check was blind to.

INC-0186. Stated idea questions that name a study offered another of that study's sentences as a wrong answer, so 178 of 378 had two defensible answers

2026-09-28, Wrong data shown or stored, `cb8eda15275e29b46629c046510c0034952521e2` *PR #190*

- **What was seen.** An LSAT stated idea item asked 'The passage states that, in the work of Scarth,' and keyed the note on Scarth's records: 'the books record every load of flour and every cart of furze and brushwood the bakers bought, with the price they paid for each'. One of its wrong answers was Scarth's finding, that the bakers went on buying flour but baked fewer batches, which the passage also states in the work of Scarth, so the item had two right answers. Measured across the live banks, three of the six stated idea asks offered a sentence their stem covers as well as the key, on 178 of the 378 GMAT and LSAT items that use them. The detail ask for the first study offered that study's finding on 31 of 72 GMAT and 23 of 54 LSAT items, and the detail ask for the second study offered the second finding on 31 of 72 and 28 of 54. The ask 'a later study of ... found that' offered the second study's detail on 38 of 72 and 27 of 54, where the passage prints the finding and the detail as one clause, 'found that A, and B', so 'found that' reads as covering both. The closing caveat, which is often a limit of one study's own records ('Only three of the town's bakehouses kept books that survive, so the other bakers are not covered'), was offered on 35 of 72 GMAT and 25 of 54 LSAT first-study detail items, where it answers 'in the work of Scarth' too. And the second study's detail ask read 'the second set of results also established that', which calls a note on the records ('the rolls set out each order of the court, the date it was made and the fines for breaking it') a result, so the key itself was misdescribed.

- **Why.** The stated idea schema offers every other sentence of the passage as a wrong answer, on the reasoning in sentences(): a sentence the passage states but the stem did not ask about is true and beside the point. That holds only when the stem picks out exactly one sentence. The two detail asks, added with the six-ask form in #45, name a study rather than a sentence, and a stem that names a study picks out everything the passage says about it: the finding, the note on its records, and the caveat when the caveat is about those records. The second finding's ask has the same problem from the other side, because text() joins that finding and its detail with ', and'. Nothing recorded what else each stem is true of, and the checks that exist for reading items measure shortcuts to the key (INC-0117) and the key's length rank, not whether a wrong answer is also right.
- **How it surfaced.** Found on September 28, 2026 while reading the items drawn from five drafted passages before adding them to the corpus, then measured across the live GMAT and LSAT banks by matching each stated idea item's stem to its ask and each of its choices to a sentence of its passage. (Found by reading the code or the output)
- **Fix.** Each stated idea ask now declares what else its stem covers, in StatedIdea.COVERS, and those sentences are left out of its wrong answers. The two detail asks leave out the same study's finding and the caveat; the second finding's ask leaves out its detail. The first finding's ask declares nothing, because the passage sets the first detail off after a semicolon as a note on the records rather than something found, and a note on the records that was not found is exactly the trap a stated idea question sets. The second study's detail ask now reads 'The passage states that, in a later study of ...,' like the first study's, rather than calling a note on the records a result. A question's id is its stem and passage, so those questions get new ids and drop out of any review queue they were in, which the trainer already allows for.
- **What stops it now.** g_rc.check_covers runs in build_banks before any bank is written. It fails when a stated idea ask has no entry in its schema's COVERS table, so a new ask cannot be added without deciding what else it is true of, and when an item built for any passage and ask offers one of the sentences its entry names. It builds every ask of every passage for the GMAT, LSAT and GRE stated idea schemas. in `src/gen/g_rc.py`
- **Lesson.** When every wrong answer is a true sentence from the same passage, each stem has to pick out exactly one sentence, and a stem that names a person or a study picks out everything said about them. Write down, for every question form, what else it is true of, and check the items against that list: the closer a wrong answer sits to the right one, the likelier it is to be right as well.

INC-0187. The stated idea question about the earlier account read 'failed to address the fact that the account could not explain', on 143 of 198 items

2026-09-28, Wrong data shown or stored, `cf751b873445a5867bbdd98944ac373f52096b66` *PR #191*

- **What was seen.** The stated idea ask for the passage's problem sentence reads 'According to the passage, the earlier account failed to address the fact that', and 49 of the 72 problem sentences open with the account itself: 'The account could not explain why bread ran short even in months when the mills had flour to sell', 'The account rested entirely on measurements taken during the brief summer', 'It could not explain why two lichens...'. So the keyed completion said the account failed to address the fact that the account could not explain something, which is circular, on 143 of the 198 GMAT, LSAT and GRE items that ask it (49 of 72 GMAT, 45 of 54 LSAT, 49 of 72 GRE). On 120 of those, one of the studies' findings was also offered, and a finding reads as the better completion: a GMAT item keyed the circular sentence and offered 'the court had forbidden the cutting of furze on the common in 1791, though furze was the bakers' only cheap fuel, and the shortages began that winter', which is exactly a fact the earlier account failed to address.
- **Why.** The problem field is written as the closing sentence of the passage's first paragraph, where a criticism naturally names what it criticises, and the same field is spliced into a stem that wants a fact about the world after 'the fact that'. The first passages' problem sentences happened to state facts ('The weather station stands two thousand metres below the ice'), so the stem read well when it was written; most later passages were written to a template that opens 'The account could not explain why', and nothing rendered the stem with them.
- **How it surfaced.** Found on September 28, 2026 while rendering the stated idea asks of every passage for INC-0186, then counted across the live banks by matching each problem item's stem and key. (Found by reading the code or the output)
- **Fix.** The ask now reads 'According to the passage, even before the studies it describes, the earlier account could be faulted because', which every one of the 72 problem sentences completes as written, whether it names the account or states a fact. The first replacement, 'the difficulty with the earlier account is that', read well with every key but was answered by the findings as well: on the glacier passage 'summer melt had changed little' is as much a difficulty for a summer warming account as the key. Reading the rebuilt items caught it, and the stem now sets the studies aside, so their findings, the revision and the notes on their records stay wrong answers and the ask covers nothing else. The GRE schema, which asks the same question of its one-paragraph rendering, uses the same stem. A question's id is its stem and passage, so the 198 problem questions take new ids.
- **What stops it now.** g_rc.check_stems renders every stated idea ask of every passage with its key, for the GMAT, LSAT and GRE schemas, on each build, and fails on a completion that asks for a fact and then names the account itself ('the fact that the account', 'the fact that it'). The wider protection is the practice INC-0115 recorded: a reused field is read in every slot it lands in. in `src/gen/g_rc.py`
- **Lesson.** A template is written against the fields that exist when it is written, and the corpus goes on growing in a style the template never saw. When a new passage is added, render every question the schemas ask of it, not the ones the passage was written for, and write each stem so that it fits what the field is, a criticism or a fact, rather than what the first few happened to be.

INC-0188. Two queued posts said a GMAT score report carries only the score you send, after GMAC's Superscore began adding the best section scores from every

attempt

2026-09-28, Wrong data shown or stored, `bd08f42e9bf660c65a45af536d883541a8d92f8e` *PR #192*

- **What was seen.** free-score-reports-by-exam, queued for December 7, said 'Each report a program receives contains only the total and section scores from the exam it is sent for', and cancel-scores-by-exam, queued for December 13, put 'Only the scores you send' in its What schools see column for the GMAT. Since August 12, 2026 a GMAT score report sent to a school includes the GMAT Superscore when one exists: the highest section scores across valid attempts of the current edition, with the exam date and delivery method of each, and the individual scores sent to that program in the past five years; test takers cannot opt out of sending it (GMAC help center, GMAT: Sending and Reporting GMAT Superscores, updated August 12, 2026). mba.com's Sending Your Score page, which the December 7 post cites, says both in consecutive sentences: a report 'will only contain the Total and Section Scores from the exam associated with the score report', then 'Where applicable for candidates with multiple attempts, the Official Score Report will include your GMAT Superscore'. Neither post was live yet.
- **Why.** The December 7 post was written from the first of those two sentences and not the second, and the December 13 post turned it into a table cell. GMAC's older help article on who has access to a score, last updated in May, still lists a report's contents without the Superscore, so the two sources a writer would read first agreed with each other and not with the page's next sentence. Nothing compares a post's claim with the rest of the paragraph it came from: the source check confirms that a fact's words are on its page, which a sentence cut short still passes, and data/exams.json held no fact about what a report contains, so no check read the Superscore at all.
- **How it surfaced.** Found on September 28, 2026 while reading GMAC's Sending Your Score page and its four Superscore help articles for a post on whether schools see every score. (Found by reading the code or the output)
- **Fix.** Both posts now say what a report carries: the score you send and, once you have tested more than once, the GMAT Superscore with the section scores behind it and the date and delivery method of each, which cannot be left off. EDITORIAL.md's GMAT score sending notes state the Superscore, and data/exams.json gains the Superscore facts, what a report sent to a school includes and that it cannot be opted out of, so the source check reads them against GMAC's article.
- **What stops it now.** The weekly source check reads the new Superscore facts in data/exams.json against GMAC's article, so a change to what a report carries is reported rather than found by accident. build_blog also refuses a post that says a GMAT score report carries only one exam's scores, or that schools see only the scores you send, without naming the Superscore. in `src/build_blog.py`
- **Lesson.** A source's first sentence is not always its answer. When a page states a rule and qualifies it in the next sentence, the qualifier is part of the fact: read to the end of the paragraph before quoting, and when a test maker changes what schools receive, reread every sentence that tells a reader what a school will see.

INC-0189. Watch for notes named answer choices by letter or position, and the shuffle that moves the key never moved them, so 124 notes called the correct answer a trap

2026-09-28, Wrong data shown or stored, `a3786888a205412571d55603ab2c8b684995fdd9` *PR #204*

- **What was seen.** LSAT reading item LC103 asks how defenders of the exclusionary rule answer the distributive objection, and its key, 'mistakes a side effect of the rule for the purpose of the rule', is E. Its Watch for note read 'B is the other objection, and C, D and E are not the response described', so it told a student who chose correctly that their answer was one of the wrong ones, and the choice it called the other objection, B, is 'would apply equally to any remedy for unlawful searches'; the other objection sits at D. Run against the banks as they stood, the check this record adds fails 124 items whose Watch for note names the key's own letter or position among the wrong answers: 57 in bank_lsat_lr3.js, 21 in bank_lsat_rc3.js, 5 in bank_lsat_lr2.js, 2 in bank_lsat_rc2.js, 12 in bank_verbal.js, 13 in bank_verbal2.js, 11 in bank_verbal4.js, and one each in bank_quant.js, bank_act_english.js and bank_act_reading.js. Notes that point by position fail the same way: LL069's explanation said 'The first option has exactly that form' when its key is E, and LC044's note said 'The second option drops the timing' of a choice that now sits third.
- **Why.** Items are written key first, because that is how a person writes one, and the notes are written against that order: the author's B is the first distractor they wrote. bank_emit.permute then shuffles the choices so the key does not sit at A (INC-0039) and recomputes the answer index, but it moves only the choices and the index. A letter or an ordinal inside expl and wrong is plain text, and nothing tied it to the order it was written against. Every key in the ten generated banks sits exactly where permute's crc32 shuffle puts the first choice as written (140 of 140 in bank_lsat_lr3.js, 90 of 90 in bank_lsat_rc3.js), so in those files every positional reference is off by that shuffle, including the ones that happen to miss the key. The three GMAT verbal files were reordered before their first commit by something that left no seed: their keys do not sit where the crc32 shuffle would put them, and V221's key has been B since its first commit while its note says '(B) actually strengthens the conclusion', which describes A. The checks on reading items measure the choices and the key, not what the notes say about them.
- **How it surfaced.** Found on September 28, 2026 while rewriting the wrong answers of word matchable stated items (the measure INC-0117 set a ceiling on), when LC103's note was read beside its choices; then measured across every hand written item by reading each note's letters against its key, and by testing every bank's key positions against the crc32 shuffle. (Found by reading the code or the output)

- **Fix.** permute now resolves positions written in braces: a note says {B} for the second choice as written, and permute replaces it with the letter that choice lands on after the shuffle. The nine generators write every choice reference that way, so their notes move with their choices; Passage A and Passage B stay plain words, which is why the markers are explicit rather than guessed, since in a comparative set 'B says' is a passage. The hand written notes in bank_verbal.js, bank_verbal2.js, bank_verbal4.js, bank_quant.js, bank_act_english.js, bank_act_reading.js, bank_act_science.js and bank_lsat_rc2.js, whose generator no longer reproduces it, were corrected by hand, choice by choice, because the order they were written against is gone. 200 items' notes changed in all, most of them letters that pointed at the wrong choice without happening to name the key.
- **What stops it now.** permute refuses a brace that names no choice, a Watch for note that names the key as written, and any bare C to J, any A or B that reads as a choice outside a comparative set, and any ordinal that points at a choice, left in a shuffled item's notes. test.js fails any item on any exam whose Watch for note names the key's letter or position among the wrong answers; run against the banks before this fix it fails the 124 above. in `src/bank_emit.py`
- **Lesson.** A position is not part of a sentence. When content is written against one order and shipped in another, anything that points at the order, a letter, 'the second option', 'the last choice', has to move with it or be written so it does not depend on it. Make the reference explicit, resolve it where the order is decided, and check the result against the key rather than trusting the text that was true when it was written.

INC-0190. A length correction clause turned an LSAT must be true distractor into a second correct answer, and its note said so

2026-09-28, Wrong data shown or stored, `a3786888a205412571d55603ab2c8b684995fdd9` *PR #204*

- **What was seen.** LSAT item LL040 gives three premises: no member of the drafting committee voted against the proposal, every member who attended the March meeting voted on it, and two members did not vote at all. It asks what must be true and keys 'At least two members of the committee did not attend the March meeting.' One of its wrong answers read 'Every member who attended the March meeting voted in favour of the proposal, since no member of the committee voted against it', which follows from the first two premises just as surely, so the item had two correct answers. Its Watch for note conceded the point: 'The second is true of attendees but the question asks what must follow, and it does: attendees voted and nobody voted against.'
- **Why.** The option was written as 'Every member who attended the March meeting voted in favour of the proposal.', which was already inferable, and mk_bank_lsat_lr2.py's EXTEND table then appended ', since no member of the committee voted against it' to carry it past the key in length. The table's header says each clause states the omission that makes an option wrong and that none makes a wrong option defensible, but this clause supplied the missing premise instead, and nothing checks what a clause does to an option's truth, only what it does to its length and its seam (INC-0119).

- **How it surfaced.** Found on September 28, 2026 while reading every note in the generated banks that points at a choice by position, for INC-0189: LL040's note describes one of its own wrong answers as following from the premises. (Found by reading the code or the output)
- **Fix.** The option now reverses the second premise, 'Every member who voted on the proposal attended the March meeting', and its clause carries that error one step further, ', since every member who attended that meeting voted on the proposal', so it is wrong in the way the table intends and still sits above the key in length. The note says what is wrong with it.
- **What stops it now.** None that reads meaning: a clause that makes an option true cannot be told from one that makes it false by its words. The check that exists is the one that found this, reading the notes: a note that says a wrong answer follows is a finding, and permute now refuses the positional wording this note used. in `src/mk_bank_lsat_lr2.py`
- **Lesson.** A clause added for length is still a claim. When text is appended to a wrong answer to change its shape, read the result as a test taker would, against the question it answers, because the easiest way to lengthen a false statement is to add the reason that would make it true.

INC-0191. Three LSAT reasoning items offered wrong answers with the key's logical force, and each note said they lacked it

2026-09-28, Wrong data shown or stored, `a3786888a205412571d55603ab2c8b684995fdd9` *PR #204*

- **What was seen.** LSAT item LL128's argument runs 'Every member of the committee who voted against the proposal is a lawyer. Ruiz is a lawyer. So Ruiz voted against the proposal', which is all P are Q, x is Q, so x is P. Its key has that form, and so did one of its wrong answers, 'Every building made of stone on the square survived the fire. The library survived the fire. So the library is made of stone', with P and Q named the other way round, which changes the content and not the form. Its note said that option 'is not the same form', and called another, which denies the antecedent, a valid contrapositive. LL007 asks what can be inferred when the library extends its hours only if the grant rises or the endowment beats four percent, the grant is frozen and the endowment returned three and a half; its key is that the hours will not be extended, and a wrong answer, 'If the library does extend its hours, the endowment must have returned more than four percent', follows as surely, while the note said it treats a necessary condition as sufficient. LL167 gives ratification leads to falling tariffs, falling tariffs lead to rising exports or lost market share, and no market share is lost, and asks what must be true if the treaty is ratified; its key is that exports will rise, and two wrong answers followed as well: 'The treaty will be ratified only if exports rise' from the chain, and 'Exports will rise only if the tariffs fall' because the question's own supposition makes the tariffs fall. The note said the premises support neither. Each item had more than one correct answer.

- **Why.** Each distractor was written to commit an error and came out valid. LL128's was made by swapping the terms of the key's conditional, on the idea that a reversed conditional makes a different argument, but the argument still moves from consequent to antecedent. LL007's and one of LL167's were meant to reverse a conditional and were written the right way round, and LL167's other was a conditional whose consequent the question's supposition makes true, so it held whatever its antecedent. The notes were written from the intention rather than from the sentence on the page, so they describe the error the option was meant to make. Nothing checks a hand written reasoning item's options against its premises: the generated parallel reasoning items prove their forms (#102), and these were never put through anything like it.
- **How it surfaced.** Found on September 28, 2026 while reading the LSAT Logical Reasoning notes that INC-0189 re-lettered, and then every parallel and must be true item in the LSAT reasoning banks, checking each option against the premises rather than against its note. (Found by reading the code or the output)
- **Fix.** LL128's option is now the valid contrapositive of the key's premise, 'Every building on the square that survived the fire is made of stone. The library is not made of stone. So the library did not survive the fire', and its note says what each option is: one denies the antecedent, two are valid, one is probabilistic. LL007's is now 'If the city had increased its grant, the library would have extended its hours', which does treat a necessary condition as sufficient. LL167's two conditionals are now 'The tariffs will fall only if domestic producers increase their market share' and 'Exports would have risen even if the treaty had not been ratified', which the premises leave open even with the treaty ratified. Each note now describes the option as it reads.
- **What stops it now.** None added. A valid option cannot be told from an invalid one without reading the logic, which the generated parallel items do by proof and the hand written items get only from review; that review, of every parallel and must be true item, found these three and no others. in `src/mk_bank_lsat_lr3.py`
- **Lesson.** Check a wrong answer against the premises, not against what it was meant to be. A note written from the author's intention describes the error the option was supposed to commit, so it cannot catch an option that came out valid; write each option out as symbols beside the premises before writing what is wrong with it.

INC-0192. Hand written 'according to the passage' questions could be answered by choosing the option made of the passage's words

2026-09-28, Degraded, `c31bd4b5ac0d1c82271aaba1d7dc0bf99361f363` *PR #205*

- **What was seen.** A test taker who never read the question and chose the option with the largest share of its words in the passage got the key on 29 of 44 hand written GMAT stated questions, 22 of 30 LSAT, 16 of 25 ACT and 3 of 9 GRE, against a blind guess of one in five or one in four. GMAT item V261 asks what set a United States town's time before the 1880s, and its key, 'the local position of the sun', is the passage's first sentence word for word; LSAT item LC179's key, 'the stomach was sterile', is the passage's phrase.

- **Why.** The key of a detail question was written by copying the sentence that answers it, and the wrong answers were written fresh, so the key was the one option built from the passage's own words. INC-0117 put a ceiling on this shortcut for the generated reading schemas and left stated idea questions out, since a stated question's answer is in the passage by definition; nothing measured the hand written banks, which is where the copying happened.
- **How it surfaced.** Found on September 28, 2026 by running INC-0117's word match measure over the hand written banks, which had never been measured, after it was noted as an open item in ROADMAP.md. (Found by measuring something)
- **Fix.** Each of those keys is now said in other words, as the test makers' own stated questions are, keeping its meaning, its place in the length order of its choices and, in the generated banks, the rank its lift table sets: V261's is now 'the sun as observed from that particular town'. Keys that are only a figure, 'six thousand years' and 'a hundred galleys', stay as they are, because finding a figure in a passage is what those questions test.
- **What stops it now.** test.js measures the shortcut on every exam's hand written stated questions each run and fails when it picks the key, and only the key, more often than a blind guess would, one in the number of choices. in `src/test.js`
- **Lesson.** A question whose answer is the passage's own sentence tests finding, not reading. Say the correct answer in other words, and measure what a reader who only matches words would score, because the shortcut is invisible item by item and obvious across a bank.

INC-0194. An LSAT reading choice said 'to particular facts to the particular facts', and the doubled phrase check missed it because the copy added 'the'

2026-09-29, Cosmetic, `c69493cfaaa32e4cf9dad245c4db96d83a4282a7` *PR #207*

- **What was seen.** LC041's first choice, a wrong answer to what the author thinks of the Hand formulation's probability term, read 'an element that courts have in practice ignored when applying the test to particular facts to the particular facts in front of them'. It scored correctly and read as a typo, in the live LSAT trainer, from September 21 to September 29, 2026. In the same set LC036's last choice was grammatical but not what its author wrote: 'from one jurisdiction to the next' was meant to follow 'reached inconsistent verdicts' and landed after 'consistent ones'.
- **Why.** src/mk_bank_lsat_rc2.py lengthened distractors with a loop of its own rather than bank_emit.extend(), and its second table placed six clauses after needles that stop short of the end of their choice, the defect INC-0119 found, so each clause landed after words it restated or did not belong to. #97 repaired four of the six in the bank. LC041's copy put 'the' into the repeated phrase, which the back to back check in test.js cannot see, because it looks for exactly the same words twice; LC036 repeats nothing at all.

- **How it surfaced.** Found on September 29, 2026 while comparing this bank with its generator's output (INC-0193): the line is the same in both. A scan for a two word phrase repeated within four words found 13 answer choices across every bank, 12 of them deliberate parallels such as 'seed sown in April but not seed sown in May'; taking out a, an and the and running the back to back rule again finds LC041 and nothing else, in any field of any item. Replaying the generator's two clause tables listed the six short needles, and reading them found LC036. (Found by reading the code or the output)
- **Fix.** LC041's choice now reads 'an element that courts have in practice ignored when applying the test to the particular facts in front of them', and LC036's clause moves to where its needle ends: 'reached inconsistent verdicts from one jurisdiction to the next until the Hand formulation gave them a method for reaching consistent ones'. Both are wrong answers, and neither change moves its key's length rank.
- **What stops it now.** test.js and build_banks.py run the said twice rule a second time on each text with a, an and the taken out, so a repeat with an article slipped into one copy fails as the exact repeat it is; measured over every bank, it finds this item and no other. The generator that made it no longer writes the bank (INC-0193), and every generator that still writes one appends through extend(), which refuses a short needle. in `src/test.js`
- **Lesson.** A check for text said twice has to allow for the copy not being exact. A seam left by an appended clause can differ from the text it repeats by an article, so take the small words out before comparing, and measure the rule over every bank to see that it finds the defect and nothing else.

INC-0195. Eight LSAT structure explanations ran a claim with a comma of its own into the next words ('where the water has warmed the most and is offered as a reason')

2026-09-29, Cosmetic, `937780969a8498133dea3a317c618485fa9a99b1` *PR #212*

- **What was seen.** The explanation shown after a main conclusion question read, for the fisheries biologist's argument, 'The claim that the decline follows water temperature rather than fishing pressure is drawn from the premise that the trout have declined most in the shallow bays, where the water has warmed the most and is offered as a reason for that conclusion', so the relative clause seemed to take 'and is offered' as its own verb. Six arguments' main conclusion explanations did this, and the transit planner's argument did it twice, once there and once in the explanation of the view it rejects: 'The argument opens with the view that the new rail line should run along the river, where land is cheapest and then rejects it'. Eight items in all, in the live LSAT trainer from September 26, 2026 (#100) to September 29. Keys, choices and stems were right; only the explanation misread.

- **Why.** The explanation templates in src/gen/g_lsat_struct.py follow some slotted claims with more sentence: '%s and is offered as a reason', '%s and then rejects it', '%s and argues that', 'The claim that %s is drawn from'. A claim with a comma clause of its own leaves that clause open, and the template's next words read as part of it. The question stems already avoided this, because asks() moves a claim with a comma to the end of its question, but that rule lived in one function as a comment and was never applied to the explanations. check_args checked each part alone (no capital, no opening pronoun, no full stop), never the sentences the parts are slotted into, so a part that passed it was taken to be safe anywhere.
- **How it surfaced.** Found on September 29, 2026 while reading all 60 items for ten new arguments before they went in: three of the new main conclusion explanations misread the same way. A scan of every structure item in the built bank for a claim with a comma followed by anything but punctuation found 12 items: the 8 above among the 30 live arguments, and 4 among the new ten. (Found by reading the code or the output)
- **Fix.** The main conclusion explanation now reads 'The argument rejects the view that X. Its main conclusion is that Y. From the premise that P, it draws the claim that Q. That claim is offered as a reason for the main conclusion, so it is a step on the way rather than the main point.', and the explanation of the rejected view reads 'The argument opens with a view only to reject it. The view is that X. The argument's own conclusion is that Y.', so every slotted claim is followed by punctuation. A first draft said 'It argues instead that Y', which doubled the word in the transit planner's conclusion ('should follow Market Street instead'); reading the rendered text caught it.
- **What stops it now.** check_args renders every text an argument produces (the stimulus, every question stem, every explanation and every wrong answer reason) and fails the build when a claim with a comma of its own is followed by anything but punctuation. Run against the old templates it names the 12 seams and nothing else; on the new templates it finds none. in `src/gen/g_lsat_struct.py`
- **Lesson.** A template that slots a clause into a sentence has to know where the clause ends. When the slotted text can carry commas of its own, end the sentence after it or set it off with punctuation, and check the rule by rendering every sentence the template makes, since a check on the pieces alone passes every piece of a sentence that misreads.

INC-0199. GRE sentence-function questions offered a second right answer: the third sentence's job, worded so that it was true of the fourth sentence too

2026-10-03, Wrong data shown or stored, `c2f9acf70447cc3eb029ccb04df2ec6e6741dfb7` *PR #219*

- **What was seen.** Each GRE reading passage is six sentences with a fixed job each, and a function question asks what one sentence does, with the other sentences' jobs as wrong answers. One of the two wordings for the third sentence, the first finding, was 'present a finding that tells against the earlier account'. The fourth sentence is the second finding and does exactly that, so on the 31 of 107 fourth-sentence questions that offered this wording as a wrong answer, a student who chose it had a defensible case. The 34 second-sentence questions that offered it carried the same risk in a weaker form, since the second sentence, the fact the earlier account cannot explain, also tells against that account.
- **Why.** The job table was written one sentence at a time, each wording checked against the sentence it describes and never against the others. The two findings share every property but their order, so any wording for either of them that does not say which one it is describes both, and a wrong answer that is also true of the asked sentence is a second key. The duplicate-job check caught two wordings of one job offered together, not one wording true of two jobs.
- **How it surfaced.** Found on October 3, 2026 while reading g_gre_rc.py as the model for generated LSAT structure questions, then counted in the built GRE bank: 65 of its 428 function questions offered the wording beside a sentence it also describes. (Found by reading the code or the output)
- **Fix.** The wording now reads 'present the first of the two findings that tell against the earlier account', which is false of every sentence but the third.
- **What stops it now.** g_gre_rc.check_corpus fails the build unless every wording for either finding says which finding it is (first, second, further, or the one before it), the one property that tells the two apart. in `src/gen/g_gre_rc.py`
- **Lesson.** When wrong answers are drawn from the jobs of the other parts of the same text, each job's wording has to be false of every part but its own, and that has to be checked across the table, not sentence by sentence. Two parts that differ only in order can only be told apart by naming the order, so any wording for either that leaves it out is true of both.

INC-0200. School pages stated averaged and approximate figures as exact: a cohort average as one class's size, and approximate tuition, work experience and scholarships without their qualifier

2026-10-05, Wrong data shown or stored, `bc230b2d898d98f3d5cc3682964f4ea3f04ea649` *PR #224*

- **What was seen.** George Washington's page said 'The 2024-2025 profile reports a class of 41' and 'The class profiled in 2024-2025 has 41 students', and its search description ended 'class of 41', where the school's figure is its average cohort size. Michigan State, Minnesota, Rice and Washington University in St. Louis read the same way about an average or typical cohort, and Louisville, Ohio State and Oklahoma stated as exact a size their schools give as approximately or about. The same pages said UC Riverside's tuition was $59,505 a year where the school prices each quarter at approximately $19,835, said Lehigh's students arrive with an average of 4 years of work experience where the page says more than 4, and the rankings list said 50% of Harvard students get a scholarship averaging $50,000 a year where Harvard says approximately half, at roughly $50,000. Each table showed the school's own note beside the figure, so the tables were right and only the sentences, the search descriptions and the list's scholarship note were not.
- **Why.** Every sentence builder took a figure's value and dropped its note, because the note was treated as provenance for the table and never read for meaning. Class size had a second problem of its own: its phrases were written for a count of one class, so 'a class of 41' and 'has 41 students' turned a size averaged over several cohorts into the size of a single named class. One helper already read the note for 'approximately', but only the whole program cost used it, and it knew one word.
- **How it surfaced.** Found on October 5, 2026 while adding Notre Dame's class size, which Mendoza's page gives as an average cohort size of 85: writing it as 'a class of 85' would have repeated the problem, and counting the library showed eight programs already worded that way. (Found by reading the code or the output)
- **Fix.** One reader takes the qualifier from a figure's own note (about or more than) and the kind of class size (one class, or an average or typical cohort), and every sentence and description that states a class size, tuition, work experience, program cost or scholarship figure goes through it. An averaged cohort size is now its own sentence ('Mendoza College of Business cohorts average 85 students') rather than a count attached to a class. Notre Dame's class size is filled from Mendoza's page as an average cohort of 85.
- **What stops it now.** The build fails if a school page states a qualified class size in any sentence without the qualifier its note carries, and fails if a figure's note carries a qualifier the reader cannot put into words (nearly, at least, up to, fewer than and the like), so a new kind of note stops the build instead of being dropped. in `src/build_rankings.py`
- **Lesson.** A figure's note is part of the figure. When a table prints the note beside the number, every sentence that restates the number has to carry what the note says about it, average, about or more than, or it states the figure more exactly than the source does. And a count of one thing and an average over several are different statistics even when the number is the same: the wording has to say which one it is.

INC-0203. A live admissions post said most full-time MBA programs run three rounds a year, which 21 of the 32 programs whose deadlines we verified do not, and its other claims about rounds cited nothing

2026-10-06, Wrong data shown or stored, `c13caebbded421c36f84b1a7a9438ab334d282d6` *PR #237*

- **What was seen.** MBA Application Timeline: A Month by Month Plan, published August 23, 2026, said 'Most full-time MBA programs run three rounds a year'. The 32 programs whose deadlines were read from their own pages on October 5 and 6, 2026 list two deadlines (Harvard and MIT Sloan), three (11 programs), four (13), five (4) or six (2), so 11 of the 32 run three. The same post said Round 1 gives reapplicants and candidates from heavily represented industries the largest number of open seats, that Round 2 remains a full-strength option at most programs, and, in its FAQ, that Round 3 is the thinnest round with fewer remaining seats and a compressed decision window at most programs. None of the four carried a source.
- **Why.** The post was written before the site had any deadline data, from general knowledge of how admissions rounds work, and the rule that a school statistic needs a source was read as covering a figure for a named school, not a claim about what most programs do. A sentence about most programs is a statistic about programs, but it has no number in it, so neither the blog's price guard nor the weekly source check, which both look for figures, could see it. When the deadline data arrived on October 5, nothing compared it with what the posts already said.
- **How it surfaced.** Found on October 6, 2026 while reading the site's existing posts on rounds before writing the January 14 post on which round to apply in. (Found by reading the code or the output)
- **Fix.** The timeline post's round sentences now say what the deadline pages say: of the 32 programs on the deadlines page, 30 list three or more deadlines and 11 exactly three; Wharton says space in the class becomes more limited for Round 3 applicants, making it a more competitive round; Stanford lists what applying in Round 1 or 2 gives that Round 3 does not. The unsourced claims about reapplicants, heavily represented industries and Round 2 are gone. Every other sentence in the posts that says what most programs or schools do was read against its basis: those our data supports name it on the fact sheet, and those it does not were rewritten.
- **What stops it now.** build_blog.py fails on any sentence in a post or its FAQ that says what most, nearly all or the majority of programs or schools do, unless the fact sheet in EDITORIAL.md lists that sentence with its basis, a count from our data or a source, so a new claim about most programs cannot ship without someone writing down what it rests on. in `src/build_blog.py`
- **Lesson.** A claim about most of a group is a statistic even when it has no number in it, and checks that look for numbers cannot see it. Treat the quantifier words themselves as the thing to check, make each such sentence point at its basis, and when new data arrives, read what the site already says against it, because the old sentences were written without it.

INC-0207. The live GRE format guide told GRE readers our trainer was built for the GMAT Focus Edition specifically, beside a button to the GRE trainer, and credited ETS with a 260 to 340 total ETS does not report

2026-10-06, Wrong data shown or stored, `ef7a4c49bd4ab68eb05bf16d1fc55f1d1464d785` *PR #263*

- **What was seen.** GRE Format 2026: Sections, Timing, and Scoring, dated September 18, 2026, closed its preparation section with 'Our own trainer, Start From Nowhere, is built for the GMAT Focus Edition specifically, but its core habit, returning missed questions on a spaced schedule, is exactly the habit to copy into GRE study.' The GRE trainer at /gre/app/ was live in the repository's first commit, September 19, 2026, and the sentence was already in the post then, so for as long as the post has been in the repository it has told GRE readers to take the product's habit elsewhere while its own Start a Free Round button opened the GRE trainer. Its first paragraph said Verbal and Quant 'are often read together as a 260 to 340 total' and ended '(ETS, 2026)', crediting ETS with the sum INC-0130 took out of the exam data because ETS's page never describes it, and its FAQ said many applicants read the scores that way, with no source. It told readers to confirm the per-section timing and question counts on ETS's site rather than giving them, though ETS's Test Structure page prints them. It made five claims with no source about what test takers or committees usually do or find: GRE quant is generally gentler than GMAT Focus quant, skip-and-return test takers usually find the GRE more comfortable, heavy readers tend to find its verbal questions friendlier than GMAT critical reasoning, spaced review matters even more on the GRE than on most exams, and admissions committees read the two scores separately. And it called the GRE's 44 percent of HBS's Class of 2027 'the largest single group of test submitters', where HBS lists 34 percent for the GMAT and 28 percent for the GMAT 10th edition beside it and says some students submitted more than one test. The GMAT vs GRE post, read for the same sum, called it 'often read as a 260 to 340 total' in its comparison table and carried the same kind of claims: that nearly every major program takes either exam and states no preference (in its first paragraph and its FAQ), that employers in consulting and finance most often expect the GMAT, that GRE math is generally gentler, that heavy readers often find text completion friendlier than critical reasoning, and that GRE submitters were HBS's largest single group, none with a source.
- **Why.** The post describes the product in prose, and the only check on what posts say about the product looks for words that call a live trainer unfinished: in development, coming soon, waitlist, not live (INC-0137, INC-0138). This sentence calls the product finished but single-exam, which none of those words describe, so it passed every build. The 260 to 340 total was removed from data/exams.json under INC-0130 by checking the exam data against its cited pages, and that record noted the site's GRE posts already called the total informal; the format guide's first sentence was never read, and it carries the ETS citation. The claims about what test takers usually find carry no number, so no figure check could see them, and the guard for claims about most programs (INC-0203) looks only at programs and schools, and only for most, nearly all, almost all and the majority of, so 'nearly every major program' passed it.

- **How it surfaced.** Found on October 6, 2026 while reading the live GRE format guide so as not to repeat it in a GRE question types post queued for March 5, 2027. The post was then read against ETS's Test Structure page and HBS's Class of 2027 profile, both read that day, and the whole site was searched for the same product claim (one hit) and for 260 to 340 (four, in two posts). (Found by reading the code or the output)
- **Fix.** The preparation section now says what the GRE trainer is: seven categories, three Verbal and four Quant, and mock sections that use ETS's question counts and times, with a link to /gre/app/. The section table gives ETS's question count and time for each of the five sections, cited to the Test Structure page. The 260 to 340 sum is described as the GRE calculator describes it, arithmetic on two scores that ETS does not report and gives no percentile for, and the ETS citation now covers only what ETS says. The five unsourced claims are gone or replaced by what ETS or HBS says, the HBS sentence gives all three shares as HBS prints them, and the GMAT vs GRE post now says ETS reports no total, quotes HBS on having no preference between the tests instead of claiming nearly every program does, compares the two tests' content and calculators from GMAC's and ETS's pages instead of calling one gentler, and drops the employer claim.
- **What stops it now.** page_checks.stale_sentences, which reads every built page and every post, held or live, now also fails a sentence that says the product as a whole, Start From Nowhere or our trainer, app or platform, is built, designed or made for an exam, is for one specifically or exclusively, or is a one-exam trainer, whenever more than one trainer is live; naming one trainer with its exam ('our GMAT trainer') passes. Its self-check confirms the old sentence fails and that the retake post's 'This is the case our trainer is built for' passes. RETIRED_CLAIMS gains the 260 to 340 sum: a GRE post that names it fails unless the same sentence or table cell says ETS does not report it. The INC-0203 pattern for claims about most programs now also reads nearly every, almost every, virtually every and virtually all, singular program and school, and leading as a qualifier; across every post it found only the GMAT vs GRE sentence. in `src/page_checks.py`
- **Lesson.** A statement about what the product is goes stale each time the product grows, and a check for one way of being stale (in development, coming soon) passes the others (built for one exam specifically). When the product changes, search every page for each way it used to be described, not just the words that said it was unfinished. And when a figure leaves the data because its source never said it, search the prose for it too, because posts repeat the data in their own words and keep the citation.

INC-0208. Posts stated what most test takers, students, applicants and employers do with no source, and one misquoted ACT, because the guard for claims about most of a group read only programs and schools

2026-10-06, Wrong data shown or stored, `cb77e2dd0dc8dfcd7c79c23720634e0b0291f126` *PR #265*

- **What was seen.** A scan on October 6, 2026 of every post for most, many, nearly all and their kin before a word for people found 52 sentences, and one for many programs or schools found 10 more. About two dozen claimed something about a group with no source: some consulting and finance employers most often expect the GMAT, and many programs weigh test scores in merit scholarships (the Executive Assessment guide, the employer line the same one INC-0207 removed from the GMAT vs GRE post); many programs weigh scores in merit aid (the waiver guide, three times); six to twelve weeks covers most test takers (the featured study-length post's description and two sentences, echoed by three other posts); most working professionals can finish a full preparation in two to three months; many first-time test takers find Data Insights the hardest; most full-time MBA students arrive with about five years of experience (a queued post whose own figures are program averages, which say nothing about most students); many schools report both GMAT editions (a queued post; 21 of the 94 programs in our library do); many law schools accept the GRE (ETS lists 128 in the United States); and rhetorical ones such as the point most applicants miss and the half most people skip. One queued post misquoted its source: 'ACT suggests most students retest only two to three times', where ACT's retesting page says that on average it takes students 2 to 3 times to achieve their testing goals. Others were the test makers' own words quoted correctly (LSAC's almost all test takers, College Board's most students get a higher score the second time, ACT's most students can sign in to MyACT), questions and idioms (how many, in so many words), or invented LSAT stimuli. Three more of the same kind used words no pattern reads: students who often prefer the ACT, the four programs applicants benchmark against most often, and most commonly the general MBA is not STEM.
- **Why.** INC-0203 made claims about most programs and schools show their basis, on the principle that a share of a group is a statistic even with no number in it. The pattern it wrote named only programs and schools, and only most, nearly all, almost all and the majority of, so the same sentence about test takers, applicants or employers, or with many, passed every build. The prose was written as a tutor's advice, in the voice of someone who has watched many students prepare, which the site has no basis for: EDITORIAL.md forbids invented anecdotes and credentials, and a claim about what most students do is the same thing in statistical form.
- **How it surfaced.** Found on October 6, 2026 after INC-0207, by scanning the live posts for usually, often, typically, tend to and most or many before a group (93 sentences), then every post, held or live, for the quantifier forms the guard now reads, and reading each hit against its source, our data or the fact sheet. (Found by reading the code or the output)
- **Fix.** Each sentence was rewritten from a source or our data (21 of the 94 programs report both editions; ETS lists 128 U.S. law schools; the programs in our library average about five years of experience), rewritten as advice that claims nothing about a group (ask each program whether scores count toward merit aid; six to twelve weeks is the window the guide plans for, since GMAC's preparation pages refuse automated reads and nothing else backs a claim about most test takers), or, where it quotes the test maker's own words, listed on the fact sheet with that source. The ACT sentence now says what ACT says. The two invented LSAT stimuli are listed as stimuli.

- **What stops it now.** build_blog.py's claims check now also reads most, many, nearly all, almost all, virtually all and the majority of before applicants, students, test takers, candidates, people, adults, employers and the like, and many before programs and schools, skipping how many, as many, so many and too many; each such sentence, in a post's body or FAQ, held or live, fails the build unless EDITORIAL.md lists it with its basis. Its self-check covers each group word, the skipped idioms, and a backed sentence. in `src/build_blog.py`
- **Lesson.** When a guard encodes a principle, write the pattern for the principle rather than for the first case it caught: a check for 'most programs' that passes 'most applicants' teaches writers that the rule is about programs. List every group the content talks about and test the pattern on each. And read each quoted claim against its source as written, because a paraphrase that adds 'most' or drops 'on average' changes what the source said while keeping its name on it.

INC-0209. The GRE trainer served about 195 trigonometry questions, though ETS says the Quantitative measure does not include trigonometry

2026-10-06, Wrong data shown or stored, `5f2897745bfb1a365dbf1b8f179846f8233228d4` *PR #266*

- **What was seen.** The GRE trainer's Geometry category held generated questions such as 'In right triangle ABC, the right angle is at C. The side opposite angle A has length 55, the side adjacent to angle A has length 48, and the hypotenuse has length 73. What is the tangent of angle A?': 2 in the starter bank and 193 in the deferred bank on October 6, 2026. ETS's overview of the Quantitative Reasoning measure says its content is high school mathematics and statistics at a level generally no higher than a second course in algebra, and 'It doesn't include trigonometry, calculus or other higher-level mathematics'. EDITORIAL.md's GRE fact line already said so. A GRE student was being drilled on content the test does not have, inside the category whose accuracy feeds the Geometry rating.
- **Why.** The GRE's quantitative questions are SAT schemas remapped by gen/mapping.py, whose rule is that a schema appears under the category whose published framework names that content. GRE_MAP's Geometry list was copied from the SAT's Geometry and Trigonometry domain, and sat_geo_trig came with it; the ACT list beside it, where trigonometry is in scope, has the same members. Nothing compared a map against what each test maker says its exam excludes, and nothing scanned a built bank for out of scope content, so the rule lived only in a comment.
- **How it surfaced.** Found on October 6, 2026 while reading generated GRE quantitative stems to match their style for a new Quantitative Comparison generator: the fourth sample printed asked for the tangent of an angle. (Found by reading the code or the output)
- **Fix.** sat_geo_trig is out of GRE_MAP, so the GRE Geometry category is filled by the schemas that remain and the new comparison schemas, and the trigonometry items leave the bank.

- **What stops it now.** gen/mapping.py now holds the content each test maker excludes, with the source's words (ETS for the GRE: no trigonometry or calculus; GMAC for GMAT Focus: algebraic and arithmetic knowledge, so no geometry schema), and the map build fails if any exam's map names an excluded schema. test.js scans every GRE quantitative item, hand written and generated, for trigonometric functions (sine, cosine, tangent of an angle) and fails on one. in `src/gen/mapping.py`
- **Lesson.** A rule that only a comment states is not enforced. When content is reused across exams by mapping, write down what each exam excludes, in the test maker's words, and check every map and every built bank against that list, because a list copied from the exam that does include the topic carries it over without anyone choosing to.

INC-0214. The GMAT reading guide told students what most people do with Verbal time, with nothing behind it, because the claims check covered blog posts only

2026-10-19, Wrong data shown or stored

- **What was seen.** The live study guide page /guide/gmat/verbal/rc-how-to-read/ opened its case with 'Most people lose Verbal time in the passage rather than the questions', followed by an unsourced 'about forty seconds' for reading a passage. Neither is something GMAC or any source we hold says; it is a claim about what most test takers do, the exact kind INC-0208 removed from the blog.
- **Why.** INC-0203 and INC-0208 built the check for claims about what most of a group does into build_blog.py, which reads posts only. The study guide is written as Python topics in src/guide/ and rendered by build_guide.py, which ran its own completeness checks but never the claims check, so the rule held in one place that publishes prose and not in the other. Task #181 had been opened to extend it and was not yet done.
- **How it surfaced.** Found on October 19, 2026 by running the blog's claims patterns over every authored field of the 120 guide topics, as task #181 set out to do; two hits, one of them this sentence and the other a worked step quoting an example argument. (Found by reading the code or the output)
- **Fix.** The sentence is rewritten as advice with no claim about what most people do and no unsourced timing. build_guide.py now runs the blog's GENERAL and GROUP patterns over every field of every topic that we write ourselves (title, idea, why, formula explanations, traps, worked steps and their why), with quoted text set aside so a step that quotes an example argument's 'many people' is not read as our claim.
- **What stops it now.** build_guide.py fails the build when any authored field of a guide topic says what most of a group does, naming the topic and field; the example questions and bank items on the ladder are not scanned, because they are fiction by design. in `src/build_guide.py`
- **Lesson.** A content rule enforced by one renderer is a rule about that renderer, not about the site. When the same kind of prose is published by more than one builder, the check belongs where every builder can call it, and a new builder should start by asking which existing content checks it skips.

Tests and guards (38)

INC-0016. The performance test waited for the load event, which waits for the thing being optimised

2026-09-19, Silent loss, `4b25169` *PR #45*

- **What was seen.** A change that made the page usable nine times faster would have reported no improvement.
- **Why.** smoke_load waited for the load event. The load event waits for the async bank, which is precisely the part the fix moved off the critical path.
- **How it surfaced.** The number refused to move when it obviously should have. (Found by measuring something)
- **Fix.** Wait for the first question to be answerable.
- **What stops it now.** smoke_load waits on the user-visible milestone in `src/smoke_load.js`
- **Cost.** would have hidden the entire win
- **Lesson.** Measure the moment the user can act, not a browser lifecycle event. A test that measures the wrong instant is worse than no test, because it produces a number people trust.

INC-0017. A build guard silently narrowed its own scope when the counts grew

2026-09-19, Silent loss, `4b25169` *PR #45*

- **What was seen.** The item-count guard kept passing while checking nothing.
- **Why.** The guard matched \d{2,4}. When the banks passed ten thousand, the two largest counts stopped matching the pattern and fell out of the check.
- **How it surfaced.** Reading the guard while raising the counts. (Found by reading the code or the output)
- **Fix.** Match any run of digits.
- **What stops it now.** the guard asserts how many counts it checked in `src/build.py`
- **Cost.** the two largest banks unchecked
- **Lesson.** A regex with a length bound is a guard with an expiry date. Assert the number of things checked, not only that the checks passed.

INC-0019. The audit reported a working link as broken

2026-09-19, Cosmetic, `a34a798` *PR #47*

- **What was seen.** A false positive in the launch audit.
- **Why.** Relative link resolution in the audit did not match the browser's.

- **How it surfaced.** Checking a reported failure by hand. (Found by reading the code or the output)
- **Fix.** Resolve links the way the browser does.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** trust in the audit
- **Lesson.** Check your checker. A tool that cries wolf gets muted, and then it is worse than nothing.

INC-0021. A test passed because nothing was configured at all

2026-09-19, Silent loss, `489363b` *PR #46*

- **What was seen.** smoke_funnel's 'nothing sent on refusal' assertion passed. It would have passed with the feature deleted.
- **Why.** The test used storage key sfn_consent instead of sfn_consent_v1, so consent was never set up, so nothing was ever sent, so the assertion held vacuously.
- **How it surfaced.** Fixing the key, which immediately exposed a real bug underneath. (Found by reading the code or the output)
- **Fix.** Use the real key, and assert the positive case as well as the negative.
- **What stops it now.** every 'nothing happens' assertion is paired with a 'something happens' assertion in `src/smoke_funnel.js`
- **Cost.** a real bug hidden behind a green check
- **Lesson.** A negative assertion passes when the system is broken in the right way. Always pair it with the positive case, or it is testing nothing.

INC-0026. I reported a regression that was seed noise

2026-09-21, Cosmetic, `c63654a` *PR #56*

- **What was seen.** I reported that SAT band coverage fell from 88 to 72 percent after an engine change.
- **Why.** I compared a single seed before and after. Across seeds 3, 7 and 11 the baseline was 52, 88 and 80 against 72, 80 and 72 for the change. The spread between seeds was larger than the effect.
- **How it surfaced.** Running more seeds because the number looked too clean. (Found by measuring something)
- **Fix.** Report across seeds, never one.
- **What stops it now.** review_bot runs multiple seeds and reports the spread in `src/review_bot.js`
- **Cost.** a false regression report, corrected in the same session
- **Lesson.** A stochastic measurement on one seed is an anecdote. If your system has randomness, a single-run before-and-after cannot distinguish a change from the weather.

INC-0030. The theme parity test matched by name prefix and caught a font size

2026-09-21, Cosmetic, `8af7188` *PR #61*

- **What was seen.** The parity guard flagged --text-xs, which is a length, not a colour.
- **Why.** Token selection was by name prefix rather than by whether the value is a colour.
- **How it surfaced.** Reading the failure. (Found by reading the code or the output)
- **Fix.** Select tokens by inspecting the value.
- **What stops it now.** smoke_charts selects by value, not name in `src/smoke_charts.js`
- **Cost.** a false failure
- **Lesson.** Classify by what a thing is, not by what it is called. Naming conventions are a hint, never a type.

INC-0031. The browser suites had never actually run, because npm install had not

2026-09-21, Silent loss, `8af7188` *PR #61*

- **What was seen.** Browser tests appeared to be part of the suite and were not being executed.
- **Why.** playwright was listed as a devDependency and node_modules did not exist in the environment.
- **How it surfaced.** Running them by hand and getting a module-not-found. (Found by measuring something)
- **Fix.** Add npm install and playwright install to CI, and check in package.json.
- **What stops it now.** CI runs npm run test:browser in `.github/workflows/ci.yml`
- **Cost.** an unknown number of regressions unnoticed
- **Lesson.** A test that is not wired into CI is a test that does not exist. Prove a suite runs in the place it is supposed to run, not on your machine.

INC-0033. The dashboard test measured a hidden element and passed

2026-09-21, Silent loss, `37241958be4dadca750f76742818b18b88e5a5b1` *PR #62*

- **What was seen.** Layout and contrast assertions passed on a view that was never displayed.
- **Why.** The admin view carries a hidden class. A display:none element still answers innerText and getComputedStyle, so every assertion held on a box that had never been laid out.
- **How it surfaced.** Screenshotting the result and seeing the dashboard was not in it. (Found by rendering it and looking)
- **Fix.** Call show('admin') and dismiss the onboarding overlay before measuring.
- **What stops it now.** smoke_business asserts the view is on screen and taller than 400px before measuring it in `src/smoke_business.js`
- **Cost.** a layout suite that tested nothing, caught the same day

- **Lesson.** Before you measure a layout, assert the thing is rendered. Hidden elements answer most DOM questions, and they answer them wrongly.

INC-0046. The error queue was 81 percent other people's blocked scripts

2026-09-21, Degraded, `c4698e0` *PR #52*

- **What was seen.** 1,202 of the first 1,490 rows in the error table were one third-party beacon being blocked by ad blockers.
- **Why.** The handler captured every error on the page, including ones from scripts the site does not control and cannot fix.
- **How it surfaced.** Clustering the rows and looking at the largest cluster. (Found by measuring something)
- **Fix.** Filter third-party script errors, and give the triage queue a permanent noise label.
- **What stops it now.** the triage queue mutes a cluster marked noise in `src/app_template.html`
- **Cost.** a monitoring surface that was 81 percent noise
- **Lesson.** An error feed with no filter is a feed nobody reads. Signal has to be defended, and the cheapest defence is a human label that mutes permanently, so the queue gets quieter as it learns.

INC-0053. The playbook's own citation guard failed CI on its first run

2026-09-21, Cosmetic, `37241958be4dadca750f76742818b18b88e5a5b1` *PR #62*

- **What was seen.** CI reported all 37 cited commits as nonexistent, with 'fatal: Not a valid object name' for every one.
- **Why.** The checkout action shallow-clones by default, so no historical commit is present in the CI working copy. The guard was correct and its environment assumption was not.
- **How it surfaced.** The first CI run of the guard. (A test caught it)
- **Fix.** fetch-depth: 0 on the checkout so the history is actually there to check.
- **What stops it now.** fetch-depth: 0 in the workflow in `.github/workflows/ci.yml`
- **Cost.** one red CI run
- **Lesson.** A guard that reads the repository needs the repository. CI checkouts are shallow by default, and anything that walks history, blames a line or resolves an old hash will fail in a way that looks like the data is wrong rather than the clone.

INC-0054. The first fix asked the wrong question and muted a working check

2026-09-21, Silent loss, `37241958be4dadca750f76742818b18b88e5a5b1` *PR #62*

- **What was seen.** After guarding on 'is this a shallow clone', the citation check stopped running locally, where it had been working perfectly and verifying all 52 citations.
- **Why.** This clone is shallow and still holds every commit the ledger cites, which is the normal case. Shallowness does not imply missing history, so the guard disabled itself in the one environment where it was useful.
- **How it surfaced.** Reading the output after the fix and noticing a check had turned into a skip. (Found by reading the code or the output)
- **Fix.** Ask whether ANY citation resolved. None at all means the history is absent; some means those particular citations are wrong.
- **What stops it now.** the check degrades only when zero citations resolve in `src/smoke_playbook.js`
- **Cost.** a working check disabled for the length of one edit
- **Lesson.** When you add a condition that skips a check, make sure it describes the failure and not something merely correlated with it. A skip is indistinguishable from a pass in the output, so the fix for a noisy check can silently delete it.

INC-0055. A new browser suite hardcoded this machine's browser directory and crashed in CI

2026-09-21, Cosmetic, `37241958be4dadca750f76742818b18b88e5a5b1` *PR #62*

- **What was seen.** ENOENT scandir /opt/pw-browsers on the first CI run of the dashboard suite, after every other suite had passed.
- **Why.** The suite read a browser directory that exists in the development sandbox and not on a CI runner, where Playwright installs browsers in its own location. The older suites pass process.env.CHROMIUM_PATH straight through, which is undefined in CI and correctly means 'you decide'.
- **How it surfaced.** The first CI run of the suite. (A test caught it)
- **Fix.** src/chromium_path.js, one shared resolver: the environment variable, then the sandbox directory if it exists, then undefined so Playwright resolves its own.
- **What stops it now.** one shared resolver, so the logic cannot differ between suites in `src/chromium_path.js`
- **Cost.** one red CI run
- **Lesson.** A path that exists on the machine you wrote the test on is not a path. Resolve environment-specific locations through one helper that falls back to the tool's own default, and return undefined rather than an empty string, because undefined means 'you decide' and an empty string means 'launch nothing'.

INC-0057. The ledger cited commits that squash merging destroys

2026-09-21, Silent loss, `724ef0ec90a8344f0a17530450d13c792b71e573` *PR #63*

- **What was seen.** CI failed with three assertions at once: six citations unresolvable, the commit count wrong, and the HTML naming the wrong build commit.
- **Why.** One cause behind all three. Each incident cites the commit that fixed it, and this repository squash merges, so a branch commit ceases to exist the moment its pull request lands. The builder treated an unresolvable citation as fatal, so it refused to rebuild; build.py catches that failure as a warning rather than stopping the deploy, so the stale committed artefacts stayed on disk and the two freshness assertions then failed against them.
- **How it surfaced.** The first CI run after a squash merge, which is the first moment the problem could exist. (A test caught it)
- **Fix.** Cite the pull request as the durable reference and treat the commit hash as best effort. A citation that no longer resolves is reported and allowed when the record carries a PR number, and is still fatal when it does not.
- **What stops it now.** the builder and smoke_playbook both distinguish a squashed commit from a wrong one in `src/build_playbook.py`
- **Cost.** one red CI run, and a document that silently stopped rebuilding
- **Lesson.** A commit hash is not a durable citation in a repository that squashes. Pull requests, issues and tags survive history rewriting; branch commits do not. Cite the thing that outlives the merge, and make any check of the other one advisory.

INC-0061. A repeat metric that gets worse when the bank gets better

2026-09-22, Cosmetic, `de67a0eaeab023593685c7e5f5a0ffee944d3599` *PR #65*

- **What was seen.** Doubling the LSAT reading bank cut the repeats a student actually sees by 46 percent, from 1875 to 1006, and the bot's avoidable repeats figure went up, from 258 to 309.
- **Why.** A repeat counts as avoidable when the student has not yet exhausted that section's pool. Enlarging the pool keeps that condition true for longer in every sitting, so the window in which a repeat is classified avoidable widens with the pool. Measured directly, only 5 of 1300 repeats in a single long run were genuinely avoidable, and both pools were exhausted.
- **How it surfaced.** The number moved the wrong way after a change that should only have helped, so the metric was read rather than trusted. (Found by measuring something)
- **Fix.** Report the repeat rate per item served alongside the avoidable count, so the student-facing number is visible next to the diagnostic one. The avoidable definition is left alone rather than quietly redefined to make a number look better.
- **What stops it now.** the bot prints repeats per item served next to the avoidable count in `src/review_bot.js`
- **Cost.** nearly a false regression, and a standing hazard for whoever reads it next

- **Lesson.** A metric that moves against you when the product improves will eventually be used to justify reverting an improvement. When a number goes the wrong way after a change that should only have helped, measure the underlying thing directly before believing either the number or your own explanation of it. Never redefine the metric in the same change that made it look bad.

INC-0065. The analysis chapter called one file eight failing guards

2026-09-22, Wrong data shown or stored, `a91b158cb151cd88b19421a525e406d568f58ee3` *PR #68*

- **What was seen.** The generated chapter reported src/build.py as a guard named by eight incidents, under a heading saying a guard named twice is one that did not hold. build.py contains dozens of unrelated guards and most of those eight are different ones.
- **Why.** The recurrence grouping keys on guard_file, which is the file a guard lives in, not the guard. A file that is the natural home for many checks therefore looks like a single check failing repeatedly, and the chapter states that reading in the heading as though it were established.
- **How it surfaced.** Reading the generated chapter after a build, to see whether it had picked up a genuine recurrence recorded minutes earlier. (Found by reading the code or the output)
- **Fix.** Group by the guard description, which names the mechanism, and report file level clustering separately with wording that says what it actually means.
- **What stops it now.** the analysis chapter groups recurrence by guard rather than by file in `src/build_playbook.py`
- **Cost.** a generated chapter making a confident and wrong claim about where to spend effort
- **Lesson.** An aggregate is a claim about whatever you grouped by. Group by the file and you have measured the file. State the grouping in the sentence that reports the result, and the overclaim becomes visible while you are writing it.

INC-0067. The browser path fix covered two suites and three others kept crashing

2026-09-22, Silent loss, `1fd4e0529fe06171c90528558b16358e25b71c6a` *PR #69*

- **What was seen.** smoke_items.js, smoke_consent.js and smoke_billing.js all died at launch with 'Executable doesn't exist at /opt/pw-browsers/chromium_headless_shell-1243/...'. The sandbox holds chromium-1194 and chromium_headless_shell-1194, so Playwright's own resolved path pointed at a build that is not installed. CI passed throughout, because a CI runner installs the browser Playwright expects.

- **Why.** INC-0055 was the same failure in smoke_business.js, and the fix was src/chromium_path.js, a shared resolver that prefers CHROMIUM_PATH, falls back to the newest /opt/pw-browsers/chromium-*, and otherwise returns undefined so Playwright decides. Two suites were changed to use it, smoke_business and smoke_fit. The three older suites were left reading process.env.CHROMIUM_PATH directly, which with the variable unset passes executablePath: undefined and hands the decision back to Playwright, which is exactly the case the resolver exists to override. A shared module only helps the callers that call it.
- **How it surfaced.** Running the three suites after a bank change, as the pre push checklist requires. They had not been run in this sandbox since the resolver was introduced. (A test caught it)
- **Fix.** All ten browser suites and playbook_pdf.js now call chromiumPath() from src/chromium_path.js. A check in src/build.py fails the build if any file that launches Playwright does not, and on its first run it named six suites beyond the three the failure had surfaced, plus four more that a grep written from its output then found.
- **What stops it now.** the build refuses a Playwright launch that reads CHROMIUM_PATH instead of calling chromiumPath() in `src/build.py`
- **Lesson.** Extracting a shared helper does not migrate the callers. The extraction fixes the file it was extracted from and leaves every sibling on the old path, which is INC-0059 and INC-0064 in a different costume: a correction applied to the instances in hand rather than to the pattern. Three suites had failed visibly and ten were wrong; the seven silent ones were found by the guard, not by reading. When a helper exists because a direct call was wrong, make the direct call fail the build, and let it enumerate the callers rather than enumerating them by hand.

INC-0077. A review bot check whose verdict was three coin flips warned on an unrelated bank change

2026-09-22, Cosmetic, `2c72da1fde8a3819ca9bda7eabe9fb75ac0754b7` *PR #71*

- **What was seen.** Widening two GMAT critical reasoning corpora turned the bot's planted weakness check from ok to warn: 2 of 3 sections correctly identified as weakest. Nothing about the diagnosis had changed. The bot threads one seeded generator through every check in an exam, so more items meant the earlier checks consumed a different number of draws and the planted weakness test started from a different point in the stream.
- **Why.** The check sat one simulated student per section, three sittings in total, and reported a pass only if all three came out right. Measured over 60 independent seeds the diagnosis is right on 179 of 180 sittings, so at three sittings the check warns on roughly three percent of seeds by sampling alone. Every other check in the bot already averages over 25 sittings; this one did not, and its verdict was therefore a property of the seed as much as of the engine.
- **How it surfaced.** The warning appeared on a change that could not plausibly have caused it, so the check was measured across seeds instead of being believed or dismissed. (Found by measuring something)

- **Fix.** The check sits five students per section and reports the rate, passing at 90 percent or better and failing below half. The threshold is stated against the measured 99 percent so the margin is visible rather than implied.
- **What stops it now.** the planted weakness check reports a rate over 15 sittings rather than a verdict over 3 in `src/review_bot.js`
- **Lesson.** A check that reports pass or fail from a handful of random draws is a check that will flip on work that has nothing to do with it, and the cost is not the false alarm. It is that the next real alarm arrives in a tool people have learned to shrug at. Before believing or dismissing a warning, run the thing it measures enough times to know its rate: that answers both whether this alarm is real and whether the check is worth keeping in its current form. Here the answer was that the engine was fine and the check was wrong, and both were worth knowing.

INC-0089. A ratchet that trips on sampling noise gets re-recorded rather than read

2026-09-22, Degraded, `9056b8daa4fe87508a82591b79adb7b8a9fd181f` *PR #82*

- **What was seen.** Widening one geometry schema from 20 items to 96 made six sibling schemas fail their recorded bias figures, every one of them by between 1 and 4 points: 55 to 57, 45 to 46, 39 to 41, 44 to 45, 36 to 40, 53 to 54. None of those schemas changed. The same thing happened one change earlier, in the other direction, when two entries cleared and two got worse by 2. Each time the response is to re-measure and re-record, which is a ratchet being rewritten rather than read.
- **Why.** A recorded figure is a property of the schema AND of the mix it was measured in, and the mix is a shared resource: a category stops at TARGET, so growing one schema shrinks its neighbours and changes which items the shared dedup set had already taken when they ran. The check then compares a fresh measurement against a figure recorded under a different mix, with no allowance for the sampling error at that size, so a difference well inside noise reads as a regression.
- **How it surfaced.** Doing the same re-recording twice in two changes and noticing that every delta was between 1 and 4 points while the schemas themselves were untouched. (Found by reading the code or the output)
- **Fix.** Every ceiling in the check is now either a line someone chose or an error bar, and only the error bar stands on its own. The policy cap and any recorded figure are lines, so a measurement fails against them only when it is over by more than the standard error of a proportion at the size it was measured at, which is about two points on a bank of five hundred and nowhere near the thirty two points the worst real tell showed. The small sample floor already is two and a half standard errors and takes no second allowance. The staleness check mirrors it: an entry is called unnecessary only when the measurement sits clear of the cap by that same margin rather than a point under it. Nothing is re-recorded for drift, so the figures stay the ones a person chose. Where the drift can be removed instead of tolerated it is: the starvation check added alongside this one measures each schema on its own fixed seeded draws rather than on whatever the bank build happened to ask of it, so its figures are exact and need no allowance at all.

- **What stops it now.** check_bias compares against recorded figures with the sampling error at the measured size, so mix drift does not fail the build and a real regression still does in `src/build_banks.py`
- **Lesson.** A ratchet is only read while it is quiet. One that fires on noise gets re-recorded as a reflex, and the re-recording is indistinguishable from accepting a real regression, so the mechanism that exists to catch regressions becomes the mechanism that launders them. Give a comparison the same error bars as the measurement it is made from, and fix the thing that fires wrongly before the habit of clearing it forms.

INC-0110. A browser suite outside CI had failed on its first click since the consent dialog shipped

2026-09-26, Degraded, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** node src/smoke_pages.js, the suite for /international/ and /apply/, timed out on 2026-09-26 trying to tick the international checkbox: the consent dialog, which is modal, sat over the page and intercepted the click. Everything the suite exists to check after that point (checklist persistence, the shortlist carried from /schools/, CSV export, overflow at phone width) had not been checked since the dialog arrived.
- **Why.** The suite was written before the consent dialog and loads pages with empty storage, which is exactly the state in which the dialog appears. Other suites pre-set a consent choice; this one was never updated. It is not in ci.yml, so nothing ran it on a schedule and its failure was invisible.
- **How it surfaced.** Found on 2026-09-26 by running the browser suites that touch the pages changed in this session, before committing. (A test caught it)
- **Fix.** The suite sets a consent choice before any page loads, the way a returning visitor arrives, and passes again end to end.
- **What stops it now.** The pattern is now in the suite itself with the reason written beside it. The larger gap stays open and is named here: suites outside ci.yml only run when someone remembers them.
- **Lesson.** Run every browser suite when a site-wide element such as a modal ships, because a test nobody runs is a claim about the past. Every suite that loads a page with empty storage meets the new element before anything it was written to check.

INC-0140. The source check read a page's 2026-27, written with an en dash, as 2026 and 27, so every figure we labelled 2026-27 looked unsupported

2026-09-27, Cosmetic, `214f8da3401b8186dff51a54ce9bbff918edecf8` *PR #127*

- **What was seen.** check_sources.py --schools reported the year 2027 as missing from Indiana Kelley's and Wisconsin's fee pages, which print their figures and the year 2026-27 with an en dash between the years. Our facts write 2026-27 with a hyphen. The check expanded our span into 2026 and 2027 but left the page's span as 2026 and 27, so a figure matched word for word was reported as unsupported in the weekly issue.
- **Why.** SPAN, the pattern that turns a year range into both years, accepted a hyphen or a slash between the years and not an en dash. Our data never contains an en dash, by house rule, but the pages we cite use one all the time, so the two sides of the comparison were normalised differently.
- **How it surfaced.** Found on September 27, 2026 while reading the tuition flags from the school source check: two of them flagged only the year, on pages that plainly print it. (Found by reading the code or the output)
- **Fix.** SPAN accepts an en dash as well as a hyphen or a slash between the years, so a page's range reads as 2026 and 2027 like ours.
- **What stops it now.** check_sources.py checks its own normaliser before it reads a page: a year range written with a hyphen, a slash or an en dash must read as both years, or the run stops. in `src/check_sources.py`
- **Lesson.** When you compare text you wrote against text someone else wrote, normalise both sides with the same rules, and remember the other side does not follow your house style: a comparison is only as fair as its most lenient reading of the source.

INC-0146. The source check's browser read returned Stanford's class profile without a single figure, because the figures are drawn in an embedded chart that renders only when scrolled into view

2026-09-27, Degraded, `b295d09de941a53b84e3d5154dcd62ed87e0ac6a` *PR #135*

- **What was seen.** check_sources.py --render exists to read pages built by JavaScript, and the weekly audit runs it over the exam guides and the school library. On Stanford GSB's class profile it timed out after 60 seconds, and the check fell back to the page as served. With the timeout avoided it still returned only the page's prose, 2,341 characters of navigation and introduction with no 3.76 average GPA, although a person reading the same page sees the figure.
- **Why.** render() waited for networkidle, which a page that keeps sending analytics requests never reaches, so Stanford's page timed out. It then read document.body.innerText of the top frame only and never scrolled. Stanford draws its class profile in an Infogram chart embedded as an iframe from e.infogram.com, and the chart writes its numbers only once it is scrolled into view: loaded without scrolling, the frame exists and holds no 3.76; scrolled, it does. The top frame's text never holds the figures at all.

- **How it surfaced.** Found on September 27, 2026 while checking Stanford GSB's average GPA before quoting it in a blog post. The sandbox's browser certificate store had been empty, so no rendered read had run here before; once the proxy's CA was added to it, Stanford's page loaded, and three runs separated the causes: without scrolling neither the page nor its frames held 3.76, and with scrolling the Infogram frame did. (Found by reading the code or the output)
- **Fix.** The browser read moves to src/render_page.js, which check_sources.py calls. It waits for the page's own content, gives the network at most ten seconds to go quiet instead of requiring it, scrolls to the bottom in steps so lazy content draws, and returns the text of every frame a reader can see, the page's own first. A hidden frame is skipped, because the text of a document that is not rendered is its raw source: on Wharton's page one hidden Infogram frame gave more than 21,000 characters of chart configuration. It refuses a response that is not HTML, since a PDF shown in Chromium's viewer held a read of a school's PDF for more than three minutes with nothing to show, and it has one deadline for the whole read so that a stalled page ends with the browser closed. check_sources.py also keeps the browser's read only when it shows at least as many of the cited figures as the page as served, so a render that comes back as a challenge page cannot hide figures the served page printed.
- **What stops it now.** src/smoke_render.js serves fixtures from a local server: a page that requests the network every 200 ms, embeds far below the fold an iframe that writes its figure only when an IntersectionObserver sees it, and hides a second iframe whose script holds a decoy figure. The smoke fails unless render_page.js returns the figure, the page's own text first, and not the decoy; it also serves a PDF, which must be refused at once, and runs one read past a short deadline, which must end with an error. It runs in npm run test:browser, so CI runs it; with networkidle restored it times out, and with hidden frames read it returns the decoy. in `src/smoke_render.js`
- **Lesson.** Loading a page is not the same as reading it. A chart can be a separate document inside the page that draws only when scrolled into view, and a page with trackers may never fall quiet, so a reader built for checking has to scroll like a person, collect every frame, and use waits that give up rather than hang.

INC-0152. The source check read text inside HTML comments as page text, and confirmed Arizona State's 43 percent women from a table row the school had commented out

2026-09-27, Wrong data shown or stored, `914cd0ddb6818c683e36ec05dae84f5b340122c7` *PR #143*

- **What was seen.** data/schools/arizona-state-carey.json gave women_pct 43, "female, incoming class of fall 2025", cited to W. P. Carey's class profile page, and check_sources.py --schools counted it as printed there. The page a reader sees has no gender figure at all: its class profile lists applications, admits, class size, age, GMAT, GRE, GPA, work experience, international share and majors. The 43 is in a row the school wrapped in an HTML comment, "<!-- ... Male 57% ... Female 43% -->". ASU's school page showed it, and the November 7 post on women in MBA classes, queued but not yet published, listed ASU at 43 percent.

- **Why.** to_text() in check_sources.py removes script, style and noscript elements, then every tag, and keeps the text between them. An HTML comment is not an element: the tag pattern removed only its opening up to the first >, and every word and number inside it survived as page text, followed by a stray "-->". A page that hides old figures in a comment therefore confirms them. Rice's class profile does the same with a Class of 23, 24, 25 block. The browser read (INC-0146) never had this gap, because innerText holds only rendered text; the check prefers the page as served whenever it finds as many figures.
- **How it surfaced.** Found on September 27, 2026 while moving Rice to its three-year averages: its page as served carried blocks ending in a literal "-->", which showed comment text reaching the check. Rerunning both checks with comments removed changed exactly one result: ASU's women share went from found to not in the source. The exam check's 79 facts came out the same. (Found by reading the code or the output)
- **Fix.** to_text() drops every HTML comment before reading tags. ASU's women share is empty, with the commented-out row and the reason in a note, and the queued November 7 post and its fact block drop ASU: 35 programs, the middle class still 43 percent women.
- **What stops it now.** check_sources.py's self-check holds a page whose only copy of a figure sits inside a comment, and fails if that figure is read as printed; it also holds the same figure outside the comment, which must still be found. in `src/check_sources.py`
- **Lesson.** A checker that reads a page has to read the page a person sees, not the file behind it. Markup can hold text no reader is shown, in comments, hidden elements and templates, and a site that retires a figure often leaves it there.

INC-0156. The source check read mba.com's bot challenge page as the page it cites and reported the GMAT fact's numbers as missing from it

2026-09-27, Degraded, `58c9567d9d403a8a70a268cea4209520fc1d095a` *PR #148*

- **What was seen.** After INC-0155 moved the free score report fact to mba.com's Sending Your Score page, python3 src/check_sources.py --render reported "5: not in the source" and "48: not in the source" against it. The page prints both. The text the check had read and cached for it was 726 characters beginning "www.mba.com - Additional security check is required Why am I seeing this page? The website you are visiting is protected and accelerated by Imperva": the challenge page, not the page. A browser read of the same URL minutes earlier returned the page itself, rule included.
- **Why.** check_sources.py treats a read as the page when it holds at least 400 characters (MIN_TEXT), a floor set to catch empty shells of pages built by JavaScript. A bot challenge is a page of its own, with a paragraph of explanation, and Imperva's is 726 characters, so it passed the floor and every number in the fact was reported as missing. It was also cached, so a later run with --cache reused the challenge. Imperva challenges mba.com's pages on some reads and not others, so the same fact could be confirmed, reported missing or reported unreadable from one run to the next.

- **How it surfaced.** Found on September 27, 2026 while confirming INC-0155's fix: the check reported both numbers missing from a page just read in a browser with both on it, and the cached text was Imperva's challenge. (Found by reading the code or the output)
- **Fix.** fetch() recognises a challenge page by its wording (Imperva's security check, Cloudflare's Just a moment and Attention Required, Verify you are human and the like) when the read is short, returns nothing for it and never caches it, so the browser read is tried and, if that is challenged too, the source is reported unreadable as a bot challenge rather than as a page missing the figures.
- **What stops it now.** check_sources.py's self-check holds Imperva's and Cloudflare's challenge text, which must be recognised, and a long page that mentions a security check in passing, which must not be. in `src/check_sources.py`
- **Lesson.** A checker has to know when it has not read its source. A response is not the page because it has text in it: a challenge, an error page or a login wall reads as a page with none of the facts on it, and every fact then looks wrong.

INC-0158. The source check filed six school pages as probably built by JavaScript; five were something else, and three of them showed newer figures than the library published

2026-09-27, Wrong data shown or stored, `f16c2b0ea0bb816c3d422465136e05477da63995` *PR #152*

- **What was seen.** Each python3 src/check_sources.py --schools run on September 27, 2026 ended with pages reported as "shows none of its N figures, probably built by JavaScript", nine in the day's first runs and six in the later ones, and ROADMAP.md explained the six as pages drawn by JavaScript or as images without alt text. Reading each page on September 27 found one that fitted. Kelley's class profile is a plain table that now describes the Class of 2028 (57 students, average GMAT 618, GPA 3.38) while the library published the Class of 2027 (106, 607, 3.48). Poets&Quants' Maryland Smith profile, updated September 25, 2026, prints female 36% and international 47% where the library published 35 and 68 cited to it, and a full-time enrollment of 115 where the library published a class size of 69. Auburn's page draws its cohort statistics with counters that show 0 until they animate, inside a closed accordion, and their values now read GPA 3.47 and 1.81 years of work experience against the library's 3.49 and 2.73, with no international share at all where the library published 19 percent. UMass Amherst's PDF prints all three of its figures glued to their labels ("STUDENTS39%", "GPA3.45"), and it is a file created in August 2025 that the library dated 2023 from its folder name. Chicago Booth's employment statistics are an Excel workbook, which the check decoded as text. Only WashU Olin's page fitted the explanation: its figures are drawn as images, whose alt text, where there is any, names the chart and none of its figures.

- **Why.** The check sets a page aside when it prints none of the two or more figures cited to it, because a page drawn by JavaScript reads as prose with no numbers, and it gave that one cause in its report. But a page that prints none of its figures is also what a page looks like when every figure on it has changed, which is how a class profile goes stale when a school moves it on to the next class: the same labels with all new numbers. Set-aside pages did not count toward the check's exit status, so the weekly issue could never open for them. Three gaps in how pages are read put more pages in the same place: a number glued to the word before it is not read as a number, a spreadsheet was decoded as if it were text, and a counter that animates shows 0 in the page as served. The diagnosis was then written into the ROADMAP without anyone reading the pages.
- **How it surfaced.** Found on September 27, 2026 while reading the six set-aside pages one by one: a browser read of Kelley's page returned every figure in a table headed Class Profile of 2028. (Found by reading the code or the output)
- **Fix.** Kelley moves to its Class of 2028 figures. Maryland's women and international shares come from the profile as updated, its class size is blank because Poets&Quants gives a full-time enrollment without saying whether it counts one class or both years, and the profile's class is recorded as not labeled. Auburn's GPA and work experience move to the page's current values, its international share is blank, and its class is recorded as not labeled. UMass's figures are dated 2025. The check now reads an Excel workbook as the values its cells display, splits a number from a word it is glued to, and reads an animated counter's final value. A page that still prints none of its figures is sorted by what it does print: numbers beside the figures' labels are reported per figure as a finding, since the page has probably moved on, and anything else is reported for a person to read. Either way it counts toward the exit status, so the weekly issue opens, unless every figure on it carries a triage entry, as WashU Olin's three now do after being read from its images, along with five more figures read from the same images and added to its record.
- **What stops it now.** check_sources.py's self-check holds a page that moved on (the labels with other numbers), which must be reported with what it prints beside each label, a page with neither, which must be reported to be read, text with numbers glued to their labels and a counter whose value sits in its data-target attribute, which must both yield their numbers, and a spreadsheet cell formatted as a percentage, which must read as the percentage it displays. A page showing none of its figures now fails the run unless every figure on it is triaged. in `src/check_sources.py`
- **Lesson.** A checker's excuse category needs the same scrutiny as its findings. "Shows none of its figures" was given one cause and the cause was believed, but to a check that looks for numbers, a page whose every figure changed looks exactly like a page that never loaded. A category that explains itself away should instead say what it saw, and keep counting until a person has looked.

INC-0161. The source check's blank-page rule judged a figure worked from two pages by its first page alone, and called mba.com's payment page blank while the second page was unreadable

2026-09-27, Degraded, `3525aa85563806e01ec2bd9cab7074f70b030615` *PR #154*

- **What was seen.** With the GMAT's fees recorded against mba.com's exam payment page and, in also_urls, the address that page loads its US fee table from, python3 src/check_sources.py --render reported the payment page as "prints none of its 5 figures and no number beside their labels" and exited 1. The payment page prints no fee until a country is chosen, which is why the table's address is read with it, and on that run the table's address had answered with a bot challenge, so it was reported unreadable in the same output.
- **Why.** The rule for a page that shows none of its figures dates from INC-0133, when every figure was read against one page: it groups figures by their url and looks for their numbers in that page alone. also_urls (INC-0154) made a figure's source the union of its pages, and the finding loop reads it that way and skips a figure any of whose pages went unread, but the blank-page rule was not changed with it. Until INC-0158 a blank page was only a line in the report, so nothing depended on it; from INC-0158 it fails the run.
- **How it surfaced.** Found on September 27, 2026 on the first run after the GMAT's fee facts were added with also_urls: the page the rule called blank was one the facts had never claimed to be read alone. (A test caught it)
- **Fix.** The blank-page rule reads a figure the way the finding loop does: against every page it names, and only when all of them were read. A figure with an unread page is left to the unreadable list.
- **What stops it now.** check_sources' self-check holds a figure whose numbers are on its second page only, which must not make its first page blank, and one whose second page went unread, which must be left out of the blank-page rule. in `src/check_sources.py`
- **Lesson.** When a check learns a new way to read a source, every rule that reads a source has to learn it too. A second reader that lags the first is found only when the data first depends on the difference.

INC-0165. The playbook's smoke test and its builder disagreed on what counts as an unresolved placeholder, so a correctly quoted one failed the smoke

2026-09-27, Degraded, `44b3f8c80bb2860eb619880931f9cca9272b3ee7` *PR #162*

- **What was seen.** After INC-0163's record quoted `{{FONTS_CSS}}` in a code span, python3 src/build_playbook.py built the book, and node src/smoke_playbook.js failed it with "no unresolved placeholders: FONTS_CSS".
- **Why.** The builder decides what is an unresolved placeholder by blanking code spans and fenced blocks first, since a quoted placeholder is an example rather than a place to substitute, and its comment says why it does not use an allowlist: one would go stale. The smoke test, written to catch a builder change that dropped that check, used exactly such an allowlist, CHROME_CSS and PLACEHOLDER, the two quoted names that existed when it was written. The first new quoted name failed it.
- **How it surfaced.** Found on September 27, 2026 when the suites were run on the font change. (A test caught it)

- **Fix.** The smoke test blanks code spans and fenced blocks the way the builder does and keeps no list of names.
- **What stops it now.** smoke_playbook's placeholder check now applies the builder's own rule, so the two cannot disagree about a quoted placeholder, and it still fails on one left in running text. in `src/smoke_playbook.js`
- **Lesson.** A test of a check should apply the check's rule, not a snapshot of the cases that existed when the test was written. An allowlist of known exceptions is a record of the past, and the first new legitimate case turns it into a false failure.

INC-0166. The standalone page smoke test still loaded pages from disk, where the site's root-relative paths point at the filesystem root, so it failed once the fonts moved onto the site

2026-09-27, Degraded, `44b3f8c80bb2860eb619880931f9cca9272b3ee7` *PR #162*

- **What was seen.** After INC-0163, node src/smoke_pages.js failed /international/ and /apply/ with five console errors, each "Failed to load resource: net::ERR_FILE_NOT_FOUND": the pages link their fonts at /vendor/fonts-2026-09-27/fonts.css, and loaded as file:// that path is the root of the disk, not of the site.
- **Why.** smoke_pages opened the built pages as file:// URLs. A page that loads anything by a root-relative path cannot load it that way, and smoke_billing and smoke_guide had already moved to a local server for that reason; smoke_pages had not, and nothing forced it to, because until now every root-relative reference those two pages make failed without writing to the console, and the one resource that did fail loudly, Google Fonts, was on its list of environment noise.
- **How it surfaced.** Found on September 27, 2026 when the suites were run on the font change. (A test caught it)
- **Fix.** smoke_pages serves the built site from a local HTTP server and opens the pages through it, as the other page smokes do, so every root-relative path resolves the way it does live.
- **What stops it now.** smoke_pages counts any resource that fails to load as an error, now that the site's own paths resolve, so a stylesheet, font or script a page names and the build did not produce fails it; Google's hosts are no longer on its noise list. in `src/smoke_pages.js`
- **Lesson.** A test harness has to load a page the way a visitor does or its passes mean less than they say. file:// quietly breaks every root-relative path, and a noise filter written for one failure hides the next one of the same kind; when a test has to ignore an error to pass, it is worth asking what else that exemption lets through.

INC-0169. The reading corpus check accepted a near miss whose only negative word opened it, and the item builder, which capitalises first, could not use it

2026-09-27, Degraded, `6ec335282a2f1056aea8af4ff21019207fcc33ac` *PR #164*

- **What was seen.** A new passage's near miss read "no water was pumped from the lower levels on the fourteenth of June 1866". check_premises passed it as the negative near miss the inference question needs, and python3 src/build.py then failed with "gmat/v_inf did not ship the questions its schemas list": every draw of that question raised "balance needs 4 wrong answers, pool has 117".
- **Why.** Both places test a near miss with the same pattern, NEGATIVE, which matches not, no, none and never in lower case only. check_premises tests the near miss as stored, lower case; the inference builder capitalises it first, because it becomes a sentence, so "No water" no longer matched and the builder found no negative near miss to offer. The balancer's error then counted every wrong answer in the pool instead of the one kind it lacked, so the message pointed away from the cause.
- **How it surfaced.** Found on September 27, 2026 when the build refused a batch of five new reading passages. (A build guard caught it)
- **Fix.** NEGATIVE matches in any case, so a near miss counts as negative however it is capitalised, in the check and in the builder alike. When the balancer lacks the answers about the same case that it must offer, its error now says that and how many it has.
- **What stops it now.** g_rc checks at import that every near miss the corpus check counts as negative still counts once capitalised as the builder prints it, and the build's existing check that every listed question ships (INC-0126) caught this one. in `src/gen/g_rc.py`
- **Lesson.** A check that tests a value in a different form from the one the code uses tests something else. Test what is printed, after every transformation the code applies, or make the test indifferent to the transformation.

INC-0173. The source check ended a tag at the first > it met, so markup inside a quoted attribute leaked into the page text, and an element id's 5 confirmed GRE's five-year score validity on a page that never states it

2026-09-27, Wrong data shown or stored, `b6727f6ba5f7be256111a4212877c1bc4980feaf` *PR #166*

- **What was seen.** /exams/gre/ showed Score Validity 5 years citing ETS's GRE scores overview page. Read on September 27, 2026, that page says nothing about how long scores last; ETS's Get Your Scores page does: "GRE scores are reportable for 5 years following your test date." The weekly exam source check had passed the figure, and the text it read from the overview page held the fragment "}}" id="text-5ae0eb3b6b" class="cmp-text">, markup no reader sees. The figure is right; the page cited for it was not the one that says it.
- **Why.** to_text removed tags with the pattern <[^>]+>, which ends a tag at the first > it meets. ETS's pages keep a copy of each text block in a data-cmp-data-layer attribute: JSON holding the block's markup with its < escaped but its > left raw, inside the attribute's quotes. The first > in that attribute ended the tag early, and the rest of the attribute, the block's markup and text again and then the element id text-5ae0eb3b6b, was read as page text. The number pattern accepts a digit after a hyphen, so the id supplied a 5.

- **How it surfaced.** Found on September 27, 2026 by an audit that listed every exam figure the check found on its page only away from all of the fact's words (INC-0174); the 5 it had matched on this page sat inside an element id. (Found by measuring something)
- **Fix.** The GRE validity figure cites ETS's Get Your Scores page. to_text reads markup with Python's HTML parser, which keeps a quoted attribute whole, so no attribute text reaches the page text.
- **What stops it now.** check_sources' self-check reads a tag whose quoted attribute holds markup, a > and a number, and fails if any of the attribute's text comes back as page text. The proximity check added for INC-0174 would also refuse a 5 standing in an id. in `src/check_sources.py`
- **Lesson.** Taking HTML apart with regular expressions works until a page puts markup where the pattern did not expect it, and then it fails silently, by producing text that looks plausible. A parser knows where a tag ends and costs nothing here, so anything whose output is used as evidence should read HTML with one.

INC-0175. The source check cached Imperva's script challenge as GMAC's policies PDF, a page with no words, so every later run with --cache reported the PDF unreadable from its own cache

2026-09-27, Degraded, `b6727f6ba5f7be256111a4212877c1bc4980feaf` *PR #166*

- **What was seen.** A run of check_sources --render --cache on September 27, 2026 reported GMAC's GMAT policies and procedures PDF unreadable with "only 1 characters of text", and a second run reported the same, although curl had just fetched the same URL as the 295,049 byte PDF with 80,078 characters of text. The cache file for the URL held a single space.
- **Why.** Some reads of the PDF's URL get Imperva's script challenge instead: 212 bytes of text/html whose only content is a script from /_Incapsula_Resource. The challenge check matches a challenge's words, and this one has none once its script is removed, so it read as an empty page. fetch() caches every read except a recognised challenge, so the empty read was kept, and the next run took it from the cache instead of reading again: a one-off refusal became a standing one. The browser read, which the check tries for short pages, fails on a PDF, so nothing replaced it.
- **How it surfaced.** Found on September 27, 2026 when a retry, meant to read the one source the first run could not, reported the same failure. (A test caught it)
- **Fix.** fetch() recognises Imperva's script challenge in a plain read, reads once more a few seconds later, as the browser read already does for challenges, and reports the source as challenged if it comes back again. It caches a read only when it holds at least MIN_TEXT characters, so a short read of any kind is tried again on the next run.
- **What stops it now.** check_sources' self-check serves fetch() the challenge as mba.com sent it, followed by the page and then by the challenge again, and fails unless the first comes back as the page and the second as a challenge; and it reads a short page through fetch() with a cache and fails if the read was kept. in `src/check_sources.py`

- **Lesson.** A cache should keep only what would be accepted as an answer. Caching a failure turns a transient fault into a permanent one, and it then looks like the source's fault rather than the cache's. And a detector that looks for a refusal's words misses a refusal that has none.

INC-0181. The ledger's harvest read only incident ids written out in full and matched commits by their short hash, so seven records fixed under a range such as "INC-0104 to INC-0110" cited nothing and 57 cited fixes came back every week as possible unrecorded defects

2026-09-27, Degraded, `7ff690443f31290b82700cd0832990934e955f57` *PR #176*

- **What was seen.** On September 27, 2026, python3 src/playbook_harvest.py --citations reported eight records that name no commit and that no commit names: INC-0105 to INC-0109, INC-0171, INC-0174 and the just-written INC-0180. The squash commits on main do name the first seven: PR #96's names "INC-0104 to INC-0110", PR #165's "INC-0170 to INC-0172" and PR #166's "INC-0173 to INC-0175". The harvest also listed 38 commits as candidate unrecorded defects, many of them commits whose records simply cited nothing. Reading why, a second fault: the candidate loop compared each commit's short hash with the ledger's citations, 57 of which were full hashes, so every commit cited in full was listed as a candidate unrecorded defect however long ago it was written up.
- **Why.** incident_commits() finds ids with the pattern INC-\d{4}, so it reads the two ends of a range and nothing between them, and it would read only the first id of a list written "INC-0130, 0131, 0132". Sessions name several incidents in one squash title that way whenever one PR fixes a run of them. The report asked a human to find those seven by hand, when git already held the answer. The candidate loop has the same shape of fault in the other direction: git prints a short hash, the backfill writes a full one, and an exact comparison between the two never matches, so the tool that fills in citations made its own weekly report noisier every time it ran.
- **How it surfaced.** Found on September 27, 2026 by running the harvest's --citations report after a merge and reading why each uncited record had no match. (Found by reading the code or the output)
- **Fix.** playbook_harvest reads an incident range ("INC-0104 to INC-0110") and a short list ("INC-0130, 0131, 0132") as every id they name, capped at 20 past the first so a year after a comma is never an id, and counts a commit as cited when either hash is a prefix of the other. --backfill then filled 77 citations from main's history, leaving only this record, which this change's own merge will name. The five candidates left were read and cleared in data/playbook/cleared.jsonl with reasons: two were validation or design language, one a claim the same change made stale, and two follow-on work on INC-0079 and INC-0057.
- **What stops it now.** playbook_harvest's self-check, run first on every invocation, reads the four forms main's commits use and fails unless each yields every id it names, rejects an over-long range and a year after a comma, and requires a commit cited in full or short to count as cited; mutants that drop the range reading, the list reading or the cap each fail it. in `src/playbook_harvest.py`

- **Lesson.** A reader of free text has to read every way people actually write the thing it looks for. Check the forms in the history, not the form you would write, before trusting that a pattern finds all of them.

INC-0182. Arizona State's class size stayed at 47 after its class profile changed it to 45, because the source check confirmed the head count with the page's "Business 47%" under "Class composition"

2026-09-28, Wrong data shown or stored, `2af0f33bc3a976c6a0d0c7012295b474d3a27132` *PR #184*

- **What was seen.** The weekly source check of September 28, 2026 (issue #182) reported three of Arizona State W. P. Carey's figures as no longer on its class profile page: 19 percent admitted, 5.3 years of work experience and 32 percent international. Read in full, the page is still headed "Incoming class of Fall 2025" but now prints 328 applications, 24% admitted, an entering class of 45, age 30, GMAT Focus 645, GRE 314, GPA 3.6, 6.2 years of work experience and 33% international. Earlier runs' cached reads of September 26 and 27 show the figures the library holds: 426 applications, 19%, 47, age 29, GMAT 690, GRE 314, GPA 3.6, 5.3 years and 32%. The library's class size of 47 was not reported, though the page no longer gives 47 as the class size.
- **Why.** The school check confirms a figure when its number appears on the page within 200 characters of one of its field's label words, and class size's words include "class", which a class profile prints in nearly every heading ("Class averages", "Class composition"). The page's undergraduate major table prints "Business 47%" a few words after "Class composition", so the old head count of 47 was found beside a label. A percentage can never be a head count, but the check matched digits without asking what kind of quantity they were.
- **How it surfaced.** Found on September 28, 2026 by reading Arizona State's page in full after the weekly issue flagged three of its figures, and comparing every figure the library cites to it, including the ones the check had passed. (Found by reading the code or the output)
- **Fix.** A number followed by a percent sign no longer confirms a school figure that is a count, a score or a length of time: class size, the GMAT and GRE scores, GPA and years of work experience. Arizona State's entry moves to the page's figures: 24 percent admitted, an entering class of 45, 6.2 years, 33 percent international, and a GMAT Focus average of 645, now that the page names the edition, with a note that the figures changed on or about September 28 under an unchanged Fall 2025 heading.
- **What stops it now.** check_sources' self-check reads a class profile row "Class composition International 33% Undergraduate major Business 47%" and requires a class size of 47 to be reported as not beside its label, while "Size of entering class 47" still confirms it; the mutant that drops the percent rule fails it. in `src/check_sources.py`
- **Lesson.** A matching number is the figure only if it is the same kind of quantity. Before asking whether a number sits near its label, ask whether it could be the figure at all: a percentage cannot be a head count, and a label word that a page prints in every heading proves nothing about any one number.

INC-0183. The source check's browser read left out collapsed accordion panels, so Berkeley Haas's and Buffalo's correct tuition figures were reported missing whenever a site refused the plain read

2026-09-28, Degraded, `2af0f33bc3a976c6a0d0c7012295b474d3a27132` *PR #184*

- **What was seen.** The weekly check of September 28, 2026 (issue #182), run on GitHub's runner, reported Berkeley Haas's $92,755 and Buffalo's $16,655 and $7,615 as not printed by their cited pages. Both pages print them: Haas as the non-resident Tuition & Fees Subtotal for the first year in its 2026-27 cost of attendance table, Buffalo in its resident and non-resident MBA charges tables. In this sandbox the check's plain read of each page finds every figure, and its browser read finds none of them.
- **Why.** render_page.js returns document.body.innerText, which leaves out any element that is not rendered. Both tables sit in accordion panels that stay collapsed until a reader clicks: Haas's in a div carrying the hidden attribute, Buffalo's in a panel with display none and aria-hidden, each named by its button's aria-controls. The check falls back to the browser when a site refuses the plain read, as these two evidently did on GitHub's runner, and the browser then reported a page without the content a reader opens with one click.
- **How it surfaced.** Found on September 28, 2026 by comparing the check's plain and browser reads of both pages after the weekly issue flagged them, and walking up from each figure to the element that hid it. (Found by measuring something)
- **Fix.** render_page.js appends the text of every collapsed panel a reader can open: an element named by an aria-controls attribute that is not rendered, and a closed details element, when it holds a digit and sits outside the page's navigation, header and footer. The browser read of both pages then finds all three figures.
- **What stops it now.** smoke_render.js serves a page with a figure inside a collapsed accordion panel named by its button's aria-controls, and one inside a closed details element, and fails unless the browser read returns both; a figure in a hidden navigation menu must still not come back. in `src/smoke_render.js`
- **Lesson.** What is on screen is not what a reader can see. Content one click away behind the page's own controls is part of the page, so a reader built for checking has to open what the page offers to open, and leave closed what it hides for other reasons, such as menus.

INC-0184. Vanderbilt's median salary and 39 schools' Bloomberg ranks stayed on Bloomberg's 2025-26 edition after its pages moved to 2026-27, and nothing could see it: Bloomberg answers most reads with a bot challenge, and the source check never reads a rank

2026-09-28, Wrong data shown or stored, `33d98288de6b2022e8b92c51dbd8164a877bcb5d` *PR #188*

- **What was seen.** A school run of the source check on September 28, 2026 reported vanderbilt-owen.profile.salary_median_usd, $150,000, as not printed by its cited page, Bloomberg's school profile for Vanderbilt (Owen). That page had answered the check with a bot challenge in every earlier run; this time it served the page, now titled "Best Business Schools & MBA Programs 2026-27", with Owen at No. 17 in the US, up 10 from 2025-26, and a median base salary across all industries of $142,800. The library still held the 2025-26 profile's $150,000 and Owen's 2025-26 rank of 27, and 39 schools carried a Bloomberg rank labeled 2025-26.
- **Why.** Bloomberg rewrites each school's page in place when it publishes a new edition, so a figure or rank read from it holds only until the next one, and nothing recorded when that would be or looked for it. The source check could not have noticed: Bloomberg answers most of its reads with a bot challenge, which it rightly reports as unreadable rather than as stale, and it does not read rank entries at all, because it collects only objects carrying a v or a text, and a rank carries rank and edition. A new edition therefore surfaced only when one read of the one profile figure cited to Bloomberg happened to get through.
- **How it surfaced.** Found on September 28, 2026, when a school run of the source check read Bloomberg's Vanderbilt page as the page rather than a challenge and reported the salary missing from it. (Found by measuring something)
- **Fix.** The source check now reads every rank entry whose edition is written like 2025-26 or 2025-2026, and reports as a failure any whose cited page names a later edition; its first run flagged 29 Bloomberg ranks and nothing else. A later read of Bloomberg's US ranking page got the page rather than the challenge, and every school's Bloomberg rank moves to the 2026-27 edition from it: 40 ranks move, 22 schools gain one, and Penn State and UC San Diego, absent from the 2026-27 list, lose theirs. Vanderbilt's salary figure moves to the 2026-27 profile's $142,800.
- **What stops it now.** check_sources' self-check requires later_edition to report 2026-27 for a rank recorded as 2025-26 on a page titled with 2026-27 that also says "up 10 from 2025-26", to report nothing for a page naming only 2025-26, and to read neither a date such as 2026-10 nor a span such as 2025-2027 as an edition. in `src/check_sources.py`
- **Lesson.** A figure copied from a page its publisher rewrites every year expires on the publisher's schedule, not yours. Record which edition a figure came from, and have the check compare it with the edition the page now names; a page that cannot be read says nothing about whether what you took from it is still true, and a field the check does not read is not checked at all.

INC-0185. Maryland's and Penn State's Financial Times ranks were years old and named no edition, so the composite scored them beside FT's 2026 ranks, and Maryland's was a US position where every other FT entry is a global one

2026-09-28, Wrong data shown or stored, `8a78cffe014ba97490cabc2dc61d47216a2d0a2f` *PR #189*

- **What was seen.** Listing every rank with an empty edition on September 28, 2026 found two, both Financial Times. Maryland's 31 cites a school news page on the 25th anniversary edition of FT's ranking, where Smith was No. 57 in the world and No. 31 in the US; the library recorded the US position, where every other FT rank it holds is a position worldwide. Penn State's 47 cites a Smeal Magazine article from its Spring 2021 issue, which reports the FT Global MBA Ranking naming Smeal No. 47 overall. Every other FT rank in the library comes from the 2026 edition, and FT's 2026 table of 100 schools names neither school.
- **Why.** A rank's edition was optional. validate_schools checked that a rank came from a known publisher and sat between 1 and 200, but never that it said which edition it came from, so two ranks entered without one were accepted, and the composite, which averages whatever publisher ranks a school carries, scored both as current. Nothing compared what a rank measured either: Maryland's was a US position among the library's worldwide positions.
- **How it surfaced.** Found on September 28, 2026, when every rank without an edition was listed after the Bloomberg refresh (INC-0184) and the two pages they cite were read. (Found by reading the code or the output)
- **Fix.** Both ranks are set to null for the 2026 edition, citing the copy of FT's 2026 Global MBA Ranking table that another school's FT rank already cites, in which neither school appears. validate_schools now refuses a published rank that names no edition or has no source url. Cornell's, UCLA's and Washington Foster's FT ranks, which Poets&Quants' report of the 2026 ranking confirms, now cite it rather than two admissions consulting blogs.
- **What stops it now.** validate_schools refuses any rank that has a value and an empty edition, which fails the build; the rule was confirmed by restoring one of the two old entries, which the validator then refused. in `src/validate_schools.py`
- **Lesson.** A rank without its edition is a number without a date, and an average cannot tell a 2021 rank from a 2026 one. Make the edition a required part of the record, so a rank that cannot say when it is from is refused rather than scored.

INC-0196. A smoke test's page errors went into the live error log, because the error beacon reports from any page it runs on, including one a test opened from disk

2026-09-29, Wrong data shown or stored, `5a6cc3abdeb9c4c5305f2058739ec7065166600b` *PR #216*

- **What was seen.** client_errors, the table behind Admin > Errors, holds five rows from 12:51 UTC on September 27, 2026, each 'Failed to load LINK' on the path /home/user/GMAT/apply/index.html from HeadlessChrome. None was a visitor's: a smoke test in a build sandbox opened the built page from disk, where its root-relative font links point at the filesystem root, and the error beacon on the page posted each failed load to the production table. They are 5 of the table's 1,499 rows, and nothing in them says they came from a test.

- **Why.** sentinel_js reports every script error and every failed load of the page's own files wherever the page is running, and the production address of the table is written into every built page. Its only filter drops third-party files, and on a page opened from disk the page's own files are file:// URLs, which pass it. INC-0166 moved smoke_pages off file:// that morning, 18 minutes after these rows, which ended this source; but 12 smoke scripts open pages in a browser with no request interception at all, so any error one of them catches, locally or in CI, would be posted to the live table the same way.
- **How it surfaced.** Found on September 29, 2026 while reading client_errors for errors visitors hit: grouped by path, five rows carried a sandbox file path, and grouping the whole table by path and user agent showed them to be the only rows from a headless browser. A search of the smoke scripts for request routes found 12 that open pages without one. (Found by reading the code or the output)
- **Fix.** The beacon now returns before reporting when navigator.webdriver is true, which a browser under automation sets and a visitor's browser does not, or when the page was opened from disk. Visitors on the live site, on a preview deploy or in an app that loads the pages are unaffected. The five rows stay in the table, for triage in Admin > Errors, rather than being deleted from production.
- **What stops it now.** build.py runs the beacon's own script in node against a stubbed page three times, as a visitor on https, as an automated browser and as a page opened from disk, fires a script error and a failed load of the page's own file in each, and fails the build unless only the visitor's run posts. in `src/build.py`
- **Lesson.** Telemetry that ships inside every page runs wherever the page runs, including every test harness and every local preview, and it writes to production from all of them. Decide in the snippet which contexts may report, and test that decision by running the real snippet in each context, because the harness that would notice the leak is the one producing it.

INC-0202. The SAT date reader took three things one reading of College Board's pages showed as rules, and when the score release page changed it refused both pages, hiding a moved 2027 date behind the refusal

2026-10-06, Wrong data shown or stored, `031ddd1c13535ce53ba56e8b8f24b981c924c3d1` *PR #235*

- **What was seen.** The weekly source check of October 5, 2026 (issue 223) reported that test_dates.py could not read College Board's SAT pages: no SAT Weekend score release table with the header Test Date, Student Score Release, Educator Score Release. College Board had split its score release table into a fall and a spring table and renamed the columns Student Score Release Date and K-12 Score Reporting Portal Score Release Date. Read with the new header, the reader still refused the pages twice over: the score page lists August 22 and September 12, 2026, which the dates page no longer lists, and June 5, 2027 releases scores to students and to the K-12 portal on the same day, June 21. While the reader refused, the check compared nothing else, so two changes on the dates page went unreported: an anticipated 2027 date moved from October 2 to October 9, marked Updated on College Board's page, and the School Day sentence the page quotes was reworded. Until this fix the published SAT test dates page showed October 2, 2027 and no score release dates for the three spring 2027 tests, which College Board's page now lists.

- **Why.** The reader was written against one reading of College Board's pages and took three things that reading showed as rules: that the weekend releases sit in one table, that the score page lists exactly the dates the dates page lists, and that educators always get scores after students. The second was never true for long: the dates page drops past dates on its own schedule (on October 6 it had dropped August 22 and September 12 while still listing October 3), and the score page keeps the whole season, so from the first week the dates page dropped a date the score page still listed, the reader would have refused both pages whatever College Board did to its layout. The third was strict ordering where the page only shows not before. And one table the reader could not take stopped the whole comparison, so the changes it could still have read were hidden behind it.
- **How it surfaced.** The weekly source job's report of October 5, 2026 (issue 223) named the missing header. The other two faults surfaced on October 6, when the reader was run with the new header and refused rows it should have read; both were in the reader from the day it was written. (A test caught it)
- **Fix.** The reader takes the rows of every table under the score release header, in page order, accepts a student and an educator release on the same day, and leaves out a release whose test day has passed and falls before the first date the dates page lists. A release for any other date the dates page does not list is still refused. The data file was rewritten from both pages: October 2, 2027 is now October 9, the School Day sentence is quoted in its new words, the three spring 2027 tests carry their release dates, and August 22 and September 12, 2026 left with the dates page.
- **What stops it now.** python3 src/test_dates.py --selfcheck reads pages built to look like College Board's two, with a fall and a spring score table, a passed date only the score page lists, a test whose two releases share a day, and a date still ahead that only the score page lists, which it must refuse. The build runs it on every change, and --write and --check run it before they read anything. Each of the three old rules, put back on its own, fails it. in `src/test_dates.py`
- **Lesson.** A reader of someone else's page should treat as rules only what the page promises, not what one reading of it happened to show. Two pages from one publisher are kept on their own schedules, so a check that both list the same things fails the week one is tidied and the other is not, and an order seen on every row so far is a pattern, not a promise. When a strict reader refuses, everything behind the refusal goes unchecked, so once the refusal is fixed, read the whole difference it was hiding.

INC-0211. Our own browser tests were 88 percent of the page views the site recorded, because the analytics beacon had no automation check

2026-10-19, Wrong data shown or stored, `235fea9250c812432a1d12efb071f7b43c01e1d4` *PR #269*

- **What was seen.** Over the 30 days to October 9, 2026, site_events held 1,292 pageviews and 1,136 of them had the same shape: path '/', a 390 pixel viewport, no referrer, a desktop user agent, two to a session, in bursts on the days the browser suites ran (September 21 and 22, 26 to 29, October 3, 5 and 6). About 156 pageviews were left for real visitors. Every count the Admin traffic view showed, and any reading of growth from it, was about eight times the real number.

- **Why.** INC-0196 taught the error beacon to stay quiet in a browser under automation (navigator.webdriver) but left the consent gated analytics beacon in partials.consent_js without the same check, because smoke_consent and smoke_funnel assert that this beacon fires and a blanket check would have failed them. The first block of smoke_consent opens the home page in a browser context that intercepts nothing, accepts analytics and reloads, so every run of the browser suites, locally and in CI on every push, sent two real pageviews to the production table. The tests that did intercept the beacon were never the problem; the one that did not was invisible because nothing compared the live table against the shape of test traffic.
- **How it surfaced.** Found on October 9, 2026 while measuring real traffic to answer the owner's question about growth: 957 of 979 pageviews in 14 days were path '/' at 390 pixels with no referrer, and the bursts matched the days the suites ran. (Found by measuring something)
- **Fix.** The analytics beacon, and the item telemetry on the trainers and the daily question pages, send nothing in a browser under automation unless the page has opted into capture with window.__sfnBeaconTest, which the suites that intercept and assert on those requests set before the page loads. A test that forgets to intercept now records nothing rather than writing to production. The rows already written stay in the table, since production rows are not deleted, and are identifiable by their shape.
- **What stops it now.** smoke_consent opens a page under automation without the opt in, accepts analytics, reloads, answers nothing else, and fails if a single pageview is sent; the existing checks that the beacon does fire run with the opt in set. in `src/smoke_consent.js`
- **Lesson.** When one beacon learns to ignore automation, every beacon on the page needs the same rule, and a test that needs a beacon to fire should opt in explicitly rather than being the reason the rule is missing. A test that can reach a production endpoint is writing production data whether it meant to or not, so check the live table for the shape of your own test traffic before trusting any count from it.

Front end (16)

INC-0001. Unescaped quotes in onclick strings took the whole app down

2026-08-19, Site down, `1f3ecec` *PR #7*

- **What was seen.** On the deployed site no tab rendered and every nav click was dead. The page looked fine; nothing worked.
- **Why.** Five onclick handlers were built as JS strings containing unescaped single quotes inside single-quoted string literals. That is a parse error, and a parse error anywhere in an inline script kills the entire script, not the one handler.
- **How it surfaced.** Reported from the live site after deploy. (A person hit it)

- **Fix.** Escape the quotes, and make the build parse every inline script with node before it will write a page.
- **What stops it now.** build.py node-parses every inline script and fails the build on a syntax error in `src/build.py`
- **Cost.** a live outage of the trainer
- **Lesson.** Generated code is code. If your build writes JavaScript into a string, the build must parse the result, because the blast radius of one bad character is the whole file, not the line.

INC-0038. One exam's progress blob overwrote another's

2026-09-16, Wrong data shown or stored, `81eecec` *PR #30*

- **What was seen.** None shipped: caught before release. A student practising two exams would have had one exam's progress destroyed by the other.
- **Why.** Progress was stored under a single key. Making the engine multi-exam added a second writer to the same slot without keying the slot by exam.
- **How it surfaced.** Reviewing the cross-exam data paths while adding the second exam. (Found by reading the code or the output)
- **Fix.** Key the stored state per exam.
- **What stops it now.** engine tests run once per exam and assert progress isolation in `src/test.js`
- **Cost.** would have destroyed real progress
- **Lesson.** The moment a single-tenant store becomes multi-tenant, every key in it is a collision waiting to happen. Enumerate the writers before you add the second tenant, not after.

INC-0043. The GRE app told GRE students to calibrate at the wrong test maker's site

2026-09-16, Wrong data shown or stored, `1a003bb` *PR #34*

- **What was seen.** A shipped application displayed a score card headed with a different exam's name and linked students to that exam's official site.
- **Why.** The template forked on the exam id in seven places and treated not-SAT as the original exam. Adding a third exam made every one of those branches wrong.
- **How it surfaced.** Reading the built page for the third exam. (Found by reading the code or the output)
- **Fix.** Per-exam copy moved into the registry and resolved from the injected exam id.
- **What stops it now.** a headless check asserts each app names itself in `src/test.js`
- **Cost.** a live app misidentifying itself
- **Lesson.** A conditional that treats not-A as the original case is a bug the day a third case exists. Resolve variants from data, and the third one costs a row rather than a search.

INC-0015. The whole bank blocked first paint: 20 seconds to the first question on 3G

2026-09-19, Degraded, `4b25169` *PR #45*

- **What was seen.** On throttled regular 3G, 20.3 seconds to the first GMAT question and 23.4 for ACT.
- **Why.** The entire item bank shipped as a blocking script. It grew past ten megabytes without anyone measuring what that cost.
- **How it surfaced.** A throttled measurement, added because the number had never been taken. (Found by measuring something)
- **Fix.** A strided starter bank plus an async remainder that pushes into the same array. ACT 7.6s, GMAT 11.0s; blocking download 11.3MB to 1.2MB.
- **What stops it now.** smoke_load measures time to first question under throttling in `src/smoke_load.js`
- **Cost.** every visitor on a slow connection, for weeks
- **Lesson.** Nobody notices a page getting slower one commit at a time. Put the number in a test the first time you care about it, not the first time somebody complains.

INC-0022. A try/catch caught nothing because the call inside it does not throw

2026-09-21, Silent loss, `c4698e0` *PR #52*

- **What was seen.** An error path never ran.
- **Why.** The wrapped call signals failure by return value, not by throwing, so the catch was unreachable.
- **How it surfaced.** A review bot playing a full exam against the real engine. (Found by a review bot or an adversarial pass)
- **Fix.** Check the return value.
- **What stops it now.** review_bot.js sits each exam repeatedly against the real engine in `src/review_bot.js`
- **Cost.** a silent failure path
- **Lesson.** try/catch around an API that returns errors is decoration. Know which convention each call uses before you wrap it.

INC-0028. The service worker precache regex did not match the chunked files

2026-09-21, Degraded, `58f3a93` *PR #59*

- **What was seen.** Chunked bank files would not have been precached offline.
- **Why.** The IMMUTABLE pattern was /bank_rest\.js$/, which does not match bank_rest1.js.
- **How it surfaced.** Reading the service worker while changing the file naming. (Found by reading the code or the output)

- **Fix.** /bank_rest\d*\.js$/.
- **What stops it now.** smoke_offline asserts every bank chunk is precached in `src/smoke_offline.js`
- **Cost.** offline would have silently degraded
- **Lesson.** When you change a filename scheme, grep for the old name as a string, including inside regexes. A pattern is a hardcoded name wearing a disguise.

INC-0032. ARR was rounded to whole dollars and lost real money at small scale

2026-09-21, Wrong data shown or stored, `37241958be4dadca750f76742818b18b88e5a5b1` *PR #62*

- **What was seen.** $159.84 of ARR printed as $160.
- **Why.** The currency formatter was called with zero decimal places for the larger tiles, a rule chosen for readability at scale and applied at a scale where cents still matter.
- **How it surfaced.** A smoke test whose expected values were computed by hand before the page was written. (A test caught it)
- **Fix.** Keep cents below a thousand dollars, drop them above.
- **What stops it now.** smoke_business asserts the exact printed figures in `src/smoke_business.js`
- **Cost.** a wrong figure on the tile the owner reads most closely
- **Lesson.** A fixed rounding rule is wrong at one end of the range or the other. Pick the precision from the magnitude, and write the expected number down before you write the code that produces it.

INC-0045. The error reporter reported its own failures, in a loop

2026-09-21, Degraded, `c4698e0` *PR #52*

- **What was seen.** 267 rows in the error table, one per page a crawler swept.
- **Why.** fetch rejects on a network error, it does not throw, so the try/catch around the reporting call caught nothing. The rejected promise reached the unhandledrejection handler, which called the reporter again. The analytics beacon had the identical trap.
- **How it surfaced.** Reading the error table and noticing the shape of the rows. (Found by measuring something)
- **Fix.** Handle the rejection, and never report from inside the reporter.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** 267 junk rows and a self-sustaining loop
- **Lesson.** Anything that reports failures must not be able to report its own. Check whether each call rejects or throws before you wrap it, and make the reporting path unable to re-enter itself.

INC-0113. The GMAT trainer's first question on a slow connection took twice as long as when the bank split shipped

2026-09-26, Degraded, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** src/smoke_load.js measured 21.7 seconds to the first GMAT question on its 3G profile (400 kbit/s, 400 ms latency) against a 15 second budget, and main failed identically. The two-stage bank load had put the same measurement at 11.0 seconds when it shipped. ACT was 13.8, LSAT 11.0, SAT 9.4 and GRE 7.0 on the same run.
- **Why.** The deferred bank chunks were written into the page as async script tags immediately after the blocking bank.js. Async only means the tag does not block parsing; the preload scanner still finds it and starts the download at once, so on a slow connection the one file a first question needs shared its bandwidth with 30 MB it does not. The GMAT remainder was later split into two chunks to stay under Cloudflare's 25 MiB asset limit, which made three concurrent downloads instead of two and is why GMAT degraded most. The suite that measures it is not in ci.yml, so nothing ran it after the chunking.
- **How it surfaced.** Found on 2026-09-26 by running every browser suite by hand before committing, including the ones CI does not run. It failed on this branch and on main, the second measured from a separate worktree built from origin/main. (A test caught it)
- **Fix.** The chunk tags are no longer in the markup. A small inline loader adds them after the page's load event, when the browser is idle, so nothing a student is waiting for competes with them. On the same profile the first question now appears in 10.1 seconds for GMAT, 7.5 for ACT, 7.4 for LSAT, 6.5 for SAT and 5.2 for GRE. Unthrottled, GMAT moved from about 0.5 to about 1.0 seconds in the suite, because the suite starts a round at the instant the page loads, which is when the idle parse of the chunks begins; a person who clicks a second later does not meet it.
- **What stops it now.** src/smoke_load.js passes within its 15 second budget for all five trainers again. The gap that let this sit is named: smoke_load is not in ci.yml. Adding it would put a timing measurement on shared CI runners, where it may be noisy, so it stays a manual pre-push suite, and this record is the reminder to run it after any change to how the bank is split or loaded.
- **Lesson.** Async on a script tag decides when it runs, not when it downloads, so an async tag still competes for bandwidth with everything the page is waiting for. Load what the first screen does not need after the first screen exists, and re-measure the slow path whenever a file is split, because splitting changes the number of downloads competing at once.

INC-0128. The trainer button on four exam guides and on the study guide hub opened the GMAT trainer

2026-09-26, Degraded, `97ea315375fb1fb12f5ebdd98b856c5a3d0b43ce` *PR #115*

- **What was seen.** The Train for It Here block at the foot of the SAT, GRE, LSAT and ACT guides (/exams/<exam>/) ended in an Open the trainer button that linked to /app/, the GMAT Focus trainer. A student reading the ACT guide who took it landed in GMAT practice. The study guide hub at /guide/, which covers all five exams, had the same button to the same place. Each exam page's other trainer button, Start Training Free in its header, went to the right trainer, which is how the wrong one went unnoticed.
- **Why.** exam_page() builds the header button from APP_PATH, the map of each exam's trainer. The block further down the same function wrote the literal /app/, as it has since build_exams.py was added on September 19 (#45), and the guide hub template added on September 22 (#89) copied the literal. A path written out by hand is right only for the exam it was written for, and nothing compared where a page's trainer links went with the exam the page is about.
- **How it surfaced.** Found on September 26, 2026 while reading exam_page() to link the new ACT score calculator from the ACT guide. The built pages confirmed it on all four. (Found by reading the code or the output)
- **Fix.** The block takes its link from APP_PATH, as the header button does, and names the exam: Open the ACT Trainer. The hub, which is about every exam, links to each live exam's trainer by name instead of to one of them.
- **What stops it now.** build_exams.py fails if any link into a trainer on an exam page goes anywhere but that exam's own trainer, or if a page for an exam without a live trainer links to one. build_guide.py fails unless the hub links to every live exam's trainer. in `src/build_exams.py`
- **Lesson.** When a site grows from one of something to several, grep for the literals the single case left behind. A link that was right when there was one trainer is wrong four times over when there are five, and it still looks right to anyone checking the page it was written for.

INC-0148. The trainer app ran its own code only after a script from cdn.jsdelivr.net had loaded, so a slow CDN stalled the app and timed out CI's games smoke

2026-09-27, Degraded, `77404eaa5c0fc0418e41c8f069825cea33a600ac` *PR #138*

- **What was seen.** CI failed on marlbrook-dev/GMAT#136 in src/smoke_games.js with "page.goto: Timeout 30000ms exceeded" navigating to /app/index.html and waiting for load, before any game check ran. The same suite passed locally on that commit and on one re-run. The pull request touched only school records and their validators, nothing the app loads.
- **Why.** The trainer app, the community page and the do-not-sell page load the supabase-js client from https://cdn.jsdelivr.net as a plain blocking script tag, and in the app it sits ahead of the item bank and the app's own code, which needs window.supabase to exist when it starts. So the browser runs none of the app until jsdelivr answers or fails, and the page's load event waits on it too. A slow response from jsdelivr, or a network that blocks it, holds the whole app for as long as the request takes, for students as well as for CI, and the content security policy had to name jsdelivr as a script source to allow it.

- **How it surfaced.** CI on September 27, 2026: build-and-test failed in the games smoke on PR 136 and passed on one re-run. Reading what the app's load event waits on found two other hosts: a Google Fonts stylesheet and supabase-js from jsdelivr, the second blocking the app's code. (A test caught it)
- **Fix.** supabase-js is served from this site at /vendor/supabase-js-2.117.2/supabase.js, with its MIT license beside it. It is the npm package's dist/umd/supabase.js for 2.117.2, the file jsdelivr was serving under @2 apart from a comment header, with one em dash in a debug message made a comma under the house rule; DEVSECOPS.md F5 records both checksums and how to upgrade. The trainer apps, the 404 page built from the app, the community page and the do-not-sell page load it from there, and the content security policy no longer lists cdn.jsdelivr.net as a script source. partials.SUPABASE_JS holds the path once and the pages take it from a placeholder. The trainers' service workers precache the file and treat /vendor/ as immutable: served as an ordinary same-origin file it was missing from the cache on the first offline launch and revalidated in the background after, and src/smoke_offline.js caught both as a request reaching the server with the network off.
- **What stops it now.** page_checks.offsite_scripts() finds any script a built page loads from another host, and build.py and build_blog.py fail on one. Run on the build from before the fix, it named every trainer, the 404 page, the community page and the do-not-sell page. in `src/build.py`
- **Lesson.** A script tag is a dependency on the host it names, with that host's speed and uptime. Code a page needs before it can run belongs on the page's own origin; another host belongs only where the page still works when that host is slow or gone.

INC-0149. A half-typed sign-in on the Account page vanished when a deferred bank chunk arrived and redrew the page, and the smoke that shows it had been failing unnoticed outside CI

2026-09-27, Degraded, `15334bab858321bb91c38ad9cb881d4194686ccb` *PR #139*

- **What was seen.** node src/smoke_signup.js failed "sends once both are chosen" and "the birth date is kept for the profile push" in all five trainers, on main as well as on the branch that found it. Traced step by step: the smoke typed an email, then chose a birth month, and by the time it chose the year the email field was empty and the sign-in form had been replaced by a new one, so sendMagicLink() refused an empty email and sent nothing.
- **Why.** The trainer loads most of its bank in deferred chunks after the page's load event, and each chunk calls window.__bankGrew(), which redraws the Account page with renderData() if it is open, because that page prints the bank size. The auth listener in Cloud.init() redraws it the same way when the sign-in state changes. renderData() builds the sign-in form's email box from state.settings.email and its month and year from state.settings.about, which are only written when the link is sent, so any redraw before Send threw away whatever the student had typed or chosen. The smoke fills the form in the seconds after load, when the chunks arrive, and loses that race; a student who opens the Account page and types at once can lose it too. The smoke is not in npm run test:browser, so nothing ran it.

- **How it surfaced.** Found on September 27, 2026 while running every smoke by hand for the supabase-js change: smoke_signup failed on that branch and on main's own build. A trace of the form's values between steps showed the fields emptied and the element replaced, and wrapping renderData() to record its caller showed the redraw coming from window.__bankGrew, called by bank_rest2.js as it arrived. The auth listener was the first suspect and the trace ruled it out as the trigger in this run. (A test caught it)
- **Fix.** renderData() now builds the sign-in form from what its email, month and year fields hold at the moment of the redraw, and falls back to the saved values only when the form is not on the page yet. The smoke passes in all five trainers, three runs in a row.
- **What stops it now.** src/smoke_signup.js fills the form, forces a redraw and fails unless the email, month and year survive; with the old renderData() all three came back empty in every trainer. The smoke now runs in npm run test:browser, so CI runs it. in `src/smoke_signup.js`
- **Lesson.** A view that is redrawn from saved state loses whatever the user has not saved yet. When a background event can redraw a form, carry the fields' current values across the redraw. And a test that is not in CI is not guarding anything: it fails quietly until someone happens to run it.

INC-0197. Every blog page carried the error beacon twice, because the blog builder appends its own after a footer that already carries one

2026-09-29, Wrong data shown or stored, `5a6cc3abdeb9c4c5305f2058739ec7065166600b` *PR #216*

- **What was seen.** Every built blog page contained two copies of the error beacon: the site footer carries one, and build_blog.py appended a second after it. Each copy listens for errors on its own, so an error on a blog page could reach client_errors twice. Three blog page loads did exactly that, each leaving two rows with the same path and signature within 30 milliseconds (2026-09-16 22:14:52, and 2026-09-17 05:23:28 and 11:25:17 UTC), and in two of the pairs one row carried a session id and the other did not, so the copies do not even group as one visit. Six of the table's 11 blog rows are those three pairs.
- **Why.** apply_chrome refuses a template that asks for both the beacon placeholder and the footer placeholder, because the footer already supplies the beacon and two would double-report every error. The blog builder does not go through apply_chrome: it writes its own page shell, calls footer_html() directly and then appends its own copy, so the assertion never saw the blog. The rule was enforced at the one entry point that could break it, not on the pages the site ships.
- **How it surfaced.** Found on September 29, 2026 while confirming that the INC-0196 change reached every built page: each blog page held the new line twice. Reading client_errors for blog paths then found the three pairs. (Found by reading the code or the output)
- **Fix.** build_blog.py no longer appends its own copy; the footer's is the one every blog page carries.
- **What stops it now.** build.py and build_blog.py count the beacon in every page they write and fail the build unless it appears exactly once, so the check reads the pages the site ships rather than the one template entry point the blog bypassed. in `src/build_blog.py`

- **Lesson.** An invariant about what a shipped page contains belongs on the shipped page. A check at one entry point protects only the pages built through it, and a builder that assembles pages its own way walks past it; counting the thing in every output file catches all of them, including builders written later.

INC-0201. The application checklist's calendar file left semicolons unescaped, because the escape was written '\;', which JavaScript reads as a bare semicolon

2026-10-06, Wrong data shown or stored, `eb538c357988599753afde1076961b325ec68739` *PR #232*

- **What was seen.** The Add to Calendar file on /apply/ wrote the Remind Your Recommenders event's description as 'Plan on sending it; most people need it.' with the semicolon bare. RFC 5545 requires a semicolon in a text value to be written as a backslash and a semicolon, and a strict calendar reader may cut the text at it. Commas and backslashes were escaped correctly in the same function.
- **Why.** The escape function was written as replace(/;/g,'\;'). In a JavaScript string literal a backslash before a character with no escape meaning is dropped, so '\;' is ';' and the replacement changed nothing. The comma rule beside it used '\\,' and worked, so reading the line, the two looked equivalent. The Python writer the function was copied from (build_test_dates.ics_text) is correct, because there the string is written with the backslash doubled.
- **How it surfaced.** Found on October 6, 2026 while reading the calendar code on /apply/ to add school rounds to it. The smoke test checked line endings, line length and event count, but never what an escaped value looks like, and no task text tested had a semicolon in its summary. (Found by reading the code or the output)
- **Fix.** The escape is written with the backslash doubled, so a semicolon becomes a backslash and a semicolon, as the Python writer already did.
- **What stops it now.** src/smoke_pages.js now fails if any SUMMARY or DESCRIPTION line in the downloaded calendar holds a semicolon or comma without a backslash before it, and the checklist's own data holds a description with a semicolon, so the check always has one to read. in `src/smoke_pages.js`
- **Lesson.** A backslash written into a string literal is a request to the language, not a character, and languages answer it differently: '\;' is a bare semicolon in JavaScript, while Python keeps an escape it does not know and gives a backslash and a semicolon. When code is copied between languages, an escape that looks identical can change meaning, so test the output rather than the source line, and give the test an input that needs the escape.

INC-0206. The Start a Free Round button at the foot of every blog post opened the GMAT trainer, including on the LSAT, GRE, SAT and ACT posts

2026-10-06, Degraded, `2c826b51fca103a1072f124057c42afb8dae4af6` *PR #260*

- **What was seen.** Every post in The Study Room ends in a Put This Into Practice box whose Start a Free Round button linked to /app/, the GMAT Focus trainer. On October 6, 2026 five live posts were about another exam: the GRE format guide, the LSAT format and scoring guide, the digital SAT format guide, the SAT study plan by domain and SAT vs ACT. A reader who finished the LSAT guide and pressed the button landed in GMAT practice. 31 queued posts would have done the same, 19 of them the LSAT question type series. The Where to Take This Next cards below the same posts already linked the right trainer, which is how the wrong button went unnoticed.
- **Why.** build_post() writes the box's link as the literal /app/, as it has since the blog had only the GMAT trainer to point at. The onward cards choose their trainer from what the post is about, through onward_for(), but nothing made the button agree with them, and nothing compared a post's trainer link with the exam its title names. It is INC-0128 again: that fix took the literal out of the exam guides and the study guide hub, and the blog template kept its copy.
- **How it surfaced.** Found on October 6, 2026 while previewing two queued GRE posts at phone width: the screenshot showed the box under a GRE post, and its link, read from the built page, was /app/. Counting every post by the first exam its title names gave the five live posts and 31 queued ones above. (Found by reading the code or the output)
- **Fix.** The button takes its link from the first exam the post's title names, through cta_href(): GMAT to /app/, GRE to /gre/app/, LSAT to /lsat/app/, SAT and PSAT to /sat/app/, ACT to /act/app/. A title that names no exam keeps /app/. The title is used rather than onward_for(), whose scoring sends the general Introducing the Study Room post to the LSAT trainer, because a button that starts practice should only change exams when the post says which exam it is about.
- **What stops it now.** build_blog.py checks cta_href() against fixed titles before it builds (an LSAT title goes to the LSAT trainer, SAT vs ACT to the SAT one, the lower-case word act to none), and after building it reads every post's Start a Free Round link back from the page and fails if it is not the trainer of the exam the title names. in `src/build_blog.py`
- **Lesson.** When you fix a literal that went stale because there came to be more than one of something, grep the whole site for that literal, not just the file you found it in. A link that points at the original trainer looks right on every page about the original exam, so it survives anywhere nobody reads a page about another one.

INC-0210. Every trainer greeted a visitor who had typed no name as Hunter, the owner's first name, and saved that name to signed-in accounts

2026-10-06, Wrong data shown or stored, `0f93ac0821defc5977b6b1aacfa24b31ca4947b6` *PR #267*

- **What was seen.** A first visit to any of the five trainers opened on 'Good morning, Hunter' with an avatar reading 'H', and the avatar's own markup said 'HR' until the script ran: a headless Chromium load of a fresh visitor on October 6, 2026 returned the heading 'Good morning, Hunter', the avatar 'H' and settings.name 'Hunter'. Hunter is the owner's first name and HR the owner's initials. The default was saved into the visitor's stored progress on the first load, so it would have outlived a fix to the default alone, and Cloud.push copied it to profiles.display_name for anyone who signed in: on October 6 the one profile in the database held 'Hunter' in display_name and in state_blob.settings.name. The admin user detail shows display_name, so an account that never typed a name was listed under the owner's name.
- **Why.** The trainer's personal defaults were the owner's own name and initials, and they were never replaced when it opened to the public: newState() in engine.js and the two settings fallbacks in app_template.html (on load and on import) set name to 'Hunter', updateHeader fell back to 'HR', the dashboard heading fell back to 'Hunter', and the avatar's static markup was 'HR'. They are present in the first commit of this repository's history (September 19, 2026). A greeting by the owner's name reads as correct to the owner, so nothing about it looked broken from the inside. And no test read what a new visitor is shown: every browser suite finishes onboarding and goes straight to items, games, consent or billing, and test.js checks a new state's skills and reviews but never its settings.
- **How it surfaced.** Found on October 6, 2026 while reading the trainer's state setup to write a screenshot script for the new GRE Quantitative Comparison items: the settings fallback named a person. A headless Chromium load of a fresh visitor confirmed the greeting and the avatar, and a read of the profiles table confirmed the synced copy. (Found by reading the code or the output)
- **Fix.** newState() and both fallbacks start with no name. The dashboard says the greeting alone and the avatar shows a plain figure when no name is set. A name typed in Account is marked nameSet, and forgetDefaultName in engine.js runs wherever a state comes in (this device's store, the account's copy on pull, an imported file): it clears an unmarked 'Hunter' and marks any other stored name as typed, because the default was the only name the trainer ever wrote for someone. Someone really called Hunter who typed it before this fix types it once more. The next sync writes the empty name to profiles.display_name as null.
- **What stops it now.** test.js checks for every exam that newState() carries no name, and that forgetDefaultName clears the old default, keeps a name typed as Hunter, marks an older typed name, and changes nothing on a second run. smoke_items.js opens every trainer as a new visitor and fails if the dashboard heading is anything but the bare greeting or the avatar shows any initials, then stores the old default the way a returning visitor has it, reloads, and fails if the name comes back. in `src/smoke_items.js`
- **Lesson.** A default is what everyone who never changes it is shown, so a personal value left as a default is published to every user, and it looks right to exactly the person who put it there. Anything a person would fill in about themselves, such as a name, initials or a goal, should default to empty, and some test should load the product as a stranger and read what it says to them. Once a bad default has been saved into people's stored state, fixing the default is not enough: the stored copies need a one-time migration, and a value the person typed needs a mark that tells it apart from the default.

Build system (14)

INC-0059. The item counter missed a whole bank file because it assumed a quoting style

2026-09-22, Silent loss, `de67a0eaeab023593685c7e5f5a0ffee944d3599` *PR #65*

- **What was seen.** A new bank added 35 items and the build reported the exam's total unchanged at 65. The engine tests, which load the bank for real, saw all 100.
- **Why.** The count is a regex over the concatenated source looking for an id in single quotes. The new file emits JSON escaped strings, so its ids are double quoted and none of them matched. Nothing compared the regex count to the number of items that actually load.
- **How it surfaced.** Noticing that the build summary and the test output disagreed about the same bank. (Found by measuring something)
- **Fix.** Accept either quoting style, and assert that every bank file listed for an exam contributes at least one counted item, so a file the pattern cannot see fails the build instead of counting zero.
- **What stops it now.** per-file contribution assertion in the bank counter in `src/build.py`
- **Cost.** a published item count 35 short, and a guard that would have kept getting quieter
- **Lesson.** A regex that counts things assumes a formatting convention, and a file that legitimately breaks the convention counts as zero rather than as an error. Any counter that can return zero for a non-empty input needs a per-source assertion, not just a total.

INC-0060. Nothing parsed the one file every user downloads

2026-09-22, Site down, `de67a0eaeab023593685c7e5f5a0ffee944d3599` *PR #65*

- **What was seen.** None shipped. A bank file with a syntax error built cleanly and the build exited zero, leaving invalid JavaScript in the bank the trainer loads on every visit.
- **Why.** The build parses every inline script in every built page, a guard added after an unescaped quote took the whole trainer down at parse time. The item bank is not an inline script. It ships as a separate file and was never in the list, so the largest generated artefact on the site was the one thing the parser never saw.
- **How it surfaced.** Deliberately breaking a bank file to test an unrelated guard, and noticing the build passed. (Found by a review bot or an adversarial pass)
- **Fix.** Parse every built bank file and every chunk of it, alongside the pages.
- **What stops it now.** the build node-parses each built bank and chunk in `src/build.py`
- **Cost.** would have broken every trainer on the next bank edit
- **Lesson.** A parse guard covers the file shapes someone thought of. When the same code moves into a new shape, a separate file, a chunk, a worker, the guard does not follow it. List what the guard covers against what the deploy actually ships, and check the difference rather than the intention.

INC-0063. The stale count guard checked two nouns and the page used a third

2026-09-22, Wrong data shown or stored, `ecc19483fbe90db9796f445d2811daeef3bb8f3d` *PR #67*

- **What was seen.** llms.txt told every model that reads it the LSAT bank holds 65 questions. It held 142. The build's count guard passed the file.
- **Why.** The guard matches a number followed by original or flashcards. The LSAT line was phrased as 65 questions today, deliberately, because the number was small and the page said so plainly. The phrasing that made the sentence honest is what put it outside the pattern.
- **How it surfaced.** Reading the file while fixing a different count the guard did catch, then watching the widened guard flag a correct sourced figure. (Found by reading the code or the output)
- **Fix.** Match a second, unambiguous pattern rather than more nouns. Widening the noun list to items and questions did catch the stale figure, and immediately raised a false positive on '64 questions', which is the real GMAT Focus question count from GMAC and not a bank size at all. A checker that cries wolf gets muted, so the guard now matches 'N original', 'N flashcards' and the specific phrase 'item bank is N', which names our bank and cannot match an exam fact.
- **What stops it now.** the count guard matches original, flashcards, items and questions in `src/build.py`
- **Cost.** a published figure less than half the true one, on the page written for machines
- **Lesson.** A guard keyed to wording is a guard on the wording, not the fact, and every synonym is a hole in it. Widening the wording is the obvious repair and it trades missed defects for false alarms, which cost more because they get the guard switched off. Match a phrase that only the thing you care about can produce, rather than every word it might happen to use.

INC-0064. The guard against a blind counter was itself blind to three exams

2026-09-22, Silent loss, `a91b158cb151cd88b19421a525e406d568f58ee3` *PR #68*

- **What was seen.** A new GRE reading bank of 26 items was registered, built, and counted as zero. The published GRE total stayed at 19,888. The guard written a day earlier to catch exactly this said nothing.
- **Why.** Two failures of the same shape, one inside the other. The GRE id pattern was G[QVE] and the new ids begin GR, so the counter could not see them. And the per-file guard added for INC-0059, which asserts that every listed bank file contributes at least one counted item, looped over only the two exams that happened to be in hand when it was written.
- **How it surfaced.** Predicting it from the id pattern before building, then watching the build confirm it by reporting the old total. (Found by reading the code or the output)
- **Fix.** Add R to the GRE pattern, and loop the guard over all five exams rather than two.
- **What stops it now.** the per-file counter guard covers every exam in APPS in `src/build.py`
- **Cost.** 26 items invisible to every published count, caught before merge

- **Lesson.** A guard that covers a subset of cases reproduces the original defect in the cases it skips, and it is more dangerous than no guard because the incident it was written for feels closed. When you add a check, enumerate everything of that kind and cover all of it, or state in the code which cases are deliberately excluded and why.

INC-0080. A build step that fails while the build exits zero, and a verification run that was a remembered subset

2026-09-22, Degraded, `2c72da1fde8a3819ca9bda7eabe9fb75ac0754b7` *PR #71*

- **What was seen.** Three commits shipped with a playbook that had not been rebuilt, because the ledger no longer checked out: INC-0077 carried an area of tooling, which is not one of the twelve the builder accepts. The site build printed WARNING and exited zero, so nothing local said no. CI said no, on the commit after the one that broke it.
- **Why.** Two things had to be true at once. The playbook build is deliberately non fatal, because playbook/ is excluded from the deploy and a broken chapter should not stop the site shipping, and it says so with a line beginning WARNING. And the run that was supposed to catch it was four smoke suites chosen from memory rather than the eight steps CI actually runs, so smoke_playbook, which does check this, was never run. Either alone is survivable. Together they mean a failure that is reported, in a form nobody was reading, to a process that had already decided what to look at.
- **How it surfaced.** The build-and-test job failed on GitHub with five playbook assertions, three commits after the one that introduced the bad row. (A test caught it)
- **Fix.** The area is corrected to testing, which is the label the builder has for a check. The non fatal line now begins ERROR rather than WARNING, because it is one: the exit code stays zero so the site still builds, and the word matches what happened so a reader scanning for failures finds it.
- **What stops it now.** the playbook build failure is reported as ERROR while still not blocking the site build, so a scan for failures catches it in `src/build.py`
- **Lesson.** Two habits, both mine rather than the code's. Verify with the sequence the pipeline runs, read out of its config, not with the subset you remember: a suite chosen from memory drifts to the parts that were failing last week. And when a step is deliberately non fatal, the word it fails with is the whole of its signal, so it has to be the word people grep for. WARNING on a line that means a deliverable did not build is an invitation to miss it, and the cost of saying ERROR while still exiting zero is nothing at all.

INC-0083. The rules digest promises to be prompt sized and its generator grows without bound

2026-09-22, Degraded, `5d7e45dc41329917f8576f90107e1eb36ebf2c89` *PR #73*

- **What was seen.** smoke_playbook failed with 'the digest stays prompt sized (4046 words)' against a 4000 word limit, on a commit whose only relation to the playbook was adding one incident. The bootstrap pack's RULES_DIGEST.md is the file a new project pastes into its prompt, so crossing the limit is the one failure that makes the deliverable useless for its purpose.
- **Why.** rules_digest prints every incident's lesson field in full, so the digest grows linearly with the ledger while the guard holds a fixed bound. At 82 incidents the lessons alone were 3,742 words against a 4,000 word budget. Nothing about the triggering commit was special: the next incident would have tripped it whoever wrote it, which is the signature of a bound asserted on an output nobody bounded.
- **How it surfaced.** test (A test caught it)
- **Fix.** The digest now prints the operative rule rather than the whole lesson: sentences from the front of the lesson until it reads as a complete thought, at least 80 characters. That is 2,119 words instead of 3,742, and 37 of the 82 lessons are already one sentence and are unchanged. The 80 character floor is not cosmetic: the companion guard asserts every lesson reaches the digest by looking for its first 60 characters, and 27 lessons have a first sentence shorter than that, so truncating at the first full stop would have satisfied one guard by breaking the other. The reasoning behind each rule stays in BUILD_PLAYBOOK.md, which the digest already tells the reader to consult.
- **What stops it now.** smoke_playbook already held the bound; what it lacked was a generator that respects one. The word count is now roughly flat per incident rather than proportional to the lesson someone happened to write in `src/build_playbook.py`
- **Lesson.** A size limit on a generated file is only a guard if something bounds the generator too; otherwise it is a delayed failure that lands on whoever commits next, and reads as their fault. When two guards constrain the same output, check the fix against both: shortening this file to satisfy the size check would have broken the completeness check that reads its first 60 characters.

INC-0084. The bootstrap digest ships incident ids to a project that has no incidents

2026-09-22, Cosmetic, `eea3d86f9e6851734eb453a94c6902afec4cb3df` *PR #74*

- **What was seen.** RULES_DIGEST.md, the file a new project pastes into its prompt, carries three rules that cite this repository's own incident ids: one reads 'which is INC-0059 and INC-0064 in a different costume' and another opens 'INC-0074 was a bare infinitive in a noun slot'. In the book those ids resolve to records a reader can turn to. In the bootstrap pack there is no ledger yet, so they resolve to nothing.
- **Why.** The digest is generated from the same lesson text as the book, and its docstring says it strips the specifics of this codebase out, because a rule competing with context is a rule applied inconsistently. An incident id is exactly such a specific and nothing stripped it. The generator was written thinking about length, which is measured and guarded, and not about audience, which is not.
- **How it surfaced.** Reading the generated digest after adding a section to it, rather than reading the code that generates it. (Found by reading the code or the output)

- **Fix.** The digest de-identifies incident references as it renders: a run of ids becomes 'two earlier defects' and a single one 'an earlier defect', capitalised when it opens a sentence. The book keeps the ids, because there they are links.
- **What stops it now.** smoke_playbook refuses an incident id anywhere in the bootstrap pack, which is the audience boundary stated as an assertion rather than as a docstring in `src/smoke_playbook.js`
- **Lesson.** A file generated for a different audience has to be read as that audience, not as the one that generated it. The size of this one was bounded and guarded; who it was for was written in a docstring and checked by nobody.

INC-0123. The Data Sufficiency answer counter outlived the run it belonged to, so a second run from the same seed made different items

2026-09-26, Degraded, `c7b1946dc4688fa7256c20309cf26f17bf85fae1` *PR #104*

- **What was seen.** All five GMAT Data Sufficiency schemas (gmat_ds_linear, gmat_ds_inequality, gmat_ds_average, gmat_ds_rectangle, gmat_ds_percent) made a different set of 300 items when framework.run was called a second time in the same process with the same seed, and clearing the schema's answer counter between the two runs brought back the first run's items exactly. The ROADMAP had recorded the difference for two of the five without a cause. Run twice at 60 items each, every other schema the build plans made the same items both times.
- **Why.** DSBase thins whichever answer is running ahead of an even share (INC-0081), counting the answers it has produced in self._seen, which lived on the schema instance for as long as the process did. The count describes one sequence of draws, but nothing reset it when a new sequence began, so a second run thinned against the first run's answers, called rng.random() at different points, and diverged from its first item on. The build was unaffected only because it runs each Data Sufficiency schema once; check_discarded, which draws from the same schemas after the bank run on a seed of its own to make its figure exact, was drawing with the bank run's count still in place.
- **How it surfaced.** Taken from the ROADMAP queue and reproduced by running each Data Sufficiency schema twice from one seed and a third time after clearing its counter; the same probe run over every schema in the build plan found no other. (Found by measuring something)
- **Fix.** The counter is tied to the rng it counts draws from. When make() is called with a different rng from last time, a new sequence of draws has begun and the count starts again from nothing. framework.run, check_discarded and any later caller each start with a fresh rng, so none of them has to remember to reset anything. The five schemas now make the same items on every run from the same seed, and the banks the build writes are byte for byte what they were.
- **What stops it now.** build_banks.check_repeatable() runs every schema the build plans twice from one seed, 60 draws each because the thinning starts after 40 answers, and fails the build naming any schema whose two runs differ. It takes about five seconds. in `src/build_banks.py`
- **Lesson.** State that describes a run has to live and die with the run. Keyed to the object that makes the draws, it cannot outlive them; kept on a long lived instance, it quietly couples every run to every earlier one, and the output depends on call history that no one reading a single run can see.

INC-0126. The runner's 400 draw cutoff dropped the last question of a small reading schema on three of four builds

2026-09-26, Silent loss, `3d9b66ecfb7150707e5bd8eaf72514aa8a7aa116` *PR #113*

- **What was seen.** Of the four builds on September 26 that added reading passages, three shipped one question fewer than the passages define. Three short passages gave the GRE 29 new items instead of 30. The first batch of five long passages left out a GMAT stated idea question, and the second left out a GRE one: what the earlier account in the guilds passage failed to address. Nothing failed. The counts were one short, and the ROADMAP recorded it as expected behaviour.
- **Why.** framework.run retires a schema after 400 duplicate draws in a row. That rule was meant for schemas whose space is unknown and large. A reading schema's space is small and exact: a fixed list of questions per passage. Near the end of a run only one question is still unseen, and each draw finds it with probability one in the size of the space. So 400 misses in a row become likely as the space grows. At the corpus's present size, the GMAT stated idea schema (252 questions) ends a build short about one time in four, and the GRE one (168) about one time in ten. Every passage added makes this worse, while the 400 stays put. The schema knew exactly what its questions were; the runner never asked.
- **How it surfaced.** Noticed across the day's passage builds, where the new item counts came out one short three times in four. It was then measured: listing every question each reading schema can ask and comparing the list with what the build shipped found the one GRE question missing and nothing else. (Found by measuring something)
- **Fix.** Each reading schema now lists its questions through asks(), per passage, and so do the two LSAT argument structure schemas, per argument, which had the same shape and the same exposure (framework.ListsQuestions). make() draws from the same list, so the list and the draws cannot differ. The schema states its space as the number of questions listed, and the mapping wrappers pass both through. The runner retires a schema that states a space once it has made that many. It gives up on the rest only after twenty draws per question in a row, where before it gave up after 400. The chance of giving up on a question that can still be drawn is then about two in a billion. Schemas that do not state a space are retired exactly as before, so no other bank changes.
- **What stops it now.** For every schema that lists its questions, build_banks.py compares the questions shipped with the list. The build fails, naming the schema and a question, when a category under its target is missing one of them, or when any category ships a question the list does not have. in `src/build_banks.py`
- **Lesson.** A heuristic stopping rule is right only when nothing better is known. Where the size of what is being collected is known exactly, stop at that size and check that it was reached. Do not stop when the collection merely seems to have gone quiet, because the odds of a quiet spell grow with the collection while the threshold stays fixed.

INC-0129. The rules digest crossed its word budget again, because the earlier fix bounded each rule and not the digest

2026-09-26, Degraded, `97ea315375fb1fb12f5ebdd98b856c5a3d0b43ce` *PR #115*

- **What was seen.** smoke_playbook failed with 'the digest stays prompt sized (4012 words)' against its 4000 word limit, on the commit that added the 128th incident, whose only relation to the playbook was that one record. The bootstrap RULES_DIGEST.md is the file a new project pastes into its prompt, so it crossing the limit makes the deliverable too long for its purpose.
- **Why.** INC-0083 fixed the same failure at 82 incidents by printing each lesson's operative rule instead of the whole lesson. That bounded the size of each rule, at about 30 words, and not the size of the digest, which still printed one rule per incident, so it kept growing by about 30 words a record and crossed again at 128. Printing every rule and staying under a fixed budget cannot both hold as the ledger grows. The companion check that every lesson reaches the digest made the budget the one to give way, and nothing in the generator decided which rules a prompt sized digest should keep.
- **How it surfaced.** smoke_playbook, run locally before pushing the INC-0128 fix. (A test caught it)
- **Fix.** The generator now holds the budget itself. It always keeps every rule the build learned more than once, then adds the rest in order of severity, oldest first within each level, while they fit. It says how many rules it left out and that the checklist in BUILD_PLAYBOOK.md carries every one, since the checklist was already checked to include every lesson. The kickoff no longer claims the digest holds one rule for every defect.
- **What stops it now.** smoke_playbook still holds the 4000 word limit. In place of every lesson reaching the digest, it checks three things: every rule learned more than once is there; the rules present are a prefix of the priority order, so none left out outranks one kept; and the count the digest states as left out equals the number of lessons missing from it. in `src/smoke_playbook.js`
- **Lesson.** When a generated file has a size limit and a completeness rule, decide which one wins before the file grows, because it will. Bound the whole output in the generator, and let the file say what it left out and where the rest is, so omission is a stated choice rather than a silent one.

INC-0162. The blog build read a fee table as one sentence and refused a correct post for calling the SAT trainer unfinished, over College Board's seat Waitlist

2026-09-27, Degraded, `75a55429214346dcc3f56bdb48603c64812db29c` *PR #158*

- **What was seen.** python3 src/build_blog.py refused the queued post "How Much Does the SAT Cost in 2026?" with "calls a live trainer unfinished", quoting a run of text that began inside its fee table ("ET before test day $44 Waitlist $0 Source: College Board's SAT test fees..."). The post says nothing about the trainer being unfinished: "Waitlist" is the name of College Board's service that places a test taker in an open seat, and it cost $0 in the table's last row.

- **Why.** The check that a post never calls a live trainer unfinished (INC-0137) strips a post's tags with a space and splits what is left at sentence punctuation, so a table's cells, which carry no punctuation, run together with the paragraph after them into one sentence. That sentence held both "wait list" in College Board's sense and "SAT", which is what the check looks for. The same check over built pages (INC-0138) had already learned that block elements end a sentence, after the site header's menu read as one run-on sentence; the post version was never given the same rule.
- **How it surfaced.** Found on September 27, 2026 when the build refused the SAT cost post on its first preview build. (A build guard caught it)
- **Fix.** The post check splits a post at block elements, table cells included, before it looks for sentences, as the built-page check does, and reads each FAQ question and answer apart.
- **What stops it now.** build_blog's self-check holds a fee table whose row names College Board's Waitlist beside an SAT fee, which must pass, and a paragraph saying the SAT trainer is in development, which must still be refused. in `src/build_blog.py`
- **Lesson.** Two copies of one check drift apart. When a check exists in two places, a fix to one is a question about the other, and the cheapest answer is to make them share the code that splits text.

INC-0193. The second LSAT reading bank had a generator that no longer wrote it, and running the generator would have put back 25 corrected lines

2026-09-29, Degraded, `c69493cfaaa32e4cf9dad245c4db96d83a4282a7` *PR #207*

- **What was seen.** src/mk_bank_lsat_rc2.py, whose docstring says it emits src/bank_lsat_rc2.js and which writes there when given no path, produced a bank that differs from the committed one on 25 lines in 23 of its 35 items. Run over the bank, it would have taken off 12 clauses added to even out answer lengths, brought back four garbled seams such as 'in every species in which the effect has been observed been observed', returned six Watch for notes to naming choices by the order they were written in, and put three keys back into the passage's own words. test.js would have failed three checks (two of the seams, two notes that name the key, and one length rank holding 46 percent of the keys against a recorded 36); the other two seams, four notes pointing at the wrong choices and three keys would have gone back without a failure.
- **Why.** The bank was generated on September 21 (#65) by a script with its own writer, which put no 'Generated by' line in the file. The next day #70 added src/bank_repair.py so that hand written banks could be corrected 'without regenerating a file that was never generated', and used it on this file, which had been generated; #97, #204 and #205 then corrected the bank as well, because that is where each defect showed. The nine generators written since stamp their banks 'Generated by ... Edit that file, not this one', but nothing compared any bank with what its generator writes, so a bank and its generator could part without anything noticing.

- **How it surfaced.** Noticed on September 28, 2026 while moving Watch for notes into the generators (INC-0189): this was the one generator whose notes were not converted, because its output no longer matched the bank. On September 29 each of the ten bank generators was run into a scratch directory and compared with its bank: nine match byte for byte, and this one differs on 25 lines, 12 of them from #70, 4 from #97, 6 from #204 and 3 from #205. Swapping its output into a scratch checkout and running test.js measured what the suite would and would not have caught. (Found by reading the code or the output)
- **Fix.** src/bank_lsat_rc2.js now says in its header that it is the source and is edited by hand, and src/mk_bank_lsat_rc2.py refuses to write it and says why. Given another path it still writes what it wrote on September 21, which keeps it as the record of the content decisions its docstring describes.
- **What stops it now.** src/check_bank_sources.py, which build.py runs, regenerates every bank whose header says 'Generated by' into a temporary directory and fails the build when the result differs from the committed bank at all, and fails when a generator's default output is a bank that does not name that generator, which is the state this bank was in. in `src/check_bank_sources.py`
- **Lesson.** A generated file that someone edits by hand has two sources, and the next run of the generator silently picks one of them. Put the edit in the generator or retire the generator in writing, and have the build regenerate every generated file and compare it, because nothing else notices a generator that no longer makes its file until someone runs it.

INC-0204. A queued post's title was checked for Title Case only once it went live, so a title the build refuses would have failed the deploy on its publish day

2026-10-06, Degraded, `c13caebbded421c36f84b1a7a9438ab334d282d6` *PR #237*

- **What was seen.** Writing the post queued for January 14, 2027, its title 'Which MBA Round Should You Apply In? What Schools Say' passed build_blog.py while the post was held. Building with BLOG_BUILD_DATE=2027-01-14 exited 1: the built page's h1 breaks the Title Case rule ('Apply in?'), so the deploy on January 14 would have stopped, holding that post and every later change until someone fixed it. The other 49 held posts pass when built as live.
- **Why.** validate() reads every post, held or live, but checks only the headings inside its body. The title becomes a page's h1 only when the page is built, and the built-page checks (headings, button labels, articles, design tokens, trainer claims, offsite scripts) read only the pages this build publishes. So a held post's title, and anything else that exists only on its built page, was first checked on its publish day, by the deploy that published it.
- **How it surfaced.** Found on October 6, 2026 by building the blog with BLOG_BUILD_DATE set to the new post's date to look at it at phone width, which printed the Title Case failure and exited 1. (Found by rendering it and looking)
- **Fix.** build_blog.py builds every held post into a temporary directory as it will publish, with every post live, and runs the same page checks on those pages that it runs on the published ones, so a held post that would fail on its day fails the build now. The new post's title now reads 'Which MBA Round Should You Choose? What Schools Say'.

- **What stops it now.** build_blog.py checks the held posts as built pages on every build, and its self-check confirms that a held post whose title breaks Title Case fails it. in `src/build_blog.py`
- **Lesson.** A check that runs only on what is published tests today's site, not tomorrow's. When content is queued to publish by date, run every publish-time check on the queued content too, built as it will be, or the first time the check sees that content is the deploy that depends on it.

INC-0212. Admin > Social showed an empty queue on the live site, because the queue is written by a script the deploy build never runs

2026-10-19, Degraded, `235fea9250c812432a1d12efb071f7b43c01e1d4` *PR #269*

- **What was seen.** https://startfromnowhere.com/social.js returned 404 on October 19, 2026. Admin > Social loads that file on demand, so on the live site the tab rendered an empty queue, and the error beacon logged 'social.js: Failed to load SCRIPT' on /app/ on October 6.
- **Why.** build.py copies src/generated/social.js to the site root only if the file exists, and only build_social.py writes it. Nothing in build.py, in CI or in the Cloudflare build command (python3 src/build.py && python3 src/build_blog.py) runs build_social.py. This sandbox had the file from a hand run on September 26, so every local check passed; the deploy builds from a fresh clone where it never existed. The 'if it exists, copy it' step turned a missing generator into silence.
- **How it surfaced.** Found on October 19, 2026 while reading three weeks of the live error log for real visitor errors: one 'social.js: Failed to load SCRIPT' row on /app/, and the file answered 404 on the live site. (Found by measuring something)
- **Fix.** build.py runs build_social.main() itself, after the banks it reads are built, and fails the build if social.js was not written to the site root.
- **What stops it now.** build.py fails the build when the social queue is missing after generation, so a fresh clone either ships the file or does not deploy. in `src/build.py`
- **Lesson.** An 'if the file exists, copy it' step makes a missing generator invisible. A file a page loads should be produced by the same build that deploys the page, and that build should fail when it is missing; a local checkout that once ran a step by hand is not evidence that the deploy runs it.

Search and metadata (12)

INC-0002. School URLs vanished from the sitemap when the data file was split

2026-08-24, Silent loss, `df14c7c` *PR #23*

- **What was seen.** Per-school pages were live and correct, and absent from sitemap.xml.
- **Why.** The sitemap generator read schools.json. When the library was split into one file per school, nothing read the new directory, and a loop over an empty list emits nothing rather than failing.

- **How it surfaced.** Noticed while reviewing the built sitemap. (Found by reading the code or the output)
- **Fix.** Generate sitemap entries from the same directory scan the pages are built from.
- **What stops it now.** the build asserts the sitemap URL count matches the page count in `src/build.py`
- **Cost.** unknown period of missing indexation
- **Lesson.** A refactor that moves data has to be followed to every reader, and a loop over nothing is the quietest failure in programming. Derive counts from one source and assert they agree.

INC-0014. A hardcoded count in the meta description went stale, and Google showed it

2026-09-19, Wrong data shown or stored, `36b1e92` *PR #43*

- **What was seen.** The meta description claimed 1,451 colleges after the number changed. og:description and twitter:description both derive from that tag, so all three were wrong.
- **Why.** A figure typed into a template instead of computed at build time, which is the exact failure build-time counting exists to prevent.
- **How it surfaced.** Reading the built page. (Found by reading the code or the output)
- **Fix.** Derive the number from the same count the pages are built from.
- **What stops it now.** no hardcoded corpus counts in templates in `src/build_rankings.py`
- **Cost.** a wrong figure in the search result snippet
- **Lesson.** Any number in user-facing copy that describes the size of something must be computed from that thing at build time. The moment it is typed, it has a half-life.

INC-0049. Two pages told the same story with different numbers after a reweighting

2026-09-19, Wrong data shown or stored, `a34a798` *PR #47*

- **What was seen.** The rankings index still showed figures from before a scoring change while the methodology page showed the new ones.
- **Why.** Two artefacts derived from one model, and only one was regenerated.
- **How it surfaced.** Reading both pages after the change. (Found by reading the code or the output)
- **Fix.** Derive both from the same computed values at build time.
- **What stops it now.** both figures are printed by the build from one source in `src/build_rankings.py`
- **Cost.** a visible contradiction between two pages
- **Lesson.** When one model feeds two pages, generate both from the model in the same pass. Two places that must agree will not, and the reader who notices is the reader you were trying to convince.

INC-0103. A 139 page section shipped with no route into it from the sitemap

2026-09-24, Silent loss, `52026960246aaac40d2d261dbf0bbb5f48a1d3f1` *PR #95*

- **What was seen.** The study guide went live on 2026-09-22 with 139 pages across five exams and thirteen sections. The sitemap carries 1,592 URLs and not one of them is a guide page. Every other content section is there: 1,453 colleges, 92 schools, 31 blog posts, 6 exams. The pages resolve and are linked from the site header, so nothing looks wrong, and a search engine's cheapest and most reliable route into the largest new body of content on the site does not exist.
- **Why.** build_sitemap opens with a hand written list of section URLs and then expands the data driven ones from data/exams.json, data/schools and data/colleges. A new section is therefore invisible to it until somebody remembers to add a line. Nothing about building 139 pages causes them to appear, and nothing about their absence causes a failure, so the omission is silent in both directions.
- **How it surfaced.** Found during a search performance audit, three days after the guide shipped, by reading the live sitemap rather than the code that writes it. No check looks at the sitemap at all: the build validates the pages it writes and never asks whether anything can find them. (Found by measuring something)
- **Fix.** The sitemap walks the built output for guide pages rather than naming them, so the section cannot fall out of step with itself, and a section added later is carried without a code change.
- **What stops it now.** build_blog compares every built index.html under the site's content directories against the sitemap it just wrote, and fails when a page that was built is not listed. The check is on the BUILT OUTPUT rather than on a list of sections, because a list of sections is the thing that was wrong.
- **Lesson.** An enumeration that has to be kept in step by memory will fall out of step, and the failure is silent because nothing downstream can tell the difference between a section that was excluded on purpose and one that was forgotten. Derive the list from what was actually produced, and when that is not practical, make something compare the two. Publishing work nobody can find is not a smaller version of publishing it.

INC-0104. Half the school pages opened with a data note pasted into a sentence, and the structured data repeated it

2026-09-26, Degraded, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** On 47 of 91 MBA school pages the opening paragraph, the first prose on the page, read like Kellogg's: 'The Class of 2027 profile reports a average GMAT Focus score, middle 80 pct 645-735 GMAT Focus of 687, a class of 534.' Notre Dame's read 'reports a avg, Class of 2026, range 560-760 (edition not labeled; range consistent with Classic) GMAT of 680 (Classic edition)'. The same text answered 'What GMAT score do you need' in the Quick Answers block and in the FAQPage JSON-LD that search and answer engines read. Separately, 70 pages said a school 'ranks #5 of 91' in the first sentence and 'number 5 of the 70 programs it scores' in the next paragraph, because 21 of the 91 programs are listed unscored. The rankings table itself cut the same note to its first three characters beside every GMAT figure, so all 50 rows that carry one showed a slice rather than a statistic: 'Focus ave' on 22 of them, and 'Focus lis', 'Classic typ' and 'Classic 202' on one each. The class label had the same problem: 22 intros pasted the stored class_year verbatim, producing 'The 2025 Profile profile reports', 'The Typical class profile (no class year stated) profile reports' and 'The Full-time MBA Graduates, 2025 (outcomes) profile reports'. And four pages reported a figure from one class inside a sentence about another: Notre Dame's GMAT comes from Poets&Quants coverage of the Class of 2026 and was stated as the Class of 2027's.
- **Why.** The stat field on a profile figure is free text written for the table's provenance column ('average GMAT Focus score, middle 80 pct 645-735'), not a word for a sentence. Three sentences on the page interpolated it. lead_paragraph was fixed for exactly this in PR 45 (commit 4b25169, whose comment reads 'only a recognised one word statistic is used and anything else becomes a reported'), and the fix was written inside that one function. The intro and the FAQ are built seventy lines further down in the function that calls lead_paragraph, and kept interpolating the raw field. The denominator is the same shape: lead_paragraph was taught the scored total and the intro was not. The rankings table is a fourth consumer: it sliced stat[:3] on the assumption that every note begins with the statistic's name. class_year is free text too, stored in more than twenty shapes, and the intro treated it as a noun phrase that always fits 'The ... profile'. Each figure also carries its own class in its note or source, which a single page-level label cannot represent.
- **How it surfaced.** Found on 2026-09-26 by reading the built text of the school pages with the most Search Console impressions, while checking what they say against the queries they are shown for. No check read the prose on a school page: validate_schools reads the data files and the build validates structure, not sentences. (Found by reading the code or the output)
- **Fix.** One helper, stat_kind, reads a note and returns average, median or nothing, and every consumer outside the provenance column calls it: the intro, the lead, each FAQ answer, the meta description and the rankings table. The note itself stays in the provenance column where it belongs. Every rank claim on the page uses the scored total. The class label is parsed into one of four forms a sentence can carry (the Class of YYYY, the class that entered in YYYY, an academic year, or 'the most recent published profile'), and each figure is reported under its own class where its source names one, with past tense for a class that has graduated.

- **What stops it now.** build_rankings checks every school page before writing it: no multi-word stat note of that school may appear in the intro, the lead or an FAQ answer, no sentence may read 'a average' or 'a avg', and every 'of N' rank claim must use the scored total. A failure names the school and the sentence. The same check fails on a pasted label ('profile profile', ') profile', 'no class year stated') and on any sentence that states a figure whose source names its class under a different 'Class of' year.
- **Lesson.** When a fix names a class of input, such as 'the stat field is free text', find every place that input is used before closing it. A correct rule written inside one caller protects that caller, and the existence of the rule makes the unprotected siblings look protected.

INC-0107. Seven college titles promised SAT scores the page does not have

2026-09-26, Cosmetic, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** Seven of the 1,451 college pages report an ACT midpoint and no SAT figure of any kind, and their title still read '...: Acceptance Rate, Cost, and SAT Scores'. A student searching for that college's SAT scores who clicked found none.
- **Why.** title_bits added 'SAT Scores' when either sat_avg or act_mid was present, treating the two tests as one field. The function's own docstring says it promises only what the page holds, and for these seven it did not.
- **How it surfaced.** Found on 2026-09-26 while reading the college title logic against the Search Console export, by counting pages whose data has act_mid and no sat field. (Found by reading the code or the output)
- **Fix.** The title names the test the page actually reports: SAT Scores when any SAT figure exists, ACT Scores when only the ACT does.
- **What stops it now.** build_colleges fails when a page's title names SAT or ACT scores and the page's data has no figure for that test.
- **Lesson.** Test every noun in a title against the page's own data, because a title is a promise to someone who has not seen the page yet. Grouping two fields under one flag because they usually travel together is how a promise gets made on the pages where they do not.

INC-0109. Two withdrawn exam guides kept their search traffic and sent it to a 404 that is the whole GMAT trainer

2026-09-26, Silent loss, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** The MCAT and Executive Assessment guides were withdrawn on 2026-09-18 because their official sources could not be re-verified. Search Console for the five weeks to 2026-09-24 still showed 220 impressions for /exams/mcat/ and 82 for /exams/executive-assessment/, the second at an average position of 9.6, with queries like 'executive assessment test cost'. Both URLs answered 404, and the 404 page is the GMAT trainer, so a searcher looking for the Executive Assessment landed in a GMAT app with no word about what had happened. llms.txt, the file written for answer engines, still listed both guides among the exam guides the site publishes.
- **Why.** Withdrawing a page was treated as deleting a file. Nothing in the build or the deploy knows which URLs used to exist, so nothing asked what a visitor arriving at one should see, and the generic 404 handling that suits a mistyped URL was applied to a URL search engines were still ranking.
- **How it surfaced.** Found on 2026-09-26 in the owner's Search Console export: both URLs appear in the top pages list with impressions after their removal date, and fetching them from this environment returned 404. (Found by measuring something)
- **Fix.** src/worker.mjs keeps a RETIRED table of withdrawn paths and answers each with a 301 to the exams hub, on either hostname, with or without the trailing slash, keeping the query string. The hub says which exams are covered. An entry is removed when its page returns. llms.txt now lists the five exams actually covered and says the other two were withdrawn and why.
- **What stops it now.** src/smoke_redirect.js asserts the retired paths redirect in one hop and that /exams/ itself and lookalike paths are served normally. The recorded practice is the larger guard: withdrawing a page means adding it to RETIRED in the same change.
- **Lesson.** A URL that search engines know is not the site's to delete quietly; it belongs partly to everyone still linking to it. Removing the content is one decision and deciding what that address now says is another, and skipping the second one sends real visitors to whatever the 404 happens to be.

INC-0131. The exam hub's structured data described every exam with a printed Python dict

2026-09-26, Degraded, `4727c2e59c09a7d1bbe45f95e917b8cfab2ee836` *PR #117*

- **What was seen.** The ItemList structured data on /exams/ described each exam as 'Scored' followed by the Python repr of its score_scale record, braces, quotes, src, year and url included: Scored {'text': 'Total score 205 to 805, always ending in 5; ...', 'src': 'GMAC', 'year': 2026, 'url': '...'}. That is the text search and answer engines read for each exam, it was on every exam, and it had been since the page was added on September 19 (#45).
- **Why.** score_scale is a sourced record of text, src, year and url, and the description interpolated the record itself with an f-string rather than its text. Python formats a dict without complaint, so the build succeeded, and structured data is not drawn on the page, so no one reading the page could see it.
- **How it surfaced.** Found on September 26, 2026 while tracing where the GRE score scale text was published, for INC-0130. (Found by reading the code or the output)
- **Fix.** The description uses the record's text.

- **What stops it now.** build.py scans every built page for the printed form of a Python dict or list, such as {'text': or 'src':, and fails naming the page and the fragment, since no page on the site has a reason to contain one. in `src/build.py`
- **Lesson.** Structured data is published text that no one reads on the page, so check it the way a page is checked. Interpolating a record where its text was meant succeeds silently in Python, and the only place the mistake shows is the output, so scan the output for the printed form of a data structure.

INC-0138. The exam guides hub told search engines only the GMAT and SAT trainers were live, eleven days after all five were

2026-09-27, Wrong data shown or stored, `a91fe6f677c6f3a45121f4e9bb1026a81663fcc5` *PR #124*

- **What was seen.** The /exams/ page's meta description, the text a search result shows under the title, read 'GMAT Focus and digital SAT training are live; more exams in development.' The GRE, LSAT and ACT trainers went live on September 16, 2026, so every search result for the hub understated the product by three trainers and promised work that was already done.
- **Why.** The description was typed into exams_index_template.html when two trainers were live, while build_exams.py already knew which exams are live (LIVE) and used it for every other status on the page. A sentence about the product's state was written once as copy instead of being built from the one place that knows it, and no check reads meta descriptions for claims about trainers.
- **How it surfaced.** Found on September 27, 2026 while adding a calculators section to the hub, the same day INC-0137 found a queued post calling the live LSAT trainer unfinished. (Found by reading the code or the output)
- **Fix.** The hub's description is now built from LIVE and the exams' names, so it changes when a trainer does.
- **What stops it now.** build_exams.py builds the hub's description from LIVE and fails if it leaves out a live trainer; page_checks.trainer_claims() reads every built page's text and meta description, treating block elements as sentence breaks, and fails the build on a sentence that names a live exam and calls something in development, coming soon or on a waitlist; build_blog.py runs the same pattern over posts. in `src/page_checks.py`
- **Lesson.** Any sentence about your own product's state is data, and data belongs in one place: build the sentence from the source that knows, and check every page that could repeat it, including the text only search engines read.

INC-0153. Five school pages were named only "School of Business", "College of Business" or "Graduate School of Management", so their titles and answers named no school

2026-09-27, Degraded, `6edd506aa79fd565dab17809a3fa936e3193bfd3` *PR #145*

- **What was seen.** data/schools/charleston.json, lehigh.json, portland-state.json, uc-davis-gsm.json and uc-riverside.json gave their name as "School of Business", "College of Business", "The School of Business", "Graduate School of Management" and "School of Business". A school page prints name in its title, heading, lead sentence and every Quick Answer, so College of Charleston's page was titled "School of Business MBA: Cost and Class Profile" and asked "What is the acceptance rate at School of Business?": a title no search for the college could match and questions that name no school.
- **Why.** The research merge copied each business school's name as the school's own site prints it in its header, where the university's name sits beside it and nothing more is needed. The library keeps the university in a field of its own, and every page that prints name prints it alone. validate_schools checked that a name was there, not that it said which school it was.
- **How it surfaced.** Found on September 27, 2026 while reading the new GMAT Quick Answer on College of Charleston's page: "What GMAT score do you need for School of Business?". Searching the library for names made only of words like School, College, Graduate, Business and Management found these five. (Found by reading the code or the output)
- **Fix.** Each of the five now carries a name its school's own site uses that says which school it is: Charleston School of Business, Lehigh College of Business, Portland State University School of Business (its header reads Portland State University, The School of Business), UC Davis Graduate School of Management and UC Riverside School of Business.
- **What stops it now.** validate_schools refuses a school name made only of generic words (school, college, graduate, business, management and the like), because a name printed on its own has to say which school it is. in `src/validate_schools.py`
- **Lesson.** A value copied from a page carries the context the page gave it, and loses that context wherever it is printed on its own. A name that is plain under its university's banner is anonymous in a page title, so store the form that stands alone.

INC-0198. The search report told us to redirect two URLs the Worker already redirects

2026-10-03, Degraded, `79ec340e6921eb8ea737c4d14963a09f18af54bc` *PR #218*

- **What was seen.** Run on the owner's October 3 Search Console export, src/gsc_report.py listed /exams/mcat/ (223 impressions) and /exams/executive-assessment/ (88 impressions) under its heading for URLs with impressions that the build does not produce, with the instruction to redirect them and see RETIRED in src/worker.mjs. Both have been in RETIRED since INC-0109 and both already answer with a 301 to /exams/. Acted on, the advice meant adding entries that already exist, or concluding that a working redirect had stopped working.
- **Why.** The check asked one question, whether the build writes a file at the URL's path, and treated every no as a page that still needs a redirect. A URL can be served two ways, by a built page or by the Worker's RETIRED table, and the check knew only the first. The table its own message told the reader to consult was the one place it never looked.

- **How it surfaced.** Found on October 3, 2026 while reading the report on the owner's October 3 export against src/worker.mjs before acting on it. (Found by reading the code or the output)
- **Fix.** gsc_report.py reads the RETIRED table from src/worker.mjs and lists retired URLs separately as already redirected, so its warning names only URLs that are neither built nor redirected.
- **What stops it now.** The report stops with an error if src/worker.mjs declares a RETIRED table it cannot parse, so a change to the table's format cannot quietly switch the exclusion off and bring the false alarm back. in `src/gsc_report.py`
- **Lesson.** A check that reports what is missing has to know every way the thing can be present. When its advice names the place where the fix belongs, it should read that place first; otherwise it keeps recommending work that was already done, and the people reading it learn to ignore it.

INC-0213. Score calculators linked a study guide page that has never existed, and 54 college pages linked a category the college list does not have

2026-10-19, Degraded, `235fea9250c812432a1d12efb071f7b43c01e1d4` *PR #269*

- **What was seen.** The 'Study Guide' button on the GRE, SAT, LSAT and ACT calculators pointed at /guide/gre/, /guide/sat/, /guide/lsat/ and /guide/act/. None of the four has an index page: the guide is published as /guide/ plus one page per section (/guide/gre/quant/ and so on), so every click on those buttons landed on the 404 page. A sweep of all 1,879 built pages on October 19, 2026 found these four and no other broken internal link. The same sweep found the 54 special focus institutions' pages linking /colleges/#special-focus. The college list picks a tab from its hash and has only four ranked categories, so that link quietly opened National Universities instead of the list the button named.
- **Why.** The calculators were written by hand, one template per exam, and each guessed the guide's address from the pattern of the exam pages (/exams/gre/ exists, so /guide/gre/ looked right). Nothing checked a hand written link against the built site: smoke_launch.js follows links only from the pages it opens and is not in CI, and the build validates blog links to sibling posts but not links from other templates.
- **How it surfaced.** Found on October 19, 2026 by a sweep written to answer the owner's request to make sure there were no bugs: every href and src in every built page, resolved against the files the deploy serves and the Worker's redirect table. (Found by reading the code or the output)
- **Fix.** Each exam's section on /guide/ carries an id named for the exam, and the four buttons point at /guide/#gre, /guide/#sat, /guide/#lsat and /guide/#act. Special focus institutions link the whole list as All Ranked Colleges, and each category tab on /colleges/ carries its key as an id, so category links are checked like anchors.
- **What stops it now.** src/check_links.py resolves every internal href and src in every built page against what the deploy serves (the .assetsignore exclusions, the .html and index.html handling, and the Worker's RETIRED redirects) checks that a #fragment on another page names an element on that page (pages that route by hash, the trainers and the forum, are listed and exempt), and fails on any that do not resolve; CI runs it after the build. in `src/check_links.py`

- **Lesson.** A link typed by hand is a guess about where something lives, and a guess that follows a pattern looks right to every reviewer. The only reliable check is to resolve every link in the built site against what is actually served, on every build, rather than following links from a sample of pages.

CSS and layout (9)

INC-0050. A landing-page icon referenced a colour token that did not exist

2026-08-19, Cosmetic, `5337dfd`

- **What was seen.** One game icon on the landing grid rendered without its colour.
- **Why.** An undefined custom property, which CSS treats as a fallback rather than an error.
- **How it surfaced.** Looking at the page on a phone during a mobile pass. (Found by rendering it and looking)
- **Fix.** Use a defined token.
- **What stops it now.** weekly_audit checks computed colour on every page in `src/weekly_audit.js`
- **Cost.** one icon, and the same root cause later cost a compliance control
- **Lesson.** The same undefined-property failure will find you repeatedly, at every severity from one icon to an invisible legal control. One audit of computed colour catches the whole class.

INC-0010. CSS comments do not nest, and one placeholder killed the palette on 1451 pages

2026-09-17, Degraded, `61c3ed2` *PR #40*

- **What was seen.** The colour palette was gone from every college page. It looked like a design problem and was chased as one for hours.
- **Why.** A `{{CHROME_CSS}}` placeholder was left inside an opening CSS comment in rankings_base.css. The first */ closes the outer comment, so everything after it was live CSS that overwrote the palette.
- **How it surfaced.** Eventually, by reading the built CSS rather than the template. (Found by reading the code or the output)
- **Fix.** Remove the nested comment.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** hours, on 1451 pages
- **Lesson.** CSS comments do not nest. When a whole page category loses its styling, read the built artefact, not the source that produced it.

INC-0048. Error recovery in the CSS parser swallowed the palette silently

2026-09-17, Degraded, `a920f5c` *PR #41*

- **What was seen.** Score bars rendered as empty gaps and panels lost their borders on 1,451 college pages and the rankings.
- **Why.** A nested comment ended early, the leftover words became a selector, and the CSS parser's error recovery discarded the :root colour block that followed it. Every grey and navy variable on those pages resolved to nothing. There is no error anywhere in this chain; CSS is specified to recover and continue.
- **How it surfaced.** The missing borders eventually gave it away, after the missing colours did not. (Found by reading the code or the output)
- **Fix.** Remove the nested comment.
- **What stops it now.** weekly_audit checks computed colour on every page in `src/weekly_audit.js`
- **Cost.** the palette on over 1,450 pages
- **Lesson.** CSS never fails loudly. A malformed rule is skipped, a bad selector eats the block after it, and an undefined variable paints as nothing. Anything that matters visually has to be asserted on the rendered page, because the parser will not tell you.

INC-0018. The Do Not Sell button was invisible from the day it shipped

2026-09-19, Degraded, `a34a798` *PR #47*

- **What was seen.** White text on a transparent background on a white page. A legally required control that nobody could see.
- **Why.** The page referenced nine colour custom properties that exist nowhere. CSS does not treat an undefined custom property as an error; it falls back to the initial value and paints.
- **How it surfaced.** An automated contrast audit across every page. (A test caught it)
- **Fix.** Define the tokens and fix seven contrast failures, all site wide because all lived in shared chrome.
- **What stops it now.** weekly_audit checks computed contrast on every page in `src/weekly_audit.js`
- **Cost.** an invisible compliance control for the life of the page
- **Lesson.** An undefined CSS custom property is silent. Audit computed colour, not authored colour, and do it on the rendered page.

INC-0029. A sequential colour ramp inverts direction between themes, and the label text did not

2026-09-21, Degraded, `8af7188` *PR #61*

- **What was seen.** White heatmap labels on a light cell in dark mode.
- **Why.** A sequential ramp runs light to dark in a light theme and dark to light in a dark theme. The label colour was hardcoded white, so it was correct in one theme and unreadable in the other.
- **How it surfaced.** Rendering the chart in dark mode and looking at it. (Found by rendering it and looking)
- **Fix.** An --sfnc-on-seq token that flips with the theme.
- **What stops it now.** smoke_charts asserts token parity between the light and dark blocks in `src/smoke_charts.js`
- **Cost.** an unreadable chart in one theme
- **Lesson.** A dark mode does not break by having a wrong colour. It breaks by missing one. Any colour paired with a ramp has to move with the ramp.

INC-0139. Every blog page drew the site header without the design tokens, so its logo sat against the screen edge on phones

2026-09-27, Cosmetic, `3f4de8317b7469acadf1f24da99af04ee00a381e` *PR #126*

- **What was seen.** On all 32 blog pages, the 31 published posts and the blog index, the shared header's wordmark sat at 0 pixels from the left edge at phone width where every other page puts it 24 pixels in, and the header and footer ignored the page width. Their stylesheet used 22 design tokens, --gutter, --page and the radii and motion values among them, that the blog pages never defined.
- **Why.** partials.apply_chrome() injects TOKENS_CSS ahead of CHROME_CSS wherever a template carries the CHROME_CSS placeholder. build_blog.py writes its own page shell and pasted partials.CHROME_CSS into it directly, so the blog got the rules without the variables they read. CSS treats a variable that was never defined by quietly falling back to the property's initial value, so nothing failed: the padding became 0 and the max width none.
- **How it surfaced.** Found on September 27, 2026 while previewing a queued post at phone width: the logo touched the screen edge, and an already published post showed the same. (Found by rendering it and looking)
- **Fix.** build_blog.py injects TOKENS_CSS with CHROME_CSS, as apply_chrome does.
- **What stops it now.** page_checks.undefined_tokens() reads every built page's inline CSS and any local stylesheet it links, and fails the build when a page uses one of TOKENS_CSS's variables without a fallback and never defines it. in `src/page_checks.py`
- **Lesson.** CSS forgives a variable that was never defined by quietly using the initial value, so a missing token never throws, it only looks slightly wrong. When shared rules depend on shared variables, inject them together through one function, and check built pages for variables used but never defined.

INC-0163. The shared site header rendered its bold labels at two weights, because each page template asked Google Fonts for its own list of IBM Plex Sans weights

2026-09-27, Cosmetic, `44b3f8c80bb2860eb619880931f9cca9272b3ee7` *PR #162*

- **What was seen.** The site header's Create Account button is styled font-weight 700, 13px, and is meant to be identical on every page. Measured in Chromium on September 27, 2026, its label drew 92.41px wide on the landing page and on /exams/, where only the 400 and 600 faces of IBM Plex Sans had loaded, and 93.61px on /daily/, where the 700 face had. The shared chrome's other 700 weight rules, the header's tags and the mobile menu's group labels, draw from the same declared faces, so they fall back the same way.
- **Why.** The header's CSS is injected once, from partials.py, but the font files it depends on were not: each of 30 page sources (28 templates, plus build_blog.py and build_colleges.py, which write their own page heads) carried its own Google Fonts link and its own list of weights. Five of them (the daily page template and the four study guide templates) asked for IBM Plex Sans 400, 500, 600 and 700; the other 25 stopped at 600. A browser asked for 700 when only 600 is loaded draws the 600 face and says nothing, so the shared rules looked right in every template on its own and differed between them. The terms and privacy pages also imported the design system's font sheet, which still requested Manrope, the retired fourth face, from Google on every load.
- **How it surfaced.** Found on September 27, 2026 while moving the fonts off Google (DEVSECOPS O6): listing the distinct Google Fonts requests across the templates turned up five different ones, and a measurement of the header button on three pages confirmed the weights differ. (Found by measuring something)
- **Fix.** The three faces are served from this site at /vendor/fonts-2026-09-27/ with one stylesheet covering every weight any page uses (Source Serif 4 600, 700 and italic 400; IBM Plex Sans 400, 500, 600 and 700; IBM Plex Mono 400 and 500). Every template links it through one placeholder, `{{FONTS_CSS}}`, filled from partials.FONTS_CSS, so no page can ask for a different set. The design system's font sheet points at the same file, and Manrope is gone from it.
- **What stops it now.** The builds fail on any page that loads a stylesheet or font from another host, including through @import in a stylesheet the page links (page_checks.offsite_styles), so a template cannot go back to its own font request, and the Content-Security-Policy no longer allows fonts.googleapis.com or fonts.gstatic.com. in `src/page_checks.py`
- **Lesson.** Shared CSS is only shared if everything it depends on travels with it. A rule that asks for a font weight depends on that weight's file being loaded, and a browser that cannot find it substitutes a near one without an error, so the difference shows only when two pages are measured side by side. Load what shared rules need from the same single place the rules come from.

INC-0164. The community page scrolled sideways on phones for visitors whose random pseudonym was long, because the line that names it could not wrap

2026-09-27, Cosmetic, `44b3f8c80bb2860eb619880931f9cca9272b3ee7` *PR #162*

- **What was seen.** At 390px wide, signed out, /community/ shows "Anonymous posts from this device appear as" and the device's pseudonym on one line. Measured on September 27, 2026 with the bold weight the page loaded before INC-0163 (600), the page scrolled 35px sideways for Crimson Kingfisher 98, the longest name the page can draw, and 16px for Steady Swallow 10; Wise Owl 2 fit. The pseudonym is picked at random per device from 20 adjectives, 20 birds and a number from 2 to 98, so whether a visitor's page slid sideways depended on the name they happened to draw.
- **Why.** The identity line is styled white-space:nowrap. In the hero's wrapping flex row it drops below the heading on a phone, but a line that cannot break is as wide as its whole sentence, and with a long name that sentence is wider than the screen.
- **How it surfaced.** Found on September 27, 2026 while checking the self-hosted fonts at 390px: the page showed 11px of sideways scroll on one load and none on the next, which is what a random name produces. (Found by measuring something)
- **Fix.** The sentence may wrap; the name inside it stays whole (white-space:nowrap on the name only).
- **What stops it now.** src/smoke_community.js, run in npm run test:browser, loads /community/ at 390px signed out with the longest and the shortest pseudonym stored, with Supabase answered locally the way it answers a visitor, and fails on sideways scroll or a name broken across lines. No smoke test measured the community page's width before: smoke_launch loads it at 390px for its heading, contrast and link checks but never measures sideways scroll, the suites that do measure it cover other pages, and where the network fails the page replaces this line with a short notice, so a run without Supabase never drew it. in `src/smoke_community.js`
- **Lesson.** A layout checked with one sample of random content is checked for that sample. When a page draws something at random, a name, a pick or a shuffle, test it with the widest value it can produce, and pin that value, since a check that passes or fails by luck is not a check.

INC-0205. The consent banner's Reject Non Essential label ran out of its button on phones narrower than 390 pixels, so the refusal looked broken beside a clean Save Choices

2026-10-06, Degraded, `f749ea8e1bd32ba10ae17980278e2cb73af5cd76` *PR #241*

- **What was seen.** On every page at phone widths, the consent banner's 'Reject Non Essential' label does not fit inside its button, while 'Save Choices' beside it does. Measured with the site's own fonts served over HTTP, the label's line runs past the space inside the button's padding by 55 pixels at 320 pixels wide, 35 at 360, 28 at 375, 20 at 390 and 8 at 414; below 390 that carries it over the button's right border, and at 390 and 414 it runs through the padding to the border. At 320 'Save Choices' overruns by 9 pixels too. The reject choice still works when tapped, but it is the control that looks broken, on the banner whose rule is that refusing is exactly as easy as accepting.

- **Why.** Below 520 pixels the two buttons share the row in equal halves (flex 1 1 0), and both keep white-space:nowrap and 20 pixels of padding on each side. Equal halves are right for the same-size rule, but a label that may not wrap overflows as soon as its half is narrower than the label, and 'Reject Non Essential' is 128 pixels of text. The consent smoke compares the two buttons only at 1280 pixels, where each has room, and measures their position and area, never whether a label fits inside its button. The first measurement taken for this record compared scrollWidth with clientWidth and said the label fitted from 390 up; text running into a box's padding is not scrollable overflow, so that measure misses a label that reaches the border.
- **How it surfaced.** Found on October 6, 2026 while screenshotting a queued post at 390 pixels wide: the banner covered the page with its reject label cut off. That first view used a fallback font, from a page read off disk, so the label was measured again with the site's fonts at 320, 360, 375, 390 and 414 pixels, from the label's own line boxes against the inside of the button's padding. (Found by rendering it and looking)
- **Fix.** Below 520 pixels the buttons keep their equal halves but take 12 pixels of side padding instead of 20 and may wrap their label, so 'Reject Non Essential' wraps onto a second line inside its button below about 400 pixels wide and sits on one line above that, and the two buttons stay the same height side by side.
- **What stops it now.** smoke_consent.js opens the banner at 320, 360, 375, 390 and 414 pixels and fails if any line of either label reaches outside its button's padding, measured from the label's own line boxes, or if reject stops sitting beside accept at the same size. Run against the old rule it reports the overruns above. in `src/smoke_consent.js`
- **Lesson.** A layout check at one desktop width says nothing about a phone. Where a rule says two controls must be equal, test it at the narrowest widths people use and measure that each label fits its box, because equal boxes with unequal labels fail on the longer label first. Measure the text's own boxes against the inside of the padding: scrollWidth ignores text that runs into padding, so it passes a label that touches the border.

Scoring and selection (7)

INC-0040. The easier module was not easier

2026-09-16, Wrong data shown or stored, `81eecec` *PR #30*

- **What was seen.** An adaptive test routed weaker students to a second module that was supposed to be easier, and was not.
- **Why.** The two modules were assembled without comparing their difficulty distributions.
- **How it surfaced.** Measuring the mean difficulty of each module. (Found by measuring something)
- **Fix.** Assemble the modules against a difficulty target and assert it.
- **What stops it now.** module difficulty is asserted at build time in `src/test.js`

- **Cost.** adaptive routing that did nothing
- **Lesson.** If your system branches on difficulty, measure that the branches actually differ. A label is not a property.

INC-0005. The adaptive engine was learning from nobody

2026-09-17, Silent loss, `fc4114c` *PR #39*

- **What was seen.** The attempts table held zero rows in production while the app was plainly being used.
- **Why.** Cloud.logAttempt returns early without a signed-in user, and the product deliberately requires no account. So the one path that recorded answers was the one path most users never took.
- **How it surfaced.** Querying the production table and finding it empty. (Found by measuring something)
- **Fix.** item_events: an unlinkable telemetry table with no user, session, device or address column, which is exactly why it needs no consent gate.
- **What stops it now.** smoke_consent asserts the telemetry body carries no identifying field in `src/smoke_consent.js`
- **Cost.** weeks of telemetry never recorded
- **Lesson.** Check that your instrumentation fired at all before you trust anything built on it. An empty table looks identical to a quiet week.

INC-0006. Sentence equivalence items were unanswerable however well you answered

2026-09-17, Wrong data shown or stored, `fc4114c` *PR #39*

- **What was seen.** Every GRE sentence equivalence item graded as wrong.
- **Why.** The items carry a two-element key and the engine graded by requiring an array of two, but answerInputs had no branch for that type, so they rendered as single pick and returned a scalar.
- **How it surfaced.** Rendering and grading one item of every schema. (A test caught it)
- **Fix.** Add the missing input branch.
- **What stops it now.** smoke_items renders and grades one item from all 182 generator schemas at phone width in `src/smoke_items.js`
- **Cost.** eleven items unusable since they shipped
- **Lesson.** A type system spread across a renderer and a grader will drift. The cheapest guard is one that exercises every variant end to end, once.

INC-0051. Fixing repeats directly left a side door through passage groups

2026-09-21, Degraded, `65bd09d` *PR #58*

- **What was seen.** After a freshness gate cut avoidable repeats from 276 to 9 on one exam, one exam still served 117 and another 941 across 25 sittings.
- **Why.** The gate picked an unserved anchor item, and then the group expansion pulled in every question sharing that passage, including the ones already answered. The gate was applied to the anchor and not to what the anchor dragged with it.
- **How it surfaced.** A review bot counting avoidable repeats across 25 sittings per exam. (Found by a review bot or an adversarial pass)
- **Fix.** The group keeps the anchor and any unseen sibling and drops the rest, so a group can never come back empty.
- **What stops it now.** the review bot counts avoidable repeats and the number is pinned in `src/review_bot.js`
- **Cost.** repeats persisting on two exams after the fix was believed complete
- **Lesson.** When you add a filter, find every path that adds items after the filter runs. A gate on the entry point is not a gate on the set.

INC-0111. The question of the day promised to be the same for everyone and was not, and its streak punished a wrong answer

2026-09-26, Degraded, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** The trainer's Question of the Day card said 'One question, picked for everyone today'. The pick was pool[hash(date) % pool.length] over the live BANK, which grows when the deferred bank chunks arrive, so a student who opened it before the chunks loaded and one who opened it after were served different questions, and any deploy that changed the bank changed the day's question under students who had already answered it. Every hand-written ACT item belongs to a passage group, and the pool excluded passage items, so the ACT question of the day was always a generated item. Separately, the streak reset to zero on a wrong answer, so the card asking people to build a daily habit counted accuracy and punished exactly the students who showed up and missed.
- **Why.** The pick was computed from whatever array was in memory at the moment of the click, and nothing about the design had a notion of a question being assigned to a date independently of the bank. The streak rule was written as a correctness streak and labelled as a habit.
- **How it surfaced.** Found on 2026-09-26 while designing public daily pages from the recurring-traffic research, which reported the reset-on-wrong rule as working against the evidence on streaks, and by reading how the pick would behave once a published archive needed the same question for a date to stay put. (Found by reading the code or the output)

- **Fix.** Questions are assigned to dates in data/daily/schedule.json, written once by src/daily.js from hand-written items only and only ever appended to. The trainer, the public /daily/ pages and their archive read the same schedule. The streak lives in src/daily_streak.js, shared by both: it counts days answered, right or wrong, keeps the accuracy run as a separate personal best, and earns a freeze every seven days, at most two, spent automatically on a missed day. Passage items are served with their passage, so ACT has hand-written daily questions.
- **What stops it now.** The build runs src/daily.js check and fails if a scheduled id is missing from its hand-written bank, can no longer stand alone, or repeats, and warns when fewer than 30 days remain scheduled. src/smoke_daily.js plays the live page on a fixed date and asserts it serves the scheduled question, that a wrong answer still counts as a day, that the share text carries no question or answer, and that a second visit cannot re-answer.
- **Lesson.** Anything a page promises is the same for everyone has to be assigned, stored and served, not recomputed from whatever happens to be loaded, because the recomputation will eventually run against different inputs. And name a metric for what it counts: a streak that resets on a wrong answer is an accuracy score, and calling it a habit discourages the people it was meant to keep.

INC-0112. Weakest-first selection starved skills for struggling students, down to none at all

2026-09-26, Degraded, `3230ba014d8f660a9783ccc633c0a132c7c7b24c` *PR #96*

- **What was seen.** In a 140 question sitting simulated with the engine's own response model, a student at about 35 percent accuracy on the fifteen-skill ACT got 0.6 questions on average on their least-practised skill, several skills got none, and 5.2 of the 15 stayed under the five attempts after which the dashboard stops saying 'calibrating'. GMAT and LSAT struggling students showed the same pattern at one question at worst. The dashboard could therefore say nothing about a third of the ACT skills for exactly the students who most needed to know. The roadmap had recorded a milder version (Two-Part Analysis at 2 attempts in 120) as a tuning question for the owner.
- **Why.** pickQuestions gives 70 percent of every round to the three lowest-rated skills. A struggling student's practised skills fall below the starting rating while unpractised ones stay at it, so the unpractised skills rank as the student's strongest and are never chosen; the remaining 30 percent goes to whichever other skill has an item closest to the target difficulty, not round-robin. The diagnostic pass exits once half the skills have three attempts, which leaves the other half to that competition. The review bot measured reach across all sittings combined, so a skill each student touched once or not at all still counted as reached.
- **How it surfaced.** Found on 2026-09-26 by simulating per-student coverage at six ability levels on all five exams, prompted by the owner's request to keep developing the algorithm and the open roadmap question. (Found by measuring something)

- **Fix.** A coverage floor: a skill whose share of the student's attempts falls below half an even split takes a slot before weakness targeting fills the round, most starved first, capped at 30 percent of the round. The least-practised skill rose to 4.8 on ACT, 5.8 on GMAT and 6 on LSAT for struggling students, with no skill under three anywhere. Across three full-length seeds per exam: on GMAT, planted-weakness diagnosis stayed at 15 of 15 and the share of reported bands containing the true score went from 85 to 91 percent; on ACT, diagnosis went from 59 to 57 of 60 sittings, both above the 90 percent bar, and bands containing the true score from 65 to 77 percent. Coverage bought band accuracy and cost diagnosis two sittings in sixty. /scoring/ now explains how the next question is chosen, including the floor.
- **What stops it now.** review_bot.js checks each sitting on its own: it fails if any simulated student gets no question on a skill the bank can serve and warns below three, reporting the least-practised skill per sitting. It failed on the engine as it was (0 at worst on ACT and GMAT) and passes on the fixed one.
- **Lesson.** Measure an adaptive policy per student, not in aggregate: a pooled statistic averages the starved students with the well-served ones and reports a system that works. A targeting rule also needs a floor it can never trade away, because the students it concentrates hardest on are the ones whose picture of themselves it leaves emptiest.

INC-0127. Generated item ids were positions, so a bank change pointed stored reviews at different questions

2026-09-26, Wrong data shown or stored, `de96fd9f1b4f2b57d5c8fb220a164fa2668ddbf7` *PR #114*

- **What was seen.** A generated item's id was its position in the build: framework.run numbered items in the order it made them, continuing across an exam's categories. The trainer keys a student's spaced review (state.review), the seen counts and the item telemetry (item_events.qid) on that id. Any change to a bank renumbers what follows it. After such a deploy, a review due for a missed question served whatever question now held that number, and the telemetry pooled answers to different questions under one qid. The runner fix just before this one (INC-0126) would alone have repointed 6826 live ids, although it removed no question. 6627 of them were GRE ids, and 6534 of those were sentence equivalence and text completion ids that each came to name the question next to the one they had named, because GRE reading, which is numbered ahead of them, gained a question.
- **Why.** The id was used as a name but assigned as an index. A position is stable only while nothing before it changes, and the banks change on nearly every build: a new passage, an edited schema or a different draw order shifts everything numbered after it. Nothing that stored an id could tell that the question behind it had changed.
- **How it surfaced.** Found while measuring what the INC-0126 fix changed. Comparing the new banks with the live ones id by id showed 6826 ids naming a different question, although no question had been removed. (Found by measuring something)

- **Fix.** A generated item's id is now derived from what the item is: the exam's prefix and the first twelve hex digits of its canon key, the hash the runner already dedups on. For a reading item that key covers the schema, the stem and the passage; for the rest, the schema, the stem and the choices. The same question keeps its id across rebuilds whatever else changes, and a changed question gets a new one. Numbered ids already stored in a browser cannot be mapped back, because each one named different questions in different builds, so the trainer drops review and seen entries of the old form once, on load, rather than keep them.
- **What stops it now.** build_banks.py recomputes every generated id from its item and fails if one differs from its canon key or two items in an exam share an id. A twelve digit prefix makes a chance collision about three in a million per exam per build. in `src/build_banks.py`
- **Lesson.** An identifier that anything outside the build stores must be derived from what it identifies, not from where it happened to land. A position is a fine name only for something nobody keeps.

Infrastructure and deploy (6)

INC-0023. www and the apex were two origins, so consent and rankings split in half

2026-09-21, Wrong data shown or stored, `b852727` *PR #54*

- **What was seen.** Two hostnames served the same site. localStorage is per origin, so a visitor who arrived on both had two consent states and their ranking signal was split.
- **Why.** No canonical host redirect. The Cloudflare connector available here has no zone-level tools, so it had to be solved in the Worker.
- **How it surfaced.** Reasoning about origin scoping while investigating traffic. (Found by reading the code or the output)
- **Fix.** src/worker.mjs 301s www to the apex, with run_worker_first true so the asset router does not answer first.
- **What stops it now.** smoke_redirect asserts the redirect and that run_worker_first stays true in `src/smoke_redirect.js`
- **Cost.** split consent state and diluted ranking signal
- **Lesson.** Two hostnames are two origins and therefore two of everything the browser scopes by origin. Pick one and redirect on the server, not in a meta tag.

INC-0024. The .git directory was being served on production

2026-09-21, Wrong data shown or stored, `07f0646` *PR #55*

- **What was seen.** https://startfromnowhere.com/.git/config returned 200.

- **Why.** .assetsignore excluded src/, data/ and supabase/ but not **/.git. Cloudflare's asset uploader walks the repository root, which is the assets directory.
- **How it surfaced.** An adversarial review of the deploy exclusion list, after the first version of that list had already shipped. (Found by a review bot or an adversarial pass)
- **Fix.** Exclude **/.git, and rewrite the guard to derive what must be served from real references rather than a hand-written allowlist.
- **What stops it now.** smoke_redirect derives the must-serve set with git check-ignore and asserts the source is excluded in `src/smoke_redirect.js`
- **Cost.** the full repository history publicly readable
- **Lesson.** An exclusion list is a denylist, and denylists are wrong by omission. Derive the allowed set from what the built pages actually reference.

INC-0025. A stale edge cache made a fixed 404 look like a live 200

2026-09-21, Cosmetic, `07f0646` *PR #55*

- **What was seen.** After the fix deployed, /.git/config still returned 200.
- **Why.** Cloudflare had cached the response with max-age=300. The fix was live; the verification was reading the cache.
- **How it surfaced.** Re-requesting with a cache-busting query string, which returned 404. (Found by measuring something)
- **Fix.** None needed in code.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** minutes, and nearly a wrong conclusion
- **Lesson.** Verify a cache-fronted fix with a cache-busting request, or you are testing the cache. This costs one query parameter and saves an hour of chasing a fix that already worked.

INC-0027. A 29.88 MiB bank file would have failed the deploy the moment it merged

2026-09-21, Site down, `58f3a93` *PR #59*

- **What was seen.** None yet. The build succeeded locally and the deploy would have rejected the file.
- **Why.** Cloudflare Workers static assets cap a single file at 25 MiB. Raising the generated banks pushed the ACT rest-bank past it.
- **How it surfaced.** Checking the platform limit against the real output size before pushing. (Found by reading the code or the output)
- **Fix.** Chunk the rest-bank at 18 MiB per file and glob the chunks in numeric order.
- **What stops it now.** the build fails on any asset over 25 MiB in `src/build.py`
- **Cost.** would have broken the deploy

- **Lesson.** Know your platform's hard limits and assert them in the build. A deploy-time rejection is a bad place to learn a number your build could have told you.

INC-0047. Excluding the design directory would have shipped the legal pages with no styling

2026-09-21, Degraded, `07f0646` *PR #55*

- **What was seen.** None: caught in draft. Terms and Privacy would have rendered with no font, no colour and no background.
- **Why.** Those two pages load a stylesheet from the design directory that is nothing but imports of the token files. Excluding the directory from the deploy removed both the stylesheet and the tokens it imports.
- **How it surfaced.** An adversarial re-read of the exclusion list against what the built pages actually reference. (Found by a review bot or an adversarial pass)
- **Fix.** Re-include the tokens and the stylesheet, and rewrite the guard to read every href and src out of the built pages and follow one level of CSS import, instead of comparing against a list kept by hand.
- **What stops it now.** smoke_redirect derives the must-serve set from the built pages in `src/smoke_redirect.js`
- **Cost.** two legal pages unstyled, caught before deploy
- **Lesson.** A guard that is a hand-kept list of safe paths is written by the same hand that made the mistake. Derive the safe set from the artefact, not from memory.

INC-0095. A safety parameter filled in from memory, which makes it a coin flip rather than a guard

2026-09-22, Cosmetic, `d7ec4a71aaf8ed28e38d45d840ea1846fa83863d` *PR #85*

- **What was seen.** A merge call carried the head commit it expected to merge, and the forty character hex in it was typed from a remembered seven character prefix with the rest supplied by whatever looked like a commit hash. GitHub compared it against the real head, found it different, and refused. Nothing was merged and nothing was lost.
- **Why.** The parameter exists so that a merge cannot land on a head other than the one that was reviewed, and its whole value is that the number in it came from somewhere other than the person making the call. Filling it in from memory keeps the shape of the check and removes the thing being checked. The standing rule says every identifier must be read from a real output first, the rule names an earlier occasion when a hex string was typed straight into a merge call, and it was broken anyway, on the fourth merge of a session where the previous three had read the value with git rev-parse.
- **How it surfaced.** The API rejected the call. That is the guard working, and it is the only reason this is a note rather than an incident. (A build guard caught it)

- **Fix.** Nothing to change in the repository. The value was read with git rev-parse and the merge went through. What is recorded here is the shape of the mistake, because the rule against it already existed and was not enough.
- **What stops it now.** the merge API compares the expected head against the real one and refuses a mismatch, which is what caught this; the rule in CLAUDE.md is the part that failed, so the record is the reinforcement in `CLAUDE.md`
- **Lesson.** An optional safety parameter is a guard only while its value is read rather than recalled. Fill it from memory and it becomes a coin flip that looks exactly like a guard from the outside, and the flips it wins are the ones where it was not needed. A value that is cheap to read is never worth remembering: reading costs one command, and the failure mode of remembering is an operation that succeeds on the wrong thing.

Payments (5)

INC-0041. A signed-in user could grant themselves a paid plan

2026-09-16, Wrong data shown or stored, `81eecec` *PR #30*

- **What was seen.** The plan column was writable by the authenticated role, so any signed-in browser session could set its own plan to the top tier.
- **Why.** Column-level grants on the profiles table included the billing columns. Row Level Security controls which rows you can touch, not which columns.
- **How it surfaced.** Auditing the grants rather than the policies. (Found by reading the code or the output)
- **Fix.** Remove the billing columns from the grants entirely. Only the service role, used by the webhook, can write them.
- **What stops it now.** the billing columns are absent from the column grants in `supabase/migrations/20260916_harden_plan_column_and_rpc_surface.sql`
- **Cost.** free access to every paid tier
- **Lesson.** Row Level Security is row-level. Which columns a role may write is a separate grant, and anything money depends on belongs to the service role alone.

INC-0042. Every trialing subscriber would have been left on the free plan

2026-09-16, Wrong data shown or stored, `81eecec` *PR #30*

- **What was seen.** A checkout that starts a free trial did not upgrade the account.
- **Why.** The handler checked for a payment status of paid. A session that starts a trial settles as no payment required, which is not paid.

- **How it surfaced.** Reading the processor's status vocabulary rather than assuming it. (Found by reading the code or the output)
- **Fix.** Accept both settled states.
- **What stops it now.** smoke_billing covers the trial path in both auth states in `src/smoke_billing.js`
- **Cost.** every trial would have failed to grant access
- **Lesson.** Enumerate every value a third-party status field can take before you branch on one of them. The value you did not think of is usually the one that matters commercially.

INC-0035. Two Stripe event types announce one new subscription

2026-09-21, Wrong data shown or stored, `37241958be4dadca750f76742818b18b88e5a5b1` *PR #62*

- **What was seen.** None: caught in design. Logging new subscriptions from both checkout.session.completed and customer.subscription.created would have doubled new MRR.
- **Why.** Stripe announces a new subscription through both events, with different event ids, in no guaranteed order.
- **How it surfaced.** Working out the idempotency key before writing the handler. (Found by reading the code or the output)
- **Fix.** A partial unique index on (source, subscription_id, kind) for the once-per-lifetime kinds, so the database decides rather than whichever event arrived first.
- **What stops it now.** billing_events_once_idx in `supabase/migrations/20260921_billing_events_ledger.sql`
- **Cost.** would have doubled the headline revenue figure
- **Lesson.** Idempotency belongs in the database, not in the handler. Handlers race; unique indexes do not.

INC-0036. charge.amount_refunded is cumulative, so partial refunds double-count

2026-09-21, Wrong data shown or stored, `37241958be4dadca750f76742818b18b88e5a5b1` *PR #62*

- **What was seen.** None: caught in design. A second partial refund reports the running total, which would have counted the first refund twice.
- **Why.** Stripe's charge object reports total refunded to date, not the amount of this refund.
- **How it surfaced.** Reading the field's semantics rather than its name. (Found by reading the code or the output)
- **Fix.** Use the most recent entry in the refunds list, falling back to the cumulative total for the ordinary single full refund.
- **What stops it now.** Nothing automated. This one is still carried by attention.

- **Cost.** would have overstated refunds
- **Lesson.** Read what a payment field means, not what it is called. Cumulative and incremental fields look identical until the second event.

INC-0037. Cancellation erases the number you need to record the cancellation

2026-09-21, Wrong data shown or stored, `37241958be4dadca750f76742818b18b88e5a5b1` *PR #62*

- **What was seen.** None: caught in design. Churned MRR would always have been zero.
- **Why.** The profile write on cancellation nulls plan_amount_cents and plan_interval. That row was the only place the departing subscriber's price still existed.
- **How it surfaced.** Tracing the order of writes in the handler. (Found by reading the code or the output)
- **Fix.** Read the prior profile state before the cancellation write, and log what they were paying.
- **What stops it now.** priorState() is called before writeProfile in both the deleted and updated branches in `supabase/functions/stripe-webhook/index.ts`
- **Cost.** would have reported zero churn forever
- **Lesson.** When a write is destructive, capture what you need from the old value first. Ask what question you will want to answer after this row is gone.

Database (4)

INC-0012. Postgres fired triggers alphabetically and revoked every adult opt-in

2026-09-19, Wrong data shown or stored, `a61eeca` *PR #42*

- **What was seen.** Legitimate adult opt-ins to the data sharing programme were silently revoked.
- **Why.** Postgres fires same-timing triggers in alphabetical order by name. profiles_sharing_eligibility sorted before profiles_sync_age_tier, so eligibility was judged against a stale age tier.
- **How it surfaced.** Testing the guarantee the triggers were supposed to provide, rather than reading the trigger code. (A test caught it)
- **Fix.** Numeric prefixes on trigger names to force the order.
- **What stops it now.** src/sql smoke tests assert the guarantee, not the implementation in `supabase/migrations/20260918203431_fix_profiles_trigger_firing_order.sql`
- **Cost.** every adult opt-in, silently
- **Lesson.** Postgres fires same-timing triggers alphabetically. If two triggers on one table have an order dependency, encode it in the name, and test the outcome rather than the code.

INC-0013. UPDATE OF fires on the columns named in the statement, not the ones that changed

2026-09-19, Wrong data shown or stored, `a61eeca` *PR #42*

- **What was seen.** Updating birth_year stored a minor's tier and skipped the eligibility check entirely.
- **Why.** UPDATE OF <columns> keys off the columns named in the UPDATE statement, not off what actually changed and not off what an earlier BEFORE trigger wrote into NEW.
- **How it surfaced.** Same guarantee test as INC-0012. (A test caught it)
- **Fix.** Fire on every update and compare OLD to NEW inside the function.
- **What stops it now.** sharing eligibility fires on every update in `supabase/migrations/20260918204223_sharing_eligibility_fires_on_every_update.sql`
- **Cost.** a minor could pass the adult gate
- **Lesson.** UPDATE OF is a statement-shape filter, not a change filter. If you need 'when this value changed', compare OLD and NEW yourself.

INC-0020. Revoking EXECUTE from anon did nothing while PUBLIC still held it

2026-09-19, Wrong data shown or stored, `a61eeca` *PR #42*

- **What was seen.** Functions stayed callable after their grants were revoked from anon and authenticated.
- **Why.** PUBLIC is a separate grantee. A leading =X/postgres in pg_proc.proacl means PUBLIC holds EXECUTE, and revoking from the named roles changes nothing.
- **How it surfaced.** Re-running the Supabase advisors instead of trusting the per-role grants. (A test caught it)
- **Fix.** Revoke from PUBLIC as well, everywhere.
- **What stops it now.** every migration revokes from public, anon, authenticated together in `supabase/migrations/20260916_revoke_public_execute_and_rls_initplan.sql`
- **Cost.** an open function surface believed closed
- **Lesson.** In Postgres, revoking from every role you can name still leaves PUBLIC. Verify with the advisors or by reading the acl, never by reading your own migration.

INC-0056. Twelve profile fields said Saved and never reached the account

2026-09-21, Silent loss, `724ef0ec90a8344f0a17530450d13c792b71e573` *PR #63*

- **What was seen.** A signed-in user fills in education, intended major, score goal, application year, budget, industry, household income, first generation, military status, study hours or their birth date, sees a Saved toast, and none of it is on their account. It survives only in that browser.

- **Why.** Two layers disagreed and nothing compared them. Row Level Security scopes every write on the profiles table to the caller's own row, and a separate column grant decides which columns any role may write at all. The column grant to authenticated covers 5 of the 17 self-reported fields the Account page offers. The page updates the other 12 anyway, the database refuses, and aboutSave wraps the call in a try/catch with an empty body, so the refusal is discarded and the toast fires regardless.
- **How it surfaced.** Reading the column privileges while adding new profile fields for something else. Nothing in the product would ever have reported it. (Found by reading the code or the output)
- **Fix.** Grant UPDATE on the self-reported columns to authenticated, which is safe because the row policy already restricts every write to the caller's own row, and stop the save path swallowing its own error.
- **What stops it now.** a SQL smoke test asserts authenticated holds UPDATE on every field the Account page writes, and none of the server-owned ones in `src/sql/smoke_profile_grants.sql`
- **Cost.** every signed-in user's demographics, for as long as the fields have existed
- **Lesson.** An empty catch block around a write is a silent-loss defect waiting to be born. If a save can fail, the person must be told; a success toast that fires regardless of the result is worse than no toast, because it actively teaches the user the data is safe. And where two layers of authorisation have to agree, something has to compare them: the one that is wrong will not announce itself.

Interface and data display (3)

INC-0034. A line chart interpolated between discrete money events

2026-09-21, Degraded, `37241958be4dadca750f76742818b18b88e5a5b1` *PR #62*

- **What was seen.** A line drawn between $13.32 on the 18th and $4.99 on the 19th, passing through every value in between.
- **Why.** Money movement was charted as a time series. A line encodes continuity, and these are discrete events on particular days.
- **How it surfaced.** Rendering it and looking at it. (Found by rendering it and looking)
- **Fix.** Bars for movement and cash. Lines are reserved for measures that genuinely vary continuously.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** a chart that implied numbers that never happened
- **Lesson.** Chart form is a claim about the data. A line claims the values in between existed. Ask whether that claim is true before choosing it.

INC-0052. Two games signalled right and wrong by colour alone

2026-09-21, Degraded, `b4742a7` *PR #57*

- **What was seen.** Outcomes in two timed games were distinguishable only by colour, and nothing was announced.
- **Why.** The games were built for speed and the feedback was styling rather than content.
- **How it surfaced.** An accessibility pass over the games. (Found by reading the code or the output)
- **Fix.** Announce outcomes in a live region, and give a flipped tile a spoken identity. Colour is now secondary to a word in both.
- **What stops it now.** smoke tests assert the live region announces outcomes in `src/app_template.html`
- **Cost.** two games unusable with a screen reader
- **Lesson.** Never encode a state by colour alone. The word also survives greyscale printing, forced-colors mode and a glance from across a room, so it is better for everyone and not only for the people it is required by.

INC-0058. The fit card told readers to fill in fields that were below it

2026-09-22, Cosmetic, `0dcabb42faceedc6c3838ff1e17151e02b1c0423` *PR #64*

- **What was seen.** With nothing entered, the comparison card said to fill the three inputs in above, and they are further down the same page.
- **Why.** The copy was written while building the card, before deciding where the inputs would live, and was never re-read against the rendered page.
- **How it surfaced.** Screenshotting the page and reading it as a user would. (Found by rendering it and looking)
- **Fix.** Name the section rather than a direction, and assert the wording in the test so a later move of either block fails loudly.
- **What stops it now.** smoke_fit asserts the card points at the section by name in `src/smoke_fit.js`
- **Cost.** a sentence sending users the wrong way
- **Lesson.** Copy that says above, below, left or right is a hard dependency on layout that nothing checks. Name the thing instead, and the sentence survives every rearrangement.

The Checklist

Generated from the defect ledger. Every line exists because something went wrong once. Nothing is here for completeness.

Read it before starting a piece of work in the matching area, and again before you push.

Build system

- [ ] **Learned 5 times over.** A guard that covers a subset of cases reproduces the original defect in the cases it skips, and it is more dangerous than no guard because the incident it was written for feels closed. When you add a check, enumerate everything of that kind and cover all of it, or state in the code which cases are deliberately excluded and why.

<small>The guard against a blind counter was itself blind to three exams (INC-0064)</small>

- [ ] **Learned 3 times over.** A regex that counts things assumes a formatting convention, and a file that legitimately breaks the convention counts as zero rather than as an error. Any counter that can return zero for a non-empty input needs a per-source assertion, not just a total.

<small>The item counter missed a whole bank file because it assumed a quoting style (INC-0059)</small>

- [ ] **Learned 3 times over.** A size limit on a generated file is only a guard if something bounds the generator too; otherwise it is a delayed failure that lands on whoever commits next, and reads as their fault. When two guards constrain the same output, check the fix against both: shortening this file to satisfy the size check would have broken the completeness check that reads its first 60 characters.

<small>The rules digest promises to be prompt sized and its generator grows without bound (INC-0083)</small>

- [ ] **Learned 3 times over.** Two copies of one check drift apart. When a check exists in two places, a fix to one is a question about the other, and the cheapest answer is to make them share the code that splits text.

<small>The blog build read a fee table as one sentence and refused a correct post for calling the SAT trainer unfinished, over College Board's seat Waitlist (INC-0162)</small>

- [ ] **Learned 2 times over.** Two habits, both mine rather than the code's. Verify with the sequence the pipeline runs, read out of its config, not with the subset you remember: a suite chosen from memory drifts to the parts that were failing last week. And when a step is deliberately non fatal, the word it fails with is the whole of its signal, so it has to be the word people grep for. WARNING on a line that means a deliverable did not build is an invitation to miss it, and the cost of saying ERROR while still exiting zero is nothing at all.

<small>A build step that fails while the build exits zero, and a verification run that was a remembered subset (INC-0080)</small>

- [ ] A parse guard covers the file shapes someone thought of. When the same code moves into a new shape, a separate file, a chunk, a worker, the guard does not follow it. List what the guard covers against what the deploy actually ships, and check the difference rather than the intention.

<small>Nothing parsed the one file every user downloads (INC-0060)</small>

- [ ] A guard keyed to wording is a guard on the wording, not the fact, and every synonym is a hole in it. Widening the wording is the obvious repair and it trades missed defects for false alarms, which cost more because they get the guard switched off. Match a phrase that only the thing you care about can produce, rather than every word it might happen to use.

<small>The stale count guard checked two nouns and the page used a third (INC-0063)</small>

- [ ] A file generated for a different audience has to be read as that audience, not as the one that generated it. The size of this one was bounded and guarded; who it was for was written in a docstring and checked by nobody.

<small>The bootstrap digest ships incident ids to a project that has no incidents (INC-0084)</small>

- [ ] State that describes a run has to live and die with the run. Keyed to the object that makes the draws, it cannot outlive them; kept on a long lived instance, it quietly couples every run to every earlier one, and the output depends on call history that no one reading a single run can see.

<small>The Data Sufficiency answer counter outlived the run it belonged to, so a second run from the same seed made different items (INC-0123)</small>

- [ ] A heuristic stopping rule is right only when nothing better is known. Where the size of what is being collected is known exactly, stop at that size and check that it was reached. Do not stop when the collection merely seems to have gone quiet, because the odds of a quiet spell grow with the collection while the threshold stays fixed.

<small>The runner's 400 draw cutoff dropped the last question of a small reading schema on three of four builds (INC-0126)</small>

- [ ] When a generated file has a size limit and a completeness rule, decide which one wins before the file grows, because it will. Bound the whole output in the generator, and let the file say what it left out and where the rest is, so omission is a stated choice rather than a silent one.

<small>The rules digest crossed its word budget again, because the earlier fix bounded each rule and not the digest (INC-0129)</small>

- [ ] A generated file that someone edits by hand has two sources, and the next run of the generator silently picks one of them. Put the edit in the generator or retire the generator in writing, and have the build regenerate every generated file and compare it, because nothing else notices a generator that no longer makes its file until someone runs it.

<small>The second LSAT reading bank had a generator that no longer wrote it, and running the generator would have put back 25 corrected lines (INC-0193)</small>

- [ ] A check that runs only on what is published tests today's site, not tomorrow's. When content is queued to publish by date, run every publish-time check on the queued content too, built as it will be, or the first time the check sees that content is the deploy that depends on it.

<small>A queued post's title was checked for Title Case only once it went live, so a title the build refuses would have failed the deploy on its publish day (INC-0204)</small>

- [ ] An 'if the file exists, copy it' step makes a missing generator invisible. A file a page loads should be produced by the same build that deploys the page, and that build should fail when it is missing; a local checkout that once ran a step by hand is not evidence that the deploy runs it.

<small>Admin > Social showed an empty queue on the live site, because the queue is written by a script the deploy build never runs (INC-0212)</small>

CSS and layout

- [ ] **Learned 3 times over.** The same undefined-property failure will find you repeatedly, at every severity from one icon to an invisible legal control. One audit of computed colour catches the whole class.

<small>A landing-page icon referenced a colour token that did not exist (INC-0050)</small>

- [ ] **Learned 2 times over.** CSS forgives a variable that was never defined by quietly using the initial value, so a missing token never throws, it only looks slightly wrong. When shared rules depend on shared variables, inject them together through one function, and check built pages for variables used but never defined.

<small>Every blog page drew the site header without the design tokens, so its logo sat against the screen edge on phones (INC-0139)</small>

- [ ] CSS comments do not nest. When a whole page category loses its styling, read the built artefact, not the source that produced it.

<small>CSS comments do not nest, and one placeholder killed the palette on 1451 pages (INC-0010)</small>

- [ ] An undefined CSS custom property is silent. Audit computed colour, not authored colour, and do it on the rendered page.

<small>The Do Not Sell button was invisible from the day it shipped (INC-0018)</small>

- [ ] A dark mode does not break by having a wrong colour. It breaks by missing one. Any colour paired with a ramp has to move with the ramp.

<small>A sequential colour ramp inverts direction between themes, and the label text did not (INC-0029)</small>

- [ ] CSS never fails loudly. A malformed rule is skipped, a bad selector eats the block after it, and an undefined variable paints as nothing. Anything that matters visually has to be asserted on the rendered page, because the parser will not tell you.

<small>Error recovery in the CSS parser swallowed the palette silently (INC-0048)</small>

- [ ] Shared CSS is only shared if everything it depends on travels with it. A rule that asks for a font weight depends on that weight's file being loaded, and a browser that cannot find it substitutes a near one without an error, so the difference shows only when two pages are measured side by side. Load what shared rules need from the same single place the rules come from.

<small>The shared site header rendered its bold labels at two weights, because each page template asked Google Fonts for its own list of IBM Plex Sans weights (INC-0163)</small>

- [ ] A layout checked with one sample of random content is checked for that sample. When a page draws something at random, a name, a pick or a shuffle, test it with the widest value it can produce, and pin that value, since a check that passes or fails by luck is not a check.

<small>The community page scrolled sideways on phones for visitors whose random pseudonym was long, because the line that names it could not wrap (INC-0164)</small>

- [ ] A layout check at one desktop width says nothing about a phone. Where a rule says two controls must be equal, test it at the narrowest widths people use and measure that each label fits its box, because equal boxes with unequal labels fail on the longer label first. Measure the text's own boxes against the inside of the padding: scrollWidth ignores text that runs into padding, so it passes a label that touches the border.

<small>The consent banner's Reject Non Essential label ran out of its button on phones narrower than 390 pixels, so the refusal looked broken beside a clean Save Choices (INC-0205)</small>

Content generation

- [ ] **Learned 7 times over.** A size threshold on a check is a silent exemption, and it grows as the corpus does: every schema written from a small authored corpus falls under it by construction, which is exactly the population most likely to carry a structural tell. When a measurement cannot be trusted at a small sample, widen the tolerance to what the sample supports rather than declining to measure, and notice when a count of what was measured does not move after you add something to measure.

<small>A shipped schema answerable at 68 percent by picking the shortest option, under the size at which anything is checked (INC-0088)</small>

- [ ] **Learned 6 times over.** A citation vouches only for what its source says, so check each figure against the source rather than checking that a citation is present. The claim most likely to slip through is the true looking one written next to real ones, arithmetic or common knowledge put in the source's mouth.

<small>The GRE exam guide credited ETS with a combined 260 to 340 score that the ETS page it cited does not describe (INC-0130)</small>

- [ ] **Learned 6 times over.** Run the source check on every corpus that cites pages, not just the one that broke. When a field is empty and the research is out of budget, leave it empty: a blank is honest, and a figure made to fit beside a real citation is the most convincing kind of wrong.

<small>School pages published figures that the articles they cite do not contain, one of them a GRE total split in half (INC-0133)</small>

- [ ] **Learned 6 times over.** Finding a number on a page is not finding the fact. A check has to look for it beside the words that say what it counts, and notice when those words are about something else: another program, another class.

<small>Columbia's MBA class was credited with five years of work experience, a figure the cited article gives for its 46-student MBAxMS cohort, and the source check passed it because the article does say five years (INC-0150)</small>

- [ ] **Learned 5 times over.** An aggregate over a mixed population reports the population, and if part of that population is flat by construction it will hide the part that is not. The rule that follows is about what the unit of the measurement should be: measure at the grain the defect can exist at, which here is the file, because a file is written by one person in one sitting with one set of habits. The section was the grain the data was convenient at.

<small>A bank a student can play at 88 percent, inside a section the check passed (INC-0069)</small>

- [ ] **Learned 5 times over.** A corpus field is written against the one sentence the author had in mind, and the schema that reuses it three templates later has no way to know which shape it is. The type system says str in both places. Two things follow. Store the field in every shape a template needs and name the shapes, rather than storing one shape and trusting the next author to notice. And guard the output, not the corpus: the generated sentence is the only place the mismatch becomes visible, and a cheap pattern over the rendered text catches a class that no check on the inputs can see.

<small>A corpus field written for one grammatical slot was spliced into another (INC-0074)</small>

- [ ] **Learned 5 times over.** Anything written ahead of its publication date is a promise about the future made from the past. Re-check a queued post's claims about your own product at build time, where the product's real state is known, rather than trusting the day it was written.

<small>A post queued to publish the next day said the LSAT trainer was still in development, eleven days after it went live (INC-0137)</small>

- [ ] **Learned 4 times over.** Two lessons, and they compound. A rule copied into code by its examples loses the clause the examples were illustrating: CLAUDE.md bans six named sites and coaching site blogs, and the list kept the six and dropped the category, which is the half that generalises. And a validator gets written for the corpus that had the problem at the time, then quietly defines what is checked: two of three published corpora were enforced and the third had never had a source read, which is not a weaker check but an absent one. When a guard exists, the question is not only whether it is strict enough but which of the things it could be pointed at it is not pointed at.

<small>Nine published exam facts cite test prep companies, in the one published corpus with no source validator (INC-0082)</small>

- [ ] **Learned 4 times over.** A counter that nothing reads is not instrumentation, it is a comment that looks like instrumentation, and it is worse than nothing because it answers the question 'is anyone watching this' with a yes. Every time a guard is written against one symptom, ask what the same failure looks like arriving another way, and count the whole category rather than the instance that prompted it.

<small>A schema threw away three draws in four, and the counter that knew was read by nobody (INC-0092)</small>

- [ ] **Learned 4 times over.** A check that infers what to expect from the same data it is checking cannot fail on a missing field: absence reads as nothing to look for. Derive the expectation from something the data cannot erase, the item's TYPE, and then look for what that type requires. A field that is set in memory, used by every in-process test and copied by hand into the shipped format has no test at all between the two, so the copying should be checked as a whole rather than field by field. And a check that runs one random draw is not a check of a random process: this one was correct for months and simply never asked often enough to see the answer.

<small>Every generated reading comprehension question shipped without its passage (INC-0099)</small>

- [ ] **Learned 4 times over.** Any consumer that describes a value in words must read the field that records what kind of value it is, never the field's name. A field name is a hope about the data, not a description of it, and a schema that allows two kinds of value will be described everywhere as the kind it was named after.

<small>Twelve average salaries were published as medians, and the social queue credited schools with figures they never published (INC-0105)</small>

- [ ] **Learned 4 times over.** An edit that appends text has to read what it is appending to. A correction step that checks only its own goal (here, that the choice got longer) will happily achieve it by making the choice worse, and every check downstream measures the goal, so nothing notices. Check the seam: the words either side of where new text meets old text.

<small>Length corrections appended clauses that repeated what the answer choices already said (INC-0119)</small>

- [ ] **Learned 4 times over.** A citation is only as current as the page it points to, and publishers leave old pages up. When a source describes itself as covering a period, check that the period is still running; a page about last year's test can be quoted perfectly and still be wrong about this year's.

<small>The LSAT guide said most test takers choose between remote and in-person testing, after LSAC had moved almost everyone into test centers (INC-0136)</small>

- [ ] **Learned 3 times over.** A module that nothing imports fails no test, and an exception raised on every draw is indistinguishable from an exception raised on a hard draw. Both are silence, and a build that reports totals hears neither. Count what each schema contributed, not what the category holds, and treat a contribution of zero as a failure rather than as a small number.

<small>A finished generator module that nothing imported, and two of its four schemas produced nothing (INC-0086)</small>

- [ ] **Learned 3 times over.** Reading the record does not prevent the defect; the practice does. This one was written hours after its own lesson was read closely enough to be catalogued as a recurrence, and it was caught by rendering three items rather than by remembering. Budget the render, not the recollection.

<small>The same corpus field in two grammatical slots, in a schema written the same day the guard was read (INC-0087)</small>

- [ ] **Learned 3 times over.** A generator's wrong answers are written as labels and read as labels, and nobody looks at the values two labels produce. Where the question type makes two misconceptions arithmetically identical the list is shorter than it reads, and because a discarded draw is the ordinary way a schema says no, a whole question form can vanish from an exam leaving nothing behind but a number nobody has a reference for. Count what a schema actually produces at each width it has to serve, and compare the widths against each other.

<small>Two of a schema's three question forms could not build, and the only sign was an item count (INC-0090)</small>

- [ ] **Learned 3 times over.** A generated item is checked as data, and this one was correct as data: the logic was valid, the key was right, the distractors were the intended errors. What was missing was a fact about the RELATION between two strings, that a name in the question also occurs in the passage, and no property of either string alone can see it. When a question and its source are assembled from separate fields, write down what has to be true of them together, because every check that looks at one field at a time will pass.

<small>Every reading inference question asked about something the passage never mentions (INC-0097)</small>

- [ ] **Learned 3 times over.** A fix scoped to where the evidence was is a fix scoped to the sample, not to the defect. When the change is to how a QUANTITY is written, the unit of work is the quantity and every place it is written, not the place where the failure happened to be visible. Finding the other call sites costs one search; the fix that covers a subset leaves a difference the reader can see and the measurement cannot.

<small>The table said 31.0 and the explanation said 31, because the fix covered the table only (INC-0098)</small>

- [ ] **Learned 3 times over.** When a defect is about a KIND of code rather than a line of code, a guard bolted to the site of the failure does not generalise, and writing one feels like closing the case. Ask instead what the next instance of this code will look like and where it will live: here the answer was anywhere an item is copied out of the bank, and the durable check is that an item asking about source material has to be carrying it, wherever it is being rendered.

<small>A new field-by-field copier reproduced the passage loss defect four days after the ledger recorded it (INC-0101)</small>

- [ ] **Learned 3 times over.** A stored phrase that goes into more than one slot has to be written for the hardest of them, and the transform has to run in the direction that cannot damage anything: capitalising a sentence opener is always safe, lowercasing one breaks proper nouns. When a field is reused, read the output of every slot it lands in, not the one it was written for.

<small>A field written to open a sentence was printed mid-sentence in a reading stem (INC-0115)</small>

- [ ] **Learned 3 times over.** A figure written twice on one page will eventually be written two ways. Where the site already holds a sourced value, a page that states it should be checked against that value, because a reader who meets $4 and $5 for the same fee trusts neither.

<small>A blog post put the ACT science add-on at $4 in its table and $5 in its own FAQ, and nothing compared a post's prices with the sourced ones (INC-0124)</small>

- [ ] **Learned 3 times over.** A presence check on a source field proves the field is filled, not that it is true. Where a figure names its source, check that the URL belongs to that source, because the easiest value to fill a required field with is the wrong one that happens to be nearby.

<small>Every College Scorecard figure in the MBA library cited the program's own website as its source URL (INC-0125)</small>

- [ ] **Learned 3 times over.** A search result's snippet is not the page it links to. A figure taken from one has to be read again on the page before it is published, and a record that admits the snippet in a note while citing the page looks fully sourced, which is why nobody rereads it.

<small>Rice's school page credited Poets&Quants with an average GMAT of 693 for the Class of 2027, a number the article never prints; the figure's own note said it was read from a search snippet (INC-0151)</small>

- [ ] **Learned 3 times over.** A report that says the same thing every week stops being read. When a checker has blind spots, record each judgement about a finding it cannot settle, so that what it prints shrinks to what is new; a list that mixes known false alarms with real errors hides the errors about as well as no list at all.

<small>The weekly source check listed Duke's GMAT medians and Michigan Ross's work experience as missing from their sources for weeks, but its 30 findings mixed real errors with pages it cannot read, so none was ever worked (INC-0154)</small>

- [ ] **Learned 3 times over.** A style rule that nothing checks holds only for the work written after someone remembered it. When a rule can be stated precisely enough to apply by hand, it can be checked by the build, and it should be, or the earliest work quietly keeps breaking it.

<small>Thirty published blog posts set their section headings in sentence case, against the house rule that headings are Title Case (INC-0171)</small>

- [ ] **Learned 3 times over.** When a check fails the same way twice with different details, guard the shape rather than the instance. Two narrow guards covered a cohort paragraph and a grade range, and the third instance was a vote counter. The question every match should answer is whether the number sits where the fact would be, and that can be asked of every fact.

<small>Two GMAT facts passed the source check on numbers that meant something else: score validity cited the retake policy page, where "5 times" supplied its 5, and the score release's "3 to 5 days"

matched a 3 in the page's helpfulness counter (INC-0174)</small>

- [ ] **Learned 2 times over.** Test your content against the strategies a lazy adversary would use, not only against whether it is correct. Measure the score of a rule that ignores the question.

<small>The longest option was the correct answer 81 percent of the time (INC-0044)</small>

- [ ] **Learned 2 times over.** A record has parts that refer to one another, and a tool that edits one part by text is editing a graph while looking at a string. The cheap guard is not to check every reference but to refuse the edit when the old text occurs anywhere else in the record, because that is the only place a reference to it can be. Refusing on a false positive costs one rewritten table entry; not refusing ships an explanation about an option nobody saw.

<small>A distractor was replaced and the explanation went on naming the old one (INC-0072)</small>

- [ ] **Learned 2 times over.** Every guard here measured the answer's place in its set, and a set of guards that all take the same kind of measurement shares a blind spot the size of everything else. The tell they could not see was the simplest one a student would find: the answer is the same answer. When adding the third check of a kind, the question worth asking is not whether it is stricter than the other two but what all three have in common, because that is what is going unmeasured.

<small>A student who always answers 1 scores 98 percent on a schema, and no check looked at the answer itself (INC-0081)</small>

- [ ] **Learned 2 times over.** A template is a promise about the grammar of what goes into it, and the promise is invisible: the code says name and the sentence needs a singular noun phrase. Whenever a stored string lands next to a verb, an article or a plural, write the requirement down beside the data rather than in the template, and make adding a new row state that it meets it. Renaming the data to fit one grammar is usually cheaper and always safer than teaching the templates to handle two.

<small>Seven variable names were plural and every sentence built around them said was (INC-0093)
</small>

- [ ] **Learned 2 times over.** A test measures what someone can get right without the skill, and there is more than one way to do that. Removing one tell does not make an item sound; it moves the question to the next shortcut. Distractors drawn from unrelated material are always wrong and therefore always free, so a wrong answer has to be wrong about the same thing the right one is about.

<small>Three reading schemas could be answered by matching names, because every distractor came from a different passage (INC-0117)</small>

- [ ] **Learned 2 times over.** A figure with a source and a year can still go stale, because the source moves and the record does not. For anything a publisher changes on its own schedule, fees above all, read the live source on a schedule instead of trusting the record because it cites one.

<small>The GRE guide still quoted a $220 fee eight weeks after ETS's fees became $249 (INC-0132) </small>

- [ ] **Learned 2 times over.** When a parser must pull one value out of free text, anchor it to the words that give the value its meaning, not to its position, and return nothing when nothing anchors it: a sentence that says less is better than one that states a guess as fact. Keep inferences out of the fields that hold a source's own words, because the inferred class name in a stat is what let one class be reported under two names.

<small>Arizona State's school page said its class entered in 2027, because the label parser took the first year it found (INC-0135)</small>

- [ ] **Learned 2 times over.** A label fixed in code is a claim about every value the field will ever hold. When the data carries its own qualifier, such as a timing or a statistic, the label has to be read from the data, and anything that compares or scores the values has to use only the ones that share the qualifier the method names.

<small>Every school's employment rate was labelled and scored as a three month figure, though 15 of 70 were measured at six months, four months, a year or a reporting date, or at a timing their notes do not give (INC-0142)</small>

- [ ] **Learned 2 times over.** A fact filed next to another fact tends to inherit its citation. When two facts come from one section of a site, check that each one's own page says it, not the page its neighbour came from.

<small>The GMAT guide cited mba.com's Official Score Reports page for the rule on five free score reports within 48 hours, a rule that page never states (INC-0155)</small>

- [ ] **Learned 2 times over.** A number's presence on a page is weak evidence for a fact, because pages are full of numbers that mean other things: menus, dates, grade ranges, footnote markers. A check that matches numbers should read them where the fact would be, and should expect the page to change what it says around them.

<small>The ACT exam guide and a live post said the ACT can be taken up to 12 times, after ACT's page had come to say there is no limit, and the source check matched the 12 in the page's K-12 menu (INC-0172)</small>

- [ ] **Learned 2 times over.** A guard written for the instance found covers only where that instance was. When a rule applies everywhere, check where every page ends up, the built output, rather than one of the several places pages are written.

<small>Headings outside blog post bodies broke the Title Case rule on every kind of page, because the build checked only the headings inside posts (INC-0176)</small>

- [ ] **Learned 2 times over.** A claim about most of a group is a statistic even when it has no number in it, and checks that look for numbers cannot see it. Treat the quantifier words themselves as the thing to check, make each such sentence point at its basis, and when new data arrives, read what the site already says against it, because the old sentences were written without it.

<small>A live admissions post said most full-time MBA programs run three rounds a year, which 21 of the 32 programs whose deadlines we verified do not, and its other claims about rounds cited nothing (INC-0203)</small>

- [ ] Any generator that claims reproducibility must be seeded from something stable across processes. hash() is not, in Python, and the failure shows up as a flaky test rather than as a wrong answer.

<small>Item banks were different on every build because Python randomises hash() (INC-0003)
</small>

- [ ] Deletion by shadowing is invisible. Any collection whose size is a fact about the product needs its size asserted, not just its contents.

<small>A shadowed variable silently deleted 3000 items (INC-0004)</small>

- [ ] A deduplication key must be canonical under every transformation the item legitimately undergoes. Ask what varies per draw before you hash.

<small>The dedup key counted a reshuffled question as a new one (INC-0007)</small>

- [ ] The same inflation arrives through a different door every time you close one. When you fix a dedup bug, ask what else shares an identity.

<small>Four passages produced five hundred fake distinct questions (INC-0008)</small>

- [ ] Dedup can be wrong in both directions. An over-broad key deletes real content as silently as a narrow one inflates it.

<small>Two conditionals hashed identically and half the inference items would have vanished (INC-0009)</small>

- [ ] Every filter needs its rejection count reported. A filter that silently drops is indistinguishable from an input that was never there.

<small>A length guard silently dropped sixteen valid items (INC-0011)</small>

- [ ] In any set of multiple-choice content, count where the answers are. A positional tell makes the whole set worthless to a test-wise user, and it is invisible item by item.

<small>225 of 302 correct answers sat at position A (INC-0039)</small>

- [ ] A guard on the extreme of a distribution can be satisfied by moving the mass next to the extreme. When you correct for a measured bias, measure the whole distribution afterwards, not the statistic you were correcting.

<small>Correcting a length tell moves it one rank over, every time (INC-0062)</small>

- [ ] A correction table is a set of claims about outcomes, and an entry that quietly fails still counts as applied. Aggregate metrics hide this well: a table where half the entries work still moves the number in the right direction, which reads as success. State the per item intent in a form the machine can check, and every entry that did nothing says so by name.

<small>Half the length tell correction did nothing and the build said it had worked (INC-0066)</small>

- [ ] A seeded shuffle is deterministic, which makes calling it twice look harmless: the same input gives the same output. What repeats is the permutation, not the randomisation, and a permutation applied to its own result is biased toward leaving things where they were. Any function whose value comes from being applied exactly once should refuse to be applied twice rather than relying on the caller to remember. The second lesson is about reading: the generator printed the defect on every run, above the line being watched.

<small>Shuffling the answers twice put 37 percent of the keys at A (INC-0068)</small>

- [ ] Writing a second tool for the same job in a different context reproduces every detail the first one learned the hard way, unless the detail is written down somewhere the second author will look. This is INC-0067 seen from the other side: there the callers were not migrated to the helper, here the helper's behaviour was not carried into the second implementation. Both are the cost of a rule living in code rather than in a statement of the rule.

<small>The second tool that appends a clause did not carry the first one's rule about the full stop (INC-0070)</small>

- [ ] A report that truncates its output invites the reader to write text that continues it, and a tool that appends will put that text somewhere else. Either the report should not truncate the field the caller has to write against, or the tool should refuse input shaped like a continuation. The cheap half is the refusal, because it is one condition and it cannot be forgotten, while remembering not to write continuations is a habit that has to hold every time.

<small>Clauses written from a truncated report were appended to the end of the wrong word (INC-0071)</small>

- [ ] A guard that takes the intent as an argument is only as good as the argument, and an argument derived by hand from the same data the guard is checking is a second implementation of the thing being checked. It fails in the direction that is hardest to see: too high an intent demands a rank the clauses cannot reach, and the author satisfies it by writing more clauses than the plan called for, which skews the distribution the other way while every check passes. Derive the intent from the data with the code that already reads it.

<small>check_lift was told a one clause entry lifted two distractors (INC-0073)</small>

- [ ] A guard written from the instance in front of you covers that instance. INC-0074 was a bare infinitive in a noun slot, so the guard looked for bare infinitives, and the sentence one screen away in the same file was a wh clause in a clause slot and went straight through. The general defect was never the infinitive; it was that a corpus field carries no record of the grammatical shape it was written in, and any template may reuse it. So the guard has to be stated over the class, every field against every slot, not over the token that happened to be wrong first. The other half of this is where it was found: the distractor version was spotted first because it is louder, and the version in the key, which is three times as damaging, was found only because the first one prompted a second look. Reading one rendered item per schema would have caught both on the day they were written, and costs less than either fix.

<small>Every correct answer on one GMAT schema was ungrammatical, and the guard written an hour earlier could not see it (INC-0075)</small>

- [ ] A standard library function whose name is a plausible description of half of what it does will be used for that half. capitalize() reads as "make this the start of a sentence" and is in fact "make this the start of a sentence and flatten everything else", and the damage is invisible until a value happens to contain a capital. The guard is not a test that the output looks right, because the output looked right for every value that had no name in it. The guard is to ban the function: the correct one is three characters of slicing, the wrong one is never what a generator wants, and a lint catches it in the diff rather than in the bank.

<small>str.capitalize() lowercased the rest of the sentence, and took the proper nouns with it (INC-0076)</small>

- [ ] Presentation rules travel with the value, and a value formatted at the point of use is formatted by whoever was writing that line. Eight schemas each wrote the same two characters and all eight omitted the same exception, which is not eight mistakes but one missing function. The give away is the shape of the defect: identical output in unrelated files means the knowledge was never in one place. The check that catches it cannot be on the arithmetic, because the arithmetic was always right, so it has to be on the rendered string.

<small>A coefficient of one was printed as 1x on 2,353 shipped items (INC-0078)</small>

- [ ] A check is scoped to a grain, and the grain is a claim about where a defect can live. INC-0069 moved the grain from the section to the file for hand written banks and stopped there, so the same defect went on living one level down in generated ones, where there are far more items. When a check finds something by being made finer, the question to ask immediately is what else is measured at the old grain. The second half is about which populations a check can see: this one ran on what the test harness loads, which is a sample chosen for a different purpose, and a sample chosen for a different purpose is not a population you can make claims about.

<small>A generated schema was playable at 100 percent by picking the third shortest option, and no check looked at generated schemas (INC-0079)</small>

- [ ] Two corpora side by side, one guarded per unit and one guarded only in total, is not two levels of rigour but one measurement and one blind spot. When a check exists for the bank, ask what else is shipped alongside it and counted only in aggregate.

<small>One flashcard for a skill, for as long as anyone cared to look (INC-0085)</small>

- [ ] A list of misconceptions is a list of labels and a student sees numbers. Where every characteristic error runs the same direction the key sits at a predictable place in the ordered options however carefully the item is shuffled, because the shuffler can only place it among the candidates it is handed. When writing wrong answers, sort them next to the right one and look at which side they fall on.

<small>Every wrong answer a schema could think of was bigger than the right one (INC-0091)</small>

- [ ] Generated data gets checked for the properties the questions need, monotone and positive and distinguishable, and not for the properties the world needs. A quantity whose name contains its own bound is stating a constraint that the generator has no way to hear unless someone writes it down as data. When a field's name says out of 100, or percent, or per 100, that is a ceiling and the generator should be told.

<small>A reading of 100.2 for a quantity the table itself calls out of 100 (INC-0094)</small>

- [ ] Fixing an instance of a defect is the moment to sweep for the rest of it, and the sweep is worth running even when it is too noisy to become a check. A hundred to one false alarm rate is useless in CI and perfectly workable for one person reading the hits once, and it found three defects that no guard would have. What ships from the sweep is the narrow check on the fields that actually failed, not the broad one that found them.

<small>Three more stored phrases in front of a verb that did not agree with them (INC-0096)</small>

- [ ] A check downgraded because a source is unreachable carries an assumption with no expiry date on it, and the assumption is usually narrower than the downgrade. Blocked was true of one URL; it was applied to the fact. Before accepting that a figure cannot be sourced, try the organisation that publishes it and not just the page you first landed on. And a warning nobody can act on is not a lesser guard, it is a comment: if the condition is genuinely acceptable, record why and clear it, and if it is not, make it fail.

<small>A blocked source turned a real check into a permanent warning, and the flagship exam published its structure table uncited for as long as the page existed (INC-0100)</small>

- [ ] When the same file already solves a problem correctly, the second implementation is the one to distrust: the reference was available and was not used, so whatever made it easy to skip will make it easy to skip again. And a generator that writes a word describing the data, rather than computing it from the data, has an assumption in it that nothing will ever check unless the check is written against the relation rather than the field.

<small>A table question asked which row rose the most when every row had fallen (INC-0102)</small>

- [ ] A provenance label is a factual claim and deserves the same checking as the number it annotates. A proxy that is right for the common case (one host per school) will be confidently wrong for every institution that publishes from more than one office, and the error flatters no one: it tells readers that official data is unofficial.

<small>Official figures from a school's own university were footnoted as secondary and 'indicative rather than official' (INC-0106)</small>

- [ ] When a page publishes a number the build computes, the page should read it from the build. A figure typed in beside a placeholder is the one that will be wrong next month, and it will be wrong on the page whose whole purpose is to say how carefully the numbers are checked.

<small>The college methodology page quoted its own checks from the day they were written, and the build had moved on (INC-0108)</small>

- [ ] When a key is derived from premises held as data, each premise has to be found in what the reader sees, and a check should prove that by searching the rendered text rather than the data. Fixing the one missing premise that was noticed is not the same as asking which premises the reader is shown: list them all, and look for each one in the output.

<small>Reading inference questions credited a key that depended on a rule the passage never stated (INC-0114)</small>

- [ ] Practice material is still published writing. A passage that is only there to be read carefully is still read by people who know the subject, and a question built on it can ask them to endorse the error.

<small>A passage described zircon dating with the vocabulary of argon dating (INC-0116)</small>

- [ ] An empty field records that a search came up empty, not that the thing does not exist. Write the absence as what was done ('not verified from the sources we could read'), never as a fact about the world, because the world is the part nobody checked.

<small>School pages said a program does not publish an acceptance rate when we had only not found one (INC-0118)</small>

- [ ] When two pieces of code each supply part of a sentence, decide which one owns each word. A template and the code that fills it both writing the unit is invisible in either one read alone; only reading the output as a sentence shows it.

<small>A generated word problem printed its rate unit twice (INC-0120)</small>

- [ ] An edit tool that throws away its input leaves the diff as the only record of what it changed. Audit such edits from the commit that made them, and keep the table that drove a bulk edit in the repository, because the table is what lets the next check see the seam.

<small>The banks lengthened in place carried the same garbled seams, and nothing recorded what had been appended (INC-0121)</small>

- [ ] A check that passes on a random draw has told you about that draw. When a schema ships a handful of items, a property the whole bank needs, such as keys spread over every length rank, has to be assigned rather than sampled. And a fix proven on one variant of a shared class belongs on the class: the variants left alone were passing on luck, and the next unrelated change spends it.

<small>Reading questions asked once of each passage passed the length check on the luck of the draw, and two new passages re-rolled it (INC-0122)</small>

- [ ] A template that puts a fixed word beside a value has decided something about a value it has not seen. Let the value choose the words that depend on it, and check the built page for the rule rather than for the word that broke last time: a guard that lists yesterday's mistake misses the same mistake spelled another way.

<small>Two school pages wrote "a 18.8% acceptance rate", because the article was chosen before the number arrived (INC-0134)</small>

- [ ] Any field that is printed on a page is copy, whatever the schema calls it. A length cap that cuts at a character count will eventually cut a word in half and publish it, so shorten text by clause or leave it to a person; and check that printed text ends the way text ends, because a check that a field is present and sourced passes a field cut off halfway through.

<small>Two school pages printed a source note cut off mid-word, because the research merge kept the first 300 characters of every note and nothing checked that printed text ends as a clause (INC-0141)</small>

- [ ] When two fields hold similar text for different purposes, enforce the difference where the data is validated. A convention that only readers of the schema know is one the next edit will not follow.

<small>Six school figures kept their source description under the key reserved for our own commentary, so their pages printed them bare and the source check never read them (INC-0143)</small>

- [ ] A caveat in a note does not travel with a number: every consumer of the field reads the field's meaning, not the note. When a value does not fit the field's meaning, give it a field that does, or leave the field blank.

<small>BYU Marriott's two-year tuition total was published as tuition per year, and the fit card doubled it again (INC-0144)</small>

- [ ] A pronoun is a claim about what came before it. In generated prose the sentence before is chosen by the data, not by the writer, so name the subject of every sentence that carries a figure.

<small>Wharton's page credited its Class of 2026 GRE scores to the Class of 2027, because the GRE sentence began with Its and followed whichever class came last (INC-0145)</small>

- [ ] A shorthand in a schema is read by code with one meaning and written by people and tools with whatever meaning they had in mind. Where a value names what something is, require it to say so in full; a bare year cannot say whether it is a graduating class, an entering class, or the day a page was read.

<small>Six school records stored a bare year as their class, which the site printed as a Class of that year, so Notre Dame's average cohort size became "The Class of 2027 has 85 students" (INC-0147) </small>

- [ ] A figure with a unit that its source leaves out is not verified until the unit is. Deducing it from the number's shape feels like evidence and is not; either the source says it, or something outside the number proves it, or the figure stays blank.

<small>Eleven GMAT figures on ten school pages were filed under an edition their sources never name, by guesses from the class year, the last digit or how high the number was (INC-0157)</small>

- [ ] A total is only as good as the rows it adds. When a page breaks a figure into parts, record which parts the figure counts, and prefer arithmetic a check can redo over a description it has to trust.

<small>Baylor's school page gave its whole MBA program's entering class of 80, with 15 part-time students in it, as the full-time class size, and a source note that named only the two full-time tracks (INC-0159)</small>

- [ ] What a page publishes has to be what the checks read. When a builder falls back to a second field to fill a gap, that field is published, and every check written for the first field has to cover it too.

<small>The GMAT guide's Cost tile showed $275 and $300 from a note that no check read, cited to a page that prints no price (INC-0160)</small>

- [ ] A qualifier recorded next to a figure has to reach every sentence that states the figure. When prose is generated from data, words like reported, median or estimated are part of the value, and a fixed word in a template silently overrides what the record says.

<small>School pages called an acceptance rate its source gives as an estimate "reported" (INC-0167)</small>

- [ ] When a source contradicts itself, pick nothing silently. Show the figure it states, show the figures it contradicts, and say which cannot be settled; a reader given the conflict can judge it, a reader given one number cannot.

<small>Emory's acceptance rate was shown beside counts that divide to a different rate, with nothing on the page saying so (INC-0168)</small>

- [ ] A fact written into prose without its source cannot be rechecked, so it stays whatever it was the day it was written. State the figure with the page it comes from, or link the page and say nothing it does not say.

<small>A published post said GMAC caps GMAT attempts across a lifetime, which its current retake policy does not say (INC-0170)</small>

- [ ] A check that reads two sides must read them the same way. Reading number words on the page but not in the fact meant any figure a writer happened to spell out was exempt, and the exemption was invisible because the check reported nothing missing rather than nothing checked.

<small>The LSAT guide said scores come out roughly three weeks after each administration, citing a dates page that says no such thing, and the source check never looked because the fact wrote its number as a word (INC-0177)</small>

- [ ] A rule with a list of places it applies needs a check for each place on the list. Finishing the headings made the buttons look done by association, and only reading the rule again, clause by clause, showed the gap.

<small>Buttons across the site named their actions in sentence case, against the house rule that a button naming a destination or action is Title Case, and no check read them (INC-0178)</small>

- [ ] Changing the case of text you did not write is only safe when you know what its first word is. A generator that lowers or raises a first letter needs a check that the word is one it may change, or the corpus has to be written so it never matters.

<small>A new reading passage's revision opened with a place name, and the stated-idea questions printed it as "ardley cheese" (INC-0179)</small>

- [ ] A check that finds nothing wrong has only checked what it could see. Count the things it had no way to test as unchecked rather than as passed, or a clean report will quietly include everything the check was blind to.

<small>The GMAT's delivery fact said appointments are available year round with no fixed testing windows, words its cited page no longer carries, and the source check passed it because 20 of the 113 exam facts carry no number for it to look for (INC-0180)</small>

- [ ] When every wrong answer is a true sentence from the same passage, each stem has to pick out exactly one sentence, and a stem that names a person or a study picks out everything said about them. Write down, for every question form, what else it is true of, and check the items against that list: the closer a wrong answer sits to the right one, the likelier it is to be right as well.

<small>Stated idea questions that name a study offered another of that study's sentences as a wrong answer, so 178 of 378 had two defensible answers (INC-0186)</small>

- [ ] A template is written against the fields that exist when it is written, and the corpus goes on growing in a style the template never saw. When a new passage is added, render every question the schemas ask of it, not the ones the passage was written for, and write each stem so that it fits what the field is, a criticism or a fact, rather than what the first few happened to be.

<small>The stated idea question about the earlier account read 'failed to address the fact that the account could not explain', on 143 of 198 items (INC-0187)</small>

- [ ] A source's first sentence is not always its answer. When a page states a rule and qualifies it in the next sentence, the qualifier is part of the fact: read to the end of the paragraph before quoting, and when a test maker changes what schools receive, reread every sentence that tells a reader what a school will see.

<small>Two queued posts said a GMAT score report carries only the score you send, after GMAC's Superscore began adding the best section scores from every attempt (INC-0188)</small>

- [ ] A position is not part of a sentence. When content is written against one order and shipped in another, anything that points at the order, a letter, 'the second option', 'the last choice', has to move with it or be written so it does not depend on it. Make the reference explicit, resolve it where the order is decided, and check the result against the key rather than trusting the text that was true when it was written.

<small>Watch for notes named answer choices by letter or position, and the shuffle that moves the key never moved them, so 124 notes called the correct answer a trap (INC-0189)</small>

- [ ] A clause added for length is still a claim. When text is appended to a wrong answer to change its shape, read the result as a test taker would, against the question it answers, because the easiest way to lengthen a false statement is to add the reason that would make it true.

<small>A length correction clause turned an LSAT must be true distractor into a second correct answer, and its note said so (INC-0190)</small>

- [ ] Check a wrong answer against the premises, not against what it was meant to be. A note written from the author's intention describes the error the option was supposed to commit, so it cannot catch an option that came out valid; write each option out as symbols beside the premises before writing what is wrong with it.

<small>Three LSAT reasoning items offered wrong answers with the key's logical force, and each note said they lacked it (INC-0191)</small>

- [ ] A question whose answer is the passage's own sentence tests finding, not reading. Say the correct answer in other words, and measure what a reader who only matches words would score, because the shortcut is invisible item by item and obvious across a bank.

<small>Hand written 'according to the passage' questions could be answered by choosing the option made of the passage's words (INC-0192)</small>

- [ ] A check for text said twice has to allow for the copy not being exact. A seam left by an appended clause can differ from the text it repeats by an article, so take the small words out before comparing, and measure the rule over every bank to see that it finds the defect and nothing else.

<small>An LSAT reading choice said 'to particular facts to the particular facts', and the doubled phrase check missed it because the copy added 'the' (INC-0194)</small>

- [ ] A template that slots a clause into a sentence has to know where the clause ends. When the slotted text can carry commas of its own, end the sentence after it or set it off with punctuation, and check the rule by rendering every sentence the template makes, since a check on the pieces alone passes every piece of a sentence that misreads.

<small>Eight LSAT structure explanations ran a claim with a comma of its own into the next words ('where the water has warmed the most and is offered as a reason') (INC-0195)</small>

- [ ] When wrong answers are drawn from the jobs of the other parts of the same text, each job's wording has to be false of every part but its own, and that has to be checked across the table, not sentence by sentence. Two parts that differ only in order can only be told apart by naming the order, so any wording for either that leaves it out is true of both.

<small>GRE sentence-function questions offered a second right answer: the third sentence's job, worded so that it was true of the fourth sentence too (INC-0199)</small>

- [ ] A figure's note is part of the figure. When a table prints the note beside the number, every sentence that restates the number has to carry what the note says about it, average, about or more than, or it states the figure more exactly than the source does. And a count of one thing and an average over several are different statistics even when the number is the same: the wording has to say which one it is.

<small>School pages stated averaged and approximate figures as exact: a cohort average as one class's size, and approximate tuition, work experience and scholarships without their qualifier (INC-0200)</small>

- [ ] A statement about what the product is goes stale each time the product grows, and a check for one way of being stale (in development, coming soon) passes the others (built for one exam specifically). When the product changes, search every page for each way it used to be described, not just the words that said it was unfinished. And when a figure leaves the data because its source never said it, search the prose for it too, because posts repeat the data in their own words and keep the citation.

<small>The live GRE format guide told GRE readers our trainer was built for the GMAT Focus Edition specifically, beside a button to the GRE trainer, and credited ETS with a 260 to 340 total ETS does not report (INC-0207)</small>

- [ ] When a guard encodes a principle, write the pattern for the principle rather than for the first case it caught: a check for 'most programs' that passes 'most applicants' teaches writers that the rule is about programs. List every group the content talks about and test the pattern on each. And read each quoted claim against its source as written, because a paraphrase that adds 'most' or drops 'on average' changes what the source said while keeping its name on it.

<small>Posts stated what most test takers, students, applicants and employers do with no source, and one misquoted ACT, because the guard for claims about most of a group read only programs and schools (INC-0208)</small>

- [ ] A rule that only a comment states is not enforced. When content is reused across exams by mapping, write down what each exam excludes, in the test maker's words, and check every map and every built bank against that list, because a list copied from the exam that does include the topic carries it over without anyone choosing to.

<small>The GRE trainer served about 195 trigonometry questions, though ETS says the Quantitative measure does not include trigonometry (INC-0209)</small>

- [ ] A content rule enforced by one renderer is a rule about that renderer, not about the site. When the same kind of prose is published by more than one builder, the check belongs where every builder can call it, and a new builder should start by asking which existing content checks it skips.

<small>The GMAT reading guide told students what most people do with Verbal time, with nothing behind it, because the claims check covered blog posts only (INC-0214)</small>

Database

- [ ] Postgres fires same-timing triggers alphabetically. If two triggers on one table have an order dependency, encode it in the name, and test the outcome rather than the code.

<small>Postgres fired triggers alphabetically and revoked every adult opt-in (INC-0012)</small>

- [ ] UPDATE OF is a statement-shape filter, not a change filter. If you need 'when this value changed', compare OLD and NEW yourself.

<small>UPDATE OF fires on the columns named in the statement, not the ones that changed (INC-0013)</small>

- [ ] In Postgres, revoking from every role you can name still leaves PUBLIC. Verify with the advisors or by reading the acl, never by reading your own migration.

<small>Revoking EXECUTE from anon did nothing while PUBLIC still held it (INC-0020)</small>

- [ ] An empty catch block around a write is a silent-loss defect waiting to be born. If a save can fail, the person must be told; a success toast that fires regardless of the result is worse than no toast, because it actively teaches the user the data is safe. And where two layers of authorisation have to agree, something has to compare them: the one that is wrong will not announce itself.

<small>Twelve profile fields said Saved and never reached the account (INC-0056)</small>

Front end

- [ ] **Learned 2 times over.** When a site grows from one of something to several, grep for the literals the single case left behind. A link that was right when there was one trainer is wrong four times over when there are five, and it still looks right to anyone checking the page it was written for.

<small>The trainer button on four exam guides and on the study guide hub opened the GMAT trainer (INC-0128)</small>

- [ ] Generated code is code. If your build writes JavaScript into a string, the build must parse the result, because the blast radius of one bad character is the whole file, not the line.

<small>Unescaped quotes in onclick strings took the whole app down (INC-0001)</small>

- [ ] Nobody notices a page getting slower one commit at a time. Put the number in a test the first time you care about it, not the first time somebody complains.

<small>The whole bank blocked first paint: 20 seconds to the first question on 3G (INC-0015)</small>

- [ ] try/catch around an API that returns errors is decoration. Know which convention each call uses before you wrap it.

<small>A try/catch caught nothing because the call inside it does not throw (INC-0022)</small>

- [ ] When you change a filename scheme, grep for the old name as a string, including inside regexes. A pattern is a hardcoded name wearing a disguise.

<small>The service worker precache regex did not match the chunked files (INC-0028)</small>

- [ ] A fixed rounding rule is wrong at one end of the range or the other. Pick the precision from the magnitude, and write the expected number down before you write the code that produces it.

<small>ARR was rounded to whole dollars and lost real money at small scale (INC-0032)</small>

- [ ] The moment a single-tenant store becomes multi-tenant, every key in it is a collision waiting to happen. Enumerate the writers before you add the second tenant, not after.

<small>One exam's progress blob overwrote another's (INC-0038)</small>

- [ ] A conditional that treats not-A as the original case is a bug the day a third case exists. Resolve variants from data, and the third one costs a row rather than a search.

<small>The GRE app told GRE students to calibrate at the wrong test maker's site (INC-0043)</small>

- [ ] Anything that reports failures must not be able to report its own. Check whether each call rejects or throws before you wrap it, and make the reporting path unable to re-enter itself.

<small>The error reporter reported its own failures, in a loop (INC-0045)</small>

- [ ] Async on a script tag decides when it runs, not when it downloads, so an async tag still competes for bandwidth with everything the page is waiting for. Load what the first screen does not need after the first screen exists, and re-measure the slow path whenever a file is split, because splitting changes the number of downloads competing at once.

<small>The GMAT trainer's first question on a slow connection took twice as long as when the bank split shipped (INC-0113)</small>

- [ ] A script tag is a dependency on the host it names, with that host's speed and uptime. Code a page needs before it can run belongs on the page's own origin; another host belongs only where the page still works when that host is slow or gone.

<small>The trainer app ran its own code only after a script from cdn.jsdelivr.net had loaded, so a slow CDN stalled the app and timed out CI's games smoke (INC-0148)</small>

- [ ] A view that is redrawn from saved state loses whatever the user has not saved yet. When a background event can redraw a form, carry the fields' current values across the redraw. And a test that is not in CI is not guarding anything: it fails quietly until someone happens to run it.

<small>A half-typed sign-in on the Account page vanished when a deferred bank chunk arrived and redrew the page, and the smoke that shows it had been failing unnoticed outside CI (INC-0149)</small>

- [ ] An invariant about what a shipped page contains belongs on the shipped page. A check at one entry point protects only the pages built through it, and a builder that assembles pages its own way walks past it; counting the thing in every output file catches all of them, including builders written later.

<small>Every blog page carried the error beacon twice, because the blog builder appends its own after a footer that already carries one (INC-0197)</small>

- [ ] A backslash written into a string literal is a request to the language, not a character, and languages answer it differently: '\;' is a bare semicolon in JavaScript, while Python keeps an escape it does not know and gives a backslash and a semicolon. When code is copied between languages, an escape that looks identical can change meaning, so test the output rather than the source line, and give the test an input that needs the escape.

<small>The application checklist's calendar file left semicolons unescaped, because the escape was written '\;', which JavaScript reads as a bare semicolon (INC-0201)</small>

- [ ] When you fix a literal that went stale because there came to be more than one of something, grep the whole site for that literal, not just the file you found it in. A link that points at the original trainer looks right on every page about the original exam, so it survives anywhere nobody reads a page about another one.

<small>The Start a Free Round button at the foot of every blog post opened the GMAT trainer, including on the LSAT, GRE, SAT and ACT posts (INC-0206)</small>

- [ ] A default is what everyone who never changes it is shown, so a personal value left as a default is published to every user, and it looks right to exactly the person who put it there. Anything a person would fill in about themselves, such as a name, initials or a goal, should default to empty, and some test should load the product as a stranger and read what it says to them. Once a bad default has been saved into people's stored state, fixing the default is not enough: the stored copies need a one-time migration, and a value the person typed needs a mark that tells it apart from the default.

<small>Every trainer greeted a visitor who had typed no name as Hunter, the owner's first name, and saved that name to signed-in accounts (INC-0210)</small>

Infrastructure and deploy

- [ ] **Learned 2 times over.** Two hostnames are two origins and therefore two of everything the browser scopes by origin. Pick one and redirect on the server, not in a meta tag.

<small>www and the apex were two origins, so consent and rankings split in half (INC-0023)</small>

- [ ] An exclusion list is a denylist, and denylists are wrong by omission. Derive the allowed set from what the built pages actually reference.

<small>The .git directory was being served on production (INC-0024)</small>

- [ ] Verify a cache-fronted fix with a cache-busting request, or you are testing the cache. This costs one query parameter and saves an hour of chasing a fix that already worked.

<small>A stale edge cache made a fixed 404 look like a live 200 (INC-0025)</small>

- [ ] Know your platform's hard limits and assert them in the build. A deploy-time rejection is a bad place to learn a number your build could have told you.

<small>A 29.88 MiB bank file would have failed the deploy the moment it merged (INC-0027)</small>

- [ ] A guard that is a hand-kept list of safe paths is written by the same hand that made the mistake. Derive the safe set from the artefact, not from memory.

<small>Excluding the design directory would have shipped the legal pages with no styling (INC-0047)</small>

- [ ] An optional safety parameter is a guard only while its value is read rather than recalled. Fill it from memory and it becomes a coin flip that looks exactly like a guard from the outside, and the flips it wins are the ones where it was not needed. A value that is cheap to read is never worth remembering: reading costs one command, and the failure mode of remembering is an operation that succeeds on the wrong thing.

<small>A safety parameter filled in from memory, which makes it a coin flip rather than a guard (INC-0095)</small>

Interface and data display

- [ ] Chart form is a claim about the data. A line claims the values in between existed. Ask whether that claim is true before choosing it.

<small>A line chart interpolated between discrete money events (INC-0034)</small>

- [ ] Never encode a state by colour alone. The word also survives greyscale printing, forced-colors mode and a glance from across a room, so it is better for everyone and not only for the people it is required by.

<small>Two games signalled right and wrong by colour alone (INC-0052)</small>

- [ ] Copy that says above, below, left or right is a hard dependency on layout that nothing checks. Name the thing instead, and the sentence survives every rearrangement.

<small>The fit card told readers to fill in fields that were below it (INC-0058)</small>

Payments

- [ ] Idempotency belongs in the database, not in the handler. Handlers race; unique indexes do not.

<small>Two Stripe event types announce one new subscription (INC-0035)</small>

- [ ] Read what a payment field means, not what it is called. Cumulative and incremental fields look identical until the second event.

<small>charge.amount_refunded is cumulative, so partial refunds double-count (INC-0036)</small>

- [ ] When a write is destructive, capture what you need from the old value first. Ask what question you will want to answer after this row is gone.

<small>Cancellation erases the number you need to record the cancellation (INC-0037)</small>

- [ ] Row Level Security is row-level. Which columns a role may write is a separate grant, and anything money depends on belongs to the service role alone.

<small>A signed-in user could grant themselves a paid plan (INC-0041)</small>

- [ ] Enumerate every value a third-party status field can take before you branch on one of them. The value you did not think of is usually the one that matters commercially.

<small>Every trialing subscriber would have been left on the free plan (INC-0042)</small>

Scoring and selection

- [ ] Check that your instrumentation fired at all before you trust anything built on it. An empty table looks identical to a quiet week.

<small>The adaptive engine was learning from nobody (INC-0005)</small>

- [ ] A type system spread across a renderer and a grader will drift. The cheapest guard is one that exercises every variant end to end, once.

<small>Sentence equivalence items were unanswerable however well you answered (INC-0006)</small>

- [ ] If your system branches on difficulty, measure that the branches actually differ. A label is not a property.

<small>The easier module was not easier (INC-0040)</small>

- [ ] When you add a filter, find every path that adds items after the filter runs. A gate on the entry point is not a gate on the set.

<small>Fixing repeats directly left a side door through passage groups (INC-0051)</small>

- [ ] Anything a page promises is the same for everyone has to be assigned, stored and served, not recomputed from whatever happens to be loaded, because the recomputation will eventually run against different inputs. And name a metric for what it counts: a streak that resets on a wrong answer is an accuracy score, and calling it a habit discourages the people it was meant to keep.

<small>The question of the day promised to be the same for everyone and was not, and its streak punished a wrong answer (INC-0111)</small>

- [ ] Measure an adaptive policy per student, not in aggregate: a pooled statistic averages the starved students with the well-served ones and reports a system that works. A targeting rule also needs a floor it can never trade away, because the students it concentrates hardest on are the ones whose picture of themselves it leaves emptiest.

<small>Weakest-first selection starved skills for struggling students, down to none at all (INC-0112)</small>

- [ ] An identifier that anything outside the build stores must be derived from what it identifies, not from where it happened to land. A position is a fine name only for something nobody keeps.

<small>Generated item ids were positions, so a bank change pointed stored reviews at different questions (INC-0127)</small>

Search and metadata

- [ ] **Learned 6 times over.** When a fix names a class of input, such as 'the stat field is free text', find every place that input is used before closing it. A correct rule written inside one caller protects that caller, and the existence of the rule makes the unprotected siblings look protected.

<small>Half the school pages opened with a data note pasted into a sentence, and the structured data repeated it (INC-0104)</small>

- [ ] **Learned 3 times over.** An enumeration that has to be kept in step by memory will fall out of step, and the failure is silent because nothing downstream can tell the difference between a section that was excluded on purpose and one that was forgotten. Derive the list from what was actually produced, and when that is not practical, make something compare the two. Publishing work nobody can find is not a smaller version of publishing it.

<small>A 139 page section shipped with no route into it from the sitemap (INC-0103)</small>

- [ ] **Learned 2 times over.** Any number in user-facing copy that describes the size of something must be computed from that thing at build time. The moment it is typed, it has a half-life.

<small>A hardcoded count in the meta description went stale, and Google showed it (INC-0014)
</small>

- [ ] **Learned 2 times over.** Any sentence about your own product's state is data, and data belongs in one place: build the sentence from the source that knows, and check every page that could repeat it, including the text only search engines read.

<small>The exam guides hub told search engines only the GMAT and SAT trainers were live, eleven days after all five were (INC-0138)</small>

- [ ] A refactor that moves data has to be followed to every reader, and a loop over nothing is the quietest failure in programming. Derive counts from one source and assert they agree.

<small>School URLs vanished from the sitemap when the data file was split (INC-0002)</small>

- [ ] When one model feeds two pages, generate both from the model in the same pass. Two places that must agree will not, and the reader who notices is the reader you were trying to convince.

<small>Two pages told the same story with different numbers after a reweighting (INC-0049)</small>

- [ ] Test every noun in a title against the page's own data, because a title is a promise to someone who has not seen the page yet. Grouping two fields under one flag because they usually travel together is how a promise gets made on the pages where they do not.

<small>Seven college titles promised SAT scores the page does not have (INC-0107)</small>

- [ ] A URL that search engines know is not the site's to delete quietly; it belongs partly to everyone still linking to it. Removing the content is one decision and deciding what that address now says is another, and skipping the second one sends real visitors to whatever the 404 happens to be.

<small>Two withdrawn exam guides kept their search traffic and sent it to a 404 that is the whole GMAT trainer (INC-0109)</small>

- [ ] Structured data is published text that no one reads on the page, so check it the way a page is checked. Interpolating a record where its text was meant succeeds silently in Python, and the only place the mistake shows is the output, so scan the output for the printed form of a data structure.

<small>The exam hub's structured data described every exam with a printed Python dict (INC-0131)</small>

- [ ] A value copied from a page carries the context the page gave it, and loses that context wherever it is printed on its own. A name that is plain under its university's banner is anonymous in a page title, so store the form that stands alone.

<small>Five school pages were named only "School of Business", "College of Business" or "Graduate School of Management", so their titles and answers named no school (INC-0153)</small>

- [ ] A check that reports what is missing has to know every way the thing can be present. When its advice names the place where the fix belongs, it should read that place first; otherwise it keeps recommending work that was already done, and the people reading it learn to ignore it.

<small>The search report told us to redirect two URLs the Worker already redirects (INC-0198)</small>

- [ ] A link typed by hand is a guess about where something lives, and a guess that follows a pattern looks right to every reviewer. The only reliable check is to resolve every link in the built site against what is actually served, on every build, rather than following links from a sample of pages.

<small>Score calculators linked a study guide page that has never existed, and 54 college pages linked a category the college list does not have (INC-0213)</small>

Tests and guards

- [ ] **Learned 4 times over.** Loading a page is not the same as reading it. A chart can be a separate document inside the page that draws only when scrolled into view, and a page with trackers may never fall quiet, so a reader built for checking has to scroll like a person, collect every frame, and use waits that give up rather than hang.

<small>The source check's browser read returned Stanford's class profile without a single figure, because the figures are drawn in an embedded chart that renders only when scrolled into view (INC-0146)</small>

- [ ] **Learned 3 times over.** A path that exists on the machine you wrote the test on is not a path. Resolve environment-specific locations through one helper that falls back to the tool's own default, and return undefined rather than an empty string, because undefined means 'you decide' and an empty string means 'launch nothing'.

<small>A new browser suite hardcoded this machine's browser directory and crashed in CI (INC-0055)
</small>

- [ ] **Learned 3 times over.** Extracting a shared helper does not migrate the callers. The extraction fixes the file it was extracted from and leaves every sibling on the old path, which is INC-0059 and INC-0064 in a different costume: a correction applied to the instances in hand rather than to the pattern. Three suites had failed visibly and ten were wrong; the seven silent ones were found by the guard, not by reading. When a helper exists because a direct call was wrong, make the direct call fail the build, and let it enumerate the callers rather than enumerating them by hand.

<small>The browser path fix covered two suites and three others kept crashing (INC-0067)</small>

- [ ] **Learned 3 times over.** A checker has to know when it has not read its source. A response is not the page because it has text in it: a challenge, an error page or a login wall reads as a page with none of the facts on it, and every fact then looks wrong.

<small>The source check read mba.com's bot challenge page as the page it cites and reported the GMAT fact's numbers as missing from it (INC-0156)</small>

- [ ] **Learned 3 times over.** A checker's excuse category needs the same scrutiny as its findings. "Shows none of its figures" was given one cause and the cause was believed, but to a check that looks for numbers, a page whose every figure changed looks exactly like a page that never loaded. A category that explains itself away should instead say what it saw, and keep counting until a person has looked.

<small>The source check filed six school pages as probably built by JavaScript; five were something else, and three of them showed newer figures than the library published (INC-0158)</small>

- [ ] **Learned 2 times over.** A ratchet is only read while it is quiet. One that fires on noise gets re-recorded as a reflex, and the re-recording is indistinguishable from accepting a real regression, so the mechanism that exists to catch regressions becomes the mechanism that launders them. Give a comparison the same error bars as the measurement it is made from, and fix the thing that fires wrongly before the habit of clearing it forms.

<small>A ratchet that trips on sampling noise gets re-recorded rather than read (INC-0089)</small>

- [ ] **Learned 2 times over.** Run every browser suite when a site-wide element such as a modal ships, because a test nobody runs is a claim about the past. Every suite that loads a page with empty storage meets the new element before anything it was written to check.

<small>A browser suite outside CI had failed on its first click since the consent dialog shipped (INC-0110)</small>

- [ ] **Learned 2 times over.** A checker that reads a page has to read the page a person sees, not the file behind it. Markup can hold text no reader is shown, in comments, hidden elements and templates, and a site that retires a figure often leaves it there.

<small>The source check read text inside HTML comments as page text, and confirmed Arizona State's 43 percent women from a table row the school had commented out (INC-0152)</small>

- [ ] **Learned 2 times over.** A figure copied from a page its publisher rewrites every year expires on the publisher's schedule, not yours. Record which edition a figure came from, and have the check compare it with the edition the page now names; a page that cannot be read says nothing about whether what you took from it is still true, and a field the check does not read is not checked at all.

<small>Vanderbilt's median salary and 39 schools' Bloomberg ranks stayed on Bloomberg's 2025-26 edition after its pages moved to 2026-27, and nothing could see it: Bloomberg answers most reads with a bot challenge, and the source check never reads a rank (INC-0184)</small>

- [ ] Measure the moment the user can act, not a browser lifecycle event. A test that measures the wrong instant is worse than no test, because it produces a number people trust.

<small>The performance test waited for the load event, which waits for the thing being optimised (INC-0016)</small>

- [ ] A regex with a length bound is a guard with an expiry date. Assert the number of things checked, not only that the checks passed.

<small>A build guard silently narrowed its own scope when the counts grew (INC-0017)</small>

- [ ] Check your checker. A tool that cries wolf gets muted, and then it is worse than nothing.

<small>The audit reported a working link as broken (INC-0019)</small>

- [ ] A negative assertion passes when the system is broken in the right way. Always pair it with the positive case, or it is testing nothing.

<small>A test passed because nothing was configured at all (INC-0021)</small>

- [ ] A stochastic measurement on one seed is an anecdote. If your system has randomness, a single-run before-and-after cannot distinguish a change from the weather.

<small>I reported a regression that was seed noise (INC-0026)</small>

- [ ] Classify by what a thing is, not by what it is called. Naming conventions are a hint, never a type.

<small>The theme parity test matched by name prefix and caught a font size (INC-0030)</small>

- [ ] A test that is not wired into CI is a test that does not exist. Prove a suite runs in the place it is supposed to run, not on your machine.

<small>The browser suites had never actually run, because npm install had not (INC-0031)</small>

- [ ] Before you measure a layout, assert the thing is rendered. Hidden elements answer most DOM questions, and they answer them wrongly.

<small>The dashboard test measured a hidden element and passed (INC-0033)</small>

- [ ] An error feed with no filter is a feed nobody reads. Signal has to be defended, and the cheapest defence is a human label that mutes permanently, so the queue gets quieter as it learns.

<small>The error queue was 81 percent other people's blocked scripts (INC-0046)</small>

- [ ] A guard that reads the repository needs the repository. CI checkouts are shallow by default, and anything that walks history, blames a line or resolves an old hash will fail in a way that looks like the data is wrong rather than the clone.

<small>The playbook's own citation guard failed CI on its first run (INC-0053)</small>

- [ ] When you add a condition that skips a check, make sure it describes the failure and not something merely correlated with it. A skip is indistinguishable from a pass in the output, so the fix for a noisy check can silently delete it.

<small>The first fix asked the wrong question and muted a working check (INC-0054)</small>

- [ ] A commit hash is not a durable citation in a repository that squashes. Pull requests, issues and tags survive history rewriting; branch commits do not. Cite the thing that outlives the merge, and make any check of the other one advisory.

<small>The ledger cited commits that squash merging destroys (INC-0057)</small>

- [ ] A metric that moves against you when the product improves will eventually be used to justify reverting an improvement. When a number goes the wrong way after a change that should only have helped, measure the underlying thing directly before believing either the number or your own explanation of it. Never redefine the metric in the same change that made it look bad.

<small>A repeat metric that gets worse when the bank gets better (INC-0061)</small>

- [ ] An aggregate is a claim about whatever you grouped by. Group by the file and you have measured the file. State the grouping in the sentence that reports the result, and the overclaim becomes visible while you are writing it.

<small>The analysis chapter called one file eight failing guards (INC-0065)</small>

- [ ] A check that reports pass or fail from a handful of random draws is a check that will flip on work that has nothing to do with it, and the cost is not the false alarm. It is that the next real alarm arrives in a tool people have learned to shrug at. Before believing or dismissing a warning, run the thing it measures enough times to know its rate: that answers both whether this alarm is real and whether the check is worth keeping in its current form. Here the answer was that the engine was fine and the check was wrong, and both were worth knowing.

<small>A review bot check whose verdict was three coin flips warned on an unrelated bank change (INC-0077)</small>

- [ ] When you compare text you wrote against text someone else wrote, normalise both sides with the same rules, and remember the other side does not follow your house style: a comparison is only as fair as its most lenient reading of the source.

<small>The source check read a page's 2026-27, written with an en dash, as 2026 and 27, so every figure we labelled 2026-27 looked unsupported (INC-0140)</small>

- [ ] When a check learns a new way to read a source, every rule that reads a source has to learn it too. A second reader that lags the first is found only when the data first depends on the difference.

<small>The source check's blank-page rule judged a figure worked from two pages by its first page alone, and called mba.com's payment page blank while the second page was unreadable (INC-0161)</small>

- [ ] A test of a check should apply the check's rule, not a snapshot of the cases that existed when the test was written. An allowlist of known exceptions is a record of the past, and the first new legitimate case turns it into a false failure.

<small>The playbook's smoke test and its builder disagreed on what counts as an unresolved placeholder, so a correctly quoted one failed the smoke (INC-0165)</small>

- [ ] A test harness has to load a page the way a visitor does or its passes mean less than they say. file:// quietly breaks every root-relative path, and a noise filter written for one failure hides the next one of the same kind; when a test has to ignore an error to pass, it is worth asking what else that exemption lets through.

<small>The standalone page smoke test still loaded pages from disk, where the site's root-relative paths point at the filesystem root, so it failed once the fonts moved onto the site (INC-0166)</small>

- [ ] A check that tests a value in a different form from the one the code uses tests something else. Test what is printed, after every transformation the code applies, or make the test indifferent to the transformation.

<small>The reading corpus check accepted a near miss whose only negative word opened it, and the item builder, which capitalises first, could not use it (INC-0169)</small>

- [ ] Taking HTML apart with regular expressions works until a page puts markup where the pattern did not expect it, and then it fails silently, by producing text that looks plausible. A parser knows where a tag ends and costs nothing here, so anything whose output is used as evidence should read HTML with one.

<small>The source check ended a tag at the first > it met, so markup inside a quoted attribute leaked into the page text, and an element id's 5 confirmed GRE's five-year score validity on a page that never states it (INC-0173)</small>

- [ ] A cache should keep only what would be accepted as an answer. Caching a failure turns a transient fault into a permanent one, and it then looks like the source's fault rather than the cache's. And a detector that looks for a refusal's words misses a refusal that has none.

<small>The source check cached Imperva's script challenge as GMAC's policies PDF, a page with no words, so every later run with --cache reported the PDF unreadable from its own cache (INC-0175) </small>

- [ ] A reader of free text has to read every way people actually write the thing it looks for. Check the forms in the history, not the form you would write, before trusting that a pattern finds all of them.

<small>The ledger's harvest read only incident ids written out in full and matched commits by their short hash, so seven records fixed under a range such as "INC-0104 to INC-0110" cited nothing and 57 cited fixes came back every week as possible unrecorded defects (INC-0181)</small>

- [ ] A matching number is the figure only if it is the same kind of quantity. Before asking whether a number sits near its label, ask whether it could be the figure at all: a percentage cannot be a head count, and a label word that a page prints in every heading proves nothing about any one number.

<small>Arizona State's class size stayed at 47 after its class profile changed it to 45, because the source check confirmed the head count with the page's "Business 47%" under "Class composition" (INC-0182)</small>

- [ ] What is on screen is not what a reader can see. Content one click away behind the page's own controls is part of the page, so a reader built for checking has to open what the page offers to open, and leave closed what it hides for other reasons, such as menus.

<small>The source check's browser read left out collapsed accordion panels, so Berkeley Haas's and Buffalo's correct tuition figures were reported missing whenever a site refused the plain read (INC-0183) </small>

- [ ] A rank without its edition is a number without a date, and an average cannot tell a 2021 rank from a 2026 one. Make the edition a required part of the record, so a rank that cannot say when it is from is refused rather than scored.

<small>Maryland's and Penn State's Financial Times ranks were years old and named no edition, so the composite scored them beside FT's 2026 ranks, and Maryland's was a US position where every other FT entry is a global one (INC-0185)</small>

- [ ] Telemetry that ships inside every page runs wherever the page runs, including every test harness and every local preview, and it writes to production from all of them. Decide in the snippet which contexts may report, and test that decision by running the real snippet in each context, because the harness that would notice the leak is the one producing it.

<small>A smoke test's page errors went into the live error log, because the error beacon reports from any page it runs on, including one a test opened from disk (INC-0196)</small>

- [ ] A reader of someone else's page should treat as rules only what the page promises, not what one reading of it happened to show. Two pages from one publisher are kept on their own schedules, so a check that both list the same things fails the week one is tidied and the other is not, and an order seen on every row so far is a pattern, not a promise. When a strict reader refuses, everything behind the refusal goes unchecked, so once the refusal is fixed, read the whole difference it was hiding.

<small>The SAT date reader took three things one reading of College Board's pages showed as rules, and when the score release page changed it refused both pages, hiding a moved 2027 date behind the refusal (INC-0202)</small>

- [ ] When one beacon learns to ignore automation, every beacon on the page needs the same rule, and a test that needs a beacon to fire should opt in explicitly rather than being the reason the rule is missing. A test that can reach a production endpoint is writing production data whether it meant to or not, so check the live table for the shape of your own test traffic before trusting any count from it.

<small>Our own browser tests were 88 percent of the page views the site recorded, because the analytics beacon had no automation check (INC-0211)</small>

Adapting This to a Different Business

Most of this book is about a test-preparation platform. This chapter separates what was specific to that from what was structural, so you know what to keep.

What was specific

The item response theory, the exam registries, the score bands, the school library, the concordance between two editions of one exam. If you are not building an assessment product, none of it applies directly.

What was structural, and transfers unchanged

The infrastructure. Edge-hosted static pages, managed Postgres with Row Level Security, serverless functions, hosted checkout. Under thirty dollars a month, and it serves a hundred thousand generated items and fifteen hundred generated pages.

The build as the guard rail. Sources in, pages out, and every guard living in the build because the build is the only thing that sees the artefact.

Authorisation in the database. RLS everywhere, privileged reads behind definer functions, revoke from `PUBLIC`, never take an id from a request body.

The event-ledger pattern. Any external system that sends you events sends them out of order, more than once, and sometimes twice under different names. Never write the payload; re-read state. Put idempotency in a unique index. Keep an append-only record of facts alongside whatever current-state row you maintain, because current state is destroyed on every update.

The honesty discipline. Every published figure carries a source, a year and a URL. Unverifiable is null, never a guess. Never present an internal threshold as official. Never show a number about a person without its error bar. Decline to compute a metric you cannot compute honestly, and say why on the page.

The interface system. One token file, three typefaces, negative tracking on headings, `min-width: 0` on every grid and flex child, a complete palette on bare `:root` with only overrides in the theme blocks, colours in charts validated under simulated colour vision deficiency rather than chosen.

The test ladder and its failure modes. Every one of the five ways a test can be silently wrong applies to any codebase.

The defect ledger itself. The single most transferable artefact in this repository is `data/playbook/incidents.jsonl` and the script that turns it into this book.

The substitution table

THIS PROJECT	YOURS	KEEP
Item bank generators	Whatever your content is	Dedup keys canonical under every legitimate transformation; counts reported per stage; filters report rejections
Exam registry	Any multi-tenant or multi-variant dimension	Resolve differences from data, never from `if (x === 'a')`, which treats "not a" as "the original"
Ability estimation	Any inferred number about a user	A range not a point, a floor on the interval, a minimum evidence threshold, the method published
School library	Any library of external facts	One file per entity, source and year and URL on every figure, banned-source list enforced at build time
Two exam editions	Any legacy unit beside a current one	Never convert, compare on a common normalised measure, cite the concordance
Stripe plus Apple	Any two sources of one truth	Leave the first alone, add a derivation on top, move readers to the derivation
Forum moderation	Any user-generated content	Automatic screen flags, a human decides, the decision is stored and mutes permanently

The first week, for something new

1. Write the rules file. Use the template in the bootstrap pack and fill in the four

blanks.

1. Create the repository, the build script, and one page. Deploy that page to the real

domain.

1. Learn the platform's limits now: per-file size, which directories the uploader walks,

whether `www` and the apex are one origin, whether your headers file applies to responses your own code generates.

1. Stand up the database with RLS on every table from creation, and write one SQL smoke

test that asserts a guarantee and rolls itself back.

1. Write the token file. All of it, both themes, before the second page exists.
2. Then build the product.

The one paragraph version

Make failure loud before you make anything else. Put every guarantee in the layer that cannot be bypassed: the database for authorisation, the build for correctness of the artefact, a pinned number for anything statistical. Never write a figure you cannot regenerate. Never show a number you cannot defend. When something breaks, write down what you believed five minutes earlier, because that belief is the actual defect and it is the first thing you will lose.

The Bootstrap Pack: Starting a New Project With This Loaded

This chapter answers one question: what do you actually paste into a fresh Claude conversation so that a new build starts with everything this one learned?

The wrong answer, first

Do not paste the book. It is roughly the length of a short novel, and pasting it has three problems. It consumes context that the actual work needs. It buries the twenty operative rules inside several thousand sentences of reasoning about a test-prep site. And most of it is *explanation*, which is what you want when you are deciding something and noise when you are executing.

Reference material and operating instructions are different artefacts and need different delivery. The book is reference. The rules are instructions. Only the second goes in the prompt.

The pack

`python3 src/build_playbook.py` writes four files into `playbook/bootstrap/`, all generated from the same ledger as the book, so none of them can drift from it.

`CLAUDE.template.md` goes in the new repository's root. Claude Code reads it on every turn. It is the constitution: the house rules, the never-invent-a-value rule, the sourcing rules, the communication rule, and a short architecture section you fill in as you build. Four bracketed blanks at the top are the only things you have to change.

`KICKOFF.md` is the literal first message for a new conversation. It sets the working agreement, the phase order, and what a session should report. Paste it, then describe your business idea underneath it.

`RULES_DIGEST.md` is every lesson in the defect ledger, compressed to one line each and grouped by area. It is about three pages. This is the highest value-per-token artefact in the whole project: 214 real defects reduced to the rules that prevent them, with the specifics of this codebase stripped out.

`incidents.jsonl` is the raw ledger, copied so the new project can start appending to it on day one rather than starting an empty one.

How to load it, by surface

Claude Code, new repository. Copy `CLAUDE.template.md` to the repo root as `CLAUDE.md`, fill in the four blanks, and copy `incidents.jsonl` to `data/playbook/incidents.jsonl`. Copy `src/build_playbook.py` too, so the new project's own ledger becomes its own book from the first defect. Then open a session with `KICKOFF.md` as the first message.

A Claude Project on claude.ai. Put `BUILD_PLAYBOOK.md` and `RULES_DIGEST.md` into Project Knowledge, where they are retrieved on demand rather than sitting in every message. Put the contents of `CLAUDE.template.md`, filled in, into the project's custom instructions. Start the first conversation with `KICKOFF.md`.

A single conversation, no project. Paste `RULES_DIGEST.md`, then `KICKOFF.md`, then your idea. Attach `BUILD_PLAYBOOK.md` as a file if the surface allows it. That ordering matters: rules first, so they frame everything after, and the long reference last or not at all.

Why this layering works

Three reasons, and they are worth understanding rather than just following.

Retrieval beats repetition. A document in project knowledge is consulted when relevant. The same document in the prompt is re-read on every single turn, at full cost, mostly when it is irrelevant. For anything longer than a few pages, retrieval is strictly better.

Instructions need to be short enough to be followed. A rule competing with ten thousand words of context is a rule that gets applied inconsistently. The digest is deliberately brutal: one line per lesson, imperative mood, no examples. The examples are in the book, where they can be looked up when a rule seems wrong.

The ledger is the part that compounds. The recipe chapters age. The rules do not, because each one is the residue of a real failure, and the failure modes of software are considerably more stable than its tooling. A new project that starts with 214 defects already prevented is genuinely ahead, and every defect it hits of its own makes the next project further ahead still.

Keeping the loop closed

The pack is only worth carrying forward if the new project feeds it back. So the working agreement in `KICKOFF.md` includes one instruction that does the whole job:

> When something breaks, append a record to `data/playbook/incidents.jsonl` before fixing > it, while you still remember what you believed was true five minutes ago.

That is the entire maintenance burden. Everything else, the checklist, the analysis, the digest, the book, is generated from those records.

What to change for a different domain

`CLAUDE.template.md` has four blanks:

1. **The product, in one sentence**, and who it is for.
2. **The house style rules you will enforce.** Keep the dash ban unless you have a reason

not to; it is free and it makes the build's style enforcement real.

1. **The sourcing regime.** If you publish external facts, keep the whole thing including

the banned-source list. If you do not, delete that block rather than leaving it as decoration.

1. **The architecture section.** Empty at the start. Fill it in as decisions are made, and

only with things that are expensive to rediscover.

Everything else in the template is domain-independent and should be kept verbatim.